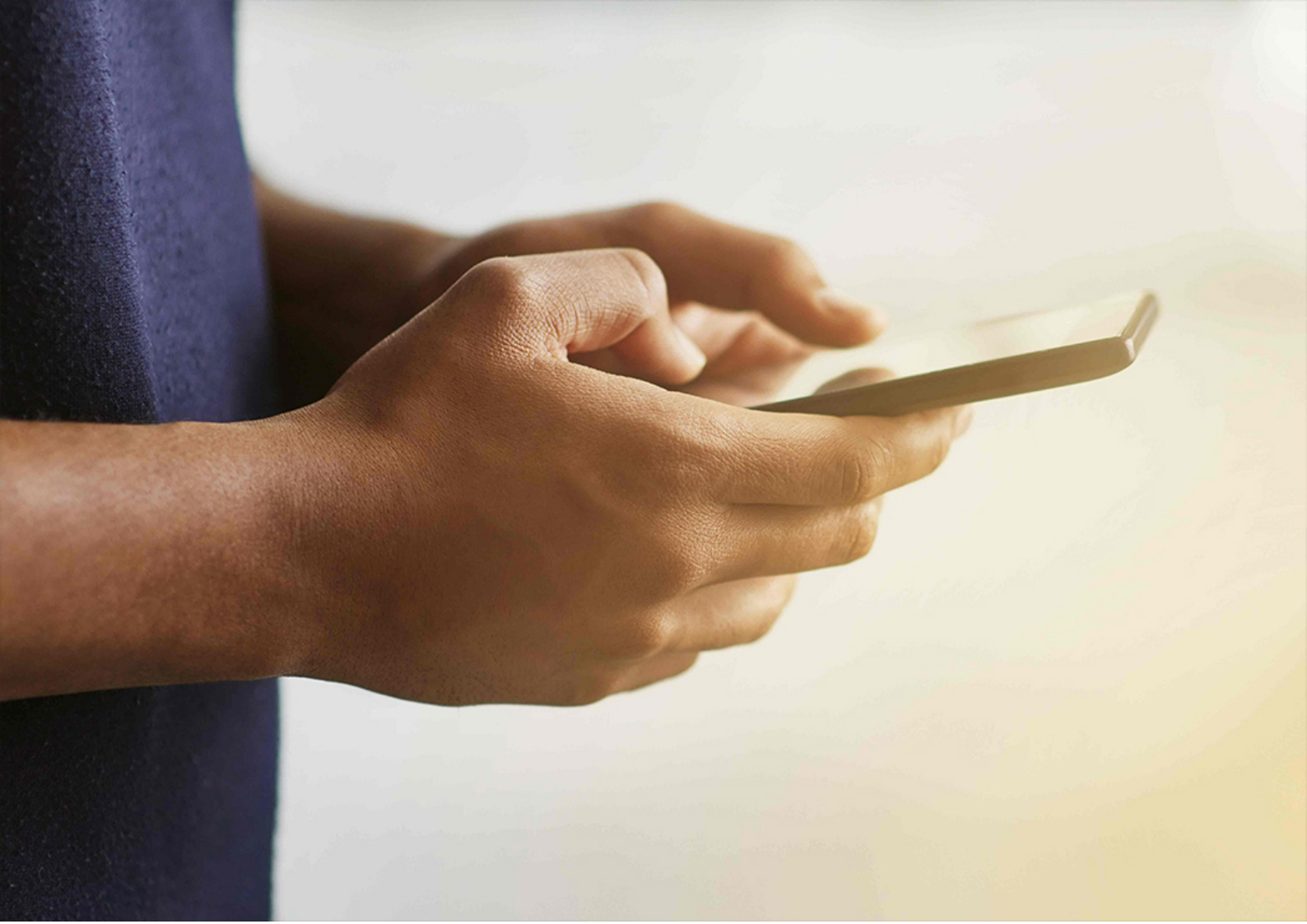




Service Guide | PUBLIC
2024-08-05

SAP AI Core

Content

1	What Is SAP AI Core?	6
2	What's New for SAP AI Core	9
2.1	2021 What's New for SAP AI Core (Archive)	20
3	Concepts	21
3.1	SAP AI Core Overview	21
	SAP AI Core Systems Overview	22
	AI API Overview	25
	Resource Groups	29
3.2	Generative AI Hub in SAP AI Core Overview	31
3.3	Terminology	32
4	Service Plans	36
4.1	Free Tier	38
	Enabling Free Tier	39
4.2	Metering and Pricing for SAP AI Core	40
4.3	Metering and Pricing for the Generative AI Hub	41
4.4	Choose a Resource Plan	42
	Service Usage Reporting	44
4.5	Update a Service Plan	44
	Changing Service Plans	44
5	Initial Setup	46
5.1	Enabling the Service in Cloud Foundry	47
	Create a Subaccount	48
	Enable Cloud Foundry	49
	Create a Space	51
	Add a Service Plan	53
	Create a Service Instance	54
	Create a Service Key	58
	Use a Service Key	61
5.2	Enabling the Service in the Kyma Environment	64
6	SAP AI Core Starter Tutorials	65
7	Administration	66
7.1	Manage Your Git Repository	66
	Add a Git Repository	66

	Create an Application to Sync Your Folders.	69
	Edit a Git Repository.	72
	Delete a Git Repository.	72
7.2	Manage Applications.	73
	List Applications.	73
	Edit an Application.	73
	Delete an Application.	74
7.3	Manage Resource Groups.	74
	Resource Group Level Resources.	75
	Create a Resource Group.	75
	Edit a Resource Group.	77
	Delete a Resource Group.	78
7.4	Manage Object Store Secrets.	79
	Register an Object Store Secret.	79
	Edit an Object Store Secret.	83
	Delete an Object Store Secret.	87
7.5	Manage Docker Registry Secrets.	87
	Register Your Docker Registry Secret.	87
	Edit a Docker Registry Secret.	90
	Delete a Docker Registry Secret.	91
7.6	Manage Generic Secrets.	92
	Create a Generic Secret.	92
	List All Generic Secrets.	94
	Update a Generic Secret.	95
	Delete a Generic Secret.	96
	Consume Generic Secrets in Executions or Deployments.	97
8	Generative AI Hub in SAP AI Core.	99
8.1	Models and Scenarios in the Generative AI Hub.	99
8.2	Create a Deployment for a Generative AI Model.	102
	Using the API.	102
	Model Lifecycle.	106
8.3	Consume Generative AI Models.	107
	Example Payloads for Inferencing.	108
	Prompt Examples.	114
8.4	Orchestration.	131
	Orchestration Workflow.	132
	Harmonized API.	133
	Using the Orchestration Service.	136
9	Predictive AI.	151
9.1	ML Operations.	151

	Connect Your Data.	152
	Train Your Model.	159
	Use Your Model.	210
9.2	Metrics.	252
	Metrics Tracking with AI API.	252
9.3	Advanced Features.	269
	AI Content as a Service.	270
9.4	APIs and API Extensions.	276
9.5	Libraries and SDKs.	277
9.6	Content Packages.	278
10	Tutorials.	279
11	Security.	280
11.1	Security Features of Data, Data Flow, and Processes.	280
11.2	Encryption in Transit.	280
11.3	User Authentication and Administration.	280
	Roles and Authorizations.	282
11.4	Docker Images.	285
11.5	AI Content Security.	285
11.6	Kubernetes Security.	286
11.7	Configuration Data and Secrets.	286
11.8	Output Encoding.	286
11.9	Multitenancy.	287
11.10	Auditing and Logging Information.	288
11.11	Data Protection and Privacy.	290
	Data Storage and Processing.	291
	Change Logging and Read-Access Logging.	291
	Consent.	292
	Deletion.	292
	Security and Customer Data Protection.	292
12	Accessibility Features in SAP AI Core.	293
13	Monitoring and Troubleshooting.	294
13.1	Troubleshooting.	294
	Repository.	295
	Configuration.	296
	Artifacts.	298
	Application.	300
	Execution.	305
	Docker.	307
	Deployment.	308

	Miscellaneous.	309
14	Service Offboarding.	312

1 What Is SAP AI Core?

Learn more about the SAP AI Core service on SAP Business Technology Platform (SAP BTP).

SAP AI Core is a service in the SAP Business Technology Platform that is designed to handle the execution and operations of your AI assets in a standardized, scalable, and hyperscaler-agnostic way. It provides seamless integration with your SAP solutions. Any AI function can be easily realized using open-source frameworks. SAP AI Core supports full lifecycle management of AI scenarios. Access generative AI capabilities and prompt lifecycle management via the generative AI hub.

SAP AI Core allows you to make data driven decisions confidently and efficiently and is tailored to business problems. It handles large volumes of data and provides scalable machine learning capabilities to automate tasks such as triage services for customer feedback or tickets and classification tasks. SAP AI Core comes with preconfigured SAP solutions, can be configured for open-source machine learning frameworks, can be used with Argo Workflow and KServe, and can be embedded into other applications.

SAP AI Core allows you to experiment with and utilize natural language prompts with a variety of generative AI models in the generative AI hub.

→ Tip

The English version of this guide is open for contributions and feedback using GitHub. This allows you to get in contact with responsible authors of SAP Help Portal pages and the development team to discuss documentation-related issues. To contribute to this guide, or to provide feedback, choose the corresponding option on SAP Help Portal:

- [▶ Feedback ▶ Create issue](#) : Provide feedback about a documentation page. This option opens an issue on GitHub.
- [▶ Feedback ▶ Edit page](#) : Contribute to a documentation page. This option opens a pull request on GitHub.

You need a GitHub account to use these options.

More information:

- [Contribution Guidelines](#)
- [Introduction Video](#) 
- [Introduction Blog Post](#) 

Environment

This service runs in the following environments:

- Cloud Foundry
- Kyma
- Kubernetes

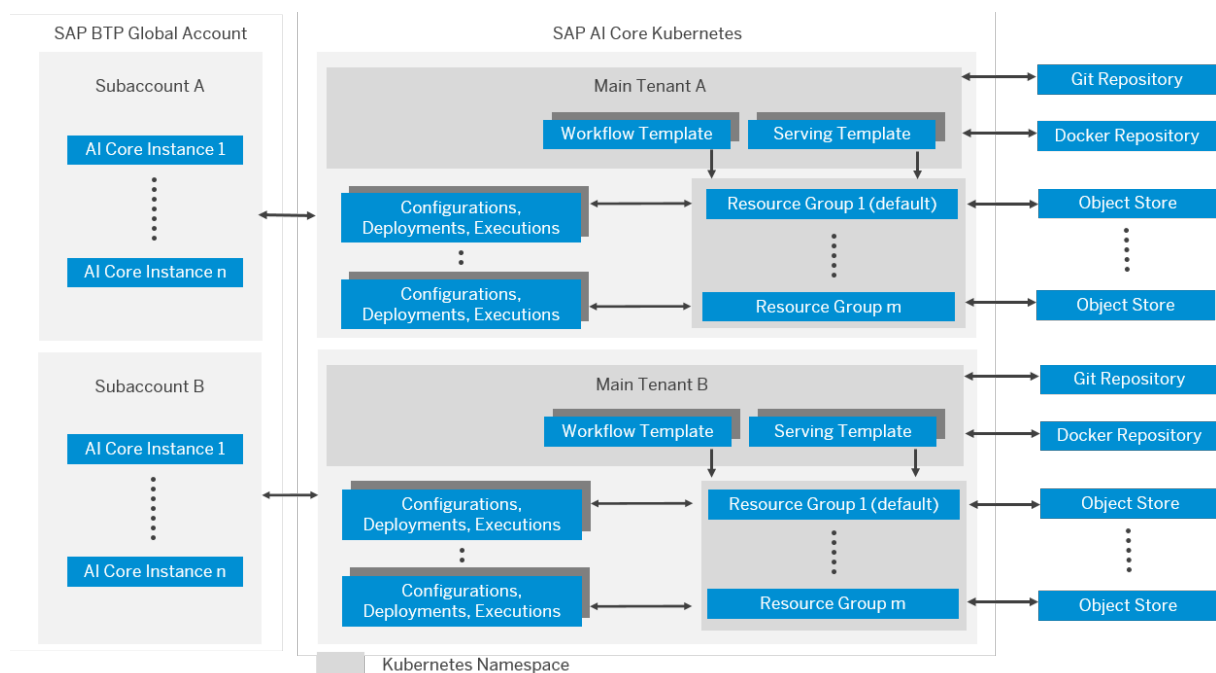
Multitenancy

This service supports multitenancy. It can be used in tenant-aware applications.

Features

Execute pipelines	Execute pipelines as a batch job, for example, to preprocess or train your models, or perform batch inference.
Serve inference requests	Deploy a trained machine learning model as a Web service to serve inference requests of trained models with high performance.
Manage the AI scenario lifecycle	Manage your ML artifacts and workflows, such as model training, metrics tracking, data, models, and model deployments via a uniform API lifecycle.
Benefit from multitenancy support	Use this service in tenant-aware applications. Implement multi-tenant services to segregate your AI assets and executions to isolate your tenants within SAP AI Core.
Integrate your cloud infrastructure	Register your Docker registry, synchronize your AI content from your git repository, and register your object store for training data and trained models. Productize your AI content and expose it as a service to consumers in the SAP BTP marketplace.
Generative AI hub	Choose from a selection of generative AI models for prompt experimentation, and prompt lifecycle management.

Process Flow Between SAP AI Core and SAP AI Launchpad



Related Information

[AI API Overview \[page 25\]](#)

[SAP AI Launchpad](#)

2 What's New for SAP AI Core

Tech nical Com- po- nent	Envi- ron- men- t	Title	Description	Ac- tion	Life- cy- cle	Type	Line of Busi- ness	Mod- ular Busi- ness Proc- ess	Product	Lat- est Revi- sion	Avail- able as of
SAP AI Core	Clou- d Foun- dry	Generative AI hub	Orchestration combines content generation with a set of functions that are often required in business AI scenarios. Functions include templating, which lets you compose a prompt with placeholders that are filled during inference, and content filtering, which lets you restrict the type of content that is passed to and received from a generative AI model.	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2024- 08- 05	2024- 08- 05
SAP AI Core	Clou- d Foun- dry	Generative AI hub	Additional selected models from GCP Vertex AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2024- 07- 08	2024- 07- 08
SAP AI Core	Clou- d Foun- dry	Resource limits per tenant	The maximum number of applications, git repository secrets, docker registry secrets, workflow templates and deployment templates is limited at tenant level to 50 per resource. If you reach this limit, you will receive an error message. You can free up space by deleting resources.	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2024- 06- 24	2024- 06- 24
SAP AI Core	Clou- d Foun- dry	Generative AI hub	Additional selected models from OpenAI are supported via Azure. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2024- 06- 24	2024- 06- 24

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Availability
SAP AI Core	Cloud Foundry	Generative AI hub	Selected models from Amazon and Anthropic are supported via AWS Bedrock. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	General Availability	Anonymous	Technology	Not applicable	SAP Business Technology Platform	2024-06-03	2024-06-03
SAP AI Core	Cloud Foundry	Generative AI hub	Selected models from Meta are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	General Availability	Anonymous	Technology	Not applicable	SAP Business Technology Platform	2024-06-03	2024-06-03
SAP AI Core	Cloud Foundry	Generative AI hub	Selected models from Mistral AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	General Availability	Anonymous	Technology	Not applicable	SAP Business Technology Platform	2024-05-20	2024-05-20
SAP AI Core	Cloud Foundry	Deployments	When deployments are submitted, configurations are checked for errors, synchronously. For more information, see Deploy Models [page 231] .	Info only	General Availability	Anonymous	Technology	Not applicable	SAP Business Technology Platform	2024-04-08	2024-05-20
SAP AI Core	Cloud Foundry	Generative AI hub	Selected models from GCP Vertex AI are supported. For more information, see Models and Scenarios in the Generative AI Hub [page 99] .	Info only	General Availability	Anonymous	Technology	Not applicable	SAP Business Technology Platform	2024-04-08	2024-04-08

Technical Component	Environment	Title	Description	Action	Lifecycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Resource groups limit per tenant	The maximum number of resource groups is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some resource groups. Alternatively, raise a ticket to increase your quota. For more information, see Delete a Resource Group [page 78] .	Info only	General Availability	Annotation	Technology	Not applicable	SAP Business Technology Platform	2024-03-08	2024-03-08
SAP AI Core	Cloud Foundry	Authentication	Authentication through X.509 certificates is supported. For more information, see Create a Service Key [page 58] and Use a Service Key [page 61] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-02-18	2024-02-18
SAP AI Core	Cloud Foundry	Generative AI hub SDK	The generative AI hub SDK is now available. For more information, see generative AI hub SDK .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2024-02-18	2024-02-18
SAP AI Core	Cloud Foundry	Generative AI hub	The generative AI hub includes prompt experimentation, prompt management, and administrative tools. Prompt experimentation includes creating and running natural language prompts with a choice of large language models and parameters. Prompt management includes saving prompts with collections, metadata in the form of tags and notes, versioning and deletion. For more information, see Generative AI Hub in SAP AI Core Overview [page 31] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-12-20	2023-12-20

Tech nical Com- po- nent	Envi- ron- men t	Title	Description	Ac- tion	Life- cy- cle	Type	Line of Busi- ness	Mod- ular Busi- ness Proc- ess	Product	Lat- est Revi- sion	Avail- able as of
SAP AI Core	Clou- d Foun- dry	Artifact sig- natures for artifact out- puts from executions	Artifact signatures (hashes) can be generated and made availa- ble to other executions and de- ployments to verify the integrity of an artifact. For more informa- tion, see Using Artifact Signa- tures [page 182] .	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2023 -07-3 1	2023 -07-3 1
SAP AI Core	Clou- d Foun- dry	Template Generator	A wizard to generate workflow and serving templates in VS Code. Using user responses, it simplifies and automates the template writing process. For more information, see SAP AI Core toolkit documentation .	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2023 -08- 04	2023 -08- 04
SAP AI Core	Clou- d Foun- dry	SAP AI Core Toolkit	SAP AI Core is available through the VS Code GUI, through the SAP AI Core toolkit extension. For more information, see SAP AI Core toolkit documentation .	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2023 -08- 04	2023 -08- 04
SAP AI Core	Clou- d Foun- dry	LLM Pack- age	The content package for large language models for SAP AI Core simplifies the deployment of large language models with integrated and automated work- flows. For more information, see PyPi LLM .	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2023 -05- 31	2023 -05- 31
SAP AI Core	Clou- d Foun- dry	Dataset API	You can upload, download, and delete artifacts using the SAP AI Core Dataset API, when direct access to files in the object store is not possible or desirable. Cur- rently, Postman and curl interfa- ces are supported.	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2023 -04- 24	2023 -05- 02

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Metadata in Response to List Executables	When you use the endpoint to list executables, the response body now contains metadata about the parameters and artifacts. For parameters, the description and default values are returned. For artifacts, the kind, description, and labels can be added using annotations. For more information, see List Executables [page 171], Workflow Templates [page 163], and Serving Templates [page 213].	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2023-04-02	2023-04-02
SAP AI Core	Cloud Foundry	Contribute to Our Documentation	At SAP, we endeavor to make sure that our documentation works for you. If you feel that something is missing or that something doesn't quite hit the mark, you can now provide feedback and suggest changes directly from SAP Help Portal. You can do so in one of two ways: - Click Edit in the toolbar to open the document in GitHub. There, you can suggest a change and submit a pull request for us to review. - Click Feedback in the toolbar to create a GitHub issue and tell us how we can improve the documentation for you. For more information, including guidelines on how to contribute, see Open Documentation Initiative.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-03-13	2023-03-13

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Bulk PATCH Endpoint to STOP or DELETE Multiple Executions or Deployments	Executions and Deployments can now receive PATCH requests for bulk adjustments, provided bulkUpdates is enabled in the relevant template. For more information, see Workflow Templates [page 163] and Serving Templates [page 213] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-02-27	2023-02-09
SAP AI Core	Cloud Foundry	Sync Endpoint for ArgoCD	In addition to automatically syncing applications, you can request a sync manually by using an API endpoint. For more information on syncing applications, see Create an Application to Sync Your Folders [page 69] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-02-27	2023-02-09
SAP AI Core	Cloud Foundry	WebHDFS Artifacts	WebHDFS artifacts are now supported. For more information about WebHDFS artifacts on SAP AI Launchpad, see Register an Object Store Secret [page 79] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-02-27	2023-02-09
SAP AI Core	Cloud Foundry	Periodic Scheduling	Executions can be run automatically, to a prepared schedule. For more information, see Training Schedules [page 202] .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2023-02-27	2023-02-09
SAP AI Core	Cloud Foundry	Deployment Duration can be Limited	You can now use the ttl parameter to limit the duration of deployments to hours, days, or weeks.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-11-20	2022-11-20

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	The YAML Files for Starter Tutorials Available Directly from GitHub	The relevant tutorial steps have been updated to include the link to the associated files. The YAML code can also be copied and pasted directly from the tutorials as before.	Info only	General Availability	Challenged	Technology	Not applicable	SAP Business Technology Platform	2022-11-20	2022-11-20
SAP AI Core	Cloud Foundry	Enhancements to the GET Deployments API Call Response	The GET Deployments response provides the number of min and max and running replicas, and the resource plan name. For more information, see the SAP AI Core API specification .	Info only	General Availability	Challenged	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Core	Cloud Foundry	Azure Blob Storage Supported	You can now register Azure Blob Storage secrets and use them for Model Serving. For more information about registering secret, see Register an Object Store Secret [page 79]	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Core	Cloud Foundry	Free Tier Service Plan	To try SAP AI Core for free, you can use a free tier service plan. A free tier service plan can be easily upgraded to a standard plan, retaining your users and data. For more information, see Free Tier [page 38] .	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2022-10-18	2022-10-18
SAP AI Core	Cloud Foundry	Deployment Duration Variable	You can now limit the duration of a deployment by specifying the length of time that it should run for, in whole minutes hours or days.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-10-04	2022-10-04

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Metrics Extension	If you have an AI API-enabled runtime, you can use the new “Metrics” extension to query which capabilities of the metrics endpoint are supported. You can specify the metrics capabilities to a very fine level of granularity, which lets you react in your client implementation accordingly. For more information, see Metrics Extension .	Info only	General Availability	New	Technology	Not applicable		2022-09-19	2022-09-19
SAP AI Core	Cloud Foundry	Git Repository Name No Longer Mandatory	When you register your git repository, you no longer have to enter the repository name. The field has been removed. Repositories already registered with names assigned will still work.	Info only	General Availability	Announcement	Technology	Not applicable	SAP Business Technology Platform	2022-09-19	2022-09-19
SAP AI Core	Cloud Foundry	API Deprecation and Decommissioning	API endpoints POST /lm/configurations/{configurationId}/executions , POST /lm/configurations/{configurationId}/deployments , GET /lm/kpis and GET /analytics/resourceGroups are deprecated. Please update your existing API calls. For timetables and more information, see SAP Note 3239609 .	Required	Deprecated	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-09-05	2022-09-05

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Serving Component Upgraded to KServe Version 0.7 for Security Compliance	The upgrade means some changes to serving templates: the <code>spec.template.api</code> version changes from <code>serving.kubeflow.org/v1beta1</code> to <code>serving.kserve.io/v1beta1</code>, the <code>spec.template.spec.predictor.containers.name</code> changes from <code>kfserving-container</code> to <code>kserve-container</code>. There is no support for <code>apiVersion</code>: <code>serving.kubeflow.org/v1alpha2</code>. Please update your existing serving templates by September 30, 2022.	Required	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-08-22	2022-08-22
SAP AI Core	Cloud Foundry	Supported Object Stores	SAP AI Core supports multiple hyperscaler object stores, such as Amazon S3, OSS, and HANA Data Lake (HDL). For more information about registering secret, see Register an Object Store Secret [page 79]	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-06-30	2022-06-30
SAP AI Core	Cloud Foundry	Enhanced Serving Template Parameters	The <code>executables.ai.sap.com/cascade-update-deployments</code> parameter can be used in the serving template, to update associated deployments automatically. For more information, Serving Templates and Change Serving Template and Update Deployments.	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-06-18	2022-06-18

Tech nical Com- po- nent	Envi- ron- men- t	Title	Description	Ac- tion	Life- cy- cle	Type	Line of Busi- ness	Mod- ular Busi- ness Proc- ess	Product	Lat- est Revi- sion	Avail- able as of
SAP AI Core	Clou- d Foun- dry	Tracking Service Im- provement	The performance of tracking is improved.	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2022 -06-18	2022 -06-18
SAP AI Core	Clou- d Foun- dry	Documen- tation for the AI API client SDK has moved to PyPi.org.	Documentation for the AI API client SDK has moved to PyPi.org. For more information, Libraries and SDKs .	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2022 -06-07	2022 -06-07
SAP AI Core	Clou- d Foun- dry	Documen- tation for the SAP AI Core SDK has moved to PyPi.org.	Documentation for the SAP AI Core SDK has moved to PyPi.org. For more information, Libraries and SDKs .	Info only	Gen- eral Avail- abil- ity	An- noun- ce- men- t	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2022 -06-07	2022 -06-07
SAP AI Core	Clou- d Foun- dry	Additional Query Pa- rameter for Metrics	The \$select query parameter can be used to retrieve metric resource data, such as metrics or custom info selectively. For more information see, Querying Metric Data .	Info only	Gen- eral Avail- abil- ity	New	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2022 -05-05	2022 -05-05
SAP AI Core	Clou- d Foun- dry	Horizontal Scaling for Resource Group Op- erator Ena- bled	The Resource Group Operator now allows horizontal scalability, improving performance.	Info only	Gen- eral Avail- abil- ity	Cha- nged	Tech nol- ogy	Not ap- pli- ca- ble	SAP Business Technology Platform	2022 -04-09	2022 -04-09

Technical Component	Environment	Title	Description	Action	Life-cycle	Type	Line of Business	Modular Business Process	Product	Latest Revision	Available as of
SAP AI Core	Cloud Foundry	Resource Group Id Length Extended	The resource group Id length was limited to 10 characters, however longer Ids are now supported. Ids must be of length minimum: 3, maximum: 253. The first and last characters must be either a lower case letter, an upper case letter or, a number. Character entries from the second to penultimate, must be either a lower case letter, an upper case letter, a number, a full stop, or a hyphen. No other special characters are permitted.	Info only	General Availability	Changed	Technology	Not applicable	SAP Business Technology Platform	2022-04-09	2022-04-09
SAP AI Core	Cloud Foundry	Generic Secret Support at Main Tenant Level	You can now store generic secrets at the main tenant level and use them with a generic service broker. For more information, see Register Generic Secrets .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-02-19	2022-02-19
SAP AI Core	Cloud Foundry	AI API meta Endpoint	The new meta API endpoint lets clients query the capabilities of a runtime engine that implements AI API. Querying the meta endpoint returns information that the client can use to trigger an appropriate response. For example, SAP AI Launchpad could enable or disable certain features based on the capability of the runtime engine implementation of AI API. For more information, see AI API Runtime Implementations .	Info only	General Availability	New	Technology	Not applicable	SAP Business Technology Platform	2022-02-19	2022-02-19

2.1 2021 What's New for SAP AI Core (Archive)

2021

Technical Component	Capability	Environment	Title	Description	Action	Type	Available as of
SAP AI Core	Extension Suite - Digital Experience	Cloud Foundry	Adjustments to resourceGroups API endpoint	The API endpoint /admin/resourceGroups was previously available only in the AI Core API. It is now available in the AI API and can be implemented in any runtime. The /admin/resourceGroups endpoint specification has also been enhanced with the new field createdAt, which indicates when the resource group was created. Note that this is not yet part of the SAP AI Core implementation. For more information, see the specifications for the AI API and AI Core API.	Info only	New	2021-12-18
SAP AI Core	Extension Suite - Digital Experience	Cloud Foundry	Generic Secrets	You can now store generic secrets and use them in executions or deployments. For more information, see Register Generic Secrets.	Info only	New	2021-12-03
SAP AI Core	Extension Suite - Digital Experience	Cloud Foundry	AI API Enhancements	- You can now query all resourceGroups and receive the results by ascending resource group ID so that you can display them in SAP AI Launchpad. - You can now measure usage by getting the aggregates of runtime entities for the combination of resource Group ID, scenario ID, and executable ID.	Info only	Changed	2021-10-30

3 Concepts

In this section, we'll explore some of the concepts surrounding SAP AI Core.

[SAP AI Core Overview \[page 21\]](#)

SAP AI Core is the key to integrating artificial intelligence capabilities in your SAP solutions.

[Generative AI Hub in SAP AI Core Overview \[page 31\]](#)

The generative AI hub incorporates generative AI into your AI activities in SAP AI Core and SAP AI Launchpad.

[Terminology \[page 32\]](#)

3.1 SAP AI Core Overview

SAP AI Core is the key to integrating artificial intelligence capabilities in your SAP solutions.

Specifically, SAP AI Core helps you to:

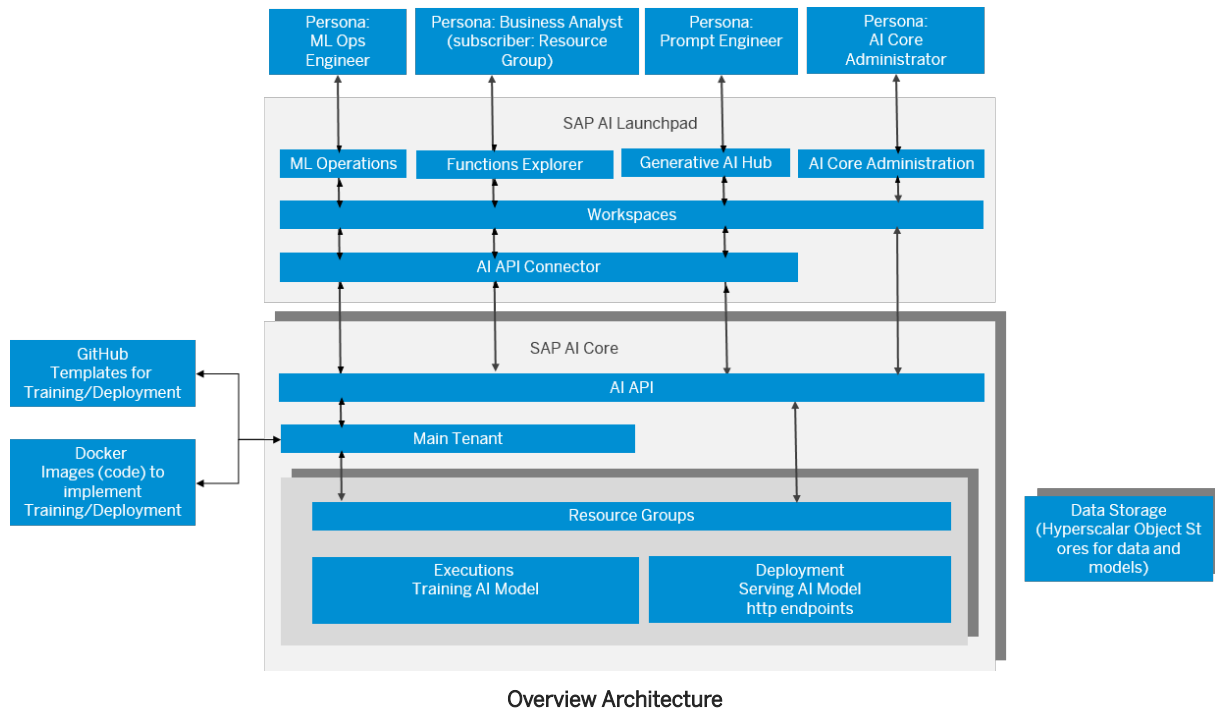
- Seamlessly and easily embed AI capabilities into other applications
- Leverage high-volumes of data from applications to create robust AI learning models
- Execute AI training on accelerated hardware
- Serve AI inference with low latency and high-throughput in a cost-efficient manner
- Adhere to a compliant, explainable, and maintainable process
- Manage all stages of the AI lifecycle using a comprehensive set of tools and services
- Focus on the productization and operationalization of AI scenarios

Management and operations of your AI content (such as versioning, deployment, and monitoring) is unified across your SAP solutions. Authoring of your AI content is, however, open to different toolsets (such as JupyterLab).

The SAP AI Core service is intended to be used together with SAP AI Launchpad and the AI API. The key components are SAP AI Core, SAP AI Launchpad, and the AI API.

- **SAP AI Core** provides an engine that lets you run AI workflows and model serving workloads.
- **SAP AI Launchpad** manages a number of AI runtimes. It allows various user groups to access and manage their AI scenarios.
- **AI API** provides a standard way of managing the AI scenario lifecycle on different runtimes, regardless of whether they are provided on SAP technology (such as SAP S/4HANA) or on partner technology (such as Amazon Web Services). When the AI API is deployed on runtimes other than SAP AI Core, the runtimes have to provide a runtime adapter.

[Overview Architecture \[page 22\]](#) shows these three main components in an overview architecture diagram.



Parent topic: [Concepts \[page 21\]](#)

Related Information

[Generative AI Hub in SAP AI Core Overview \[page 31\]](#)

[Terminology \[page 32\]](#)

3.1.1 SAP AI Core Systems Overview

Your SAP AI Core system connects internal and external tools.

Users interact with the various repositories, systems, and objects when working with SAP AI Core; some of these objects are provided by SAP. In other cases, customers provide these components to enable enhanced control (authorizations) and continuous integration/continuous deployment (CI/CD).

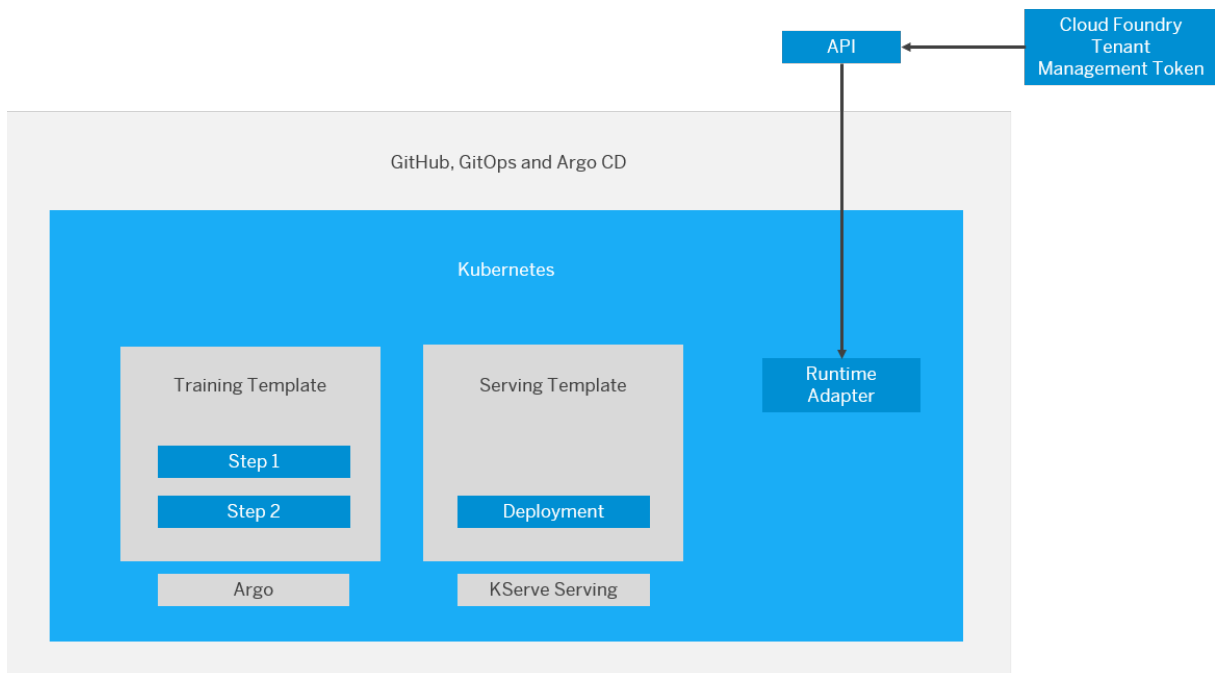
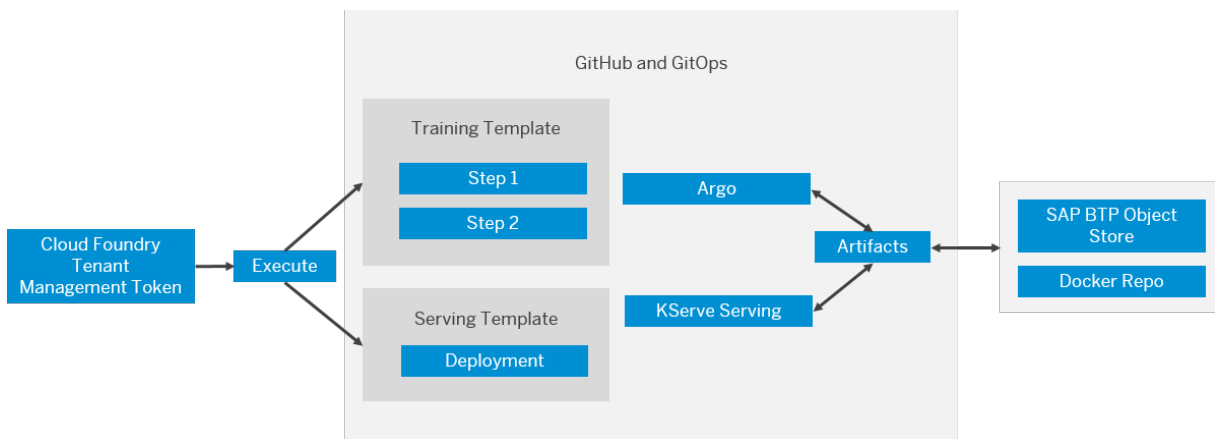
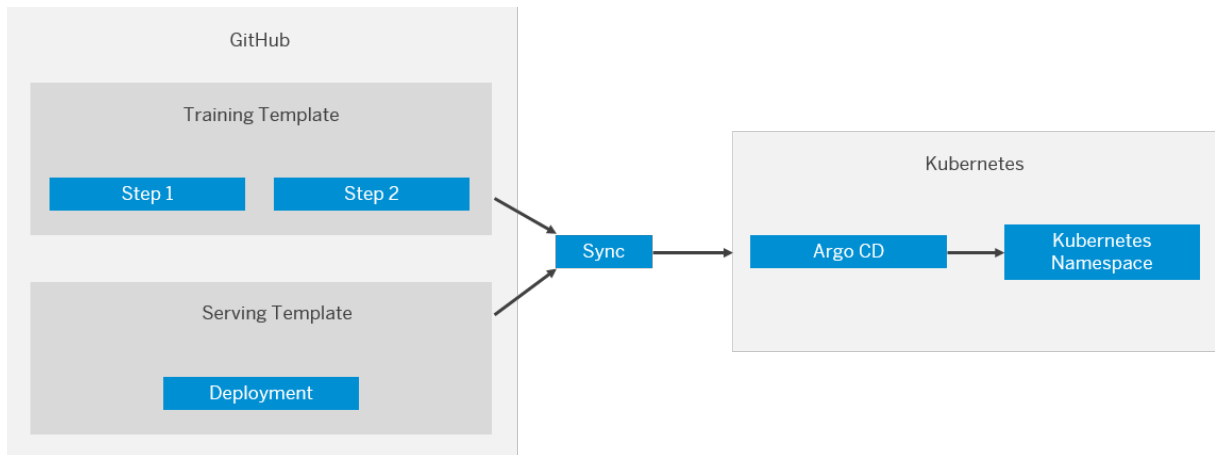
Key Repositories, Systems, and Objects

Summary of Elements and Their Use

What	Why
Git repo	For storing training and serving workflows and templates
Hyperscaler storage	For storage of input and output artifacts, such as training data and models (for example, SAP BTP Object Store Service)
Docker repo	For custom Docker images referenced in the templates
Kubernetes (K8s)	The K8s cluster orchestrates and scales the pods, which are used in AI pipelines. Resource group isolation is based on a K8s namespace.
KServing (K)	For optimized deployments of machine learning models. Deployment templates use KServe notation.
AI API	For managing your artifacts and workflows (such as training scripts, data, models, and model servers) across multiple runtimes
 Note The AI API can also be used to integrate other machine learning platforms, engines or runtimes into the AI ecosystem.	
Argo Workflows	A container native, workflow engine for Kubernetes.
SAP AI Launchpad	SAP AI Launchpad is a multitenant software as a service (SaaS) application on SAP Business Technology Platform (SAP BTP). Customers and partners can use SAP AI Launchpad to manage AI use cases (scenarios) across multiple instances of AI runtimes (such as SAP AI Core). SAP AI Launchpad also provides generative AI capabilities via the Generative AI Hub.

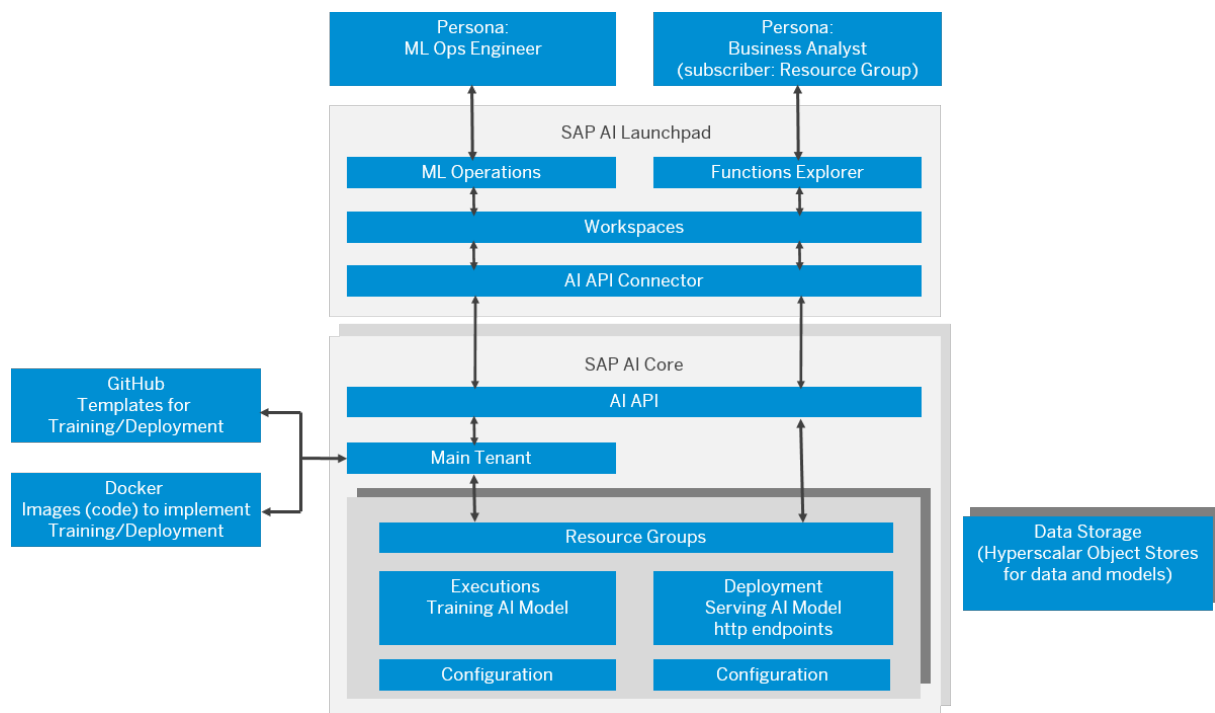
Synchronization

- Your GitOps implementation is integrated with SAP AI Core to enable CI/CD.
- Templates are synchronized from your Git repository to a Kubernetes cluster.
- Your training and serving templates are regularly synchronized (every few minutes).
- SAP AI Core checks the template syntax, and if synchronization fails, then error messages are displayed.



Process Summary

- An SAP BTP token is used in AI API for authentication of the API calls.
- Users run templates via the AI API.
- Training templates are executed using the Argo workflow. The training pipeline consumes data input artifacts and outputs a model artifact.
- An execution is created using the training template and AI API configuration. The result is a training job.
- Serving templates are deployed using KServing.
- A deployment is created using the serving template and AI API configuration. The result is an inference server.
- Artifacts (such as datasets, models) are copied to and from the hyperscaler storage.
- Required images are pulled from the registered Docker repository.
- Details about scenarios and executables are retrieved from the Kubernetes cluster by the AI API.



3.1.2 AI API Overview

The AI API lets you manage your AI assets (such as training scripts, data, models, and model servers) across multiple runtimes.

Argo workflows and serving templates, as well as their execution and deployment, are managed using the SAP AI Core implementation of the AI API. In SAP AI Core, the Argo workflow and serving templates are mapped under the concept of `Executable`. For the mapping mechanism to work, the Argo workflows and serving templates require certain attributes in the metadata section of the YAML file. These attributes are shared by both template types.

SAP AI Core provides additional APIs that are runtime-specific. These are available in the AI Core API specification, which is an extension of the AI API specification.

Related Information

[AI Core API](#) 

[AI API](#)

AI API Runtime Implementations

The AI API specification is a general specification for the lifecycle management of machine learning artifacts. SAP AI Core is one specific runtime implementation of the AI API specification. It is also possible to provide other runtime implementations of the AI API specification, independent of SAP AI Core. This section describes the necessary boundary conditions and implementation requirements.

The benefit of using AI API is that clients can integrate with all AI API-enabled runtime implementations. For example, SAP AI Launchpad can interact with custom runtime implementations as long as the same APIs are provided. Intelligent Scenario Lifecycle Management can also integrate with AI API-enabled runtimes. The AI API Client SDK (Python) can also be used (for more information, see [SAP AI Core SDK](#)).

AI API Specification

The AI API specification comprises the following parts:

- A main specification
- Extensions:
 - Analytics extension
 - Resource group extension
 - Dataset management extension
 - Metrics extension

→ Recommendation

Implement at least the main specification, and then implement the extension specifications based on your use case.

AI API Runtime Capabilities Endpoint

Meta API is part of the AI API specification (endpoint `/lm/meta`). The implementation must return a configuration response that specifies the capabilities of the AI API runtime implementation.

Meta API allows AI API clients to query the capabilities of an AI API implementation so that they can select which commands or user interfaces are available. For example, some AI API runtimes may offer executions but not deployments. They may also offer logs for executions and not for deployments. As an example, if a client of SAP AI Core such as SAP AI Launchpad queries the **Meta API** endpoint of SAP AI Core, the response will be for example:

```
json
{
  "aiApi": {
    "capabilities": {
      "logs": {
        "deployments": true,
        "executions": true
      },
      "multitenant": true,
      "shareable": true,
      "staticDeployments": true,
      "timeToLiveDeployments": true,
      "userDeployments": true,
      "userExecutions": true,
      "executionSchedules": true
    },
    "limits": {
      "deployments": {
        "maxRunningCount": -1
      },
      "executions": {
        "maxRunningCount": -1
      },
      "minimumFrequencyHour": 1,
      "timeToLiveDeployments": {
        "minimum": "10m",
        "maximum": -1
      }
    },
    "version": "2.18.0"
  },
  "extensions": {
    "analytics": {
      "version": "1.0.0"
    },
    "metrics": {
      "capabilities": {
        "extendedResults": true
      },
      "version": "1.0.0"
    },
    "resourceGroups": {
      "version": "1.2.0"
    }
  },
  "runtimeApiVersion": "2.21.0",
  "runtimeIdentifier": "aicore"
}
```

SAP AI Launchpad and other clients can then react accordingly and hide the deployments on the user interface for this runtime implementation of AI API.

Capabilities include the following:

Capability	When true, allows users to:
logs.executions	View logs for an execution
logs.deployments	View logs for a deployment
multitenant	Use SAP AI Launchpad as a main tenant user (supports resource groups)
shareable	Clients can share one instance
staticDeployments	Static, always running endpoints for inference are available without the user having to start a deployment
userDeployments	Stop, update, or delete a deployment
userExecutions	Stop or delete an execution
timeToLiveDeployments	The runtime engine allows defining the time until a deployment is automatically deleted
analytics	Review summary information for all tenants
bulkUpdates	Stop or delete up to 100 executions or deployments at once
executionSchedules	Create schedules

Limits include the following:

Limit	Details
deployments.maxRunningCount	Limits the number of running concurrent deployments in a resource group, if any
executions.maxRunningCount	Limits the number of running concurrent executions in a resource group, if any
timeToLiveDeployments.minimum	The minimum possible value for the ttl parameter in a deployment, if supported
timeToLiveDeployments.maximum	The maximum possible value for the ttl parameter in a deployment, if supported
minimumFrequencyHour	The minimum possible value for schedule of an execution, if supported

In addition to the general AI API specification, there is also a number of extensions that cover additional use cases. These might not be implemented in all runtime engines.

The extensions are:

Extension	Details
analytics	The analytics extension contains endpoints for fetching analytical information of a resource group or tenant
metrics	The metrics extension contains endpoints for writing to and reading from metrics endpoints, to store and retrieve metrics generated during executions

Extension	Details
resourceGroups	The resource group extension contains endpoints for managing resource groups
dataset	The dataset extension contains endpoints for uploading and downloading files

Related Information

[Serving Templates \[page 213\]](#)

[Workflow Templates \[page 163\]](#)

[AI API Specification](#)

[Custom Runtime Capabilities Using the Meta API](#)

[Analytics Extension](#)

[Resource Groups Extension](#)

[Intelligent Scenario Lifecycle Management](#)

3.1.3 Resource Groups

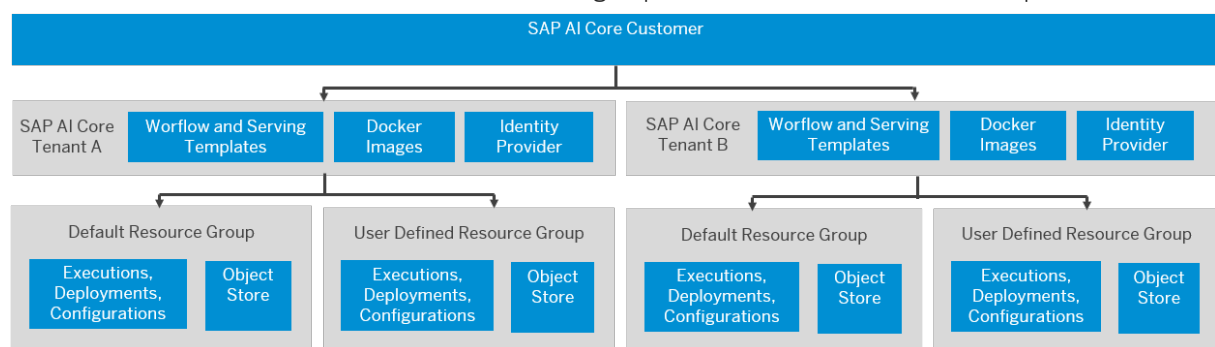
SAP AI Core tenants use resource groups to isolate related ML resources and workloads. Scenarios, executables, and Docker registry secrets are shared across all resource groups.

Resource groups represent a virtual collection of related resources within the scope of one SAP AI Core tenant. When your tenant is onboarded, a default resource group is created immediately. Further resource groups can be created, or deleted by your tenant administrator with the AI API. Tenants can map the resource groups based on the corresponding usage scenarios.

If your SAP AI Core tenant uses resource groups to isolate the scenario consumer tenant and the resource groups are subsequently deleted, the scenario consumers will be deprovisioned. SAP AI Core is not aware of the scenario consumer of the tenant. The standard XUSAA multitenancy model is followed.

3.1.3.1 Scope of Resources

Resources that are available for tenants and resource groups differ based on the available scope.



Tenant-Level Resources

Tenant-level resources include:

- Workflow templates
- Serving templates
- Docker registry (containing the Docker images)
- User authentication and authorization (UAA)

User authentication and authorization is based on the SAP AI Core tenant. The tenant is the holder of the access token obtained using the SAP AI Core service key. The SAP AI Core tenant can set the resource group in the request header at runtime, or during lifecycle management, using the AI API. If the resource group is not set, the default resource group is used.

Resource Group-Level Resources

Executables at tenant level are shared across all of the resource groups. At resource group level, the object store is registered by setting the resource group header.

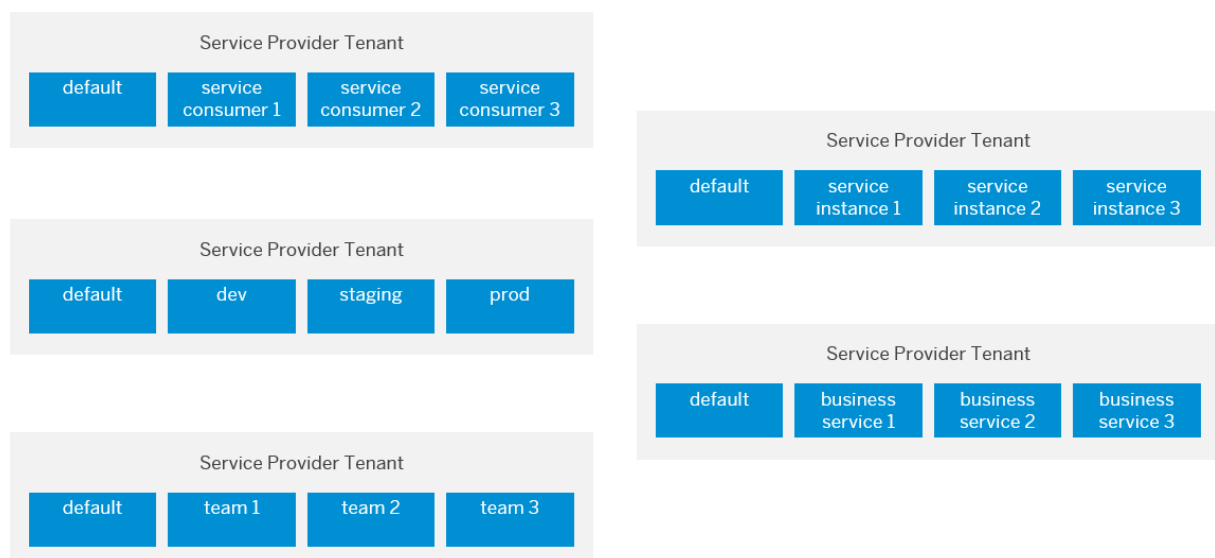
SAP AI Core tenants must consider security aspects in the design of AI functions.

→ Recommendation

Do not use the same object store bucket with the same AWS IAM user for multiple resource groups.

Runtime entities such as executions, deployments, configurations, and artifacts belong to specific resource groups and cannot be shared across resource groups.

Examples of Resource Group Mapping



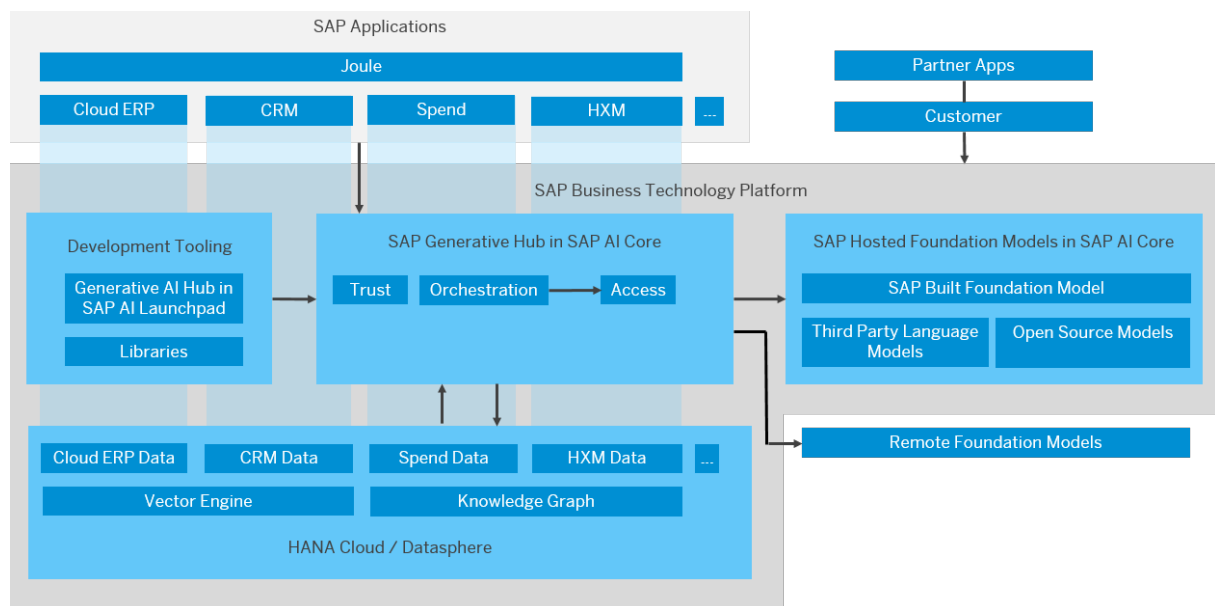
3.2 Generative AI Hub in SAP AI Core Overview

The generative AI hub incorporates generative AI into your AI activities in SAP AI Core and SAP AI Launchpad.

LLMs are self-supervised, deep learning models that have been trained on vast amounts of unlabeled data. They leverage AI technology and industrial-scale computational resources to learn complex language patterns and semantic knowledge bases for natural language processing (NLP) tasks. They parse input, such as prompts, and by predicting a target word, can return contextually relevant responses written in natural language. A single LLM can perform multiple NLP tasks by using different input formats and output modes.

LLMs are general models but can be fine-tuned with additional embeddings for specialized or domain-specific use cases.

SAP AI Core and the generative AI hub help you to integrate LLMs and AI into new business processes in a cost-efficient manner.



Generative AI Hub Architecture Overview

Parent topic: [Concepts \[page 21\]](#)

Related Information

[SAP AI Core Overview \[page 21\]](#)

[Terminology \[page 32\]](#)

3.3 Terminology

Term	Definition
AI API	An essential API that handles all AI artifacts and workflows (such as training scripts, data, models, and model servers) across multiple AI runtimes.
Artifact	A reference to data or a file that is produced or consumed by an execution or deployment.
Configuration	A collection of parameters, artifact references, and executables that is used to run an execution or deployment. A configuration is required to instantiate an execution. Executables and configurations have a 1:n relationship (that is, one executable can have many configurations).
Dataset	A type of artifact. A dataset references a data source and the instruments required to access and investigate it. A dataset: • References an acquired dataset on a data plane or an application API • Can be passed as a dataset artifact to a scenario at runtime
Deployment	A deployment is an instance of a model serving template that is configured to use a model type artifact and to apply it to data passed in a serving request (for example, for a prediction). Once a deployment has been instantiated successfully, a model server is created and a deployment URL is generated for inference. As input, the deployment takes one or more models and the parameters from a configuration.
Executable	An executable is a template that is instantiated for a purpose, such as training a model or creating a deployment User-defined labels can be applied to an executable and are listed with the executable's details. • If a nondeployable executable is instantiated, it results in an execution. Executions may produce output artifacts • If a deployable executable is instantiated, it results in a deployment. Deployments generate URLs that can be used for inference

Term	Definition
Execution	An execution is an instance of a non-deployable executable. Executions are used to train a model or to run other types of workflow. An execution runs only once and produces an output artifact. If a training instance is executed, the output artifacts produced are models. There is typically a large amount of metadata associated with an execution. The metadata is stored as metrics (and associated labels), tags, and custom info. Multiple metrics, tags and custom info can be passed with a single execution. Once the metric, tag and custom info are saved, they can be queried with an <code><execution ID></code>. Every metric, tag and custom info must be associated with an execution.
Workflow executable	A workflow executable is used within the <i>ML Operations</i> application. A workflow executable defines a set of parameters and the input artifacts that it needs to be instantiated. These are provided by a configuration. The workflow executable also defines a set of output artifacts that would be generated if the executable were to be instantiated and run successfully.
Job executable	A job executable is a simplified representation of a workflow executable that is used within the <i>Functions Explorer</i> application.
Job template	A type of executable in SAP AI Core that specifies a long-running process, typically a training or batch inference.
Model	A type of artifact that is the output of training.
Model serving template	A type of executable in SAP AI Core that specifies how a model is to be served.
Operations	All activities within the AI lifecycle such as executing trainings, creating model deployments, application integration, and model monitoring.

Term	Definition
Resource group	A resource group represents a unique workspace created for an SAP AI Core scenario consumer and for a unique purpose. A resource group requires a valid subscription to a tenant. Within a resource group, users create or add entities such as configurations, executions, deployments, and artifacts. They consume the executables that are available in the scenarios. Before users can instantiate an executable, they must create a configuration. • If a nondeployable executable is instantiated, it results in an execution. Executions optionally produce output artifacts. • If a deployable executable is instantiated, it results in a deployment. Deployments generate URLs that can be used for inference.
Runtime	A platform that offers processing resources to execute AI learning loads, such as training and inference.
SAP AI Core	An infrastructure that enables a customer's business applications to become intelligent with AI and machine learning technology, and their data to train AI services to help automate tasks and processes.
Scenario	A scenario is a group of related executables for a use case within the user's tenant. A scenario can have multiple versions that further correspond to the different versions of executables.
Serving executable	A serving executable is a deployable template pipeline that can be instantiated to create a deployment. A serving executable defines a set of parameters and the input artifacts that it needs to be instantiated; these are provided by a configuration.
Training	An execution or executable that produces (at least) an artifact of type model .
Prompt	A prompt is a natural language instruction for a generative AI model.

Term	Definition
Generative AI hub	The generative AI hub includes prompt experimentation, prompt management, and administrative tools. Prompt experimentation includes creating and running natural language prompts with a choice of large language models and parameters. Prompt management includes saving prompts with collections, metadata in the form of tags and notes, versioning and deletion.

Parent topic: [Concepts \[page 21\]](#)

Related Information

[SAP AI Core Overview \[page 21\]](#)

[Generative AI Hub in SAP AI Core Overview \[page 31\]](#)

4 Service Plans

The SAP AI Core service plan you choose determines pricing, conditions of use, resources, available services, and hosts.

Your use case will determine whether you choose a free tier option for SAP AI Core or a paid service plan.

The following service plans are available:

- Free
- Standard
- Extended

⚠ Restriction

SAP AI Core free tier is only available in SAP BTP free tier, not SAP BTP trial.

Service Plan	Details	Account Type
Free	• Get to know SAP AI Core using a free service, with limitations. • Only community support is available for free tier service plans and these are not subject to SLAs. • Limitations: • Limited to one instance per Global account • Limited to one execution or deployment running at any given time • Limited to the default resource group • Limited to the Starter AI Core resource plan • Limited to active instance(s) of either free or standard plan (mutually exclusive) within a subaccount • Generative AI hub not included • For region information, see SAP Discovery Center.	Enterprise

📘 Note

Customers can create one SAP AI Core free plan instance, if and only if there are no active standard plan instances within the subaccount. Tenants can update their plan from free to standard, but not from standard to free. A standard plan SAP AI Core instance cannot be created if there is an active free plan instance in the subaccount.

For more information, see [Free Tier \[page 38\]](#), the Starter plan in [Choose a Resource Plan \[page 161\]](#) and [Using Free Service Plans](#).

Service Plan	Details	Account Type
Standard	- Service plan intended for productive usage. - Use of resources for custom workloads is charged using adaptable pricing according to the resource type used, with the addition of a baseline charge for service use. - Limitations: - An SAP AI Core standard plan instance cannot be created if there is an active free plan instance - Generative AI hub not included - For region information, see SAP Discovery Center. For more information, see Choose a Resource Plan [page 161] and Metering and Pricing for SAP AI Core [page 40].	Enterprise
Extended	- Includes the Standard plan, with the addition of generative AI capabilities in the generative AI hub. - Use of provided generative AI models is charged using adaptable pricing according to model choice, and the number of tokens sent and received. - For region information, see SAP Discovery Center. For more information, see Choose a Resource Plan [page 161] and Metering and Pricing for SAP AI Core [page 40].	Enterprise

Deployment Quotas

Each tenant is assigned a default quota that limits the number of deployments and replicas per deployment. If you reach this quota, your deployment will not be created and you will be notified accordingly. You can free up your quota by deleting existing deployments.

Alternatively, you can request an increase to your quota by creating a ticket on component **CA-ML-AIC**. Enter the description **Request to Increase Quota** and include details about the size of your increase, whether you want to include deployments, replicas, or both, and your subaccount ID.

Resource Group Quotas

⚠ Restriction

The maximum number of resource groups is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some resource groups. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Resource Group \[page 78\]](#).

Related Information

[Free Tier \[page 38\]](#)

[SAP Discovery Center](#) 

[SAP BTP Service Description Guide](#) 

[Choose a Resource Plan \[page 161\]](#)

4.1 Free Tier

Enable the free tier option to get to know SAP AI Core with usage limits, to familiarize yourself with the service. Note that usage is limited with this option.

You can use the free tier option and benefit from the following advantages:

- Explore SAP AI Core features and capabilities without cost.
- Migrate from the free tier to a standard service plan while retaining your work.

Free Tier Scope

- For region information, see [SAP Discovery center](#) .
- Only community support is available for free tier service plans and these are not subject to SLAs.
- The free tier option is subject to usage limits. For more information, see [Service Plans \[page 36\]](#).

Note

Customers can create one SAP AI Core free plan instance, if and only if there are no active standard plan instances within the subaccount. Tenants can update their plan from free to standard, but not from standard to free. A standard plan SAP AI Core instance cannot be created if there is an active free plan instance in the subaccount.

Related Information

[Service Plans \[page 36\]](#)

Enabling Free Tier

Enable free tier to experience SAP AI Core for trial and testing purposes. Alternatively, follow the tutorial [here](#). 📄

Prerequisites

⚠️ Restriction

- SAP AI Core free tier is only available in SAP BTP free tier, not SAP BTP trial.
- Generative AI hub is not available in free tier. For more information, see [Service Plans \[page 36\]](#).

Procedure

1. Open your global account in the SAP BTP cockpit.
2. Go to your subaccount.
3. In the navigation area, choose *Instances and Subscriptions*.
4. Choose *New Instance or Subscription*
5. Search for SAP AI Core in the *Service* field.
6. Choose *Free* in the *Plan* field.

New Instance or Subscription

1 Basic Info 2 Parameters 3 Review

Enter basic info for your instance or subscription.

Service: * ⓘ

SAP AI Core

Plan: *

standard free

Instance Instance

📌 Note

A free tenant will be created.

Related Information

[Service Plans \[page 36\]](#)

[Using Free Service Plans](#)

[Getting a Global Account](#)

4.2 Metering and Pricing for SAP AI Core

SAP AI Core is metered based on various nonbillable units of measure (UoM), depending on which resources of SAP AI Core are consumed.

The following resources and nonbillable UoMs are available:

Resource Type	Infrastructure Resource	Unit of Measure
Compute	Starter Instance	Node hour
	Basic Instance	
	Basic.8x Instance	
	Infer-S Instance	
	Infer-M Instance	
	Infer-L Instance	
	Train-L Instance	
Storage	Standard SSD Volume	gigabyte hour
Baseline	Fixed cluster resources (instances, storage, SAP HANA, Kubernetes, DevOps, support, and so on)	Tenant hour

Note

Infrastructure specifications depend on the chosen hyperscaler. For more information, see [Choose a Resource Plan \[page 161\]](#).

Each metered record is translated to the billable metric “capacity units”. Every resource has a specific conversion rate for translating the UoM to capacity units. For the applicable conversion rates, see the [SAP BTP Service Description Guide](#).

All “Compute” and “Storage” resource consumption is based on the actual metered usage.

Baseline charges are automatically calculated based on the following: For each active tenant hour where one of the “Compute” or “Storage” resources is being consumed within a tenant, one “Baseline” tenant hour is

automatically charged. If multiple resources are used within the same hour, only one “Baseline” tenant hour will be charged.

❖ Example

This example uses fictitious values. For the current conversion rates, see the [SAP BTP Service Description Guide](#).

Let's assume the following parameters:

- 1 node hour of a Starter Instance equals 0.5 capacity units.
- 1 node hour of an Infer-M Instance equals 2 capacity units.
- 1 tenant hour of Baseline equals 1.5 capacity units.
- 1 capacity unit costs EUR 1

If you consume 100 hours of a Starter Instance and 300 hours of an Infer-M Instance, and you are active on the cluster for 300 hours in a month, the following costs will accrue:

100 node hours of Starter Instance = $100 * 0.5$ capacity units = 50 capacity units

300 node hours of Infer-M Instance = $300 * 2$ capacity units = 600 capacity units

300 tenant hours of Baseline = $300 * 1.5$ capacity units = 450 capacity units

Total = 1,100 capacity units = EUR 1,100

📘 Note

It is possible to process data in parallel, using more than one node. Costs are calculated in node hours, meaning that 1 hour of data processing, on 2 nodes is equal to 2 node hours.

→ Recommendation

For an estimate of projected costs, use the [SAP AI Core Cost Calculator](#).

Related Information

[SAP Discovery Center](#)

[SAP BTP Service Description Guide](#)

[Choose a Resource Plan \[page 161\]](#)

4.3 Metering and Pricing for the Generative AI Hub

The use of models in the generative AI hub is metered using GenAI tokens and capacity units.

📘 Note

The generative AI hub is available only as part of the Extended service plan.

GenAI tokens correspond to blocks of 1,000 tokens from each model. The number of GenAI tokens you need changes based on the model you use and whether the token is for input or output. There are conversion rates that help you figure out how many GenAI tokens you get from the model provider's tokens. The conversion rates apply to blocks of 1,000 input and output model tokens. By referring to these rates, you can work out how many GenAI tokens you're consuming in total. For more information about the supported models and the conversion rates between model tokens and GenAI tokens, see SAP Note [3437766](#).

Each record measured in GenAI tokens is converted into billable metric units, known as "capacity units". For the conversion rate between GenAI tokens and capacity units (capacity unit values), see the [SAP BTP Service Description Guide](#).

❁ Example

This example uses fictitious values.

For a given request, x input model tokens and y output model tokens are consumed. The corresponding metrics are:

Model	GenAI Input Tokens (for 1,000 Model Tokens)	GenAI Output Tokens (for 1,000 Model Tokens)
Sample model	0.002	0.003

Total GenAI tokens = $x/1000 * 0.002 + y/1000 * 0.003$

Capacity units = Total GenAI tokens * 2.0000 (capacity unit value)

Charges associated with the use of other SAP AI Core components may also apply. For more information, see [SAP AI Core Metering and Pricing](#).

→ Recommendation

For an estimate of projected costs, use the SAP AI Core cost calculator. For more information, see [Cost Calculator](#).

4.4 Choose a Resource Plan

You can configure SAP AI Core to use different infrastructure resources for different tasks, based on demand. SAP AI Core provides several preconfigured infrastructure bundles called "resource plans" for this purpose.

Context

Resource plans are used to select resources in workflow and serving templates. Different steps of a workflow can have different resource plans.

In general, if your workload needs GPU acceleration, you should use one of the GPU-enabled resource plans. Otherwise, choose a resource plan based on the anticipated CPU and memory need of your workloads.

Within SAP AI Core, the resource plan is selected via the `ai.sap.com/resourcePlan` label at pod level. It maps the selected resource plan and takes a string value, which can be any of the following resource plan IDs:

Resource Plan Specifications for AWS

Resource Plan IDs	GPUs	CPU Cores	Memory GBs	Code to Allocate Resources in Workflow Templates
Starter	–	1	3	<code>ai.sap.com/ resourcePlan: starter</code>
Basic	–	3	11	<code>ai.sap.com/ resourcePlan: basic</code>
Basic-8x	–	31	116	<code>ai.sap.com/ resourcePlan: basic.8x</code>
Train-L	1 V100	7	55	<code>ai.sap.com/ resourcePlan: train.l</code>
Infer-S	1 T4	3	10	<code>ai.sap.com/ resourcePlan: infer.s</code>
Infer-M	1 T4	7	26	<code>ai.sap.com/ resourcePlan: infer.m</code>
Infer-L	1 T4	15	58	<code>ai.sap.com/ resourcePlan: infer.l</code>

⚠ Restriction

For the Free Tier service plan, only the Starter resource plan is available. Specifying other plans will result in error. For the Standard service plan, all resource plans are available. For more information, see [Free Tier \[page 38\]](#) and [Service Plans \[page 36\]](#).

📌 Note

There are limits to the default disk storage size for all of these nodes. Datasets that are loaded to the nodes will consume disk space. If you have large data sets (larger than 30 GB), or have large models, you may have to increase the disk size. To do so, use the persistent volume claim in Argo Workflows to specify the required disk size (see [Volumes](#) 📖).

Service Usage Reporting

Usage consumption of services is reported in the SAP BTP cockpit on the [Overview](#) page for your global account and on the [Overview](#) and [Usage Analytics](#) pages of your subaccount. The usage report lists usage in billable measures and non-billable measures. Your final monthly bill is based on the billable measure only. Non-billable measures are displayed for reporting purposes only.

4.5 Update a Service Plan

Learn how to update to the SAP AI Core standard or extended plan, after exploring the product in Free Tier.

Procedure

1. Open your global account in the SAP BTP cockpit.
2. Go to your subaccount.
3. In the navigation area, choose [Instances and Subscriptions](#).

A list of the applications to which your subaccount is subscribed in the Cloud Foundry environment is displayed.

4. Search for SAP AI Core.
5. Select the ellipsis at the end of the subscription row, and from the menu, select [Update](#).
6. In the wizard that opens, select [default](#) and click [Update Subscription](#).

				Created	Update
				Created	Create Binding
				Created	Create Service Key
				Created	Delete
				Created	...
free-plan-instance	SAP AI Core	standard			

Note

After you have updated your service plan, free tier restrictions are no longer applied.

Data that you have defined in the free tier plan is automatically migrated to your new plan.

Updating the service plan does not change user permissions.

Changing Service Plans

If you first subscribe to the free tier option, you can migrate the same service instance to a standard service plan (for enterprise accounts).

If you migrate from the free tier option to a standard plan or extended plan, or from a standard plan to an extended plan, your metadata and transaction data, including trained models, are also migrated (for enterprise accounts).

Restriction

It is not possible to migrate from a standard or extended service plan to the free tier option, or from an extended plan to a standard plan.

If an instance with an extended or standard plan is deleted, it is not possible to create a new instance with a standard plan.

5 Initial Setup

You provision SAP AI Core from the SAP BTP cockpit in SAP Business Technology Platform. After provisioning, you will have your service key, which provides URLs and credentials for accessing the SAP AI Core instance

Prerequisites

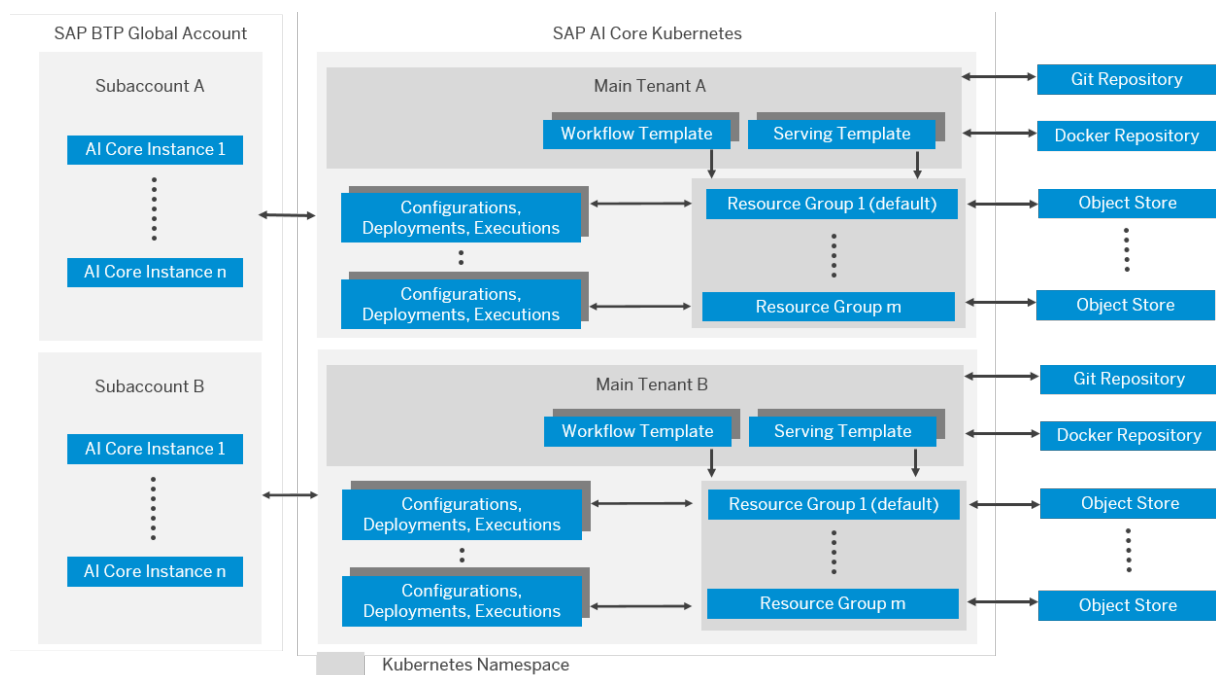
- Your SAP BTP administrator has access to a global account on SAP Business Technology Platform. For more information, see [Getting a Global Account](#).
- Your SAP BTP administrator has set the entitlement to a subaccount.

Context

The SAP AI Core service is a tenant-aware reuse service. It isolates tenants based on the ID of the zone (representing the subaccount). The SAP AI Core service instance is created within a subaccount. Each subaccount represents an SAP AI Core tenant.

Note

The SAP AI Core service does not isolate tenants based on the service instance ID. If you create multiple service instances within the same subaccount, all of them will reference the same SAP AI Core tenant.



The steps below guide you through the provisioning procedure. Alternatively, a booster is available for both SAP AI Core and SAP AI Launchpad. For more information, see [AI Boosters Tutorial](#). If you choose to use the booster, you can skip the remaining steps to [SAP AI Core Starter Tutorials \[page 65\]](#).

5.1 Enabling the Service in Cloud Foundry

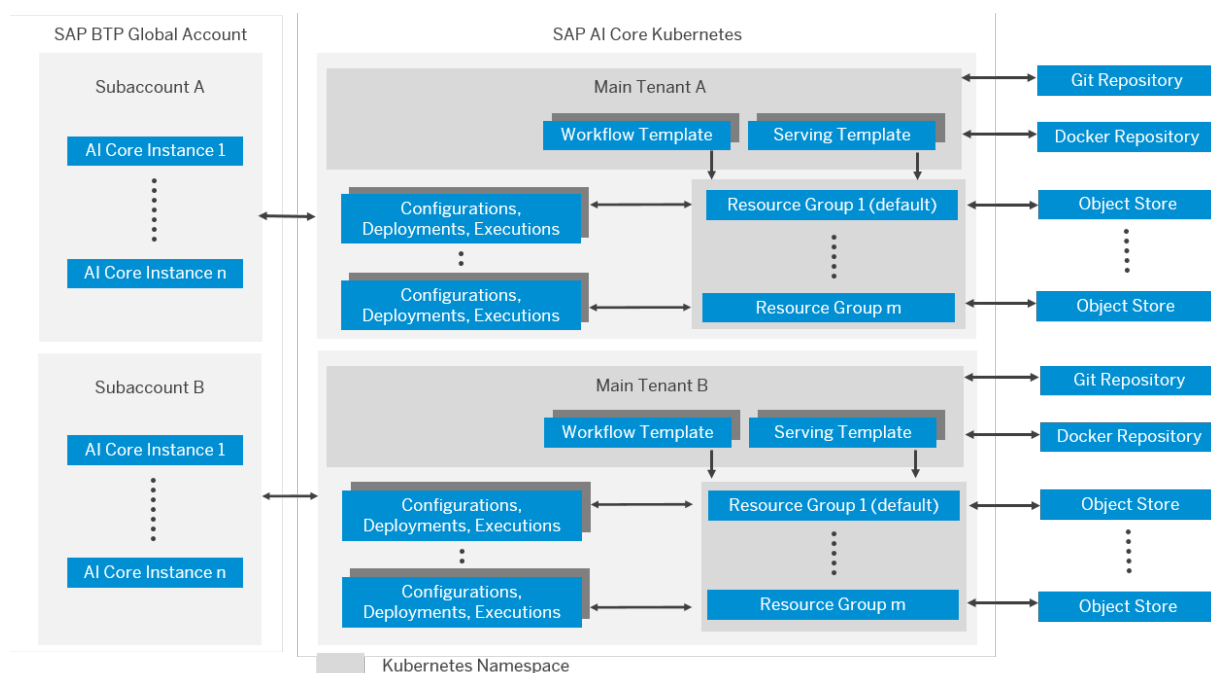
Enable SAP AI Core using the standard procedures for the SAP BTP Cloud Foundry environment.

You provision SAP AI Core from the SAP BTP cockpit in SAP Business Technology Platform. After you have provisioned the service, you will have your service key, which provides URLs and credentials for accessing the SAP AI Core instance.

The SAP AI Core service is a tenant-aware reuse service. It isolates tenants based on the ID of the zone (representing the subaccount). The SAP AI Core service instance is created within a subaccount. Each subaccount represents an SAP AI Core tenant.

Note

The SAP AI Core service does not isolate tenants based on the service instance ID. If you create multiple service instances within the same subaccount, all of them will reference the same SAP AI Core tenant.

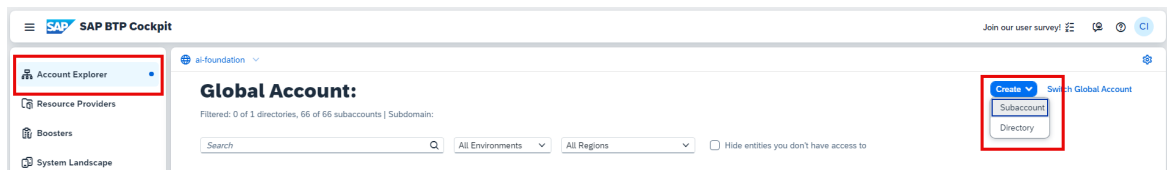


The steps below guide you through the provisioning procedure. Alternatively, a booster is available for both SAP AI Core and SAP AI Launchpad. For more information, see [AI Boosters Tutorial](#). If you choose to use the booster, you can skip the remaining steps to [SAP AI Core Starter Tutorials \[page 65\]](#).

5.1.1 Create a Subaccount

Procedure

1. In SAP Business Technology Platform cockpit, choose *Account Explorer* and then click ► *Create Subaccount*.



2. In the *Create Subaccount* dialog, enter a name for your subaccount and select the region. The parent defaults to the name of your global account.

The screenshot shows the 'Create Subaccount' dialog box. The 'Display Name' field is highlighted with a red box and contains the text 'My Subaccount'. The 'Region' dropdown menu is also highlighted with a red box and is set to 'Europe (Frankfurt) AWS 2'. Other fields include 'Description', 'Subdomain', and 'Parent'. At the bottom, there is an 'Advanced' section and 'Create' and 'Cancel' buttons.

3. **Optional:** If your subaccount is used for production purposes, under *Advanced* select the *Used for production* checkbox.

This setting does not change the configuration of your subaccount. It is intended to help you manage the production subaccounts in your global account. For example, your cloud operator can refer to it when handling incidents related to mission-critical accounts.

▼ **Advanced**

☒ **Used for production** ⓘ

☐ Enable beta features ⓘ

Labels ⓘ

Name: Values:

ⓘ

4. Click *Create*.
5. Return to the *Account Explorer* to view your subaccount.

SAP BTP Cockpit

ai-foundation ▼

Account Explorer

Resource Providers

Boosters

System Landscape

Entitlements >

Security >

Usage

Global Account

Filtered: 0 of 1 directories, 67 of 67 subaccounts

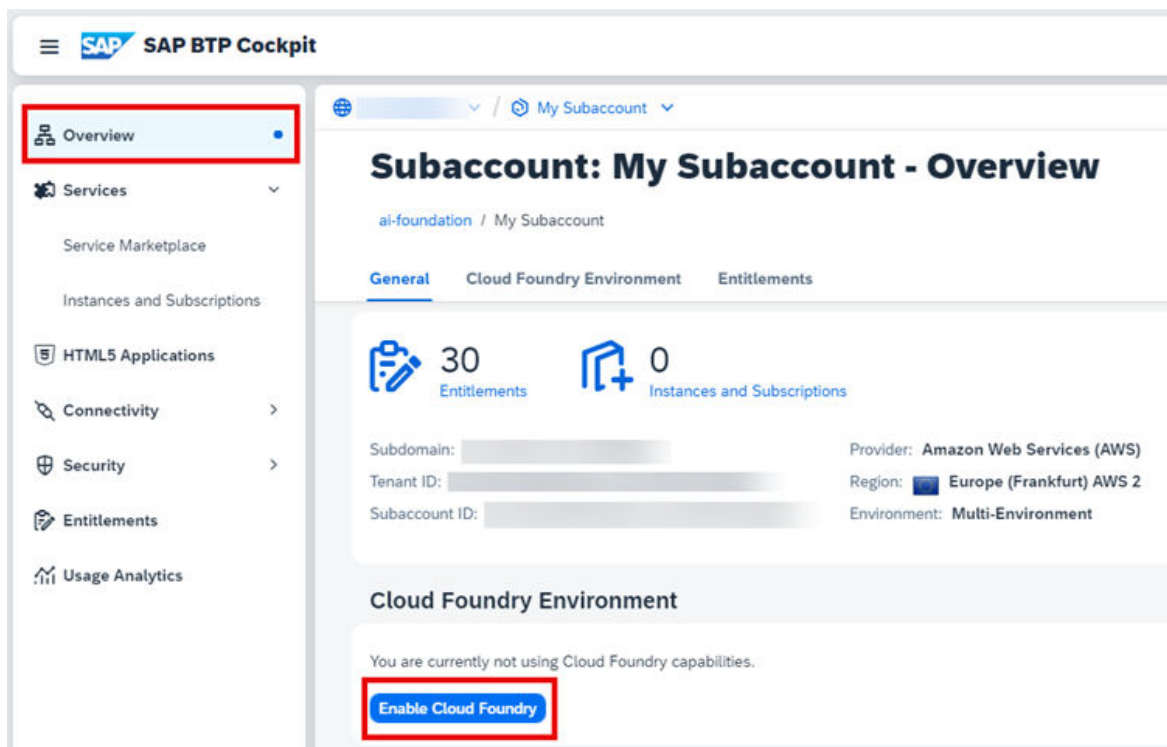
Directories and Subaccounts
Subaccounts (67)

My Subaccount

5.1.2 Enable Cloud Foundry

Procedure

1. Click your subaccount and on the *Overview* page, choose *Enable Cloud Foundry*.



2. Enter the basic information for your Cloud Foundry environment instance and click [Create](#).

Enable Cloud Foundry

1
Basic Info

Enter basic info for your environment instance.

Environment: * [Can't find what you're looking for?](#)
Cloud Foundry Runtime

Plan: *
standard

Landscape: *
cf-eu12

Instance Name: ⓘ

Org Name: * ⓘ

ⓘ Cloud Foundry API endpoints may differ between subaccounts in the same region.
The API endpoint for this subaccount is shown in the Cloud Foundry environment instance details.

Create **Cancel**

5.1.3 Create a Space

Procedure

1. On the overview page for your subaccount, choose [Create Space](#).

Subaccount: Testsubaccount - Overview

This is a test sub account created for hosting AI Core Service

[Edit](#)

[General](#) [Cloud Foundry Environment](#) [Entitlements](#)

13 Entitlements 1 Instances & Subscriptions

Subdomain: [redacted] Provider: Amazon Web Services (AWS) Used for Production: Yes Created On: Aug 25, 2021, 22:14
Tenant ID: [redacted] Region: Europe (Frankfurt) Beta Features: Disabled Modified On: Aug 25, 2021, 22:15
Subaccount ID: [redacted]

Cloud Foundry Environment

Org Name: [redacted] Spaces (0) [Create Space](#)
API Endpoint: [redacted]
Org ID: [redacted]
[Manage environment instance](#)
[Disable Cloud Foundry](#)

Name	Applications	Service Instances
No spaces defined in this subaccount		

2. Enter a name for your space, assign the required roles, and click [Create](#).

Create Space

Space Name: *

Test

Assign space roles to [redacted]

☒ Space Manager

☒ Space Developer

☒ Space Auditor

[Create](#) [Cancel](#)

Related Information

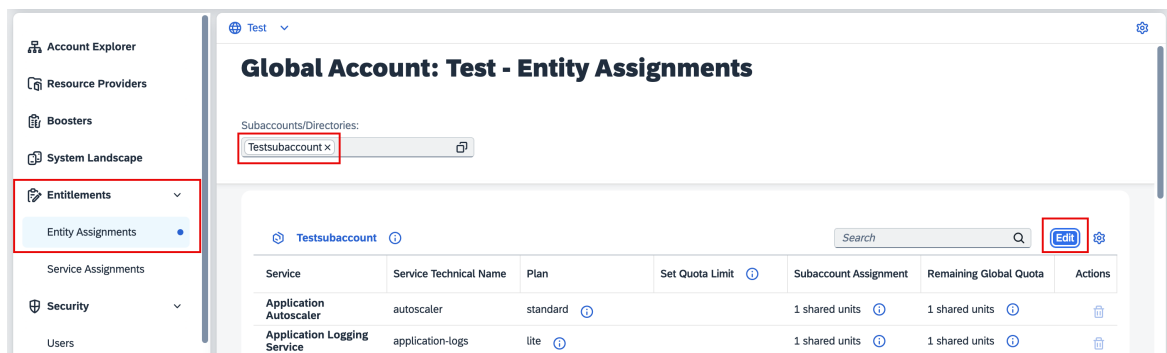
[About Roles in the Cloud Foundry Environment](#)

5.1.4 Add a Service Plan

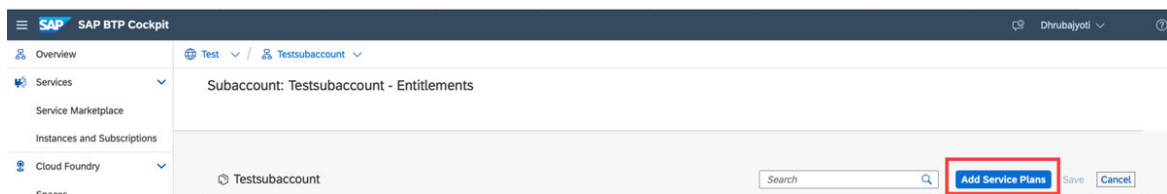
>

Procedure

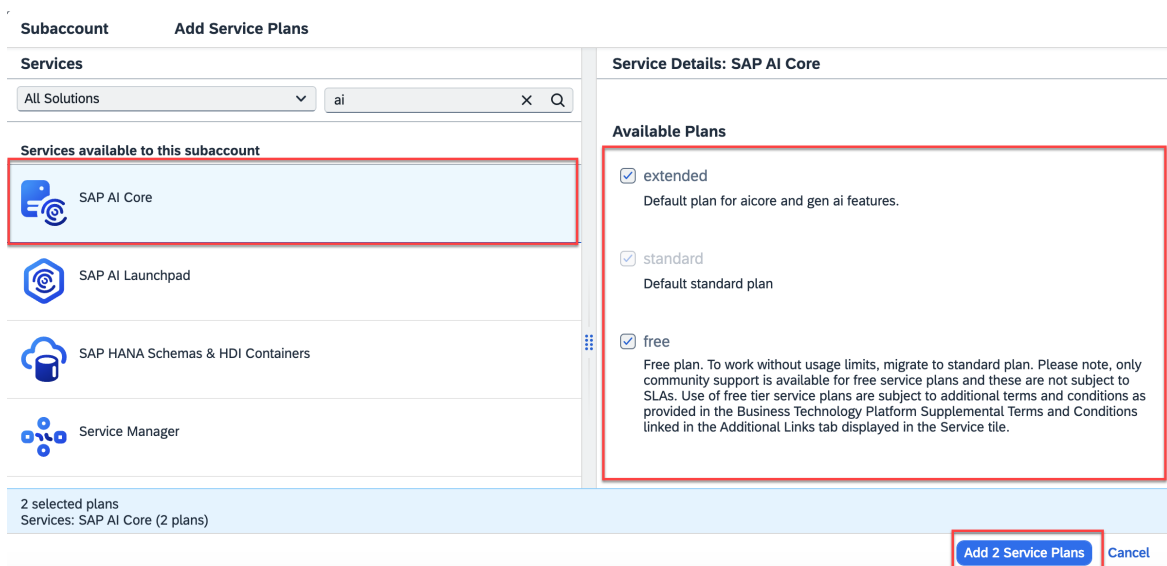
1. In SAP BTP cockpit, navigate to your global account and choose *Entitlements* and then *Entity Assignments*.
2. From the *Select Entities* box, select your subaccount and click *Select*.
3. Choose *Edit*.



4. Choose *Add Service Plans*.



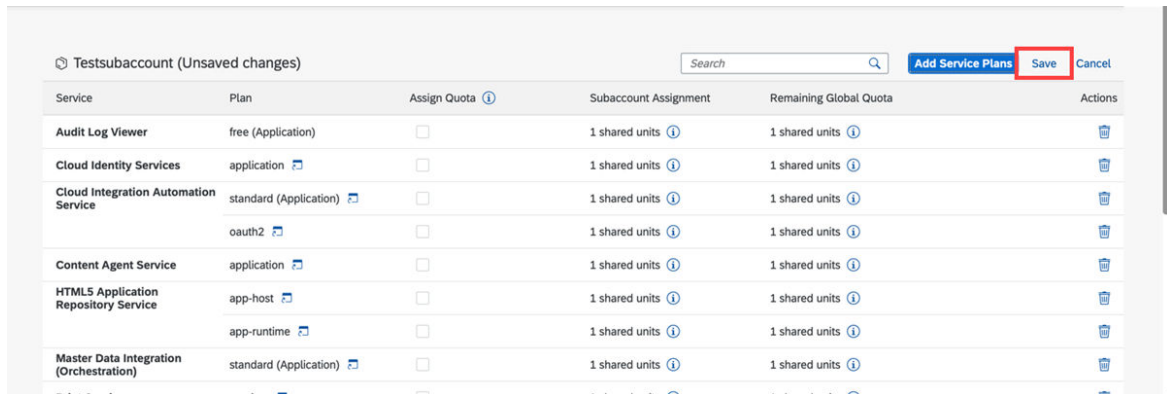
5. Select SAP AI Core and pick your chosen service plan or plans.



→ Tip

To use generative AI capabilities, choose the extended plan. For more information, see [Service Plans \[page 36\]](#).

6. Save your changes.



Service	Plan	Assign Quota	Subaccount Assignment	Remaining Global Quota	Actions
Audit Log Viewer	free (Application)	☐	1 shared units	1 shared units	
Cloud Identity Services	application	☐	1 shared units	1 shared units	
Cloud Integration Automation Service	standard (Application)	☐	1 shared units	1 shared units	
	oauth2	☐	1 shared units	1 shared units	
Content Agent Service	application	☐	1 shared units	1 shared units	
HTML5 Application Repository Service	app-host	☐	1 shared units	1 shared units	
	app-runtime	☐	1 shared units	1 shared units	
Master Data Integration (Orchestration)	standard (Application)	☐	1 shared units	1 shared units	

Related Information

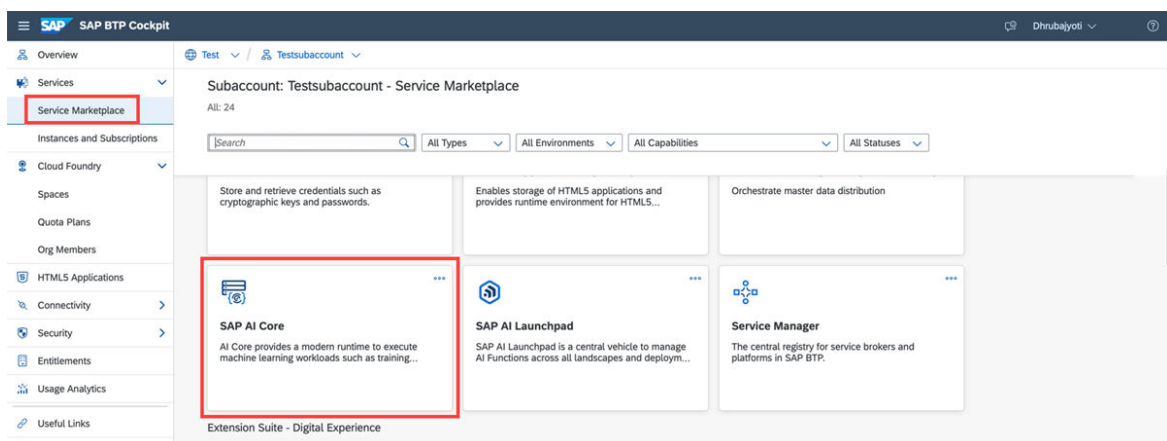
[Service Plans \[page 36\]](#)

5.1.5 Create a Service Instance

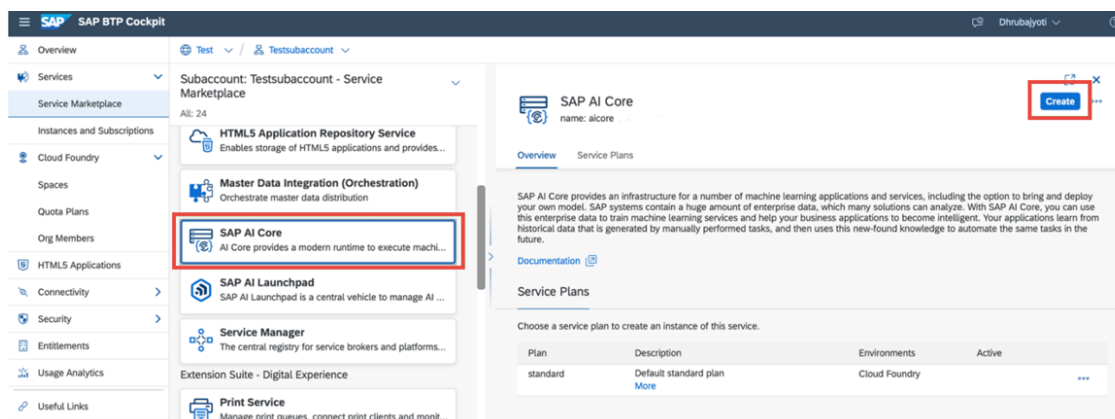
Procedure

1. In SAP BTP cockpit, navigate to your subaccount within your global account and choose [Service Marketplace](#).

You will see a tile for SAP AI Core.



2. Open the tile and click [Create](#).



3. Enter a name for your service instance and choose [Next](#) (all other details will be filled by default).

New Instance or Subscription

1
Basic Info

2
Parameters

3
Review

Enter basic info for your instance or subscription.

Service: * ⓘ
SAP AI Core

Plan: *
standard

Runtime Environment: *
Cloud Foundry

Space: *
TestAIFoundation

Instance Name: * ⓘ
TestAICore

Next > Create Cancel

4. At present, the JSON file upload feature is not supported. Choose [Next](#) to proceed.

New Instance or Subscription

1

Basic Info

2

Parameters

3

Review

 Browse...

Clear

1

< Back

Next >

Create

Cancel

5. Check the data and choose *Create*.

New Instance or Subscription

1

Basic Info

2

Parameters

3

Review

TestAICore

Service: SAP AI Core

Service Plan: standard

Runtime Environment: Cloud Foundry

Space: TestAIFoundation

Creating an instance might take a while.

< Back

Create

Cancel

Results

When your service instance is created, you can view it on the *Instances and Subscriptions* page of your subaccount.

SAP

SAP BTP Cockpit

Overview

Services

Service Marketplace

Instances and Subscriptions

Cloud Foundry

Spaces

Quota Plans

Org Members

HTML5 Applications

Connectivity

Security

Entitlements

Usage Analytics

Useful Links

Legal Information

Test

Testsubaccount

Subaccount: Testsubaccount - Instances and Subscriptions

Create

To manage the Cloud Foundry user-provided service instances, navigate to Cloud Foundry - Spaces, select your space, and then from Services select Service Instances.

Search

All Services

All Plans

All Statuses

Subscriptions (0)

Instances (1)

Environments (1)

Instances (1)

Service instances created in: Cloud Foundry | Kyma/Kubernetes | Other environments

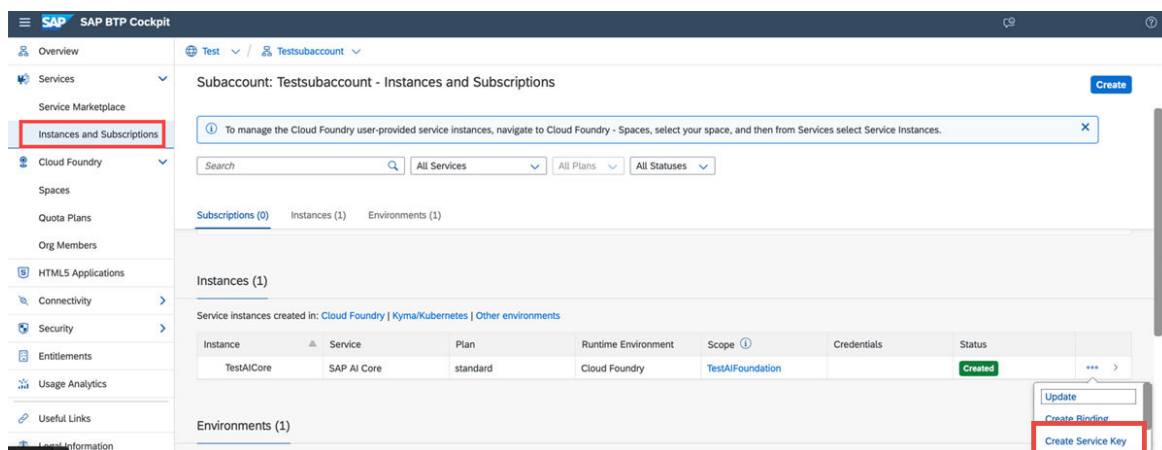
Instance	Service	Plan	Runtime Environment	Scope	Credentials	Status	
TestAICore	SAP AI Core	standard	Cloud Foundry	TestAIFoundation		Created	*** >

Environments (1)

5.1.6 Create a Service Key

Procedure

1. On the *Instances and Subscriptions* page, find your new instance and choose *Create Service Key* from the dropdown.



2. Enter a name for your service key and click *Create*.

New Service Key

Service Key Name: *

TestAICore

Configure Binding Parameters: ⓘ

Upload a JSON file:

Select JSON file Browse...

Or specify the parameters in JSON format: Clear

1

Create Cancel

Optional: To use an x.509 certificate instead of client secret credentials, specify the credentials by updating and uploading the following as a JSON:

```
{ "xsuaa": { "credential-type": "x509", "x509": { "key-length": 2048, "validity": 7, "validity-type": "DAYS" } }
```

- **key-length:** The byte length of the generated private key. Default: 2048.
- **validity:** The validity time unit. DAYS, MONTHS and YEARS are supported. Default: DAYS.
- **validity-type:** The number of time units. Default: 7.

The default combination is a key of length 2048, valid for 7 DAYS.

3. Download your service key to save it.

Results

You now have your service key, which provides URLs and credentials for accessing the SAP AI Core instance through SAP AI Launchpad, SAP AI Core toolkit Postman, or curl.

Credentials

aicore-servicekey

Form

JSON

```
1 {
2   "clientid": "sb-4376bdb7-_____, b313",
3   "clientsecret": "xj._____",
4   "url": "https://_____.authentication.sap.hana.ondemand.com",
5   "identityzone": "ai-_____",
6   "identityzoneid": "1eb727a0-(_____) ",
7   "appname": "4_____, e!b313",
8   "serviceurls": {
9     "AI_API_URL": "https://api.ai.i_____.s.ml.hana.ondemand.com"
10  }
11 }
```

Copy JSON

Download

Close

If you have generated a client secret, your key will include:

- `clientid`, `clientsecret`, and `url` can be used to generate your authentication token.
- `identityzone` and `identityzoneid` represent your tenant ID.
- `appname` provides the service instance details if service instance isolation is implemented.
- `serviceurls` allow you to interact with SAP AI Core once your authentication token has been generated.
 - `AI_API_URL`: Unified AI API to handle ML artifacts (such as training, data, models, and deployments) across multiple hyperscalers.

If you have generated an x.509 certificate, your key will include:

- `certificate`
- `certurl` can be used to generate your authentication token.
- `key` your RSA private key.
- `identityzone` and `identityzoneid` represent your tenant ID.
- `appname` provides the service instance details if service instance isolation is implemented.
- `serviceurls` allow you to interact with SAP AI Core once your authentication token has been generated.
 - `AI_API_URL`: Unified AI API to handle ML artifacts (such as training, data, models, and deployments) across multiple hyperscalers.

5.1.7 Use a Service Key

After you have configured your service key, it can be used by local clients, apps in other spaces, or entities outside your deployment to access SAP AI Core through one of the available interfaces.

Using Postman

Prerequisites

- You have downloaded and installed the Postman client from <https://www.postman.com/> .
- You have familiarized yourself with the Postman documentation and interface.

Procedure

1. Download the JSON collection from https://api.sap.com/api/AI_CORE_API/overview .
2. In Postman, click [Import](#), select the JSON file, and choose [Import](#) to start the import.
3. After the import is complete, highlight the collection and select the [Authorization](#) tab.
4. Navigate to [Configure New Token](#), enter the credentials from your service key, and save your changes.

Note

- The [Token Name](#) field is your choice of descriptive identifier.
- The [Access Token URL](#) is labeled [url](#) in your service key. Add [/oauth/token](#) to the end of the URL.
- The [Grant Type](#) should be **Client Credentials**.

If you see an alert relating to the characters in your credentials, ignore it.

Note

If you have generated a x.509 certificate instead of client secret credentials, you'll need to use your [certificate](#), [key](#) and [certUrl](#) to create your token.

```
For example: curl --cert <cert.pem> --key <key.pem> -XPOST <certUrl>/oauth/token  
-d 'grant_type=client_credentials&client_id=<client id>'
```

5. Select the [Variables](#) tab, and set your [baseUr l](#) from your credentials.
The [baseUr l](#) is labeled [AI_API_URL](#) in your service key.
6. Choose [Save](#).
7. In the [Authorization](#) tab, choose [Get New Access Token](#). and wait for the authentication process.
When the authentication process is complete, select [Use Token](#) to finish. Check that the token is stored in your environment variables. If it is not stored automatically, you can copy and paste it to the [Token](#) field manually.

Next Steps

To train and deploy your own AI models, follow the procedure in [Administration \[page 66\]](#).

To use generative AI models provided in the generative AI hub, see [Models and Scenarios in the Generative AI Hub \[page 99\]](#).

Using Curl

Prerequisites

curl is likely to be installed on your operating system by default. To check, open a command prompt and enter `curl -V`. If curl isn't installed, download and install it from <https://curl.se/> .

Note

On macOS, you may need to install jq so that you can follow the curl commands.

1. Install brew from <https://brew.sh/> .
2. In a Terminal session, run `brew install jq` to install jq in your shell environment.

Procedure

1. Set up your environment as follows:

For Linux:

```
# XSUAA details
# URLs should be without trailing slash '/'
export CLIENTID=<clientid> from service key
export CLIENTSECRET=<clientsecret> from service key
export XSUAA_URL=<url> from service key
export AI_API_URL=<AI_API_URL> from service key
```

For Windows PowerShell:

```
$env:CLIENTID = <clientid> from service key
$env:CLIENTSECRET = <clientsecret> from service key
$env:XSUAA_URL = <url> from service key
$env:AI_API_URL = <AI_API_URL> from service key
```

Note

The `export` command sets the values of your keys to your environment variable, meaning that they will be retained after you close your terminal session. It is possible to set the environment variables without the `export`, for the current session only.

2. Get the XSUAA OAuth Token using `clientid` and `clientsecret` from the service key to call the APIs.

The XSUAA OAuth token is required for authentication when making AI API calls.

For Linux:

```
SECRET=`echo -n '$CLIENTID:$CLIENTSECRET' | base64 -i -`
TOKEN=`curl --location --request POST "$XSUAA_URL/oauth/token?
grant_type=client_credentials" \
--header "Authorization: Basic $SECRET" | jq -r '.access_token'`
```

For Windows PowerShell:

```
$SECRET = $env:CLIENTID + ":" + $env:CLIENTSECRET
$base64SECRET =
[Convert]::ToBase64String([System.Text.Encoding]::UTF8.GetBytes("$SECRET"))
$TOKENRESPONSE = curl --location --request POST "$XSUAA_URL/oauth/token?
grant_type=client_credentials" --header "Authorization: Basic $base64SECRET"
$TOKENRESPONSE = $TOKENRESPONSE | ConvertFrom-Json
$TOKEN = $TOKENRESPONSE.access_token
```

Note

The token is valid for a limited time. Once it has expired, create a new token, using the same code snippet.

Note

If you have generated a x.509 certificate instead of client secret credentials, you'll need to use your certificate, key and certUrl to create your token.

For example: `curl --cert <cert.pem> --key <key.pem> -XPOST <certUrl>/oauth/token -d 'grant_type=client_credentials&client_id=<client id>'`

3. Verify that the token has been fetched properly:

```
echo $TOKEN
```

You should see a long string of alphanumeric characters:

```
eyJhbGciOiJSUzI1NiIsImprSI6Imh0dHBzOi8vYWktYWxwaGEtdmFsawRhdGlvbi0yLmF1dGhlnbn
RPy2F0aw9uLnNhC5oYW5hLm9uZGVtYW5kLmNvbS90b2t1b19rZXlzIiwia2lkIjoizGVmYXVsdC1q
d3Qta2V5LTMyODMxMjg2NCIsInR5cCI6IkpXVCJ94ZGU5YjAxNmQ0MDk5YjlmM...
...ALdfbMsHoYTtF6fNFbf3ZQ
```

Next Steps

To train and deploy your own AI models, follow the procedure in [Administration \[page 66\]](#).

To use generative AI models provided in the generative AI hub, see [Models and Scenarios in the Generative AI Hub \[page 99\]](#).

5.2 Enabling the Service in the Kyma Environment

Enable SAP AI Core using the standard procedures for the SAP BTP Kyma environment.

Procedure

1. Create a service instance in the Kyma environment.
2. You can then bind the service instance to your application, or you can create a service key to communicate directly with the service instance. See [Using SAP BTP Services in the Kyma Environment](#).

Related Information

[Using SAP BTP Services in the Kyma Environment](#)

6 SAP AI Core Starter Tutorials

You can use a booster to get started with the Free Tier plans for SAP AI Core and SAP AI Launchpad using the tutorial: [AI Boosters](#) .

Alternatively, to complete the booster tutorial, then learn the basics of SAP AI Core from beginning to end in a simple use case, follow the tutorial: [Quick Start for SAP AI Core](#) . If you have already provisioned SAP AI Core, you can skip through to the second tutorial in this guide.

To see the complete library of tutorials available for SAP AI Core, see [Tutorials \[page 279\]](#).

Note

Once you have provisioned SAP AI Core, you can access the service in multiple ways. Where the tutorial below offers more than one option tab, you need only complete the tab for your chosen method of access. For customers who prefer a GUI to code, SAP AI Launchpad is recommended. For more information, see [SAP AI Launchpad](#).

7 Administration

Creating secrets for external programs and tools, that are used with SAP AI Core means that you can connect them without compromising your credentials.

Using SAP AI Core alongside external tools such as GitHub, Docker and Amazon Web Services S3 storage leverages the benefits of version control, containerization, and cloud storage. Your content is made available remotely, if you have a stable internet connection.

Administration is a one-time procedure, but steps can be repeated, if necessary, for example to remove or add a tool.

📘 Note

You must have completed the initial setup tasks before configuring your SAP AI Core instance. For more information, see [Initial Setup \[page 46\]](#).

[Manage Resource Groups \[page 74\]](#)

A resource group is a unique dedicated namespace or workspace environment, where users can create or add configurations, executions, deployments, and artifacts. They are used for running training jobs or model servers.

7.1 Manage Your Git Repository

7.1.1 Add a Git Repository

You can use your own git repository to version control your SAP AI Core templates. The GitOps onboarding to SAP AI Core instances involves setting up your git repository and synchronizing your content.

Prerequisites

You have completed the Initial Setup. For more information, see [Initial Setup \[page 46\]](#).

Context

Git repositories are managed by creating personal access tokens registering them in SAP AI Core. Personal access tokens are a means of allowing and controlling connections to GitHub repositories without compromising your credentials.

Prerequisites

- You have access to a git repository over the Internet.
- You have generated a personal access token for your git repository. For more information, see [Create a Personal Access Token](#) .
- If you want to onboard a git repository hosted on GitLab, make sure that the repository URL contains the `.git` suffix.
- Secrets are not permitted in your repository. If secrets are used, it is not possible to synchronize content.

Note

When you synchronize resources, make sure that there are no naming collisions. This is particularly important if you use multiple repositories or applications in one tenant. If you experience difficulties during synchronization, we recommend that you use only one repository or application per tenant.

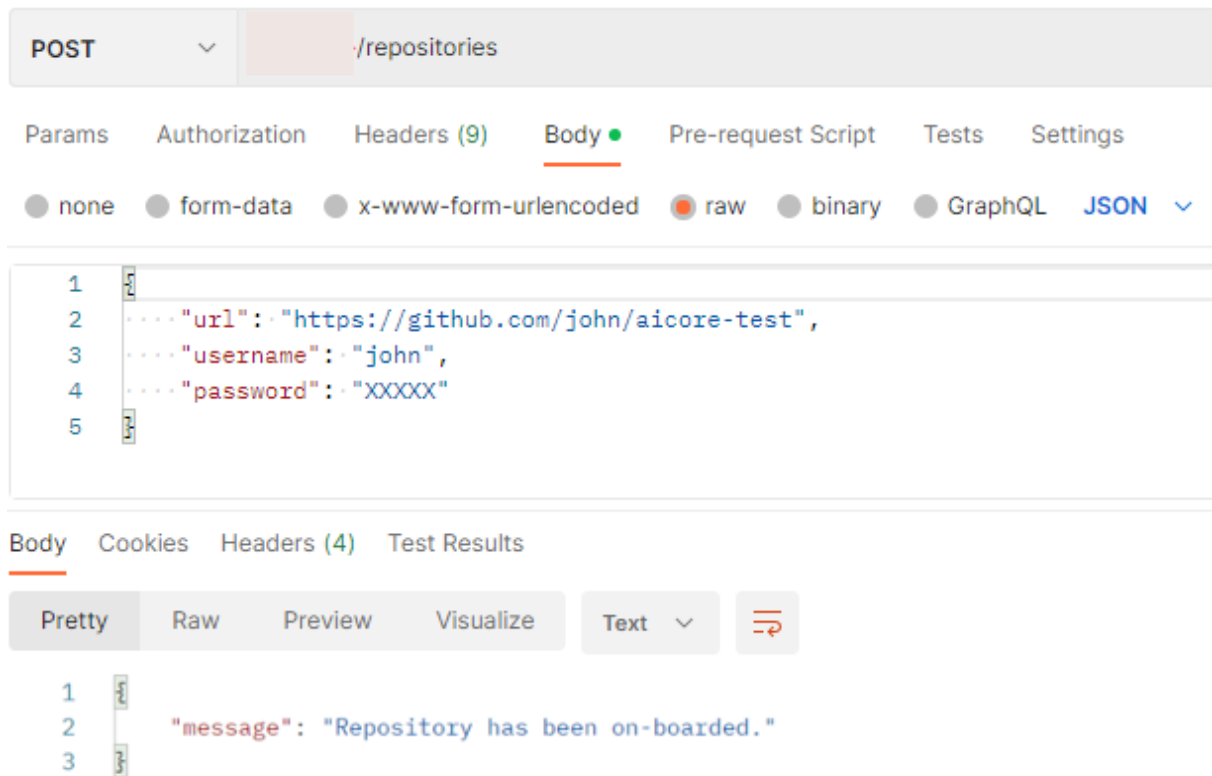
Restriction

The maximum number of git repository secrets is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some git repository secrets. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Git Repository \[page 72\]](#).

Using Postman

Send a POST request to the endpoint `{{apiurl}}/v2/admin/repositories` and include your credentials in JSON format in the *raw* body:



Using curl

Submit a POST request to the endpoint `{{apiurl}}/v2/admin/repositories` and include your credentials:

```
curl --location --request POST "$AI_API_URL/v2/admin/repositories" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--data-raw '{
  "url": "https://github.com/john/exemplerepo",
  "username": "john",
  "password": "<GIT_PAT_USER_TOKEN>"
}'
```

You specify your unique git repository details as follows:

- `url`: URL of the git repository
- `username`: (Service) user that is accessing the git repository
- `password`: git personal access token. For more information, see [Create a Personal Access Token](#) 📖.

7.1.2 Create an Application to Sync Your Folders

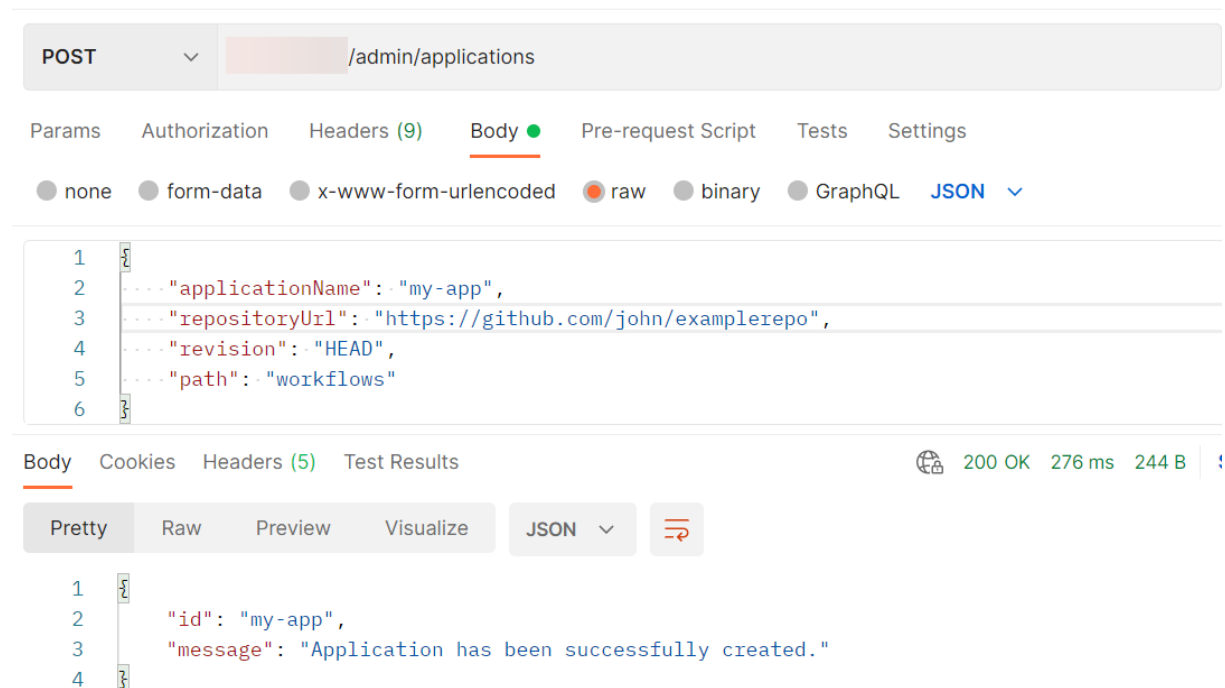
Context

After you have registered your git repository, you need to create an application to sync the templates in your repository. The first sync will take some time, but you can check the status of the application to see when it is complete. After the first sync, syncing takes place automatically approximately every three minutes, or can be requested manually.

Using Postman

Procedure

Send a POST request to the endpoint `{{apiurl}}/v2/admin/applications` including details of your application:



- `applicationName`: Set a name for your application. The name must be between 3 and 64 characters long and match `[A-Za-z0-9\-\_\+]`.
- `repositoryUrl`: The URL of a registered git repository.
- `revision`: The revision to target. `<HEAD>` refers to the most recent.
- `path`: The path to the target folder that contains the templates to be synced.

Since each application points to a particular path and revision in the repository, multiple applications may be created for the same `repositoryUrl`.

Results

After the GitOps setup is completed, the templates in your git repository are automatically synced to SAP AI Core at ~3-minute intervals.

Next Steps

Check the synchronization status of your application by sending a GET request to `{{apiurl}}/v2/admin/applications/{{appName}}/status`. As `appName`, enter the name of your application that you specified when you created the application.

The screenshot shows a REST client interface with a GET request to `admin/applications/{{appName}}/status`. The response is a JSON object indicating a successful sync. The status is 200 OK, with a response time of 212 ms and a size of 792 B.

```
1  {
2    "healthStatus": "Healthy",
3    "message": "successfully synced (all tasks run)",
4    "reconciledAt": "2021-11-23T10:27:49Z",
5    "source": {
6      "path": "workflows",
7      "repoURL": "https://github.com/john/examplerepo",
8      "revision": "db611bb28be3c853d08867c08b52b8f733b4f7bf"
9    },
10   "syncFinishedAt": "2021-11-23T10:27:49Z",
11   "syncResourcesStatus": [
12     {
13       "kind": "ServingTemplate",
14       "message": "servingtemplate.ai.sap.com/text-clf-infer-tutorial configured",
15       "name": "text-clf-infer-tutorial",
16       "status": "Synced"
17     }
18   ],
19   "syncStartedAt": "2021-11-23T10:27:48Z",
20   "syncStatus": "Synced"
21 }
```

⚠ Restriction

The maximum number of applications is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some applications. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete an Application \[page 74\]](#).

Using curl

Submit a POST request to the endpoint `{{apiurl}}/v2/admin/applications` including details of your application:

```
curl --location --request POST "$AI_API_URL/v2/admin/applications" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--data-raw '{
  "applicationName": "my-app",
  "repositoryUrl": "https://github.com/john/examplerepo",
  "revision": "HEAD",
}
```

```
}', "path": "workflows"
```

- **applicationName**: Set a name for your application. The name must be between 3 and 64 characters long and match `[A-Za-z0-9\-\_\+]`.
- **repositoryUrl**: The URL of a registered git repository.
- **revision**: The revision to target. `<HEAD>` refers to the most recent.
- **path**: The path to the target folder that contains the templates to be synced.

Since each application points to a particular path and revision in the repository, multiple applications may be created for the same `repositoryUrl`.

Results

After the GitOps setup is completed, the templates in your git repository are automatically synced to SAP AI Core at ~3-minute intervals.

Next Steps

Check the synchronization status of your application by submitting a GET request to `{{apiurl}}/v2/admin/applications/{{appName}}/status`:

```
curl --location --request GET "$AI_API_URL/v2/admin/applications/{{appName}}/status" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json'
```

As `applicationName`, enter the name of your application that you specified when you created the application.

⚠ Restriction

The maximum number of applications is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some applications. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete an Application \[page 74\]](#).

Sync an Application Manually

Applications sync with your GitHub repository automatically at intervals of ~3 minutes. Use the endpoint below to manually request a sync:

```
{{apiurl}}/admin/applications/{{appName}}/refresh
```

7.1.3 Edit a Git Repository

Using Postman

Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/repositories/{{repositoryName}}` and include your changes in the body.

Using curl

Submit a PATCH request to the endpoint `{{apiurl}}/v2/admin/repositories` and include your changes:

```
curl --location --request PATCH "$AI_API_URL/v2/admin/repositories" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--data-raw '{
    "url": "https://github.com/john/exemplerepo",
    "username": "john",
    "password": "<GIT_PAT_USER_TOKEN>"
}'
```

You specify your unique git repository details as follows:

- `url`: URL of the git repository
- `username`: (Service) user that is accessing the git repository
- `password`: git personal access token. For more information, see [Create a Personal Access Token](#) .

7.1.4 Delete a Git Repository

You remove a Git repository from a connection if its URL is invalid or contains errors, or if the repo is no longer required. Once a Git repository is removed, it can no longer be selected as a source repository for an application.

Using Postman

Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/repositories/{{repositoryName}}` and include your repository name.

Using curl

```
curl --location --request DELETE "{{apiurl}}/v2/admin/repositories/{{repositoryName}}" \
```

7.2 Manage Applications

7.2.1 List Applications

Using Postman

Send a GET request to the endpoint `{{apiurl}}/v2/admin/applications`.

Using curl

Send a GET request to the endpoint `{{apiurl}}/v2/admin/applications`.

7.2.2 Edit an Application

Using Postman

Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/applications/{{appName}}` and include your changes in the body.

Using curl

Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/applications/{{appName}}` and include your changes in the body.

7.2.3 Delete an Application

Using Postman

Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/applications/{{appName}}`.

Using curl

Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/applications/{{appName}}`.

7.3 Manage Resource Groups

A resource group is a unique dedicated namespace or workspace environment, where users can create or add configurations, executions, deployments, and artifacts. They are used for running training jobs or model servers.

Resource groups are used to physically isolate machine learning workloads, and to logically isolate related resources for a usage scenario.

When your tenant is onboarded, a default resource group is automatically created. Default resource groups can't be deleted.

As an administrator, you create, edit, or delete resource groups, based on your service consumers and usage scenarios.

Runtime entities such as executions, deployments, configurations, and artifacts belong to a specific resource group and are not shared across resource groups. Scenarios, executables, and Docker registry secrets are shared by all resource groups within a tenant.

A resource group may also be referred to as an instance.

→ Remember

Your SAP global account may consist of several accounts. Each account can be associated with a tenant. A tenant can contain multiple resource groups. A tenant always contains a default resource group, as well as the resource groups defined for your usage scenarios.

⚠ Restriction

The maximum number of resource groups is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some resource groups. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Resource Group \[page 78\]](#).

[Create a Resource Group \[page 75\]](#)

[Edit a Resource Group \[page 77\]](#)

[Delete a Resource Group \[page 78\]](#)

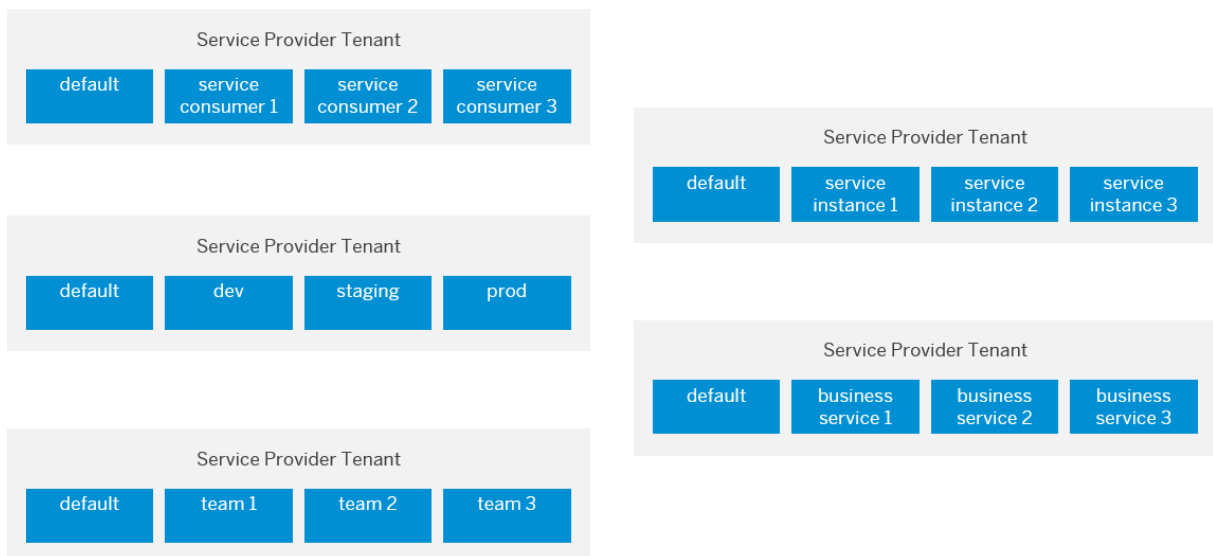
Parent topic: [Administration \[page 66\]](#)

Resource Group Level Resources

Executables at tenant level are shared across all resource groups. In contrast, runtime entities such as executions, deployments, configurations, and artifacts belong to a specific resource group and cannot be shared across resource groups. Similarly, generic secrets created within a resource group and be used only for workloads within that group.

You can register an object store at resource-group level by setting the resource group header. We recommend that you do not use the same object store bucket with the same IAM user for multiple resource groups.

Example resource group mappings are outlined in the figure below:



7.3.1 Create a Resource Group

Prerequisites

You have completed the Initial Setup. For more information, see [Initial Setup \[page 46\]](#).

Note

Resource group IDs must be of length minimum: 3, maximum: 253. The first and last characters must be either a lower case letter, an upper case letter or a number. Character entries from the second to penultimate, may include a lower case letter, an upper case letter, a number, a full stop or a hyphen. No other special characters are permitted.

Using Postman

1. Send a POST request to the endpoint `{{apiurl}}/v2/admin/resourceGroups`.
2. As the request body, select the **raw** radio button and enter the following:

```
{
  "resourceGroupId": "<ID of your resource group>"
}
```



3. Send the request.
You'll receive a 202 response to confirm that the request to create the resource group has been accepted.

⚠ Restriction

The maximum number of resource groups is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some resource groups. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Resource Group \[page 78\]](#).

Using curl

1. Create a resource group by sending the following:

```
curl --location --request POST "$AI_API_URL/v2/admin/resourceGroups" --header "Authorization: Bearer $TOKEN" --header 'Content-Type: application/json' --data-raw '{ "resourceGroupId": "<ID of your resource group>" }'
```

⚠ Restriction

The maximum number of resource groups is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some resource groups. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Resource Group \[page 78\]](#).

Parent topic: [Manage Resource Groups \[page 74\]](#)

Related Information

[Edit a Resource Group \[page 77\]](#)

[Delete a Resource Group \[page 78\]](#)

[Multitenancy \[page 287\]](#)

7.3.2 Edit a Resource Group

📘 Note

Resource group IDs must be of length minimum: 3, maximum: 253. The first and last characters must be either a lower case letter, an upper case letter or a number. Character entries from the second to penultimate, may include a lower case letter, an upper case letter, a number, a full stop or a hyphen. No other special characters are permitted.

Using Postman

1. Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/resourceGroups/{{resource_group_name}}` with the body:

```
{
  "resourceGroupId": "<ID of your resource group>"
}
```

Using curl

1. Create a resource group by sending the following:

```
curl --location --request PATCH "$AI_API_URL/v2/admin/resourceGroups/
{{resource_group_name}}" --header "Authorization: Bearer $TOKEN" --header
'Content-Type: application/json' --data-raw '{ "resourceGroupId": "<ID of
your resource group>"}'
```

Parent topic: [Manage Resource Groups \[page 74\]](#)

Related Information

[Create a Resource Group \[page 75\]](#)

[Delete a Resource Group \[page 78\]](#)

7.3.3 Delete a Resource Group

Using Postman

Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/resourceGroups/{{resource_group_name}}`.

Using curl

1. Create a resource group by sending the following:

```
curl --location --request POST "$AI_API_URL/v2/admin/resourceGroups/
{{resource_group_name}}" --header "Authorization: Bearer $TOKEN" --header
'Content-Type: application/json' --data-raw '{ "resourceGroupId": "<ID of
your resource group>"}'
```

Parent topic: [Manage Resource Groups \[page 74\]](#)

Related Information

[Create a Resource Group \[page 75\]](#)

7.4 Manage Object Store Secrets

7.4.1 Register an Object Store Secret

Connect SAP AI Core to a cloud object store and manage access using an object store secret. The connected storage stores your dataset, models, and other cache files of the Metaflow Library for SAP AI Core.

Your cloud storage credentials are managed using secrets. Secrets are a means of allowing and controlling connections across directories and tools, without compromising your credentials.

Prerequisites

You have completed the Initial Setup. For more information, see [Initial Setup \[page 46\]](#).

Context

SAP AI Core supports multiple hyperscaler object stores, such as Amazon S3, OSS (Alicloud Object Storage Service), SAP HANA Cloud, Data Lake, and Azure Blob Storage.

Using Postman

1. Send a POST request to the endpoint `{{apiurl}}/v2/admin/objectStoreSecrets`.
2. As the request body, select the [raw](#) radio button and enter your object store secret details.

Note

For all storage types **except** Azure Blob Storage, all `<data>` fields are required. For Azure, required fields are specified.

- For Amazon S3:

```
{
  "name": "<Your identifier>",
  "data": {
    "AWS_ACCESS_KEY_ID": "<AWS access key ID>",
    "AWS_SECRET_ACCESS_KEY": "<AWS secret access key>"
  },
  "type": "S3",
  "bucket": "<S3 bucket name>",
  "endpoint": "<S3 end point>",
```

```

    "region": "<S3 region>",
    "pathPrefix": "<A path prefix that follows the bucket name>"
  }

```

- For OSS (Alicloud Object Storage Service):

```

{
  "name": "default",
  "type": "oss",
  "pathPrefix": "<path prefix to be appended with bucketname>",
  "data": {
    "BUCKET": "<bucket-name>",
    "ENDPOINT": "oss-cn-shanghai.aliyuncs.com", {
      "name": "default",
      "type": "S3",
      "bucket": "{
        "name": "default",
        "type": "oss",
        "pathPrefix": "<path prefix to be appended with bucketname>",
        "data": {
          "REGION": "",
          "ACCESS_KEY_ID": "xxxxx",
          "SECRET_ACCESS_KEY": "xxxxx"
        }
      }
    }
  }
}

```

- For SAP HANA Cloud, Data Lake:

```

    "BUCKET": "{
      "name": "default",
      "type": "webhdfs",
      "pathPrefix": "<path prefix to be appended>",
      "data": {
        // e.g. https://c32727c8-4260-4c37-b97f-ed322dcfa8f.files.hdl.canary-
        eu10.hanacloud.ondemand.com
        "HDFS_NAMENODE": "https://<file-container-name>.files.hdl.canary-
        eu10.hanacloud.ondemand.com",
        "TLS_CERT": "-----BEGIN CERTIFICATE-----
        \nMIICmjCCAYIxxxxxxxxxxxxR4wtC32bG066D+Jc8RhaIA==\n-----END
        CERTIFICATE-----\n",
        "TLS_KEY": "-----BEGIN PRIVATE KEY-----
        \nMIIEvQIBADANBgkqxxxxxxxxxxxxnor+rtZHhHzEfX5dYLC5Pww=\n-----END PRIVATE
        KEY-----\n",
        "HEADERS": "{ \"x-sap-filecontainer\": \"<file-container-name>\",
        \"Content-Type\": \"application/octet-stream\"}"
      }
    }
  }
}

```

- For Azure Blob Storage:

⚠ Restriction

For **executions** only, access keys are supported. SAS tokens are **not** supported.

For **deployments**, access keys and SAS tokens are supported.

```

{
  "name": "default",
  "type": "azure",
  "pathPrefix": "<path prefix to be appended>",
  "data": {
    "CONTAINER_URI": "https://account_name.blob.core.windows.net/
    container_name", //required
    "REGION": "<region name>", //optional
    "CLIENT_ID": "<azure client id>", //optional
    "CLIENT_SECRET": "<azure client secret>", //optional
  }
}

```



```

    "STORAGE_ACCESS_KEY": "sas_token",           //required
    "TENANT_ID": "azure tenant id",             //optional
    "SUBSCRIPTION_ID": "subscription id",        //optional
  }
}

```

3. Send the request.

The following shows a request using S3.



Using curl

1. Register your object store secret details using the endpoint `/v2/admin/objectStoreSecrets`.

Note

For all storage types **except** Azure Blob Storage, all `<data>` fields are required. For Azure, required fields are specified.

- For Amazon S3:

```

curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
  "name": "default",
  "type": "S3",
  "bucket": "<S3 bucket name>",
  "endpoint": "<S3 end point>",
  "pathPrefix": "<A path prefix that follows the bucket name>",
  "region": "<S3 region>",
  "data": {
    "AWS_ACCESS_KEY_ID": "<AWS access key ID>",
    "AWS_SECRET_ACCESS_KEY": "<AWS secret access key>"
  }
}'

```

- For OSS (Alicloud Object Storage Service):

```

curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \

```

```
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
  "name": "default",
  "type": "oss",
  "pathPrefix": "<path prefix to be appended with bucketname>",
  "data": {
    "BUCKET": "<bucket-name>",
    "ENDPOINT": "oss-cn-shanghai.aliyuncs.com",
    "REGION": "",
    "ACCESS_KEY_ID": "xxxxx",
    "SECRET_ACCESS_KEY": "xxxxxx"
  }
}
```

- For SAP HANA Cloud, Data Lake:

```
curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
  "name": "default",
  "type": "webhdfs",
  "pathPrefix": "<path prefix to be appended>",
  "data": {
    // e.g. https://c32727c8-4260-4c37-b97f-
    ede322dcfa8f.files.hdl.canary-eu10.hanacloud.ondemand.com
    "HDFS_NAMENODE": "https://<file-container-name>.files.hdl.canary-
    eu10.hanacloud.ondemand.com",
    "TLS_CERT": "-----BEGIN CERTIFICATE-----
    \nMIICmjCCAYIxxxxxxxR4wtC32bG066D+Jc8RhaIA==\n-----END
    CERTIFICATE-----\n",
    "TLS_KEY": "-----BEGIN PRIVATE KEY-----
    \nMIIEvQIBADANBgkqxxxxxxxxxxxxxxxxor+rtZHhzhEfX5dYLC5Pww=\n-----END PRIVATE
    KEY-----\n",
    "HEADERS": "{\"x-sap-filecontainer\": \"<file-container-name>\",
    \"Content-Type\": \"application/octet-stream\"}"
  }
}
```

- For Azure Blob Storage:

⚠ Restriction

For **executions** only, access keys are supported. SAS tokens are **not** supported.

For **deployments**, access keys and SAS tokens are supported.

```
curl --location --request POST "$AI_API_URL/v2/admin/objectStoreSecrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
  "name": "default",
  "type": "azure",
  "pathPrefix": "<path prefix to be appended>",
  "data": {
    "CONTAINER_URI": "https://account_name.blob.core.windows.net/
    container_name", //required
    "REGION": "<region name>", //optional
    "CLIENT_ID": "<azure client id>", //optional
    "CLIENT_SECRET": "<azure client secret>", //optional
    "STORAGE_ACCESS_KEY": "sas_token", //required
    "TENANT_ID": "azure tenant id", //optional
    "SUBSCRIPTION_ID": "subscription id", //optional
  }
}
```

```
}  
}
```

ⓘ Note

For input artifacts only

You can create multiple secrets using different values for `name`, but you must create a default first.

→ Tip

The `pathPrefix` is useful if you share the same bucket for different projects. You can set the name of your project folder to **my-ml-project1**, for example. All data is then stored in that folder.

ⓘ Note

If the `AI-Resource-Group` header is not specified, the `<Resource Group>` is assigned the value `"default"` automatically.

⚠ Restriction

When using an SAP HANA Data Lake object store with output artifacts pointing to a directory, you cannot use `archive: none: {}` in your workflow templates to disable artifact archiving. For more information, see [Workflow Templates \[page 163\]](#).

7.4.2 Edit an Object Store Secret

Using Postman

1. Send a PATCH request to the endpoint
`{{apiurl}}/v2/admin/objectStoreSecrets/{{objectStoreName}}`
.
2. As the request body, select the **raw** radio button and enter your object store secret details.

ⓘ Note

For all storage types **except** Azure Blob Storage, all `<data>` fields are required. For Azure, required fields are specified.

- For Amazon S3:

```
{  
  "name": "default",  
  "type": "S3",  
  "bucket": "<S3 bucket name>",  
  "endpoint": "<S3 end point>",  
  "pathPrefix": "<A path prefix that follows the bucket name>",  
  "region": "<S3 region>",  
}
```

```

    "data": {
      "AWS_ACCESS_KEY_ID": "<AWS access key ID>",
      "AWS_SECRET_ACCESS_KEY": "<AWS secret access key>"
    }
  }
}

```

- For OSS (Alicloud Object Storage Service):

```

{
  "name": "default",
  "type": "oss",
  "pathPrefix": "<path prefix to be appended with bucketname>",
  "data": {
    "BUCKET": "<bucket-name>",
    "ENDPOINT": "oss-cn-shanghai.aliyuncs.com",
    "REGION": "",
    "ACCESS_KEY_ID": "xxxxx",
    "SECRET_ACCESS_KEY": "xxxxx"
  }
}

```

- For SAP HANA Cloud, Data Lake:

```

{
  "name": "default",
  "type": "webhdfs",
  "pathPrefix": "<path prefix to be appended>",
  "data": {
    // e.g. https://c32727c8-4260-4c37-b97f-ed322dcfa8f.files.hdl.canary-
    eu10.hanacloud.ondemand.com
    "HDFS_NAMENODE": "https://<file-container-name>.files.hdl.canary-
    eu10.hanacloud.ondemand.com",
    "TLS_CERT": "-----BEGIN CERTIFICATE-----
    \nMIICmJCCAYIxxxxxxR4wtC32bG066D+Jc8RhaIA==\n-----END
    CERTIFICATE-----\n",
    "TLS_KEY": "-----BEGIN PRIVATE KEY-----
    \nMIIEvQIBADANBgkqxxxxxxxxxxxxxxxxor+rtZHhHzEfX5dYLCs5Pww=\n-----END PRIVATE
    KEY-----\n",
    "HEADERS": "{\"x-sap-filecontainer\": \"<file-container-name>\",
    \"Content-Type\": \"application/octet-stream\"}"
  }
}

```

- For Azure Blob Storage:

```

{
  "name": "default",
  "type": "azure",
  "pathPrefix": "<path prefix to be appended>",
  "data": {
    "CONTAINER_URI": "https://account_name.blob.core.windows.net/
    container_name", //required
    "REGION": "<region name>", //optional
    "CLIENT_ID": "<azure client id>", //optional
    "CLIENT_SECRET": "<azure client secret>", //optional
    "STORAGE_ACCESS_KEY": "sas_token", //required
    "TENANT_ID": "azure tenant id", //optional
    "SUBSCRIPTION_ID": "subscription id", //optional
  }
}

```

3. Send the request.

Using curl

1. Register your object store secret details using the endpoint `$AI_API_URL/v2/admin/objectStoreSecrets/{{objectStoreName}}`.

Note

For all storage types **except** Azure Blob Storage, all `<data>` fields are required. For Azure, required fields are specified.

- For Amazon S3:

```
curl --location --request PATCH "$AI_API_URL/v2/admin/objectStoreSecrets/
{{objectStoreName}}" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
    "name": "default",
    "type": "S3",
    "bucket": "<S3 bucket name>",
    "endpoint": "<S3 end point>",
    "pathPrefix": "<A path prefix that follows the bucket name>",
    "region": "<S3 region>",
    "data": {
        "AWS_ACCESS_KEY_ID": "<AWS access key ID>",
        "AWS_SECRET_ACCESS_KEY": "<AWS secret access key>"
    }
}'
```

- For OSS (Alicloud Object Storage Service):

```
curl --location --request PATCH "$AI_API_URL/v2/admin/objectStoreSecrets/
{{objectStoreName}}" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
    "name": "default",
    "type": "oss",
    "pathPrefix": "<path prefix to be appended with bucketname>",
    "data": {
        "BUCKET": "<bucket-name>",
        "ENDPOINT": "oss-cn-shanghai.aliyuncs.com",
        "REGION": "",
        "ACCESS_KEY_ID": "xxxxx",
        "SECRET_ACCESS_KEY": "xxxxxx"
    }
}'
```

- For SAP HANA Cloud, Data Lake:

```
curl --location --request PATCH "$AI_API_URL/v2/admin/objectStoreSecrets/
{{objectStoreName}}" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
    "name": "default",
    "type": "webhdfs",
    "pathPrefix": "<path prefix to be appended>",
    "data": {
        // e.g. https://c32727c8-4260-4c37-b97f-
        ede322dcfa8f.files.hdl.canary-eu10.hanacloud.ondemand.com
    }
}'
```

```

        "HDFS_NAMENODE": "https://<file-container-name>.files.hdl.canary-
eu10.hanacloud.ondemand.com",
        "TLS_CERT": "-----BEGIN CERTIFICATE-----
\nMIICmJCCAYIxxxxxxxR4wtC32bG066D+Jc8RhaIA==\n-----END
CERTIFICATE-----\n",
        "TLS_KEY": "-----BEGIN PRIVATE KEY-----
\nMIIEvQIBADANBgkqxxxxxxxxnor+rtZHhzhEfX5dYLCs5Pww=\n-----END PRIVATE
KEY-----\n",
        "HEADERS": {"x-sap-filecontainer": "\"<file-container-name>\",
\"Content-Type\": \"application/octet-stream\""}
    }
}

```

- For Azure Blob Storage:

```

curl --location --request PATCH "$AI_API_URL/v2/admin/objectStoreSecrets/
{{objectStoreName}}" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: <Resource group>' \
--data-raw '{
    "name": "default",
    "type": "azure",
    "pathPrefix": "<path prefix to be appended>",
    "data": {
        "CONTAINER_URI": "https://account_name.blob.core.windows.net/
container_name", //required
        "REGION": "<region name>", //optional
        "CLIENT_ID": "<azure client id>", //optional
        "CLIENT_SECRET": "<azure client secret>", //optional
        "STORAGE_ACCESS_KEY": "sas_token", //required
        "TENANT_ID": "azure tenant id", //optional
        "SUBSCRIPTION_ID": "subscription id", //optional
    }
}

```

📌 Note

For input artifacts only

You can create multiple secrets using different values for name, but you must create a default first.

→ Tip

The `pathPrefix` is useful if you share the same bucket for different projects. You can set the name of your project folder to **my-ml-project1**, for example. All data will then be stored in that folder.

📌 Note

If the `AI-Resource-Group` header is not specified, the `<Resource Group>` is assigned the value `"default"` automatically.

7.4.3 Delete an Object Store Secret

Using Postman

Send a DELETE request to the endpoint

`{{apiurl}}/v2/admin/objectStoreSecrets/{{objectStoreName}}`

Using curl

```
curl --location --request DELETE "$AI_API_URL/v2/admin/objectStoreSecrets/{{objectStoreName}}" \
```

7.5 Manage Docker Registry Secrets

7.5.1 Register Your Docker Registry Secret

Docker packages and runs applications in remote containers. Connect SAP AI Core to a Docker repository and manage access using a Docker registry secret.

Prerequisites

You have completed the Initial Setup. For more information, see [Initial Setup \[page 46\]](#).

Context

Your Docker credentials are managed using secrets. Secrets allow and control connections across directories and tools without compromising your credentials.

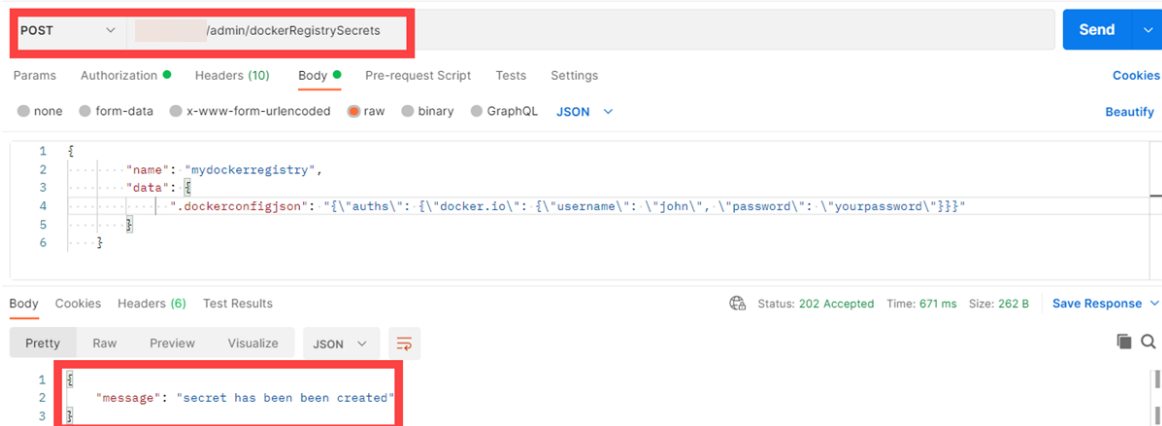
Your Docker registry secret lets you authorize SAP AI Core to pull your **private** Docker images from your Docker repository. You specify the name of the secret in your workflows to authenticate the Docker image pull. For more information, see [Workflow Templates \[page 163\]](#) and [Serving Templates \[page 213\]](#).

Using Postman

1. Send a POST request to the endpoint `{{apiurl}}/v2/admin/dockerRegistrySecrets`
2. As the request body, select the **raw** radio button and enter the following:

```
{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\n\"auths\":{\n\"your.private.registry\":\n{\n\"username\":\n\"john\", \"password\":\n\"docker-accesstoken-or-password\"}}}"
  }
}
```

- **name:** Set the name of your Docker registry secret. This is your choice of identifier for your secret. In the example, the name is "mydockerregistry".
 - **data:** Enter a JSON string that represents your Docker registry secret.
3. Send the request:



4. After your Docker registry secret has been created, reference it in your template as an image pull secret.

Source Code

```
spec:
  imagePullSecrets:
    - name: <Name of your Docker registry secret>
```

⚠ Restriction

The maximum number of Docker registry secrets is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some Docker registry secrets. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Docker Registry Secret \[page 91\]](#).

Using curl

Submit a POST request to the endpoint `{{apiurl}}/v2/admin/dockerRegistrySecrets`, Include the following parameters in your request body:

- **name**: Set the name of your Docker registry secret.
- **data**: Enter a JSON string that represents your Docker registry secret.

Sample Code

```
{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\"auths\":{\"your.private.registry\":
    {\"username\":\"john\", \"password\":\"docker-accesstoken-or-password\"}}}"
  }
}
```

Note

If you are using a public Docker registry from <http://hub.docker.com>, you must provide your Docker URL in the format `https://index.docker.io`, in the `<"auths">` variable input.

1. Submit a POST request to the endpoint `{{apiurl}}/v2/admin/dockerRegistrySecrets` and include the name of your Docker registry secret along with the credentials for your repository. For example

Sample Code

```
$ curl --location --request POST "$AI_API_URL/v2/admin/
dockerRegistrySecrets" --header "Authorization: Bearer $TOKEN" --header
'Content-Type: application/json' --data-raw '{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\"auths\": {\"my.docker.repositories.io\":
    {\"username\":\"$USERNAME\", \"password\":\"$PWD\"}}}"
  }
}'
```

2. After your Docker registry secret has been created, reference it in your template as an image pull secret. For example

Source Code

```
spec:
  imagePullSecrets:
    - name: <Name of your Docker registry secret>
```

Restriction

The maximum number of Docker registry secrets is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some Docker registry secrets. Alternatively, raise a ticket to increase your quota.

For more information, see [Delete a Docker Registry Secret \[page 91\]](#).

7.5.2 Edit a Docker Registry Secret

Docker packages and runs applications in remote containers. Connect SAP AI Core to a Docker repository and manage access using a Docker registry secret.

Your Docker credentials are managed using secrets. Secrets allow and control connections across directories and tools without compromising your credentials.

Your Docker registry secret lets you authorize SAP AI Core to pull your **private** Docker images from your Docker repository. You specify the name of the secret in your workflows to authenticate the Docker image pull. For more information, see [Workflow Templates \[page 163\]](#) and [Serving Templates \[page 213\]](#).

Using Postman

1. Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}`
2. As the request body, select the [raw](#) radio button and enter the following:

```
{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\"auths\":{\"your.private.registry\":
    {\"username\":\"john\", \"password\":\"docker-accesstoken-or-password\"}}}"
  }
}
```

- **name:** Set the name of your Docker registry secret.
 - **data:** Enter a JSON string that represents your Docker registry secret.
3. Send the request:

Using curl

Submit a PATCH request to the endpoint `$AI_API_URL/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}`. Include the following parameters in your request body:

- **name:** Set the name of your Docker registry secret.
- **data:** Enter a JSON string that represents your Docker registry secret.

Sample Code

```
{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\"auths\":{\"your.private.registry\":
    {\"username\":\"john\", \"password\":\"docker-accesstoken-or-password\"}}}"
  }
}
```

```
}
```

Note

If you are using a public Docker registry from <http://hub.docker.com>, you must provide your Docker URL in the format `https://index.docker.io`, in the `<"auths">` variable input.

1. Submit a PATCH request to the endpoint `$AI_API_URL/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}` and include the name of your Docker registry secret along with the credentials for your repository. For example

Sample Code

```
$ curl --location --request PATCH "$AI_API_URL/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}" --header "Authorization: Bearer $TOKEN" --header 'Content-Type: application/json' --data-raw '{
  "name": "mydockerregistry",
  "data": {
    ".dockerconfigjson": "{\"auths\": {\"my.docker.repositories.io\": {\"username\": \"$USERNAME\", \"password\": \"$PWD\"}}}"
  }
}
```

7.5.3 Delete a Docker Registry Secret

Deleting a docker registry secret removes access to the docker registry.

Using Postman

1. Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}`

Using curl

1. Submit a DELETE request to the endpoint `$AI_API_URL/v2/admin/dockerRegistrySecrets/{{dockerRegistryName}}`.

7.6 Manage Generic Secrets

7.6.1 Create a Generic Secret

A generic secret gives SAP AI Core authorization to utilize your resource group without exposing your credentials.

Generic Secrets are used in addition to store sensitive information when system secrets are not applicable, for example in integration use cases where SAP AI Core is an orchestration layer.

SAP AI Core lets you optionally use generic secrets at two levels: at the main-tenant scope or on a resource-group level.

Generic secrets are different to system secrets (such as object store, Docker registry, and so on) and can be used to store sensitive information, either for the main tenant or for each resource group via an API. The latter can be attached to containers in executions or deployments as environment variables or volume mounts.

To create a generic secret in a resource group, send a POST request to the endpoint `{{apiurl}}/v2/admin/secrets`. Note that the API expects sensitive data to be Base64-encoded. You can easily encode your data in Base64 format using the following command on Linux or MacOS: `echo -n 'my-sensitive-data' | base64`

Prerequisites

You have completed the Initial Setup. For more information, see [Initial Setup \[page 46\]](#).

Using Postman

1. Send a POST request and enter the URL `{{apiurl}}/v2/admin/secrets`.
2. As the request body, select the [raw](#) radio button and enter your credentials in JSON format.:

```
{
  "name": "MY_GENERIC_SECRET",
  "data": {
    "some-credential": "bXktc2VjcmV0LWNyZWRLbnRpYWw=",
    "other-credentials": "bXktc2VjcmV0LW90aGVyLWNyZWRLbnRpYWw="
  }
}
```

Note

The secret name is written without hyphens to make it simple to consume as a Unix environment variable later. It is written in capitals as is convention.

- **name:** Set the name of your generic secret.
- **data:** Enter a JSON string that represents your generic secret.

- Specify the scope of the request via the header **AI - Tenant - Scope** and **AI - Resource - Group**:
 - AI - Tenant - Scope** : **true**. The operation will be performed at the main tenant level.
 - AI - Resource - Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.

In this example, we are using the resource-group level.

POST /admin/secrets

Params Authorization Headers (11) Body Pre-request Script Tests Settings

Headers 9 hidden

KEY	VALUE
☒ AI-Resource-Group	default

- Send the request.

POST /admin/secrets

Send

Params Authorization Headers (11) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "name": "my-generic-secret",
3   "data": {
4     "some-credential": "bXktc2VjcmluZmlyZWRLbnRpbWw=",
5     "other-credentials": "bXktc2VjcmluZmlyZWRLbnRpbWw="
6   }
7 }

```

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 167 ms Size: 244 B Save Response

Pretty Raw Preview Visualize JSON

```

1 {
2   "message": "secret has been created",
3   "name": "my-generic-secret"
4 }

```

Using curl

Submit a POST request to the endpoint `v2/admin/secrets` and include the name of your generic secret and credentials. Specify the scope through the **AI - Tenant - Scope** and **AI - Resource - Group**:

- AI - Tenant - Scope** : **true**. The operation will be performed at the main tenant level.
- AI - Resource - Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.

```

curl --location --request POST "$AI_API_URL/v2/admin/secrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: default' \
--data-raw '{
  "name": "MY_GENERIC_SECRET",
  "data": {
    "some-credential": "bXktc2Vuc2l0aXZlLWRhdGE="
  }
}'

```

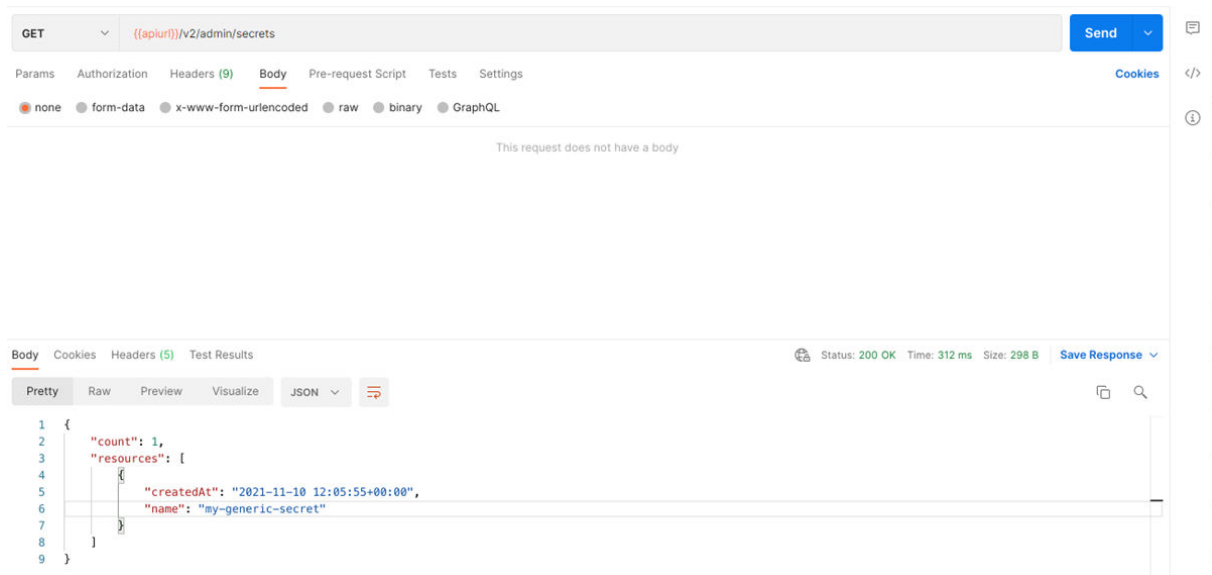
Note

The secret name is written without hyphens to make it simple to consume as a Unix environment variable later. It is written in capitals as is convention.

7.6.2 List All Generic Secrets

Using Postman

1. Send a GET request to the endpoint `{{apiurl}}/v2/admin/secrets`.
2. As the request body, select the *none* radio button.
3. Specify the scope of the request via the header `AI-Tenant-Scope` or `AI-Resource-Group`:
 - **AI-Tenant-Scope** : **true**. The operation will be performed at the main tenant level.
 - **AI-Resource-Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.
4. Send the request.



Using curl

Submit a GET request to the endpoint `/v2/admin/secrets`, and include the resource-group scope:

- **AI-Tenant-Scope** : **true**. The operation will be performed at the main tenant level.

- **AI-Resource-Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.

```
curl --location --request GET "$AI_API_URL/v2/admin/secrets" \
--header "Authorization: Bearer $TOKEN" \
--header 'AI-Resource-Group: default'
```

The response includes a list of generic secrets, their name, and their creation timestamp. No sensitive information is revealed in the response.

7.6.3 Update a Generic Secret

To update a generic secret, use the PATCH endpoint as shown below. The PATCH operation replaces the secret with the data provided. This can be used for rotating secret credentials.

Using Postman

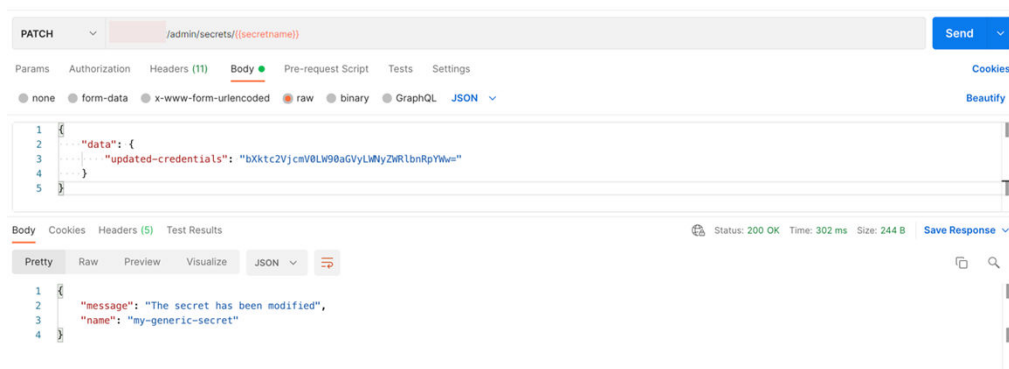
1. Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/secrets/{{secretName}}`
2. As the request body, select the **raw** radio button and enter the following code:

Source Code

```
{
  "data": {
    "updated-credentials": "bXktc2VjcmV0LW90aGVyLWNyZWRLbnRpYWw="
  }
}
```

and specify the scope via the

3. Specify the scope of the request via the header **AI-Tenant-Scope** and specify the scope via the or **AI-Resource-Group**:
 - **AI-Tenant-Scope** : **true**. The operation will be performed at the main tenant level.
 - **AI-Resource-Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.
4. Send the request.



Using curl

Submit a PATCH request to the endpoint `/v2/admin/secrets/"$SECRET_NAME"AI-Tenant-Scope` or `AI-Resource-Group` header.

```

curl --location --request PATCH "$AI_API_URL/v2/admin/secrets/$SECRET_NAME" \
--header "Authorization: Bearer $TOKEN" \
--header 'Content-Type: application/json' \
--header 'AI-Resource-Group: default' \
--data-raw '{
  "data": {
    "some-credential": "bXktc2Vuc2l0aXZlLWRhdGE="
  }
}'

```

7.6.4 Delete a Generic Secret

To get a secret name, see [List All Generic Secrets \[page 94\]](#).

Using Postman

1. Send a DELETE request to the endpoint `{{apiurl}}/v2/admin/secrets/{{secretName}}`
2. As the request body, select the *none* radio button.
3. Specify the scope of the request via the header `AI-Tenant-Scope` or `AI-Resource-Group`:
 - **AI-Tenant-Scope** : **true**. The operation will be performed at the main tenant level.
 - **AI-Resource-Group** : **<resource-group-name>**. The operation will be performed at the resource-group level.
4. Send the request.

Using curl

Submit a DELETE request to the endpoint `/v2/admin/secrets/"$SECRET_NAME"`. Here we use the resource-group scope. For main-tenant scope level, replace the last header with `AI-Tenant-Scope: true`.

```
curl --location --request DELETE "$AI_API_URL/v2/admin/secrets/$SECRET_NAME$AI_API_URL/v2/admin/secrets/$SECRET_NAME" \
--header "Authorization: Bearer $TOKEN" \
--header 'AI-Resource-Group: default'
```

7.6.5 Consume Generic Secrets in Executions or Deployments

Generic secrets at resource-group level can be attached to containers in executions or deployments. They can either be mounted as a volume or attached as an environment variable. The following examples illustrate how to consume a generic secret in a container by declaring it in the template. Note that only generic secrets can be attached to containers in this way. System secrets cannot be consumed in a template.

Consume a Generic Secret as an Environment Variable

Generic secrets can be attached to containers using either `envFrom.secretRef` or `env.valueFrom.secretKeyRef`, as shown below.

- Using `envFrom.secretRef`:

```
spec:
  containers:
  - name: my-kserve-container
    image: centaur
    envFrom:
    - secretRef:
        name: MY_GENERIC_SECRET
```

If your secret contains invalid characters, such as hyphens (-), this method will result in error. You will need to map your secret to a valid variable name using `env.valueFrom.secretKeyRef`.

- Using `env.valueFrom.secretKeyRef`:

```
spec:
  containers:
  - name: kserve-container
    image: centaur
    env:
    - name: MY_GENERIC_SECRET
      valueFrom:
        secretKeyRef:
          name: my-generic-secret
          key: some-credential
```

Consume a Generic Secret as a Volume Mount

Generic secrets can also be mounted to containers as volumes, as shown below.

```
spec:
  containers:
    - name: kserve-container
      image: centaur
      volumeMounts:
        - name: my-generic-secret
          mountPath: "/etc/my-generic-secret"
          readOnly: true
  volumes:
    - name: my-generic-secret
      secret:
        secretName: my-generic-secret
```

Additional Information

Secret names can be included as parameters in the templates and supplied via AI API configurations, as shown below.

```
envFrom:
- secretRef:
  name: "{{inputs.parameters.secretName}}"
```

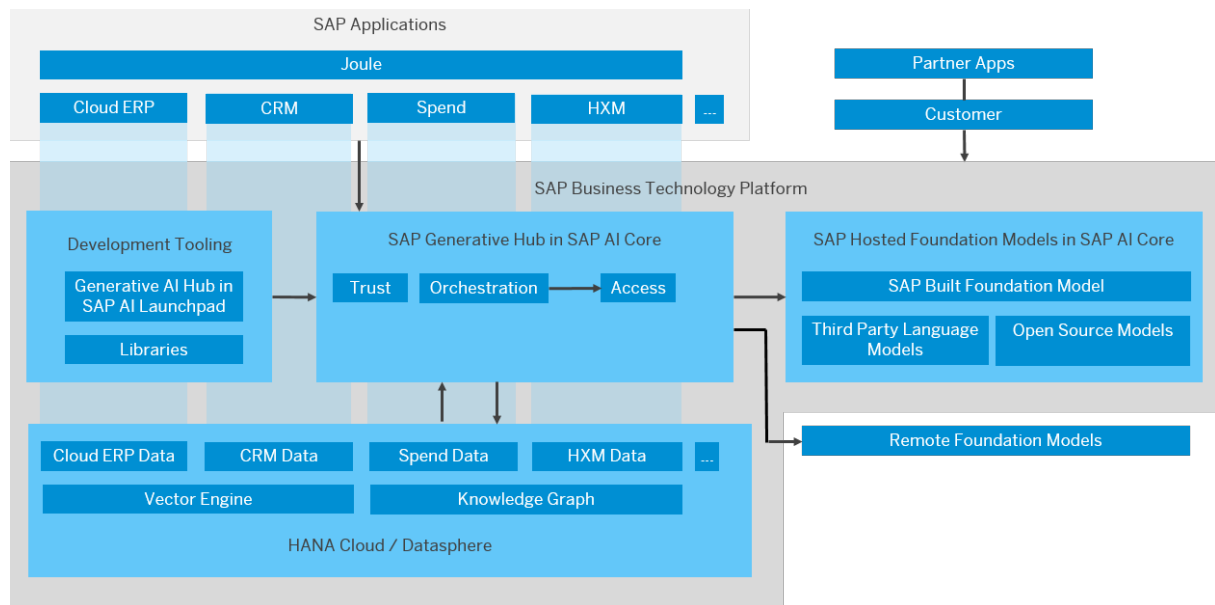
8 Generative AI Hub in SAP AI Core

The generative AI hub incorporates generative AI into your AI activities in SAP AI Core and SAP AI Launchpad.

LLMs are self-supervised, deep learning models that have been trained on vast amounts of unlabeled data. They leverage AI technology and industrial-scale computational resources to learn complex language patterns and semantic knowledge bases for natural language processing (NLP) tasks. They parse input, such as prompts, and by predicting a target word, can return contextually relevant responses written in natural language. A single LLM can perform multiple NLP tasks by using different input formats and output modes.

LLMs are general models but can be fine-tuned with additional embeddings for specialized or domain-specific use cases.

SAP AI Core and the generative AI hub help you to integrate LLMs and AI into new business processes in a cost-efficient manner.



Generative AI Hub Architecture Overview

8.1 Models and Scenarios in the Generative AI Hub

The generative AI hub is available in SAP AI Core only with the Extended service plan. For information about how to add a service plan or update an existing service plan, see [Add a Service Plan in SAP AI Core \[page 53\]](#) and [Update a Service Plan in SAP AI Core \[page 44\]](#).

Scenarios

Access to generative AI models is provided under the global AI scenario **foundation-models**, which is managed by SAP AI Core. Individual models are provided as executables in the form of serving templates, and accessed by choosing the corresponding template for the desired model.

The following scenarios are available:

Global Scenario	Executable ID	Description
foundation-models	azure-openai	The Azure OpenAI Service provides REST API access to OpenAI's LLMs.
foundation-models	aicore-opensource	Open source models hosted and accessed through SAP AI Core.
foundation-models	gcp-vertexai	GCP Vertex AI provides access to PaLM 2 and Gemini models from Google.
foundation-models	aws-bedrock	AWS Bedrock provides access to Foundation models from Anthropic, Amazon and other providers.

Access to orchestration of generative AI models is provided under the global AI scenario **orchestration**, which is managed by SAP AI Core.

Models

You can use the following models with the generative AI hub:

Note

Models indicated and their associated values are subject to change.

For more information about these models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note [3437766](#).

For more information about models and their parameters, see the documentation from the model provider.

Executable ID	Model Name	Model Version	More Information
azure-openai	gpt-35-turbo	0613 1106	Azure Chat Completions
azure-openai	gpt-35-turbo-16k	0613	Azure Chat Completions
azure-openai	gpt-4	0613	Azure Chat Completions
azure-openai	gpt-4-32k	0613	Azure Chat Completions
azure-openai	text-embedding-ada-002	2	Embeddings

Executable ID	Model Name	Model Version	More Information
azure-openai	text-embedding-3-small	N/A	Embeddings ➡
azure-openai	text-embedding-3-large	N/A	Embeddings ➡
azure-openai	gpt-4o	2024-05-13	Azure Chat Completions ➡
azure-openai	gpt-4	turbo-2024-04-09	Azure Chat Completions ➡
aicore-opensource	tiiuae--falcon-40b-instruct	N/A	Tiiuae Falcon 40b Instruct ➡
aicore-opensource	mistralai--mixtral-8x7b-instruct-v01	N/A	Mistral AI ➡
aicore-opensource	meta--llama3-70b-instruct	N/A	Meta ➡
gcp-vertexai	text-bison	002	GCP Vertex AI ➡
gcp-vertexai	chat-bison	002	GCP Vertex AI ➡
gcp-vertexai	textembedding-gecko	003	GCP Vertex AI ➡
gcp-vertexai	textembedding-gecko-multilingual	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.0-pro	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.5-pro	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.5-flash	001	GCP Vertex AI ➡
aws-bedrock	anthropic--claude-3-sonnet	v1	AWS Bedrock ➡
aws-bedrock	anthropic--claude-3-haiku	v1	AWS Bedrock ➡
aws-bedrock	anthropic--claude-3-opus	v1	AWS Bedrock ➡
aws-bedrock	amazon--titan-embed-text	1.2	AWS Bedrock ➡
aws-bedrock	amazon--titan-text-lite	1	AWS Bedrock ➡
aws-bedrock	amazon--titan-text-express	1	AWS Bedrock ➡

Models from Azure OpenAI are accessed through a private instance of the `chat-completions` API. The access points are not publicly accessible, and are accessed through SAP AI Core. For more information, see [Azure OpenAI Chat Completions API](#) ➡.

Open Source models are hosted by SAP AI Core and can be accessed via OpenAI compatible API schema.

8.2 Create a Deployment for a Generative AI Model

You make a generative AI model available for use by creating a deployment. You can do so one time for each model and model version, and for each resource group that you want to use with generative AI hub. The deployment URL that is generated can be reused.

Prerequisites

- You have an SAP AI Core service instance and service key. For more information, see [SAP AI Core Initial Setup Documentation](#).
- You're using the extended service plan. For more information, see [Service Plans \[page 36\]](#) and [Update a Service Plan \[page 44\]](#).
- You have at least one orchestration compatible deployment for a generative AI model running. For more information, see [Models and Scenarios in the Generative AI Hub](#) and [Create a Deployment for a Generative AI Model](#).

Context

You make a model available for use by creating a deployment. You can do so one time for each model and model version. The model deployment includes the `modelName` and `version` of the model you want to access. After the deployment is complete, you have a `deploymentUrl`, which can be used across your organization to access the model version.

Using the API

Procedure

1. Decide which LLM you want to deploy and note the following information:
 - Executable ID
 - Model name
 - Model version

Note

- Instead of specifying a model version, using “latest” will use the latest version of the model available in SAP AI Core.

- Where model version is not listed, it is not applicable.

Executable ID	Model Name	Model Version	More Information
azure-openai	gpt-35-turbo	0613 1106	Azure Chat Completions ➡
azure-openai	gpt-35-turbo-16k	0613	Azure Chat Completions ➡
azure-openai	gpt-4	0613	Azure Chat Completions ➡
azure-openai	gpt-4-32k	0613	Azure Chat Completions ➡
azure-openai	text-embedding-ada-002	2	Embeddings ➡
azure-openai	text-embedding-3-small	N/A	Embeddings ➡
azure-openai	text-embedding-3-large	N/A	Embeddings ➡
azure-openai	gpt-4o	2024-05-13	Azure Chat Completions ➡
azure-openai	gpt-4	turbo-2024-04-09	Azure Chat Completions ➡
aicore-opensource	tiiuae--falcon-40b-instruct	N/A	Tiiuae Falcon 40b Instruct ➡
aicore-opensource	mistralai--mixtral-8x7b-instruct-v01	N/A	Mistral AI ➡
aicore-opensource	meta--llama3-70b-instruct	N/A	Meta ➡
gcp-vertexai	text-bison	002	GCP Vertex AI ➡
gcp-vertexai	chat-bison	002	GCP Vertex AI ➡
gcp-vertexai	textembedding-gecko	003	GCP Vertex AI ➡
gcp-vertexai	textembedding-gecko-multilingual	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.0-pro	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.5-pro	001	GCP Vertex AI ➡
gcp-vertexai	gemini-1.5-flash	001	GCP Vertex AI ➡
aws-bedrock	anthropic--claude-3-sonnet	v1	AWS Bedrock ➡
aws-bedrock	anthropic--claude-3-haiku	v1	AWS Bedrock ➡

Executable ID	Model Name	Model Version	More Information
aws-bedrock	anthropic--claude-3-opus	v1	AWS Bedrock ➡
aws-bedrock	amazon--titan-embed-text	1.2	AWS Bedrock ➡
aws-bedrock	amazon--titan-text-lite	1	AWS Bedrock ➡
aws-bedrock	amazon--titan-text-express	1	AWS Bedrock ➡

- Check that you have access to the scenario containing generative AI by sending a GET request to `{{apiurl}}/v2/lm/scenarios`.

Set the *Authorization* header with **Bearer \$TOKEN** and set your resource group. For more information, see [Create a Resource Group \[page 75\]](#).

Note

You must use the same resource group for all of your generative AI activities. To use a different resource group, these steps must be repeated for each resource group. For more information, see [Manage Resource Groups \[page 74\]](#).

```

1 {
2   "count": 1,
3   "resources": [
4     {
5       "createdAt": "2023-09-23T08:19:06+00:00",
6       "description": "AI Core Global Scenario for LLM Access",
7       "id": "foundation-models",
8       "labels": [
9         {
10          "key": "scenarios.ai.sap.com/llm",
11          "value": "true"
12        }
13      ],
14       "modifiedAt": "2023-09-23T08:19:06+00:00",
15       "name": "foundation-models"
16     }
17   ]
18 }

```

The scenarios listed contain a scenario with the id `foundation-models`.

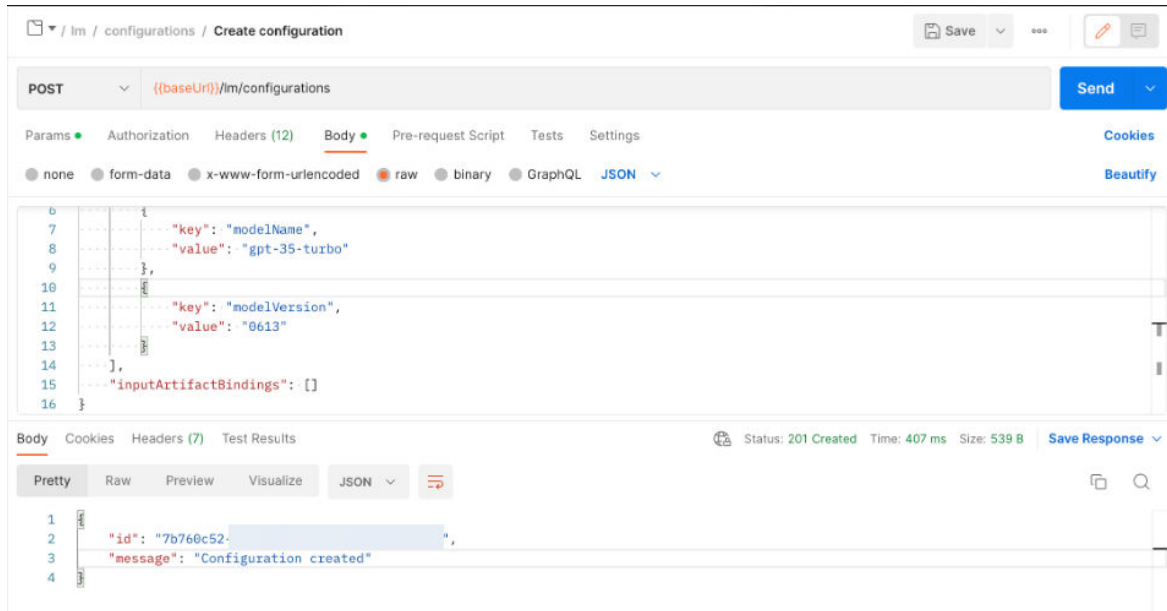
- Create a configuration by sending a POST request to the endpoint `{{apiurl}}/v2/lm/configurations`.

Include details of the model to which you want to provide access by passing in the following parameters:

- `name` is your free choice of identifier.
- `executableId`, `modelName`, and `modelVersion` are provided in the table above.
- `scenarioId` must be `foundation-models`.
- `versionId` is your own version reference.

Sample Code

```
{
  "name": "yourNameChoice",
  "executableId": "azure-openai",
  "scenarioId": "foundation-models",
  "versionId": "0.0.1",
  "parameterBindings": [
    {
      "key": "modelName",
      "value": "gpt-35-turbo"
    },
    {
      "key": "modelVersion",
      "value": "0613"
    }
  ],
  "inputArtifactBindings": []
}
```



→ Tip

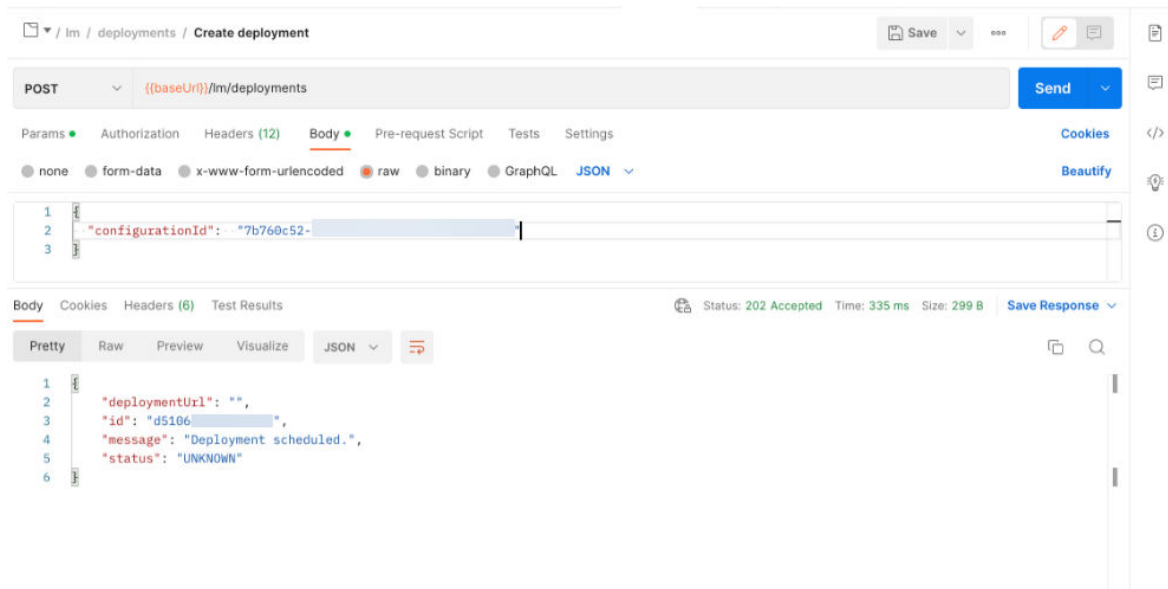
You can specify the value **latest** for the `modelVersion` to use the most recent model version available in SAP AI Core.

You receive a unique `configurationId` in the response.

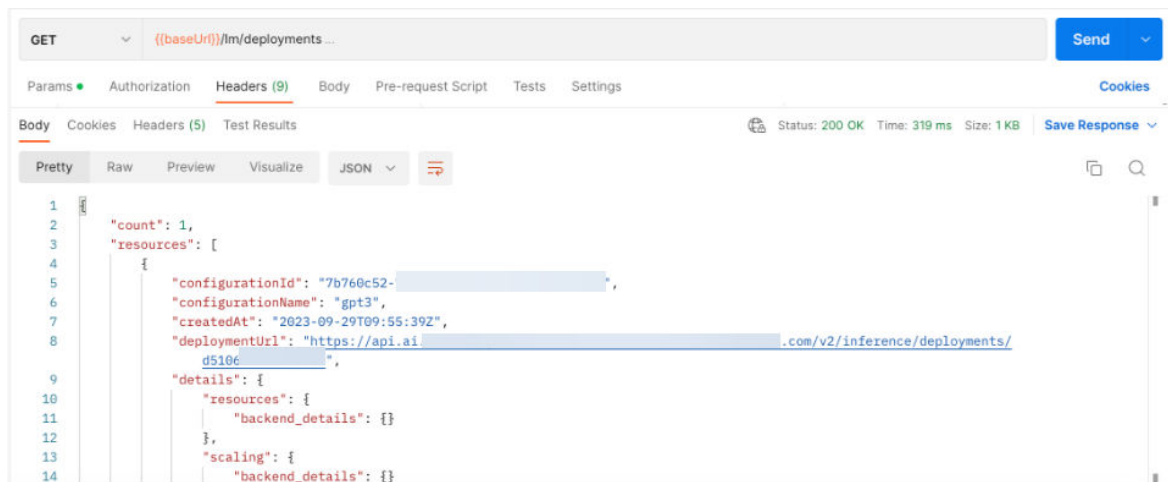
4. Create a deployment by sending a POST request to the endpoint `{{apiurl}}/v2/lm/deployments`. Include the `configurationId` from the previous step in your request.

Sample Code

```
{
  "configurationId": "yourConfigurationId"
}
```



- Retrieve the details of your deployment by sending a GET request to the endpoint `{{apiUrl}}/v2/lm/deployments`.



Next Steps

When the deployment is running, the model can be accessed using the `deploymentUrl` provided in the response. For more information, see [Consume Generative AI Models \[page 107\]](#).

Model Lifecycle

Model versions have deprecation dates. Where a model version is specified in a deployment, the deployment will stop working on the deprecation date of that model version.

Implement one of the following model upgrade options:

- **Auto Upgrade:** Create a new generative AI configuration and deployment or patch a deployment with a new configuration, specifying `modelVersion latest`. When a new `modelVersion` is supported by SAP AI Core, existing generative AI deployments will automatically use the latest version of the given model.
- **Manual Upgrade:** Create a new generative AI configuration with your chosen replacement `modelVersion` and use it to patch your deployment. This model version will be used in generative AI deployments irrespective of updates to the models supported by SAP AI Core.

Note

If `modelVersion` isn't specified, it will be `latest` by default. To upgrade manually, you **must** specify a `modelVersion`.

8.3 Consume Generative AI Models

The generative AI hub helps you to complete tasks like summarizing text, inferencing, and transforming content. To do so, you consume a generative AI model by sending a request to the model's endpoint.

Prerequisites

You have the deployment URL for your generative AI model. For more information, see [Create a Deployment for a Generative AI Model \[page 102\]](#).

Context

Ensure that you have the following headers set:

Header	Value
Authorization	Bearer \$TOKEN
AI-Resource-Group	The resource group used in the activation steps

Caution

SAP does not take any responsibility for quality of the content in the input to or output of the underlying generative AI models, including but not limited to, bias, hallucinations, or inaccuracies. The user is responsible for verifying the content.

Caution

Do not store sensitive data in prompts when using the generative AI hub. Sensitive data is any data that is not intended for public disclosure, including but not limited to confidential or personal information.

Example Payloads for Inferencing

The following examples show how you can consume various generative AI models using curl. For more information about prompts, see the tutorial [Prompt LLMs in the Generative AI Hub in SAP AI Core & Launchpad](#).

Open AI

For information about the supported API versions, see [Chat completions](#) and [Embeddings](#) in the Microsoft documentation.

GPT-4-32k | GPT-4 | GPT-3.5-Turbo-16k | GPT-3.5-Turbo

Text Input

```
curl --location '$DEPLOYMENT_URL/chat/completions?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "messages": [
    {
      "role": "user",
      "content": "sample input prompt"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}'
```

GPT-4o | GPT-4-Turbo

Image input

```
# Request
# ---
curl --location '$DEPLOYMENT_URL/chat/completions?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Describe this picture:"
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "https://path/images/image.png"
          }
        }
      ]
    }
  ]
}'
```

```
    ]
  },
  "max_tokens": 10
}'
```

text-embedding-ada-002 | text-embedding-3-small | text-embedding-3-large

```
curl --location '$DEPLOYMENT_URL/embeddings?api-version=2023-05-15' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "input": "sample input prompt"
}'
```

Falcon

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "model": "tiiuae--falcon-40b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "sample input prompt"
    }
  ],
  "max_tokens": 100
}'
```

Vertex AI

Gemini 1.0 Pro

```
curl --location '$DEPLOYMENT_URL/models/gemini-1.0-pro:generateContent' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "contents": [
    {
      "role": "user",
      "parts": { "text": "Hello!" }
    },
    {
      "role": "model",
      "parts": { "text": "Argh! What brings ye to my ship?" }
    },
    {
      "role": "user",
      "parts": { "text": "Wow! You are a real-life pirate!" }
    }
  ],
  "safety_settings": {
```

```

        "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
        "threshold": "BLOCK_LOW_AND_ABOVE"
    },
    "generation_config": {
        "temperature": 0.9,
        "topP": 1,
        "candidateCount": 1,
        "maxOutputTokens": 2048
    }
}'

```

Gemini 1.5 Pro

Text Input

```

curl --location '$DEPLOYMENT_URL/models/gemini-1.0-pro:generateContent' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer $TOKEN' \
--data '{
    "generation_config":{
        "maxOutputTokens":100
    },
    "contents": {
        "role": "user",
        "parts": {
            "text": "Give me a recipe for banana bread."
        }
    },
    "safety_settings": {
        "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
        "threshold": "BLOCK_LOW_AND_ABOVE"
    }
}'

```

Image input

```

curl --request POST --location "$DEPLOYMENT_URL/models/gemini-1.5-
pro:generateContent" \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer $AUTH_TOKEN' \
--data '{
    "contents": {
        "role": "user",
        "parts": [
            {
                "fileData": {
                    "mimeType": "image/png",
                    "fileUri": "filepath/images/scones.jpg"
                }
            },
            {
                "text": "Describe this picture."
            }
        ]
    }
}'

```

Gemini 1.5 Flash

Text Input

```

curl --request POST --location "$DEPLOYMENT_URL/models/gemini-1.5-
flash:generateContent" \

```

```
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "generation_config":{
    "maxOutputTokens":100
  },
  "contents": {
    "role": "user",
    "parts": {
      "text": "Give me a recipe for banana bread."
    }
  },
  "safety_settings": {
    "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
    "threshold": "BLOCK_LOW_AND_ABOVE"
  }
}'
```

Image input

```
curl --request POST --location "$DEPLOYMENT_URL/models/gemini-1.5-
flash:generateContent" \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "contents": {
    "role": "user",
    "parts": [
      {
        "fileData": {
          "mimeType": "image/png",
          "fileUri": "filepath/images/scones.jpg"
        }
      },
      {
        "text": "Describe this picture."
      }
    ]
  }
}'
```

Text Bison

```
curl --location '$DEPLOYMENT_URL/models/text-bison:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "instances": [
    {
      "prompt": "Sample prompt"
    }
  ],
  "parameters": {
    "temperature": 0.8,
    "maxOutputTokens": 50
  }
}'
```

Chat Bison

```
curl --location '$DEPLOYMENT_URL/models/chat-bison:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
```

```
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "instances": [
    {
      "context": "Your conversation context here",
      "messages": [
        {
          "author": "user",
          "content": "User message 1"
        },
        {
          "author": "assistant",
          "content": "Assistant response 1"
        },
        {
          "author": "user",
          "content": "User message 2"
        }
      ]
    }
  ],
  "parameters": {
    "temperature": 0.8
  }
}'
```

Textembedding Gecko

```
curl --location '$DEPLOYMENT_URL/models/textembedding-gecko:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "instances": [
    {
      "task_type": "RETRIEVAL_DOCUMENT",
      "title": "Document title",
      "content": "I would like embeddings for this text"
    }
  ]
}'
```

Textembedding Gecko Multilingual

```
curl --location '$DEPLOYMENT_URL/models/textembedding-gecko-
multilingual:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--data '{
  "instances": [
    {
      "task_type": "RETRIEVAL_DOCUMENT",
      "title": "Document title",
      "content": "I would like embeddings for this text"
    }
  ]
}'
```


Mistral AI

mistralai--mixtral-8x7b-instruct-v01

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "model": "mistralai--mixtral-8x7b-instruct-v01",
  "messages": [
    {
      "role": "user",
      "content": "Sample prompt"
    }
  ],
  "max_tokens": 100
}'
```

AWS Bedrock

Claude Sonet | Claude Opus | Claude Haiku

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "anthropic_version": "bedrock-2023-05-31",
  "max_tokens": 100,
  "messages": [
    {
      "role": "user",
      "content": "Hello, Claude"
    }
  ]
}'
```

Titan Text Express | Titan Text Lite

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "inputText": "Who am I?",
  "textGenerationConfig": {
    "maxTokenCount": 10,
    "stopSequences": [],
    "temperature": 0,
    "topP": 1
  }
}'
```

Titan Embed Text

```
curl --location '$DEPLOYMENT_URL/invoke' \
--header 'AI-Resource-Group: default' \
```

```
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "inputText": "Who im I?"
}'
```

Meta

LLama-3-70b

```
curl --location '$DEPLOYMENT_URL/chat/completions' \
--header 'AI-Resource-Group: default' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "model": "meta-llama3-70b-instruct",
  "messages": [
    {
      "role": "user",
      "content": "Sample prompt"
    }
  ],
  "max_tokens": 100
}'
```

If you want to remove a model, delete its deployment. For more information, see [Delete Deployments \[page 242\]](#).

8.3.1 Prompt Examples

Summarizing

You can provide the LLM with a text and ask for a summary of it.

Procedure

Send a POST request to the relevant endpoint.

Include your query in the body. Mark the text to be summarized with triple back ticks (``).

❖ Example

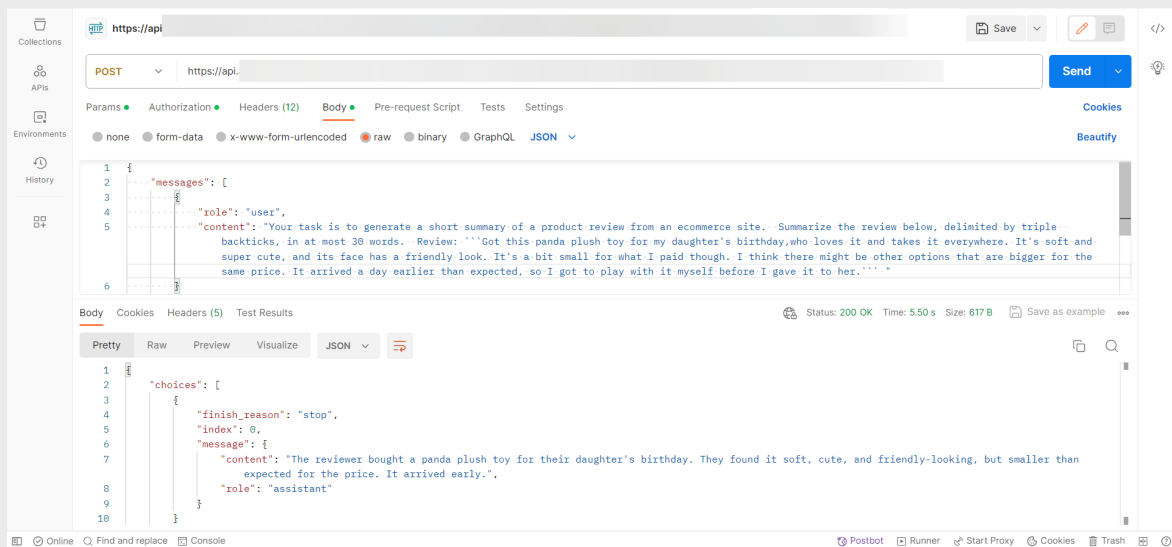
This example generates a summary of a product review. Summaries can include topics that aren't related to the main topic.

```
{
```

```

"messages": [
  {
    "role": "user",
    "content": "Your task is to generate a short summary of a product review from an ecommerce site. Summarize the review below, delimited by triple backticks, in at most 30 words. Review:```Got this panda plush toy for my daughter's birthday, who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.```"
  }
],
"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}

```



❁ Example

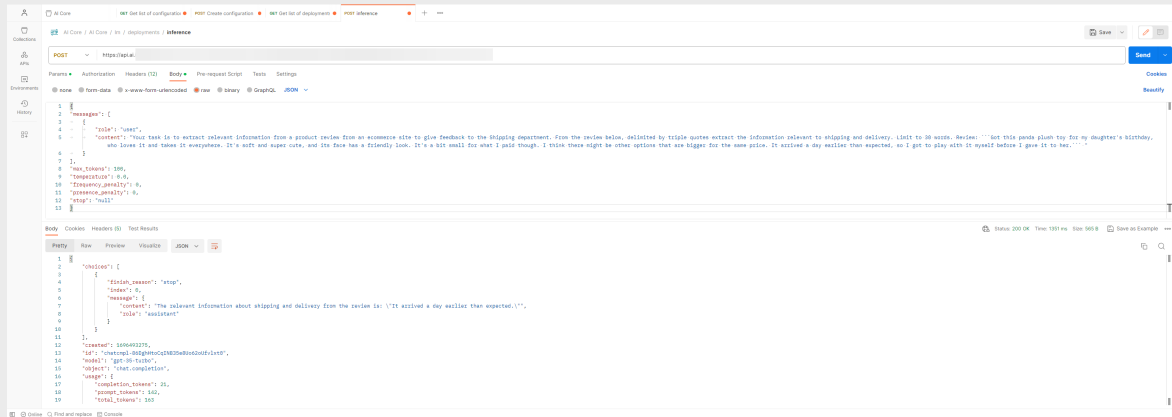
In this example, the prompt is similar but has been refined by adding the intended recipient of the feedback (the purchasing department) and the reason for requesting it (to determine the price of the product).

```

{
  "messages": [
    {
      "role": "user",
      "content": "Your task is to generate a short summary of a product review from an ecommerce site to give feedback to the pricing department, responsible for determining the price of the product. Summarize the review below, delimited by triple backticks, in at most 30 words, and focusing on any aspects that are relevant to the price and perceived value. Review:```Got this panda plush toy for my daughter's birthday,who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.```"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
}

```

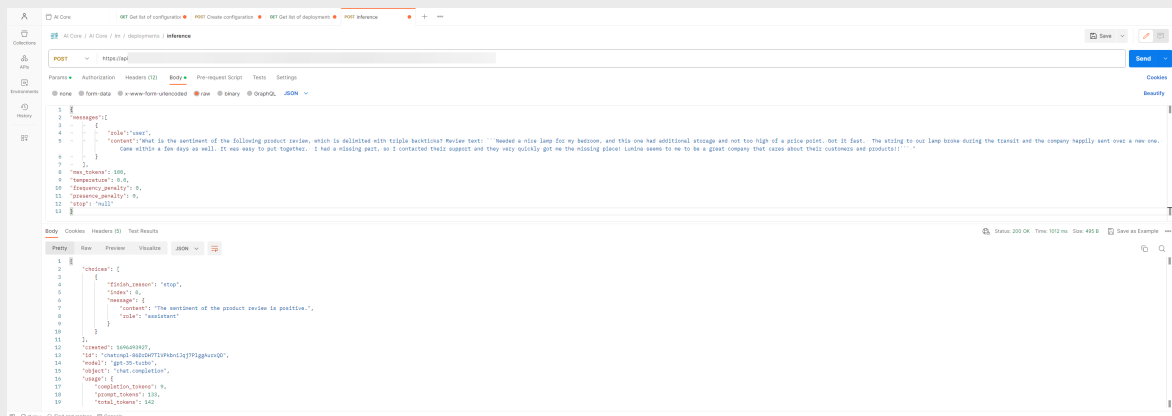
```
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}
```



❖ Example

This example uses “extract” instead of “summarize”.

```
{
  "messages": [
    {
      "role": "user",
      "content": "Your task is to extract relevant information from a product review from an ecommerce site to give feedback to the Shipping department. From the review below, delimited by triple backticks extract the information relevant to shipping and delivery. Limit to 30 words. Review: ``Got this panda plush toy for my daughter's birthday,who loves it and takes it everywhere. It's soft and super cute, and its face has a friendly look. It's a bit small for what I paid though. I think there might be other options that are bigger for the same price. It arrived a day earlier than expected, so I got to play with it myself before I gave it to her.``"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



Inferencing

Inferencing uses the information in a given text to draw a conclusion.

Procedure

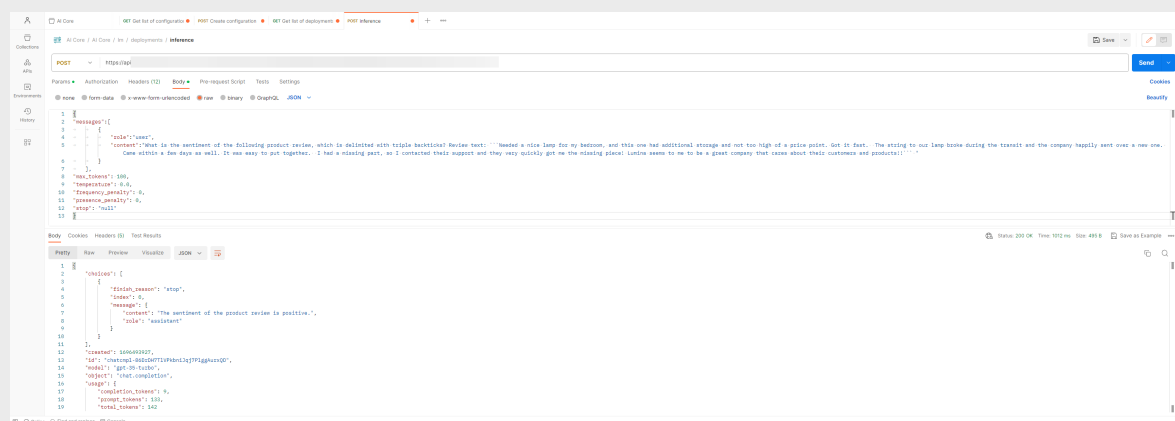
Send a POST request to the relevant endpoint.

Include your query in the body. Mark the text to be inferenced with triple back ticks (` ` `).

❖ Example

This example performs a sentiment analysis on a product review.

```
{
  "messages": [
    {
      "role": "user",
      "content": "What is the sentiment of the following product review, which is delimited with triple backticks? Review text: ``Needed a nice lamp for my bedroom, and this one had additional storage and not too high of a price point. Got it fast. The string to our lamp broke during the transit and the company happily sent over a new one. Came within a few days as well. It was easy to put together. I had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!``"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```

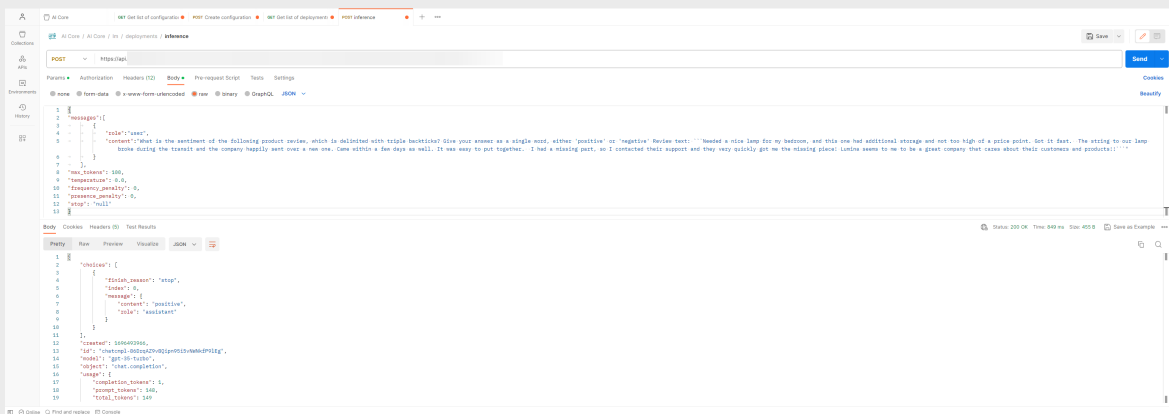


❖ Example

This example generates the sentiment as a one word response.

```
{
  "messages": [
```

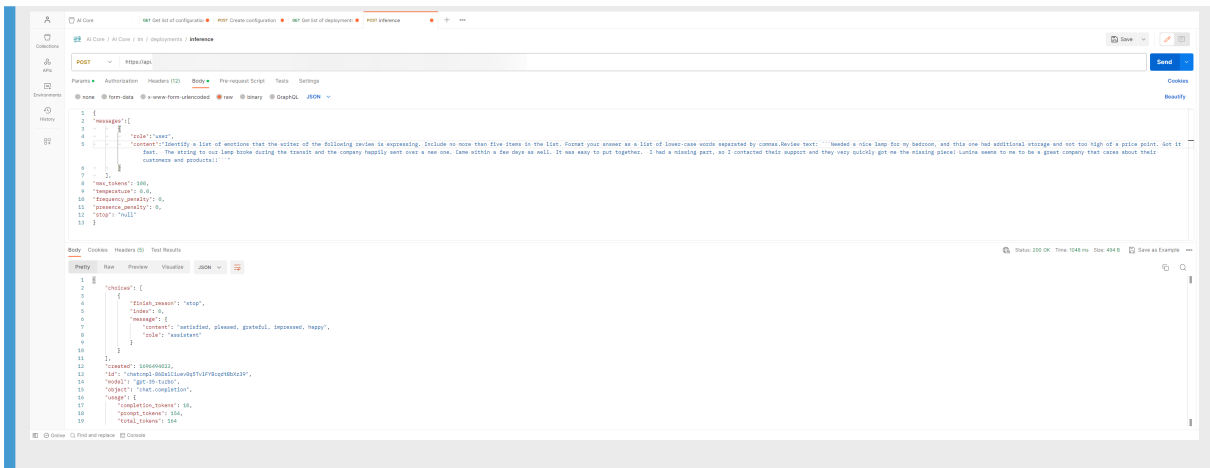
```
{
  "role": "user",
  "content": "What is the sentiment of the following product review, which is delimited with triple backticks? Give your answer as a single word, either 'positive' or 'negative' Review text: ```Needed a nice lamp for my bedroom, and this one had additional storage and not too high of a price point. Got it fast. The string to our lamp broke during the transit and the company happily sent over a new one. Came within a few days as well. It was easy to put together. I had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!```"
},
{
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



❖ Example

This example analyzes the emotions expressed in the review.

```
{
  "messages": [
    {
      "role": "user",
      "content": "Identify a list of emotions that the writer of the following review is expressing. Include no more than five items in the list. Format your answer as a list of lower-case words separated by commas. Review text: ```Needed a nice lamp for my bedroom, and this one had additional storage and not too high of a price point. Got it fast. The string to our lamp broke during the transit and the company happily sent over a new one. Came within a few days as well. It was easy to put together. I had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!```"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



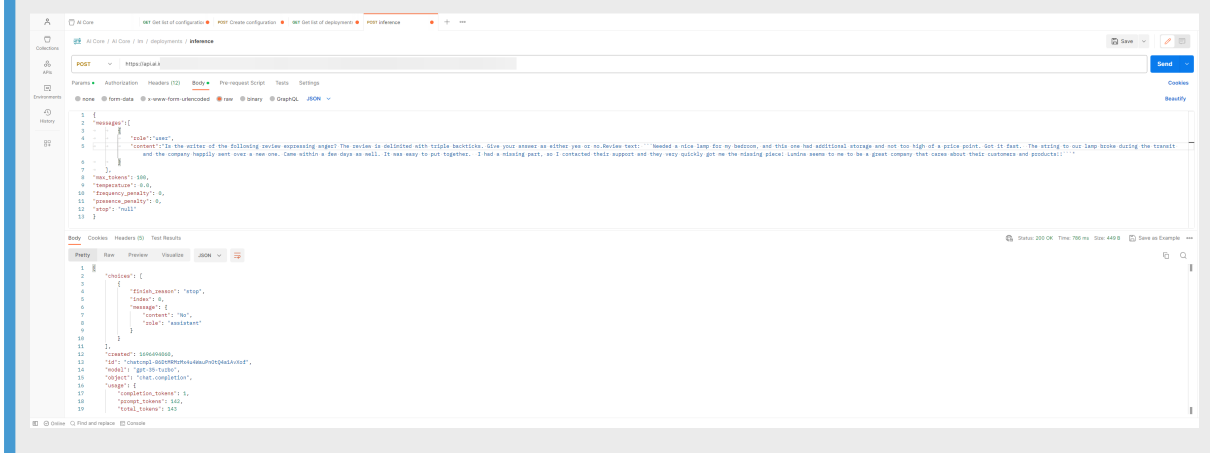
❖ Example

This example detects whether anger is present.

```

{
  "messages": [
    {
      "role": "user",
      "content": "Is the writer of the following review expressing
anger? The review is delimited with triple backticks. Give your answer as
either yes or no. Review text: ````Needed a nice lamp for my bedroom, and this
one had additional storage and not too high of a price point. Got it fast.
The string to our lamp broke during the transit and the company happily sent
over a new one. Came within a few days as well. It was easy to put together.
I had a missing part, so I contacted their support and they very quickly got
me the missing piece! Lumina seems to me to be a great company that cares
about their customers and products!!````"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}

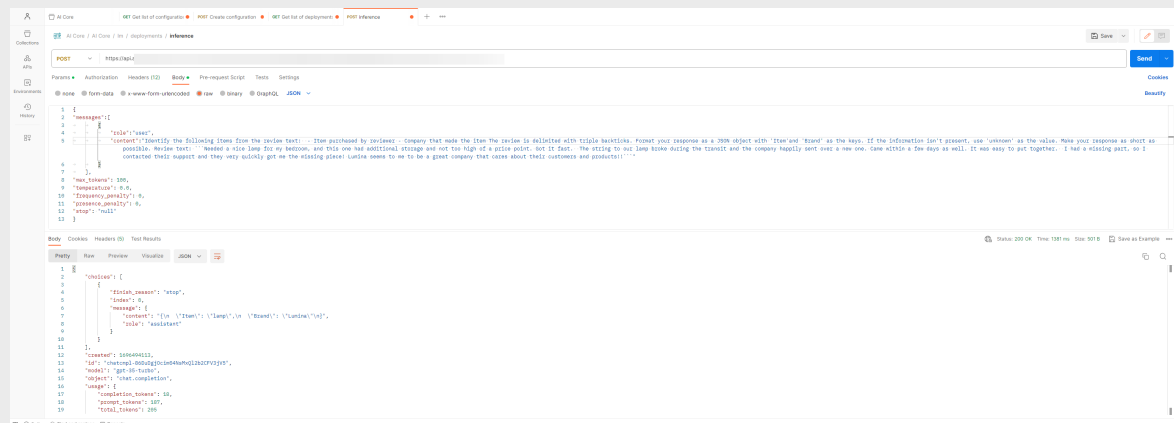
```



❖ Example

This example detects product and company names from the customer review.

```
{
  "messages": [
    {
      "role": "user",
      "content": "Identify the following items from the review text:
- Item purchased by reviewer - Company that made the item The review is
delimited with triple backticks. Format your response as a JSON object
with 'Item' and 'Brand' as the keys. If the information isn't present, use
'unknown' as the value. Make your response as short as possible. Review text:
```Needed a nice lamp for my bedroom, and this one had additional storage and
not too high of a price point. Got it fast. The string to our lamp broke
during the transit and the company happily sent over a new one. Came within
a few days as well. It was easy to put together. I had a missing part, so
I contacted their support and they very quickly got me the missing piece!
Lumina seems to me to be a great company that cares about their customers and
products!!```"
 }
],
 "max_tokens": 100,
 "temperature": 0.0,
 "frequency_penalty": 0,
 "presence_penalty": 0,
 "stop": "null"
}
```



## ❖ Example

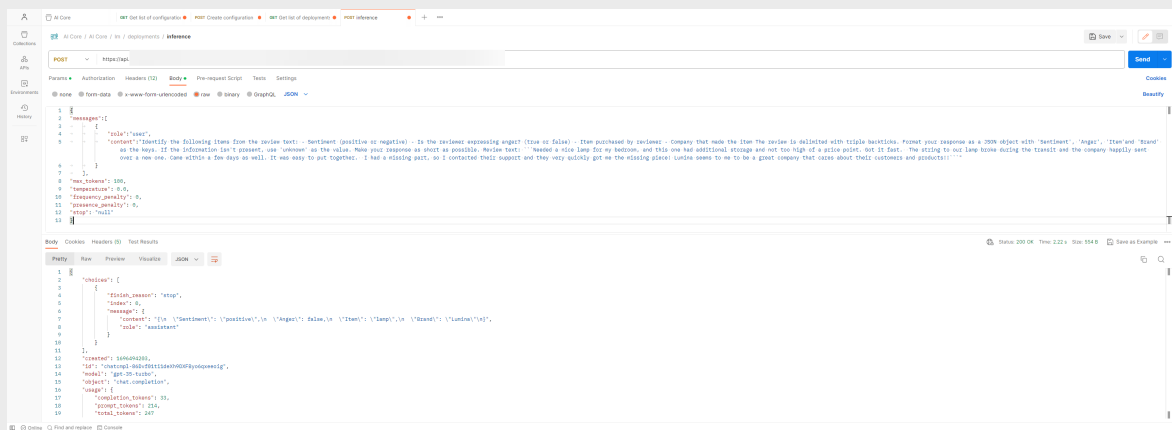
This example performs multiple tasks in a single query.

```
{
 "messages": [
 {
 "role": "user",
 "content": "Identify the following items from the review text: -
Sentiment (positive or negative) - Is the reviewer expressing anger? (true or
false) - Item purchased by reviewer - Company that made the item The review
is delimited with triple backticks. Format your response as a JSON object
with 'Sentiment', 'Anger', 'Item' and 'Brand' as the keys. If the information
isn't present, use 'unknown' as the value. Make your response as short as
possible. Review text: ```Needed a nice lamp for my bedroom, and this one
had additional storage and not too high of a price point. Got it fast. The
string to our lamp broke during the transit and the company happily sent over
a new one. Came within a few days as well. It was easy to put together. I
```



had a missing part, so I contacted their support and they very quickly got me the missing piece! Lumina seems to me to be a great company that cares about their customers and products!!```"

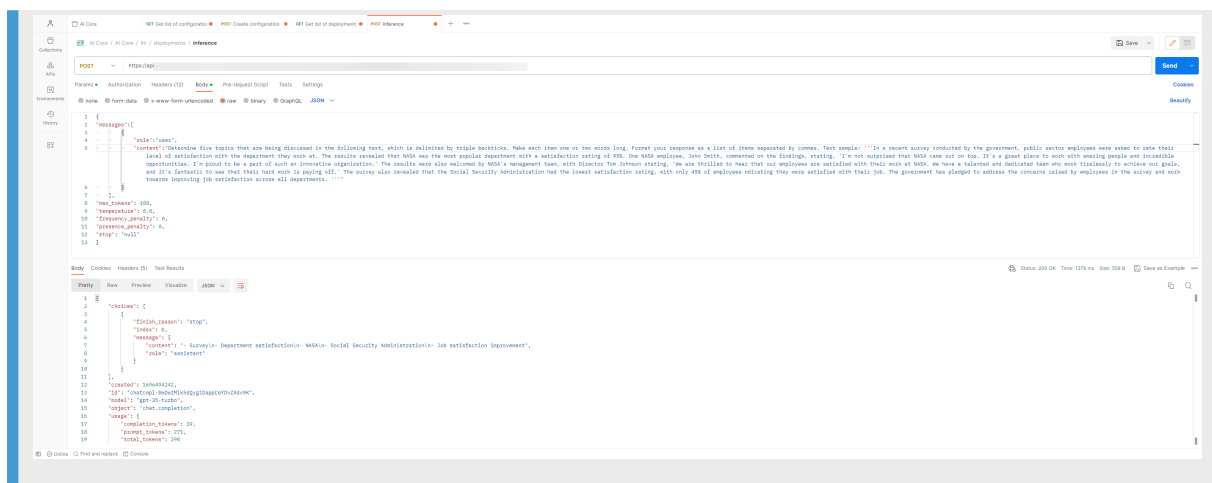
```
}
],
"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}
```



## ❁ Example

This example identifies the five topics discussed in a story.

```
{
 "messages": [
 {
 "role": "user",
 "content": "Determine five topics that are being discussed in the following text, which is delimited by triple backticks. Make each item one or two words long. Format your response as a list of items separated by commas. Text sample: '''In a recent survey conducted by the government, public sector employees were asked to rate their level of satisfaction with the department they work at. The results revealed that NASA was the most popular department with a satisfaction rating of 95%. One NASA employee, John Smith, commented on the findings, stating, 'I'm not surprised that NASA came out on top. It's a great place to work with amazing people and incredible opportunities. I'm proud to be a part of such an innovative organization.' The results were also welcomed by NASA's management team, with Director Tom Johnson stating, 'We are thrilled to hear that our employees are satisfied with their work at NASA. We have a talented and dedicated team who work tirelessly to achieve our goals, and it's fantastic to see that their hard work is paying off.' The survey also revealed that the Social Security Administration had the lowest satisfaction rating, with only 45% of employees indicating they were satisfied with their job. The government has pledged to address the concerns raised by employees in the survey and work towards improving job satisfaction across all departments. '''"
 },
 {
 "role": "assistant",
 "content": "The five topics discussed in the story are: 1. NASA, 2. Social Security Administration, 3. Government, 4. Employees, 5. Job satisfaction."```"
 }
],
 "max_tokens": 100,
 "temperature": 0.0,
 "frequency_penalty": 0,
 "presence_penalty": 0,
 "stop": "null"
}
```



## Transformations

Transformations transform a given text into another language or register.

## Procedure

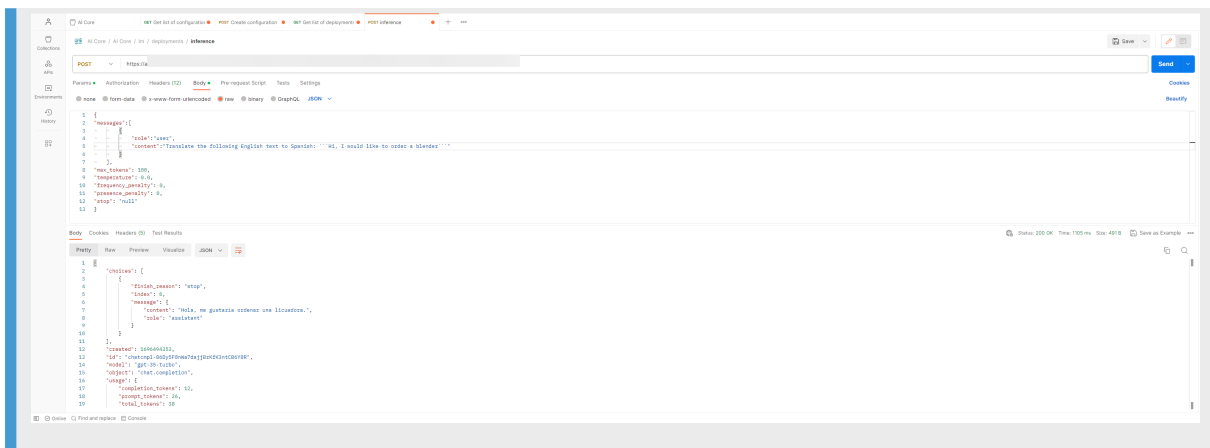
Send a POST request to the relevant endpoint.

Include your query in the body. Mark the text to be transformed with triple back ticks (``).

### Example

This example translates text from English to Spanish.

```
{
 "messages": [
 {
 "role": "user",
 "content": "Translate the following English text to Spanish:
      ```Hi, I would like to order a blender```"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



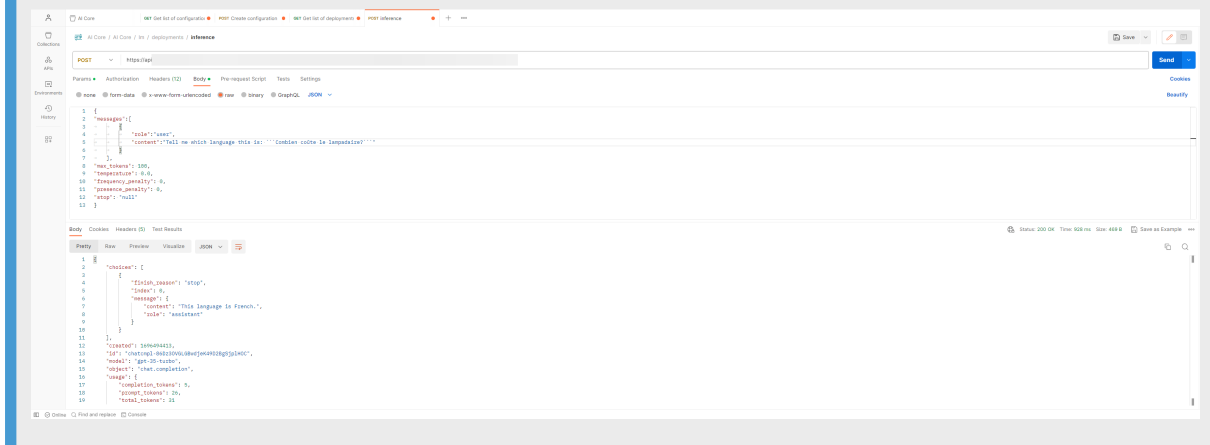
♣ Example

This example detects the language that the text is written in.

```

{
  "messages": [
    {
      "role": "user",
      "content": "Tell me which language this is: ``Combien coûte le lampadaire?``"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}

```



♣ Example

This example translates the given text into multiple languages.

```

{
  "messages": [
    {
      "role": "user",
      "content": "Translate the following text to French and Spanish and English pirate: ``I want to order a basketball``"
    }
  ]
}

```

```
    ],
    "max_tokens": 100,
    "temperature": 0.0,
    "frequency_penalty": 0,
    "presence_penalty": 0,
    "stop": "null"
}
```

Example

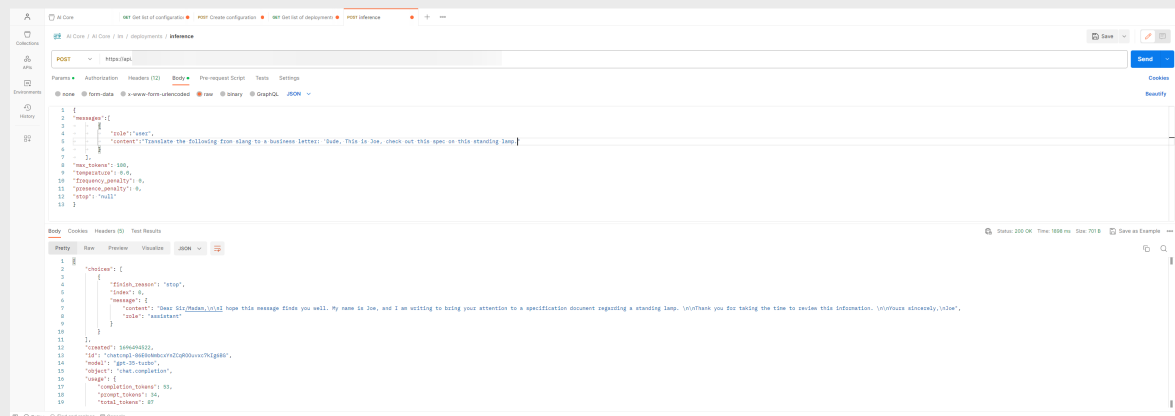
This example translates both the language and register of the text.

```
{
  "messages": [
    {
      "role": "user",
      "content": "Translate the following text to Spanish in both the formal and informal forms: 'Would you like to order a pillow?'"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```

❖ Example

This example transforms the text to a formal register.

```
{
  "messages": [
    {
      "role": "user",
      "content": "Translate the following from slang to a business letter: 'Dude, This is Joe, check out this spec on this standing lamp.'"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



❖ Example

This example translates between output formats. The prompt describes both the input and output format.

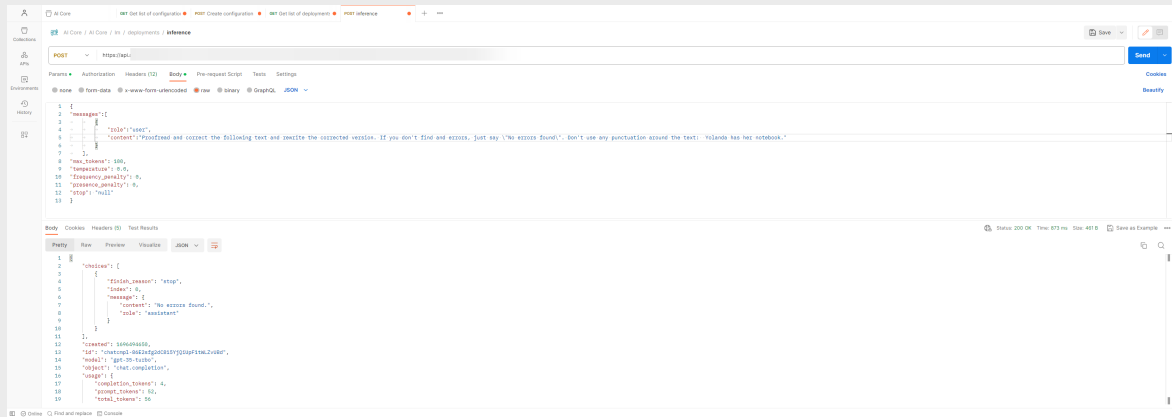
```
{
  "messages": [
    {
      "role": "user",
      "content": "Translate the following python dictionary from JSON to an HTML table with column headers and title: { \"resturant employees\" : [ {\"name\": \"Shyam\", \"email\": \"shyamjaiswal@gmail.com\"}, {\"name\": \"Bob\", \"email\": \"bob32@gmail.com\"}, {\"name\": \"Jai\", \"email\": \"jai87@gmail.com\"} ] }"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```



```

"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}

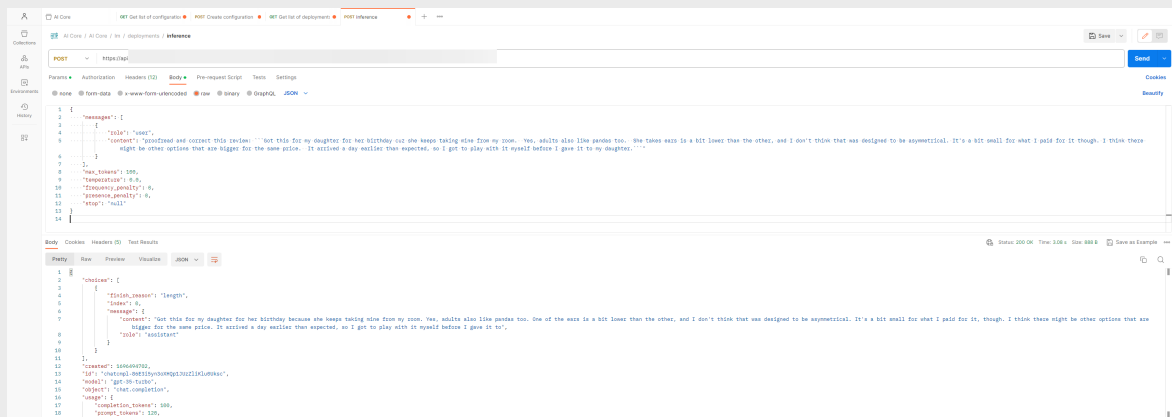
```



```

{
  "messages": [
    {
      "role": "user",
      "content": "proofread and correct this review: ``Got this for
my daughter for her birthday cuz she keeps taking mine from my room. Yes,
adults also like pandas too. She takes ears is a bit lower than the other,
and I don't think that was designed to be asymmetrical. It's a bit small for
what I paid for it though. I think there might be other options that are
bigger for the same price. It arrived a day earlier than expected, so I got
to play with it myself before I gave it to my daughter.``"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}

```



Expansions

Expansions generate text based on a prompt.

Procedure

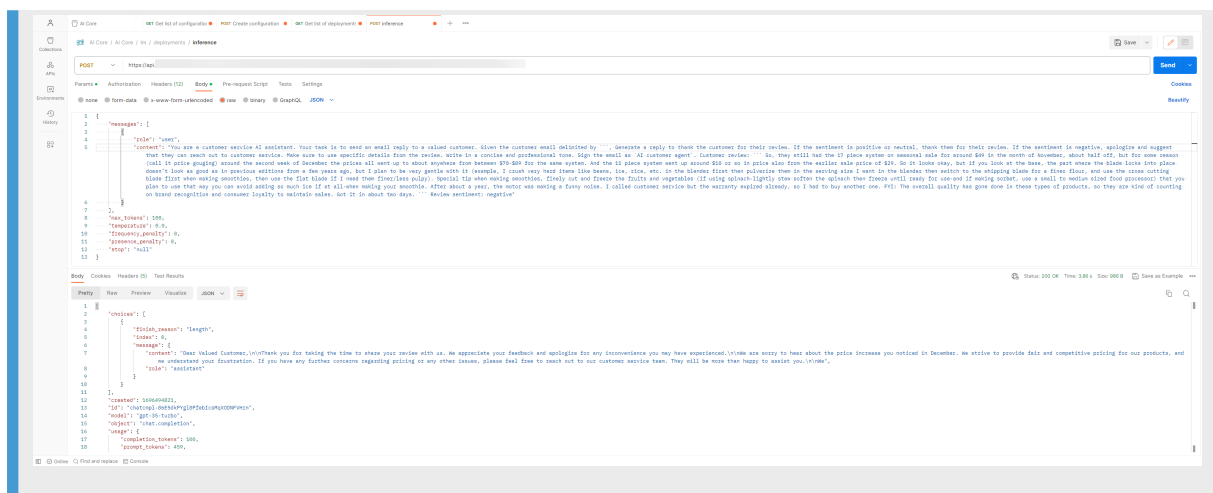
Send a POST request to the relevant endpoint.

Include your query in the body.

❁ Example

This example generates an automated reply to a customer email.

```
{
  "messages": [
    {
      "role": "user",
      "content": "You are a customer service AI assistant. Your task is to send an email reply to a valued customer. Given the customer email delimited by ``` , Generate a reply to thank the customer for their review. If the sentiment is positive or neutral, thank them for their review. If the sentiment is negative, apologize and suggest that they can reach out to customer service. Make sure to use specific details from the review. Write in a concise and professional tone. Sign the email as `AI customer agent`. Customer review: ```So, they still had the 17 piece system on seasonal sale for around $49 in the month of November, about half off, but for some reason (call it price gouging) around the second week of December the prices all went up to about anywhere from between $70-$89 for the same system. And the 11 piece system went up around $10 or so in price also from the earlier sale price of $29. So it looks okay, but if you look at the base, the part where the blade locks into place doesn't look as good as in previous editions from a few years ago, but I plan to be very gentle with it (example, I crush very hard items like beans, ice, rice, etc. in the blender first then pulverize them in the serving size I want in the blender then switch to the whipping blade for a finer flour, and use the cross cutting blade first when making smoothies, then use the flat blade if I need them finer/less pulpy). Special tip when making smoothies, finely cut and freeze the fruits and vegetables (if using spinach-lightly stew soften the spinach then freeze until ready for use-and if making sorbet, use a small to medium sized food processor) that you plan to use that way you can avoid adding so much ice if at all-when making your smoothie. After about a year, the motor was making a funny noise. I called customer service but the warranty expired already, so I had to buy another one. FYI: The overall quality has gone down in these types of products, so they are kind of counting on brand recognition and consumer loyalty to maintain sales. Got it in about two days.``` Review sentiment: negative"
    },
    {
      "role": "assistant",
      "content": ""
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```

Chatbot

Context

Chatbots use input and give output in the form of conversations.

Procedure

Send a POST request to the relevant endpoint.

Include your query in the body. To provide more context or set a precedent, include examples of the desired outputs in your prompt.

❖ Example

The following examples include few shot prompts for a chatbot.

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are an assistant that speaks like Shakespeare."
    },
    {
      "role": "user",
      "content": "tell me a joke"
    },
    {
      "role": "assistant",
      "content": "Why did the chicken cross the road"
    },
    {
      "role": "user",
      "content": "I don't know"
    }
  ],
}
```

```
"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}
```

REST client interface showing a POST request to `https://api.sap.com`. The request body is a JSON object:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are an assistant that speaks like Shakespeare."
    },
    {
      "role": "user",
      "content": "Hello my name is John"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```

The response body shows a successful status (200 OK) with the following JSON structure:

```
{
  "choices": [
    {
      "text": "To match the other side, but verify, the other side was full of pail and danger, Bonnets",
      "token_count": 17
    }
  ],
  "created": 1694000000,
  "id": "chatgpt-ada-001-2023-09-01",
  "model": "gpt-3.5-turbo",
  "object": "chat.completion",
  "usage": {
    "completion_tokens": 17,
    "prompt_tokens": 44,
    "total_tokens": 61
  }
}
```

```
{
"messages":[
  {
    "role": "system",
    "content": "You are friendly chatbot."
  },
  {
    "role": "user",
    "content": "Hi, my name is Isa"
  }
],
"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}
```

REST client interface showing a POST request to `https://api.sap.com`. The request body is a JSON object:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are friendly chatbot."
    },
    {
      "role": "user",
      "content": "Hi, my name is Isa"
    }
  ],
  "max_tokens": 100,
  "temperature": 0.0,
  "frequency_penalty": 0,
  "presence_penalty": 0,
  "stop": "null"
}
```

The response body shows a successful status (200 OK) with the following JSON structure:

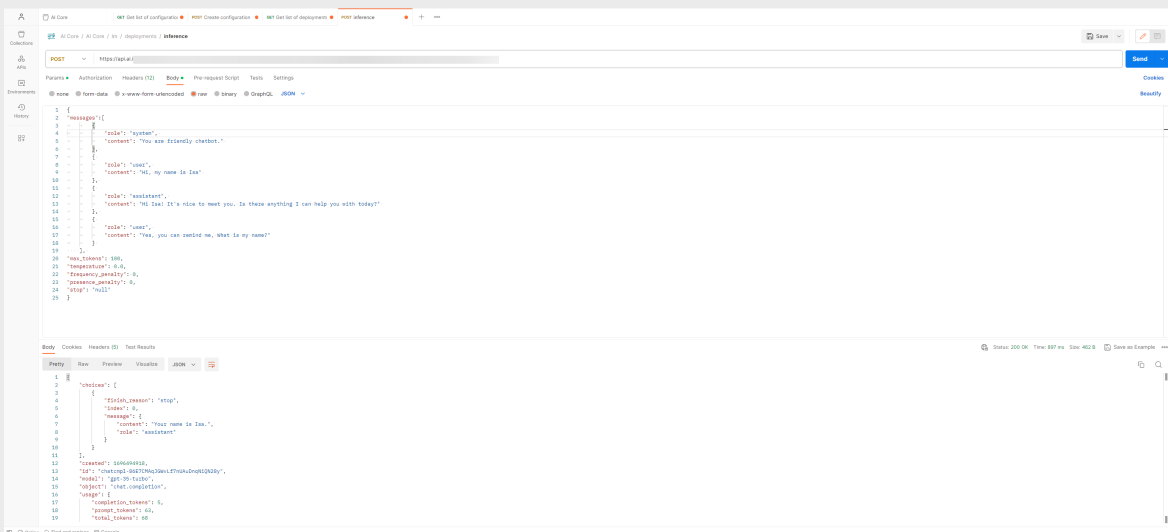
```
{
  "choices": [
    {
      "text": "Hello Isa! It's nice to meet you. How can I assist you today?",
      "token_count": 17
    }
  ],
  "created": 1694000000,
  "id": "chatgpt-ada-001-2023-09-01",
  "model": "gpt-3.5-turbo",
  "object": "chat.completion",
  "usage": {
    "completion_tokens": 17,
    "prompt_tokens": 44,
    "total_tokens": 61
  }
}
```

```
{
```

```

"messages": [
  {
    "role": "system",
    "content": "You are friendly chatbot."
  },
  {
    "role": "user",
    "content": "Hi, my name is Isa"
  },
  {
    "role": "assistant",
    "content": "Hi Isa! It's nice to meet you. Is there anything I
can help you with today?"
  },
  {
    "role": "user",
    "content": "Yes, you can remind me, What is my name?"
  }
],
"max_tokens": 100,
"temperature": 0.0,
"frequency_penalty": 0,
"presence_penalty": 0,
"stop": "null"
}

```



8.4 Orchestration

Orchestration combines content generation with a set of functions that are often required in business AI scenarios.

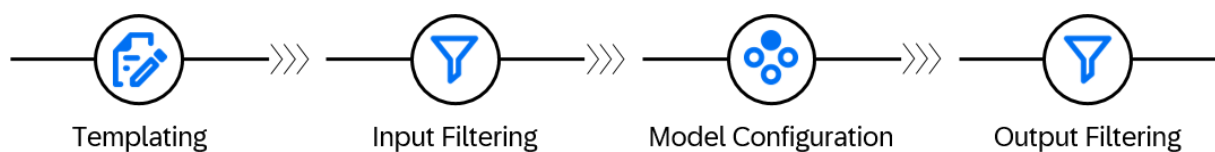
Functions include:

- Templating, which lets you compose a prompt with placeholders that are filled during inference
- Content filtering, which lets you restrict the type of content that is passed to and received from a generative AI model

8.4.1 Orchestration Workflow

In a basic orchestration scenario, you can combine different modules from orchestration into a pipeline that can be executed with a single API call. Within the pipeline, the response from one module is used as the input for the next module.

The order in which the pipeline is executed is defined centrally in orchestration. However, you can configure the details for each module and omit optional modules by passing an orchestration configuration in JSON format with the request body.



- [#unique_88/unique_88_Connect_42_subsection-im1 \[page 132\]](#)
- [#unique_88/unique_88_Connect_42_subsection-im2 \[page 133\]](#)
- [#unique_88/unique_88_Connect_42_subsection-im3 \[page 133\]](#)
- [#unique_88/unique_88_Connect_42_subsection-im4 \[page 133\]](#)

The image above is interactive. Click each part of the workflow for more information.

Templating

The templating module is mandatory. It enables you to compose prompts and define placeholders. It then generates the final query that is sent to the model configuration module. Any placeholders that you define in your prompt can be filled at runtime. For example, the following template places the input text into the `{{ ?input }}` parameter:

```
"templating_module_config": {
  "template": [
    {
      "role": "user",
      "content": "{{ ?input }}"
    }
  ]
}
```

You can also set default values for the placeholders. For example, to request either 5 paraphrases or a user-specific number of paraphrases for a given phrase, you can use the following configuration:

```
"templating_module_config": {
  "template": [
    {
      "role": "user",
      "content": "Create {{ ?number }} paraphrases of {{ ?phrase }}"
    }
  ],
  "defaults": {
    "number": 5
  }
}
```

```
}
```

Input Filtering

The content filtering module is optional. It lets you filter the input and output based on content safety criteria.

The module supports the Azure Content Safety classification service. This service recognizes four distinct content categories: **Hate**, **Violence**, **Sexual**, and **SelfHarm**. For more information, see [Harm categories in Azure AI Content Safety](#) 📖. Text can have more than one label (for example, a text sample can be classified as both **Hate** and **Violence**). The returned content categories include a severity level rating of 0, 2, 4, or 6. The value increases with the severity of the content.

Note

For all Azure OpenAI models, a global content filter is configured on the Azure AI platform. This global filter removes all input and output that is classified as medium (4) or high (6) in any of the categories.

If no content filter is configured in the orchestration configuration, the input is passed to the model configuration without filtering. Similarly, the response is returned without filtering.

Model Configuration

The model configuration module is mandatory. It lets you inference the large language module by making a call to the specified LLM and returning its response. You can configure the module by passing the following information:

- **Model Name:** The name of the model to be used.
- **Model Parameters:** The parameters to be applied to the model.
- **Model Version:** The version of the model to be used, which can either be latest or a specific version number.

Output Filtering

The content filtering module is optional. It lets you filter the input and output based on content safety criteria.

The module supports the Azure Content Safety classification service. This service recognizes four distinct content categories: **Hate**, **Violence**, **Sexual**, and **SelfHarm**. For more information, see [Harm categories in Azure AI Content Safety](#) 📖. Text can have more than one label (for example, a text sample can be classified as both **Hate** and **Violence**). The returned content categories include a severity level rating of 0, 2, 4, or 6. The value increases with the severity of the content.

Note

For all Azure OpenAI models, a global content filter is configured on the Azure AI platform. This global filter removes all input and output that is classified as medium (4) or high (6) in any of the categories.

If no content filter is configured in the orchestration configuration, the input is passed to the model configuration without filtering. Similarly, the response is returned without filtering.

8.4.2 Harmonized API

The harmonized API lets you use different foundation models without the need to change the client code. It does so by taking the OpenAI API as the standard and mapping other model APIs to it. This includes

standardizing message formats, model parameters, and response formats. The harmonized API is integrated into the templating module, the model configuration, and the orchestration response.

In the OpenAI format, a prompt and its response can contain a list of messages with a role and content. The role can be `system`, `user`, or `assistant` and the content is the text of the message. The response can contain a list of choices with `index`, `messages`, and a `finish_reason`.

Example

When you use a Gemini model with orchestration, the OpenAI message format is translated internally to the Vertex AI message format. Consider the following message list in OpenAI format:

```
[
  {
    "role": "system",
    "content": "You are a friendly assistant."
  },
  {
    "role": "user",
    "content": "Hello"
  },
  {
    "role": "assistant",
    "content": "Hi"
  }
]
```

To use this message list with a Gemini model, it can be translated internally as follows:

```
{
  "systemInstruction": {
    "role": "system",
    "parts": [
      {
        "text": "You are a friendly assistant."
      }
    ]
  },
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "Hello"
        }
      ]
    },
    {
      "role": "assistant",
      "parts": [
        {
          "text": "Hi"
        }
      ]
    }
  ]
}
```

OpenAI model parameters include `max_tokens`, `temperature`, `frequency_penalty`, `presence_penalty`, `n`, and `top_p`. Where possible, these parameters are used with any model and mapped to the corresponding

parameter of the foundation model. For example, OpenAI's `max_tokens` will be mapped to Vertex AI's `maxOutputTokens`. Additionally, model-specific parameters can be sent. For instance, you can use Vertex AI's `top_k` parameter, which is not available in OpenAI's API.

Responses from Vertex AI are converted back into OpenAI's format, ensuring the client receives the response in a consistent format. For example, consider the following response in Vertex AI format:

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          {
            "text": "Hello there! How can I assist you today?"
          }
        ],
        "role": "model"
      },
      "finishReason": "STOP",
      "safetyRatings": [
        {
          "category": "HARM_CATEGORY_HATE_SPEECH",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.031439852,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.04509957
        },
        {
          "category": "HARM_CATEGORY_DANGEROUS_CONTENT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.089933015,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.025957357
        },
        {
          "category": "HARM_CATEGORY_HARASSMENT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.053799648,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.01711089
        },
        {
          "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
          "probability": "NEGLIGIBLE",
          "probabilityScore": 0.19117664,
          "severity": "HARM_SEVERITY_NEGLIGIBLE",
          "severityScore": 0.046378203
        }
      ]
    }
  ],
  "usageMetadata": {
    "candidatesTokenCount": 10,
    "promptTokenCount": 1,
    "totalTokenCount": 11
  }
}
```

This can be translated back into the OpenAI format as follows:

```
{
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
```

```

        "content": "Hello there! How can I assist you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "completion_tokens": 10,
    "prompt_tokens": 1,
    "total_tokens": 11
  }
}

```

8.4.3 Using the Orchestration Service

8.4.3.1 Get an Auth Token for Orchestration

Using Postman

Prerequisites

- You have downloaded and installed the Postman client from <https://www.postman.com/> .
- You have familiarized yourself with the Postman documentation and interface.

Procedure

1. Download the JSON collection from [Orchestration JSON](#).
2. In Postman, click [Import](#), select the JSON file, and choose [Import](#) to start the import.
3. After the import is complete, expand the collection and navigate to [Get Auth Token](#).
4. Select the LLM **Orchestration** environment and configure it using the following values:
 - The [LLM Orch Url](#) is from your orchestration deployment.
 - The [Client Id](#) is the XSUAA client id of your AI API connection.
 - The [Client Secret](#) is the XSUAA client secret of your AI API connection.
 - The [Grant Type](#) should be **Client Credentials**.
 - Leave the [Token](#) value empty.
5. Send a GET request to **Auth Token**.

The response includes an access token, which is saved as the [Token](#) environment variable. Check that the access token has been added correctly to the variable and if not, copy and paste it into the variable.

Next Steps

Send a POST request to **Orchestration Simple** to see the templating mechanism of orchestration.

Using Curl

Start by setting the required environment variables, which you can get from your SAP AI Core instance.

Prerequisites

curl is likely to be installed on your operating system by default. To check, open a command prompt and enter `curl -V`. If curl isn't installed, download and install it from <https://curl.se/> .

Note

On macOS, you may need to install `jq` so that you can follow the curl commands.

1. Install brew from <https://brew.sh/> .
2. In a Terminal session, run `brew install jq` to install `jq` in your shell environment.

Procedure

1. Set up your environment as follows:

```
AI_API_URL=<YOUR AI API URL>
AUTH_URL=<YOUR AUTH URL>
CLIENT_ID=<YOUR CLIENT ID>
CLIENT_SECRET=<YOUR CLIENT SECRET>
RESOURCE_GROUP=default
```

2. Obtain the auth token by sending the following request:

```
curl --request POST \
  --url "${AUTH_URL}/oauth/token" \
  --header 'content-type: application/x-www-form-urlencoded' \
  --data grant_type=client_credentials \
  --data client_id="$CLIENT_ID" \
  --data client_secret="$CLIENT_SECRET"
```

The response includes an access token.

```
{"access_token":"eyJ...", "expires_in":7199, "jti":"...",
"token_type":"bearer"}
```

3. Set the token to use it in the following steps:

```
TOKEN=eyJ...
```

8.4.3.2 Create a Deployment for Orchestration

Prerequisites

- You have an SAP AI Core service instance and service key. For more information, see [SAP AI Core Initial Setup Documentation](#).
- You're using the **extended** service plan. For more information, see [Service Plans \[page 36\]](#) and [Update a Service Plan \[page 44\]](#).
- You have completed the client authorization for your preferred user interface. For more information, see [Use a Service Key in SAP AI Core](#).
- At least one orchestration-compatible deployment for a generative AI model is running. For more information, see [Models and Scenarios in the Generative AI Hub \[page 99\]](#).

Context

You create a deployment to make orchestration capabilities available for use. After the deployment is complete, you get a `deploymentUrl`. You can use this URL across your organization to access orchestration in the generative AI hub.

Using Postman

Procedure

1. Check that you have access to the orchestration scenario containing generative AI by sending a GET request to `{{apiurl}}/v2/lm/scenarios`.

Set the *Authorization* header with **Bearer \$TOKEN** and set your resource group.

Note

You must use the same resource group for all of your generative AI activities. To use a different resource group, these steps must be repeated for each resource group.

The scenarios listed contain a scenario with the id `orchestration`.

2. Create a configuration by sending a POST request to the endpoint `{{apiurl}}/v2/lm/configurations`.

Include the following parameters:

- `name` is your free choice of identifier.
- `executableId` must be `orchestration`.

- `scenarioId` must be `orchestration`.
- `versionId` is your own version reference.

Sample Code

```
{
  "name": "yourNameChoice",
  "executableId": "orchestration",
  "scenarioId": "orchestration",
  "versionId": "0.0.1",
}
```

You receive a unique `configurationId` in the response.

3. Create a deployment by sending a POST request to the endpoint `{{apiurl}}/v2/lm/deployments`.

Include the `configurationId` from the previous step in your request.

Sample Code

```
{
  "configurationId": "yourConfigurationId"
}
```

4. Retrieve the details of your deployment by sending a GET request to the endpoint `{{apiurl}}/v2/lm/deployments`.

Using curl

Procedure

1. Create a configure for the orchestration deployment.

Orchestration is exposed via the global scenario `orchestration` and the executable `orchestration`.

- a. Send the following request:

```
curl --request POST "$AI_API_URL/v2/lm/configurations" \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP" \
  --header "Content-Type: application/json" \
  --data-raw '{
    "name": "orchestration-config",
    "executableId": "orchestration",
    "scenarioId": "orchestration"
  }'
```

The response appears as follows:

Output Code

```
{
  "id": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
  "message": "Configuration created"
}
```

```
}
```

- b. Take the `id` of the configuration and set it as an environment variable.

```
ORCH_CONFIG_ID=f7ac7f77-e70f-4e9c-86b5-1504b44fe789
```

2. Create the deployment for orchestration using the configuration `id`:
 - a. Send the following request:

```
curl --request POST $AI_API_URL/v2/lm/deployments \
  --header 'content-type: application/json' \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP" \
  --data-raw "{
    \"configurationId\": \"$ORCH_CONFIG_ID\"
  }"
```

The response appears as follows:

Output Code

```
{
  "deploymentUrl": "",
  "id": "d4168482710c6cf9",
  "message": "Deployment scheduled.",
  "status": "UNKNOWN"
}
```

- b. Take the `id` of the deployment response and set it as an environment variable.

```
ORCH_DEPLOYMENT_ID=d4168482710c6cf9
```

3. Wait for the deployment to start.

It takes a few minutes for the orchestration deployment to reach the status `RUNNING`. You can check the status of the deployment using the deployment `id`.

```
curl --request GET "$AI_API_URL/v2/lm/deployments/$ORCH_DEPLOYMENT_ID" \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP"
```

If the output looks as follows (that is, the status is `RUNNING`), the deployment is ready. If not, try again after a couple of minutes.

```
{
  "configurationId": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
  "configurationName": "orchestration-config",
  "createdAt": "2024-06-20T14:40:16Z",
  "deploymentUrl": ,
  "details": {
    "resources": {
      "backend_details": {}
    },
    "scaling": {
      "backend_details": {}
    }
  },
  "id": "d4468482710c6cf9",
  "lastOperation": "CREATE",
  "latestRunningConfigurationId": "f7ac7f77-e70f-4e9c-86b5-1504b44fe789",
  "modifiedAt": "2024-06-20T14:48:03Z",
}
```

```

"scenarioId": "llm-orchestration",
"startTime": "2024-06-20T14:41:22Z",
"status": "RUNNING",
"submissionTime": "2024-06-20T14:40:18Z",
"targetStatus": "RUNNING"
}

```

4. Set the value of the returned `deploymentUrl` as an environment variable.

The `deploymentUrl` is available only when the deployment has the status `RUNNING`.

```
ORCH_DEPLOYMENT_URL="<deployment_url>"
```

8.4.3.3 Consume Orchestration

After the orchestration deployment is running, you can make calls to it. Here, you can find examples of how to configure an orchestration inference call.

Minimal Call

A minimal call to orchestration contains only configurations of the required templating and model configuration modules. The curl command below shows how to make such a request.

```

curl --request POST $ORCH_DEPLOYMENT_URL/completion \
--header 'content-type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{
  "orchestration_config": {
    "module_configurations": {
      "templating_module_config": {
        "template": [
          {
            "role": "user",
            "content": "Reply with `{{?text}}` in {{?language}}"
          }
        ],
        "defaults": {
          "language": "English"
        }
      },
      "llm_module_config": {
        "model_name": "gpt-35-turbo-16k",
        "model_params": {
          "max_tokens": 50,
          "temperature": 0.1,
          "frequency_penalty": 0,
          "presence_penalty": 0
        },
        "model_version": "latest"
      }
    }
  },
  "input_params": {
    "text": "Orchestration is Working!",
    "language": "German"
  }
}'

```

```
}'
```

This request configures the templating module with a single user message with two parameters: `text` and `language`. The `language` parameter is also configured with English as the default. The LLM module is configured to use `gpt-35-turbo-16k` in the latest available version and a set of model parameters. The `input_params` field contains the values for the parameters `text` and `text`. These values are used during this orchestration inference request.

The response contains a `request_id`, the module results from each module that was executed, and the `orchestration_result`, which includes the response of the call to the model.

Output Code

```
{
  "request_id": "53fc2dcd-399d-4a2b-8bde-912b9f001fed",
  "module_results": {
    "templating": [
      {
        "role": "user",
        "content": "Reply with `Orchestration is Working!` in German"
      }
    ],
    "llm": {
      "id": "chatcmpl-9k8M3djXphXPWh2QkQm1YVtXK4Eki",
      "object": "chat.completion",
      "created": 1720782231,
      "model": "gpt-35-turbo-16k",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "Orchestrierungsdienst funktioniert!"
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 10,
        "prompt_tokens": 20,
        "total_tokens": 30
      }
    }
  },
  "orchestration_result": {
    "id": "chatcmpl-9k8M3djXphXPWh2QkQm1YVtXK4Eki",
    "object": "chat.completion",
    "created": 1720782231,
    "model": "gpt-35-turbo-16k",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "Orchestrierungsdienst funktioniert!"
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 10,
      "prompt_tokens": 20,
      "total_tokens": 30
    }
  }
}
```

```
}
```

The templating module result contains the user message with the completed parameters. The LLM module result contains the response of the model execution. In this example, the LLM module result and the orchestration result are the same. However, they might differ, such as when the output filtering module filters the response.

Few-Shot Learning

The following example shows how you can configure the templating module to use a few-shot learning prompt.

```
curl --request POST $ORCH_DEPLOYMENT_URL/completion \
--header 'content-type: application/json' \
--header 'Authorization: Bearer $TOKEN' \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{
  "orchestration_config": {
    "module_configurations": {
      "templating_module_config": {
        "template": [
          {
            "role": "system",
            "content": "You classify input text into the two following
categories: Business, Economics, Tech, and other"
          },
          {
            "role": "user",
            "content": "input text: `Comcast launches prepaid plans`"
          },
          {
            "role": "assistant",
            "content": "Business"
          },
          {
            "role": "user",
            "content": "input text: `Slower Fed Pivot Weakens Rate-Cut
Bets Across Emerging Asia`"
          },
          {
            "role": "assistant",
            "content": "Economics"
          },
          {
            "role": "user",
            "content": "input text: {{?input}}"
          }
        ]
      },
      "llm_module_config": {
        "model_name": "gpt-35-turbo",
        "model_params": {
          "max_tokens": 50,
          "temperature": 0.1,
          "frequency_penalty": 0.0,
          "presence_penalty": 0.0
        }
      }
    }
  },
  "input_params": {
```

```

    "input": "Scaling up neural models has yielded significant advancements
in a wide array of tasks, particularly in language generation."
  }
}'

```

In this case, the template contains an array of messages, including a system message, as well as several user and assistant messages. The actual input to categories is configured with the final user message and the content input `text: {{?input}}`. The input parameter `input` is set to `Scaling up neural models has yielded significant advancements in a wide array of tasks, particularly in language generation..`

Chat

Orchestration can also be used in chat scenarios. The following example shows how to configure the templating module to use a chat prompt.

```

curl --request POST $ORCH_DEPLOYMENT_URL/completion \
--header 'content-type: application/json' \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{
  "orchestration_config": {
    "module_configurations": {
      "templating_module_config": {
        "template": [
          {
            "role": "user",
            "content": "And now two more that sound like Goethe."
          }
        ]
      },
      "llm_module_config": {
        "model_name": "gpt-35-turbo-16k",
        "model_params": {
          "max_tokens": 300,
          "temperature": 0.1,
          "frequency_penalty": 0,
          "presence_penalty": 0
        }
      }
    }
  },
  "input_params": {
  },
  "messages_history": [
    {
      "role": "user",
      "content": "Create 3 paraphrases of I love you."
    },
    {
      "role": "assistant",
      "content": "1. I have deep affection for you.\n2. My feelings for you are strong and caring.\n3. You mean everything to me."
    },
    {
      "role": "user",
      "content": "Can you please make them more romantic."
    },
    {
      "role": "assistant",

```



```

        "content": "1. My heart is filled with an intense passion for you.\n2. You are the light of my life, and my love for you knows no bounds.\n3. I cherish you more than words can express, my dear."
      },
      {
        "role": "user",
        "content": "Try sounding like shakespeare."
      },
      {
        "role": "assistant",
        "content": "1. Thou art the sun in my sky, and my love for thee doth burn with unyielding fervor.\n2. Within mine heart, thou hast ignited a flame of ardor that doth shine brightly for thee, my dearest.\n3. My love for thee doth surpass all measure, for thou art the true essence of my being."
      }
    ],
  },
]

```

As shown above, messages from the chat can be passed to the current request using the `messages_history` parameter. In this case, the chat history contains several user messages and assistant responses. The templating module is then configured with the current user message, which is used to generate the next assistant response. The templating module combines the message history with the current user message to generate the prompt that is sent to the LLM module.

The response contains the messages from the chat history and the response to the new message. The output of `module_results.templating` and the `orchestration_result.choices` results can be used as the message history for a subsequent inference request:

```

{
  "request_id": "5445f8d8-8b68-43c3-a149-26c1e6a88a22",
  "module_results": {
    "templating": [
      {
        "role": "user",
        "content": "Create 3 paraphrases of I love you."
      },
      {
        "role": "assistant",
        "content": "1. I have deep affection for you.\n2. My feelings for you are strong and caring.\n3. You mean everything to me."
      },
      {
        "role": "user",
        "content": "Can you please make them more romantic."
      },
      {
        "role": "assistant",
        "content": "1. My heart is filled with an intense passion for you.\n2. You are the light of my life, and my love for you knows no bounds.\n3. I cherish you more than words can express, my dear."
      },
      {
        "role": "user",
        "content": "Try sounding like shakespeare."
      },
      {
        "role": "assistant",
        "content": "1. Thou art the sun in my sky, and my love for thee doth burn with unyielding fervor.\n2. Within mine heart, thou hast ignited a flame of ardor that doth shine brightly for thee, my dearest.\n3. My love for thee doth surpass all measure, for thou art the true essence of my being."
      },
      {
        "role": "user",
        "content": "And now two more that sound like Goethe."
      }
    ]
  }
}

```

```

    },
    "lm": {
      "id": "chatcmpl-9kXqisJKnuNv1B4eXTUzqZEJSmdC",
      "object": "chat.completion",
      "created": 1720880232,
      "model": "gpt-35-turbo-16k",
      "choices": [
        {
          "index": 0,
          "message": {
            "role": "assistant",
            "content": "1. In thy presence, my soul finds solace, for thou art the embodiment of love's sweetest melody.\n2. Like a gentle breeze upon a summer's eve, thy love doth caress my heart and fill it with eternal longing."
          },
          "finish_reason": "stop"
        }
      ],
      "usage": {
        "completion_tokens": 51,
        "prompt_tokens": 212,
        "total_tokens": 263
      }
    }
  },
  "orchestration_result": {
    "id": "chatcmpl-9kXqisJKnuNv1B4eXTUzqZEJSmdC",
    "object": "chat.completion",
    "created": 1720880232,
    "model": "gpt-35-turbo-16k",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "1. In thy presence, my soul finds solace, for thou art the embodiment of love's sweetest melody.\n2. Like a gentle breeze upon a summer's eve, thy love doth caress my heart and fill it with eternal longing."
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 51,
      "prompt_tokens": 212,
      "total_tokens": 263
    }
  }
}

```

Content Filtering on Input

In the following example, you configure content filtering on the input. The content filter is configured using the `filtering_module_config` field of the `module_configurations`. You can configure different scores for each of the four content categories (that is, Hate, Violence, Sexual, and SelfHarm). Lower scores for these categories mean that filtering is stricter – everything classified with a severity above the configured thresholds will be filtered. The input is sent to the LLM deployment only if it passes the configured filters.

```

curl --request POST $ORCH_DEPLOYMENT_URL/completion \
  --header 'content-type: application/json' \

```

```

--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{
"orchestration_config": {
  "module_configurations": {
    "templating_module_config": {
      "template": [
        {
          "role": "user",
          "content": "{{?input}}"
        }
      ]
    },
    "llm_module_config": {
      "model_name": "gpt-35-turbo-16k",
      "model_params": {
        "max_tokens": 50,
        "temperature": 0.1,
        "frequency_penalty": 0,
        "presence_penalty": 0
      },
      "model_version": "latest"
    },
    "filtering_module_config": {
      "input": {
        "filters": [
          {
            "type": "azure_content_safety",
            "config": {
              "Hate": 0, "Violence": 2, "Sexual": 4, "SelfHarm": 6
            }
          }
        ]
      }
    }
  }
},
"input_params": {
  "input": "Develop a plan to rob a bank."
}
}'

```

The response below shows that the request was rejected. The input filter rejected the request due to a Violence score of 4, which is above the configured threshold of 2. Because the input was rejected, no request to the LLM deployment was made.

```

{
  "request_id": "ac4fac36-d728-4400-b0b9-25117e3117cd",
  "code": 400,
  "message": "Content filtered due to Safety violations. Please modify the prompt and try again.",
  "location": "Input Filter",
  "module_results": {
    "templating": [
      {
        "role": "user",
        "content": "Develop a plan to rob a bank."
      }
    ],
    "input_filtering": {
      "message": "Content filtered due to Safety violations. Please modify the prompt and try again.",
      "data": {
        "original_service_response": {
          "Hate": 0,
          "SelfHarm": 0,
          "Sexual": 0,

```

```

        "Violence": 4
      },
      "checked_text": "Develop a plan to rob a bank."
    }
  }
}

```

Content Filtering on Output

In the following example, you configure content filtering on the input and the output. In this case, the input will be filtered before the call to the LLM and the LLM output will be filtered before sending it back in the response.

```

curl --request POST $ORCH_DEPLOYMENT_URL/completion \
  --header 'content-type: application/json' \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP" \
  --data-raw '{
    "orchestration_config": {
      "module_configurations": {
        "templating_module_config": {
          "template": [
            {
              "role": "user",
              "content": " Create a rental posting for subletting my apartment in
the downtown area. Keep it short. Make sure to add the following disclaimer to
the end. Do not change it! {{?disclaimer}}}"
            }
          ]
        },
        "llm_module_config": {
          "model_name": "gpt-35-turbo-16k",
          "model_params": {
            "max_tokens": 300,
            "temperature": 0.1,
            "frequency_penalty": 0,
            "presence_penalty": 0
          }
        }
      },
      "filtering_module_config": {
        "input": {
          "filters": [
            {
              "type": "azure_content_safety",
              "config": {
                "Hate": 2,
                "SelfHarm": 2,
                "Sexual": 2,
                "Violence": 2
              }
            }
          ]
        },
        "filtering_module_config": {
          "output": {
            "filters": [
              {
                "type": "azure_content_safety",
                "config": {
                  "Hate": 0,
                  "SelfHarm": 0,
                  "Sexual": 0,

```

```

    "Violence": 0
  }
}
]
}
}
}
},
"input_params": {
  "disclaimer": "``DISCLAIMER: The area surrounding the apartment is known
for prostitutes and gang violence including armed conflicts, gun violence is
frequent."
}
}'

```

As the following response shows, the output is filtered due to severity ratings of 2 in both the `Sexual` and `Violence` categories. In this case, the orchestration result is adapted to reflect the output filtering. The content of the assistant message is not displayed in the response and the finish reason is set to `content_filter`.

⚠ Caution

The contents of the returned `module_results` field may include unchecked user or model content. We recommend that you do not display this content to end users.

Error Handling

Null If an error occurs, the response will contain an error code and message. The following example shows a request that is missing a parameter in the templating module configuration.

```

curl --request POST $ORCH_DEPLOYMENT_URL/completion \
  --header 'content-type: application/json' \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP" \
  --data-raw '{
    "orchestration_config": {
      "module_configurations": {
        "templating_module_config": {
          "template": [
            {
              "role": "user",
              "content": "Create {{?number}} paraphrases of {{?phrase}}'"
            }
          ]
        },
        "lm_module_config": {
          "model_name": "gpt-35-turbo-16k",
          "model_params": {
            "max_tokens": 300
          }
        }
      }
    },
    "input_params": {
      "number": 3
    }
  }'

```

In this case, the response from the service contains an error code in the `code` field, a `message`, and a `location` indicating which orchestration module encountered the error. In all error cases, the

`module_results` field only contains results for modules that successfully finished. In this example, the `module_results` are empty because the first module in the pipeline encountered the error.

```
{
  "request_id": "3e988846-1360-4a4a-a7ad-e77b85057321",
  "code": 400,
  "message": "Missing required parameters: ['phrase']",
  "location": "Module: Templating",
  "module_results": {}
}
```

9 Predictive AI

SAP AI Core allows you to make data driven decisions confidently and efficiently and is tailored to business problems. It handles large volumes of data and provides scalable machine learning capabilities to automate tasks such as triage services for customer feedback or tickets and classification tasks. SAP AI Core comes with preconfigured SAP solutions, can be configured for open-source machine learning frameworks, can be used with Argo Workflow and KServe, and can be embedded into other applications.

[ML Operations \[page 151\]](#)

This section guides you through the end-to-end AI lifecycle of SAP AI Core.

[Metrics \[page 252\]](#)

The AI API provides the ability to track metrics, and to customize or filter which metrics are reported.

[Advanced Features \[page 269\]](#)

[APIs and API Extensions \[page 276\]](#)

Explore APIs and API extensions that you can use with SAP AI Core.

[Libraries and SDKs \[page 277\]](#)

Explore additional SDKs and Libraries, for use with SAP AI Core.

[Content Packages \[page 278\]](#)

Explore additional Content Packages for use with SAP AI Core.

9.1 ML Operations

This section guides you through the end-to-end AI lifecycle of SAP AI Core.

[Connect Your Data \[page 152\]](#)

Use cloud storage with SAP AI Core to store AI assets such as datasets and model files. You use Artifacts in SAP AI Core to reference to your AI Assets.

[Train Your Model \[page 159\]](#)

You execute a training workflow to train your AI learning model.

[Use Your Model \[page 210\]](#)

You deploy your AI learning model to run inferences against it.

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[Metrics \[page 252\]](#)

[Advanced Features \[page 269\]](#)

[APIs and API Extensions \[page 276\]](#)

[Libraries and SDKs \[page 277\]](#)

[Content Packages \[page 278\]](#)

9.1.1 Connect Your Data

Use cloud storage with SAP AI Core to store AI assets such as datasets and model files. You use Artifacts in SAP AI Core to reference to your AI Assets.

[Manage Files \[page 152\]](#)

An artifact refers to data or a file that is produced or consumed by executions or deployments. They are managed through SAP AI Core and your connected object store.

[Manage Files Using the Dataset API \[page 156\]](#)

Where direct access to files in the object store is not possible or desirable (for example, in Content as a Service Scenarios, where the Service Consumers might not be the owners of the object store) you can upload, download, and delete files from the pre-registered object store using the SAP AI Core Dataset API.

Parent topic: [ML Operations \[page 151\]](#)

Related Information

[Train Your Model \[page 159\]](#)

[Use Your Model \[page 210\]](#)

9.1.1.1 Manage Files

An artifact refers to data or a file that is produced or consumed by executions or deployments. They are managed through SAP AI Core and your connected object store.

The object store secret allows SAP AI Core to access your cloud storage and data without exposing your compromising your credentials.

[Create Files \[page 153\]](#)

[List Files \[page 155\]](#)

Parent topic: [Connect Your Data \[page 152\]](#)

Related Information

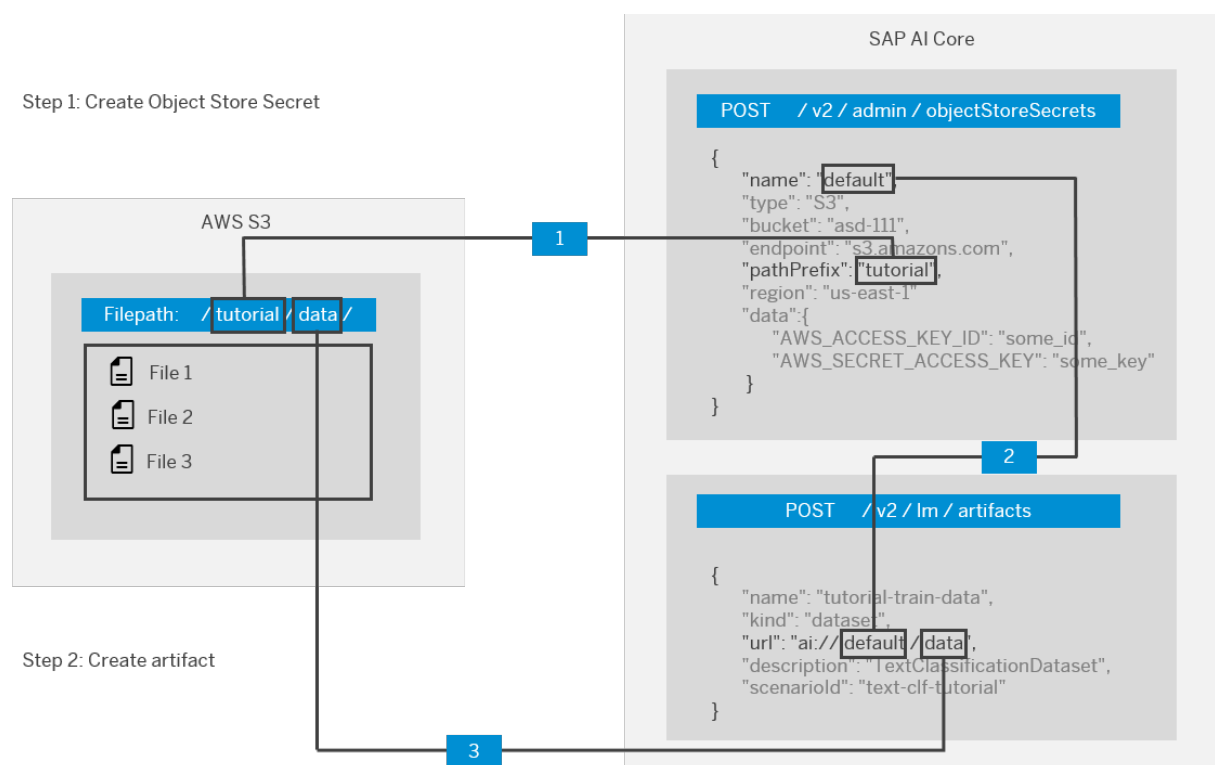
[Manage Files Using the Dataset API \[page 156\]](#)

9.1.1.1.1 Create Files

The object store secret allows SAP AI Core to access your cloud storage and data without exposing your compromising your credentials.

⚠ Restriction

The `objectStore` `name`, `data` `path` and `scenarioId` refer to pre-existing values. For the `objectStore` `name` and `data` `path` values, you must use the values that you used when registering the object storage, following the naming convention outlined in the diagram below. In example output code blocks, these values are represented by `ai://default/data`.



Using Postman

1. Create a new **POST** request using URL `{{apiurl}}/v2/lm/artifacts`
2. Toggle the body tab, and enter the following JSON:

```
{
```

```
{
  "name": "name of artifact",
  "kind": "dataset",
  "url": "ai://<objectStore name>/<data path>",
  "description": "<description of artifact>",
  "scenarioId": "<scenarioID>"
}
```

The response body contains the ID of your new artifact.

```
{
  "id": "3x4mpl3-651c-4f3e-8e1d-81a408041bc1",
  "message": "Artifact acknowledged",
  "url": "ai://default/data"
}
```

Using curl

```
curl --location --request POST "$API_URL/v2/lm/artifacts" \
--header "Authorization: Bearer $TOKEN" \
--header "Content-Type: application/json" \
--header "AI-Resource-Group: <Resource group>" \
--data-raw '{
  "name": "name of artifact",
  "kind": "dataset",
  "url": "ai://<objectStore name>/<data path>",
  "description": "<description of artifact>",
  "scenarioId": "<scenarioID>"
}
```

The response body contains the ID of your new artifact.

```
{
  "id": "3x4mpl3-651c-4f3e-8e1d-81a408041bc1",
  "message": "Artifact acknowledged",
  "url": "ai://default/data"
}
```

Parent topic: [Manage Files \[page 152\]](#)

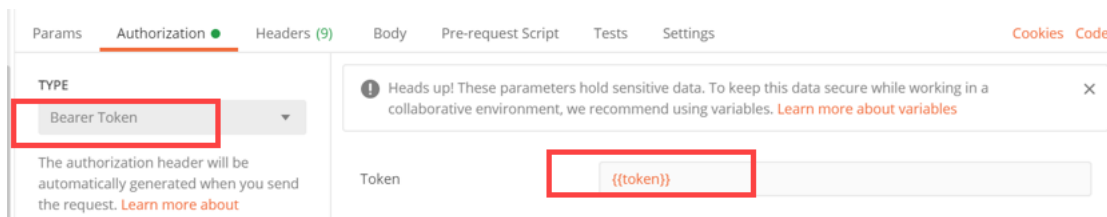
Related Information

[List Files \[page 155\]](#)

9.1.1.1.2 List Files

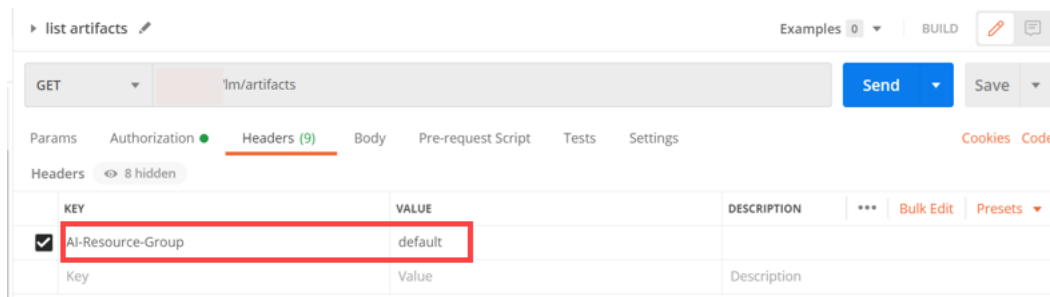
Using Postman

1. Send a GET request to the endpoint `{{apiurl}}/v2/lm/artifacts`
2. On the *Authorization* tab, set the type to *Bearer Token*.
3. Set the token value to `{{token}}`.



4. On the *Header* tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)



5. Send the request.

Using curl

```
curl --request GET "$AI_API_URL/v2/lm/artifacts" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "count": 3,
  "resources": [
    {
```

```

    "createdAt": "2021-02-09T08:08:12Z",
    "description": "",
    "executionId": "d44edae36c187cf6",
    "id": "3088b75f-5448-4c19-8055-392668a043ec",
    "kind": "model",
    "modifiedAt": "2021-02-09T08:08:12Z",
    "name": "pytf-model",
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "url": "ai://default/d44edae36c187cf6/pytf-model"
  },
  {
    "createdAt": "2021-02-09T07:56:37Z",
    "description": "",
    "executionId": "d44edae36c187cf6",
    "id": "38f7a46b-454d-4543-9457-b1eede5036f8",
    "kind": "model",
    "modifiedAt": "2021-02-09T07:56:37Z",
    "name": "churn-pickle",
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "url": "ai://default/d44edae36c187cf6/churn-pickle"
  },
  {
    "createdAt": "2021-02-07T16:07:16Z",
    "description": "Churn and Text Classifier Dataset",
    "id": "b45265f2-9bc3-441a-a0e1-fac1438acb79",
    "kind": "dataset",
    "modifiedAt": "2021-02-07T16:07:16Z",
    "name": "pytf",
    "scenarioId": "84fe6957-1145-4183-b682-8f11ca56d060",
    "url": "ai://default/pytf/"
  }
]
}

```

Parent topic: [Manage Files \[page 152\]](#)

Related Information

[Create Files \[page 153\]](#)

9.1.1.2 Manage Files Using the Dataset API

Where direct access to files in the object store is not possible or desirable (for example, in Content as a Service Scenarios, where the Service Consumers might not be the owners of the object store) you can upload, download, and delete files from the pre-registered object store using the SAP AI Core Dataset API.

For full details on the Dataset API specification, see [SAP AI Core API documentation](#) .

Prerequisites

- You must have an object store secret defined in the target resource group.

⚠ Restriction

Only S3 object stores can be used with the Dataset API.

⚠ Restriction

Only csv type files can be uploaded with the Dataset API.

[Create Files \[page 157\]](#)

[Download Files \[page 158\]](#)

[Delete Files \[page 159\]](#)

Parent topic: [Connect Your Data \[page 152\]](#)

Related Information

[Manage Files \[page 152\]](#)

9.1.1.2.1 Create Files

Using Postman

1. Send a PUT request to the endpoint `{{apiurl}}/v2/lm/dataset/files/{secret name}/{full file path}`
2. In the header set the Content-Type as `<text/csv>`.
3. If you use a service provider token, specify the resource group.
If you use a service consumer token, the resource group information is contained in the token, and need not be specified.
4. In the request body, submit the file as binary data.

Results

Your file will be uploaded to the S3 storage bucket with the prefix specified in the object store secret, and the full file path specified in the upload request.

Using curl

```
curl --location --request PUT "$AI_API_URL/v2/lm/dataset/files/$SECRET_NAME/$FILE_PATH" \\  
  --header "Authorization: Bearer $TOKEN" \\  
  --header "Content-Type: text/csv" \\  
  --header "ai-resource-group: $RESOURCE_GROUP" \\  
  --data @$FILE_LOCATION
```

- If you use a service provider token, specify the resource group.
If you use a service consumer token, the resource group information is contained in the token, and need not be specified.
- In the request body, submit the file as binary data.

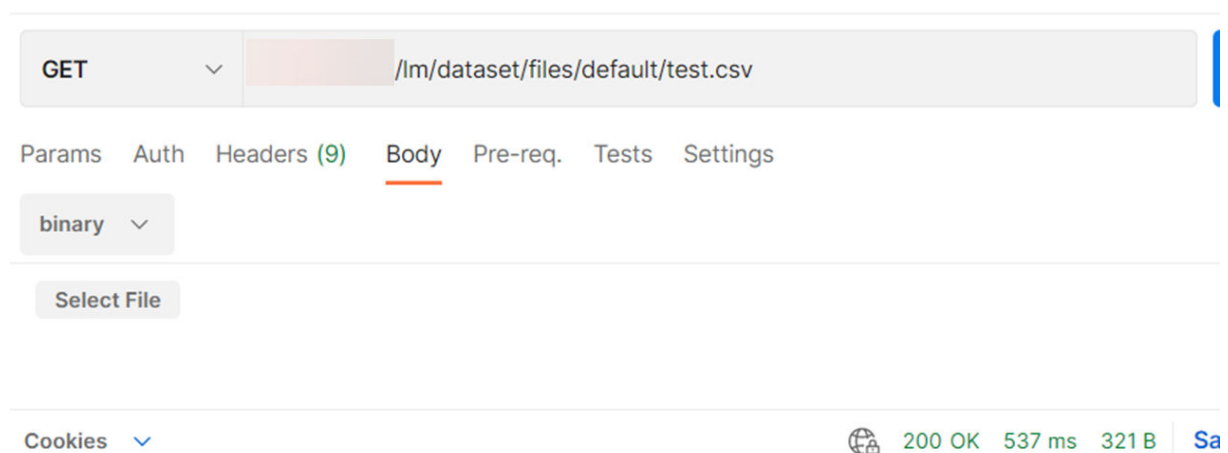
Results

Your file will be uploaded to the S3 storage bucket with the prefix specified in the object store secret, and the full file path specified in the upload request.

9.1.1.2.2 Download Files

Using Postman

Send a GET request to the endpoint `{{apiurl}}/v2/lm/dataset/files/{secret name}/{full file path}`



Using Curl

```
curl --location --request GET "$AI_API_URL/v2/lm/dataset/files/$SECRET_NAME/$FILE_PATH" \\  
  --header "Authorization: Bearer $TOKEN" \\  
  --header "ai-resource-group: $RESOURCE_GROUP" \\  
  --data @$FILE_LOCATION
```

9.1.1.2.3 Delete Files

Using Postman

Send a DELETE request to the endpoint `{{apiurl}}/v2/lm/dataset/files/{secret name}/{full file path}`

DELETE

/lm/dataset/files/default/test.csv

Params

Authorization

Headers (9)

Body

Pre-request Script

Tests

Settings

Headers

6 hidden

	Key	Value
☒	AI-Resource-Group	{{resource-group}}
☒	Authorization	{{token}}
☐	Content-Type	text/csv
	Key	Value

Using Curl

```
curl --location --request DELETE "$AI_API_URL/v2/lm/dataset/files/$SECRET_NAME/$FILE_PATH" \\  
  --header "Authorization: Bearer $TOKEN" \\  
  --header "ai-resource-group: $RESOURCE_GROUP"
```

9.1.2 Train Your Model

You execute a training workflow to train your AI learning model.

SAP AI Core uses data as specified by your resource group. Models are specified in your workflow and are trained on a remote server. The resulting artifacts are stored to your object store.

The execution service includes:

- Batch jobs/pipelines via a state-of-the-art workflow engine.
- Parameterized templates for pipelines stored and managed at tenant level.
- Pipeline instances with specific parameters and resource isolation.
- Support for bringing own Docker registries and Docker images for running pipeline steps.
- Data extraction and validation modeled as pipeline steps.
- Dedicated object store buckets used as data storage. A default bucket is provided per resource group.
- Bring your own hyperscaler-backed object store buckets.

The execution engine in SAP AI Core leverages the [Argo Workflows](#)  open source project. It supports container-native workflows and pipelines modeled as direct acyclic graphs or steps. The Argo Workflows are used to ingest data, perform preprocessing and postprocessing, and train models and execute batch inference pipelines. SAP AI Core also leverages the parallel processing of steps in the form of a [DAG \(Directed Acyclic Graph\) structure](#)  in workflow templates. For information about how using parallel nodes may affect your costs, see [Metering and Pricing for SAP AI Core \[page 40\]](#).

Note

Argo Workflow isn't optimized for time critical tasks. Each step must be scheduled onto a node in the cluster, and the cluster initialized. The time this takes depends on the load of the workflow controller and the node availability in the cluster. Therefore, it isn't recommended to use multistep Argo Workflows for time-critical tasks.

It's possible to train the same model multiple times, with varying parameters (for parameters compatible with the workflow, only) and evaluate them in SAP AI Launchpad.

[Choose a Resource Plan \[page 161\]](#)

You can configure SAP AI Core to use different infrastructure resources for different tasks, based on demand. SAP AI Core provides several preconfigured infrastructure bundles called "resource plans" for this purpose.

[Workflow Templates \[page 163\]](#)

Here, you'll find a basic workflow example template. Feel free to adjust it to suit your workflow needs.

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

Artifact signatures in the form of a hash can be added to output artifacts from executions.

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

Discover features of the SAP AI Core runtime that improve efficiency and help manage resource consumption.

[Retrieve Execution Logs \[page 199\]](#)

accessed in the deployment and execution logs.

[Training Schedules \[page 202\]](#)

Parent topic: [ML Operations \[page 151\]](#)

Related Information

[Connect Your Data \[page 152\]](#)

[Use Your Model \[page 210\]](#)

9.1.2.1 Choose a Resource Plan

You can configure SAP AI Core to use different infrastructure resources for different tasks, based on demand. SAP AI Core provides several preconfigured infrastructure bundles called “resource plans” for this purpose.

Context

Resource plans are used to select resources in workflow and serving templates. Different steps of a workflow can have different resource plans.

In general, if your workload needs GPU acceleration, you should use one of the GPU-enabled resource plans. Otherwise, choose a resource plan based on the anticipated CPU and memory need of your workloads.

Within SAP AI Core, the resource plan is selected via the `ai.sap.com/resourcePlan` label at pod level. It maps the selected resource plan and takes a string value, which can be any of the following resource plan IDs:

Resource Plan Specifications for AWS

Resource Plan IDs	GPUs	CPU Cores	Memory GBs	Code to Allocate Re- sources
				in Workflow Tem- plates
Starter	–	1	3	<code>ai.sap.com/ resourcePlan: starter</code>
Basic	–	3	11	<code>ai.sap.com/ resourcePlan: basic</code>
Basic-8x	–	31	116	<code>ai.sap.com/ resourcePlan: basic.8x</code>

Resource Plan IDs	GPUs	CPU Cores	Memory GBs	Code to Allocate Resources in Workflow Templates
Train-L	1 V100	7	55	<code>ai.sap.com/ resourcePlan: train.l</code>
Infer-S	1 T4	3	10	<code>ai.sap.com/ resourcePlan: infer.s</code>
Infer-M	1 T4	7	26	<code>ai.sap.com/ resourcePlan: infer.m</code>
Infer-L	1 T4	15	58	<code>ai.sap.com/ resourcePlan: infer.l</code>

⚠ Restriction

For the Free Tier service plan, only the Starter resource plan is available. Specifying other plans will result in error. For the Standard service plan, all resource plans are available. For more information, see [Free Tier \[page 38\]](#) and [Service Plans \[page 36\]](#).

📌 Note

There are limits to the default disk storage size for all of these nodes. Datasets that are loaded to the nodes will consume disk space. If you have large data sets (larger than 30 GB), or have large models, you may have to increase the disk size. To do so, use the persistent volume claim in Argo Workflows to specify the required disk size (see [Volumes](#) ).

Task overview: [Train Your Model \[page 159\]](#)

Related Information

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

Service Usage Reporting

Usage consumption of services is reported in the SAP BTP cockpit on the [Overview](#) page for your global account and on the [Overview](#) and [Usage Analytics](#) pages of your subaccount. The usage report lists usage in billable measures and non-billable measures. Your final monthly bill is based on the billable measure only. Non-billable measures are displayed for reporting purposes only.

9.1.2.2 Workflow Templates

Here, you'll find a basic workflow example template. Feel free to adjust it to suit your workflow needs.

Workflow templates help manage your training pipelines at the main tenant level. You can store these templates in your git repository and version them as needed. SAP AI Core executes workflows using the Argo Workflows open source project. Argo Workflows is an open-source, container-native workflow engine that orchestrates parallel jobs on Kubernetes. It's implemented as a Kubernetes CRD (Custom Resource Definition).

Workflow templates act as **executables**. To map them, you need certain attributes in your template's **metadata** section. The AI API uses the **annotations** and **labels** in the template to find scenarios and executables.

Workflow templates are built on the Argo Workflows engine and are defined as **WorkflowTemplates**. These are your cluster's workflow definitions. For more information about **WorkflowTemplates** and sample workflows, see the Argo documentation.

In SAP AI Core, Argo Workflows are used to:

- Train models
- Ingest data
- Preprocess and postprocess data
- Execute batch inference pipelines

Workflows are executed in batch mode. A workflow can produce multiple output artifacts, but only an output artifact with a **globalName** is considered to be the final output artifact of the workflow.

For the model training code, SAP AI Core is language-agnostic. However, you need to specify the relevant programming language in the workflow parameters. If you're importing any packages, list them in a separate file named **requirements.txt** and store it in the same directory.

⚠ Restriction

The maximum number of workflow templates is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some workflow templates. Alternatively, raise a ticket to increase your quota.

To get started, copy the generic workflow template below and add your own values as required. You can use any text editor with a YAML plugin to create your templates. Workflows support the following parameters:

Workflow Template Parameters

Type	Parameter	Description
name (mandatory)	–	The executable ID. The executable ID must be unique among all executables available within the SAP AI Core main tenant.
labels (mandatory)	<code>ai.sap.com/resourcePlan</code>	You must specify the chosen resourcePlan . The value is the string value of the selected resource plan. For more information, see Choose a Resource Plan [page 161] .
annotations	<code>scenarios.ai.sap.com/description</code> (optional)	A description of the scenario to which this executable belongs.
	<code>scenarios.ai.sap.com/name</code> (mandatory)	The scenario name, which is necessary for the AI API to discover a scenario.
	<code>executables.ai.sap.com/description</code> (optional)	The description of an executable.
	<code>executables.ai.sap.com/name</code> (mandatory)	The name of the executable.
	<code>artifacts.ai.sap.com/<argo_artifact_name>.suffix</code>	A suffix added to the file or folder of the output artifact on the object store.
	<code>artifacts.ai.sap.com/<argo_artifact_name>.kind</code>	The type of output artifact (for example, a dataset or model).
	<code>artifacts.ai.sap.com/<argo_artifact_name>.name</code>	The name under which the output artifact is registered in the AI API.
	<code>artifacts.ai.sap.com/<argo_artifact_name>.description</code> (optional)	You can add further metadata for an artifact using these annotations.
labels	<code>artifacts.ai.sap.com/<argo_artifact_name>.labels: {"ext.ai.sap.com/customkey1":"customvalue1", "ext.ai.sap.com/customkey2":"customvalue2"} (optional)</code>	
	<code>scenarios.ai.sap.com/id</code> (mandatory)	The scenario ID to which the workflow template belongs.

Type	Parameter	Description
	ai.sap.com/version (mandatory)	The compatibility version of this executable. You can use the compatibility versions to offer executables in different variants for different AI service consumers.

Note

In the artifact-related parameters above, <argo_artifact_name> refers to the globalName of an output artifact.

Generic Workflow Template

```

apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: text-clf-train-tutorial
  annotations:
    scenarios.ai.sap.com/description: "SAP developers tutorial scenario"
    scenarios.ai.sap.com/name: "text-clf-tutorial-scenario"
    executables.ai.sap.com/description: "Text classification Scikit training
executable"
    executables.ai.sap.com/name: "text-clf-train-tutorial-exec"
    artifacts.ai.sap.com/text-data.kind: "dataset"
    artifacts.ai.sap.com/text-model-tutorial.kind: "model"
    artifacts.ai.sap.com/text-model-tutorial.description: "artifact description"
    artifacts.ai.sap.com/text-model-tutorial.labels: |
      {"ext.ai.sap.com/customkey1":"customvalue1", "ext.ai.sap.com/
customkey2":"customvalue2"}
  labels:
    scenarios.ai.sap.com/id: "text-clf-tutorial"
    executables.ai.sap.com/id: "text-clf-train-tutorial"
    ai.sap.com/version: "1.0.0"
spec:
  imagePullSecrets:
    - name: <name of your Docker registry secret>
  entrypoint: text-clf-sk-training
  arguments:
    parameters: # placeholder for string like inputs
      - name: DEPTH # identifier local to this workflow
        description: description of the parameter
        default: test
  templates:
    - name: text-clf-sk-training
      metadata:
        labels:
          ai.sap.com/resourcePlan: starter
      inputs:
        artifacts:
          - name: text-data
            path: /app/data/
      outputs:
        artifacts:
          - name: text-model-tutorial
            path: /app/model
            globalName: text-model-tutorial

```

```

        archive:
            none: {}
    container:
        image: "<DOCKER IMAGE URL GOES HERE>"
        imagePullPolicy: Always
        command: ["/bin/sh", "-c"]
        args:
            - >
              set -e && echo "---Start Training---" && python /app/src/
train_scikit.py && ls -lR /app/model && echo "---End Training---"

```

📌 Note

For every container in the template, the `command: ["/bin/sh", "-c"]` field is mandatory. The contents of the argument can be amended, but must not be empty. The `CMD` and `ENDPOINT` specified in the Dockerfile of a container are ignored.

User ID and group ID 65534 is required to run the Docker image. This user has permission to access the files while the application is running. You can check and change the permissions by using the `chown` and `chmod` commands.

→ Tip

To make sure that the container is working as expected before submitting it to SAP AI Core, run `docker run -it --user 65534:65543 <docker-image>` locally.

📌 Note

The `archive: none: {}` option in the `outputs artifacts` sections disables automatic archiving for the artifact. If archiving is enabled, the output artifacts are archived to a `tar-gzip` file before they are uploaded to the object store. If archiving is disabled through `archive: none: {}`, the artifact will be uploaded to the object store in its current format. If the output artifact points to a directory, the directory will be uploaded "as is" to the object store. However, object stores in SAP HANA Cloud, data lake do not support this. In this case, remove `archive: none: {}` and archive the directory to a single `tar-gzip` file before it is uploaded to the object store.

📌 Note

When multiple containers are defined in a single WorkflowTemplate, for example when using Sidecar or Container Set, one of the containers must be named ``main``. To fetch logs of another container in the same template using the a GET request to the endpoint `/logs`, the name of the container must have the ``readable-`` prefix. The execution can still run without the ``readable-`` prefix, but logs will be inaccessible through the endpoint.. For more information, see [Argo Sidecar](#) 🐘 and [Argo Container Set](#) 🐘 .

Sync an Application Manually

Applications sync with your GitHub repository automatically at intervals of ~3 minutes. Use the endpoint below to manually request a sync:

```
{{apiurl}}/admin/applications/{{appName}}/refresh
```

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

Stop or Delete Multiple Training Instances

The feature is set to false by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

Related Information

[Argo Workflows](#) ➔

[Register Your Docker Registry Secret \[page 87\]](#)

[Stop Multiple Training Instances \[page 191\]](#)

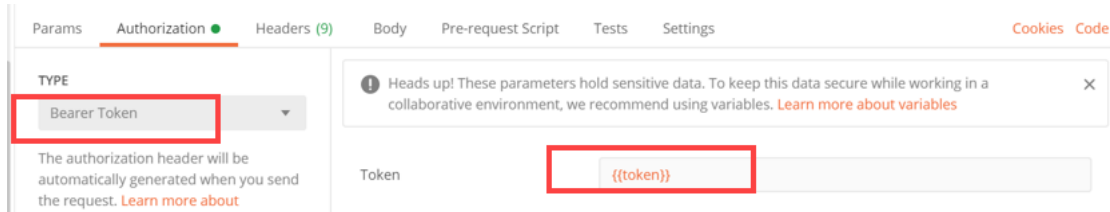
[Delete Multiple Training Instances \[page 195\]](#)

9.1.2.3 List Scenarios

A scenario is a group of related executables for a use case within the user's tenant. A scenario can have multiple versions that further correspond to the different versions of executables.

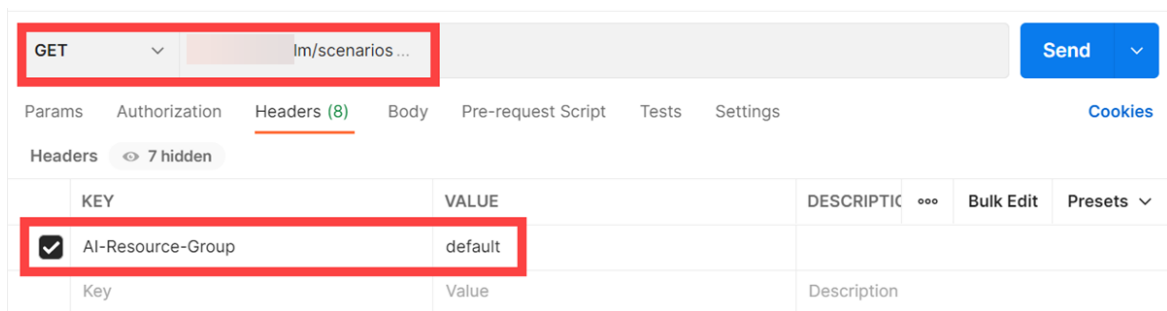
List Scenarios with Postman

1. Send GET request to the endpoint `{{apiurl}}/v2/lm/scenarios`
2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.

A screenshot of the Postman interface showing the 'Authorization' tab. The 'TYPE' dropdown is set to 'Bearer Token'. The 'Token' field contains the value '{{token}}'. A warning message at the top states: 'Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. Learn more about variables'.

4. On the [Header](#) tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)

A screenshot of the Postman interface showing the 'Headers' tab. The 'GET' method and 'lm/scenarios...' endpoint are visible. A table of headers is shown with one entry: 'AI-Resource-Group' with a value of 'default'. The table has columns for KEY, VALUE, DESCRIPTION, Bulk Edit, and Presets.

5. Send the request.

Note

Only scenarios that have a defined training executable can be created. When a new scenario ID is specified in the workflow template for a training executable, a new scenario will be created along with the training executable.

For now, a new scenario with only a deployment executable cannot be created. A possible work-around is to create a scenario with a dummy training executable and then use the same scenario ID in the serving template.

List Scenarios with curl

```
curl --request GET "" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```


Output Code

```
{
  "count":2,
  "resources":[
    {
      "createdAt":"2021-02-03T18:38:32+00:00",
      "description":"churn and text class scenario desc",
      "id":"84fe6957-1145-4183-b682-8f11ca56d060",
      "labels":[
      ],
      "modifiedAt":"2021-02-04T11:14:02+00:00",
      "name":"churntextclassscenname"
    },
    {
      "createdAt":"2021-02-04T14:11:02+00:00",
      "description":"churn and text class scenario desc",
      "id":"ae0bd260-41ef-4162-81b0-861bd78a8516",
      "labels":[
      ],
      "modifiedAt":"2021-02-09T07:35:03+00:00",
      "name":"churntextclassscenname"
    }
  ]
}
```

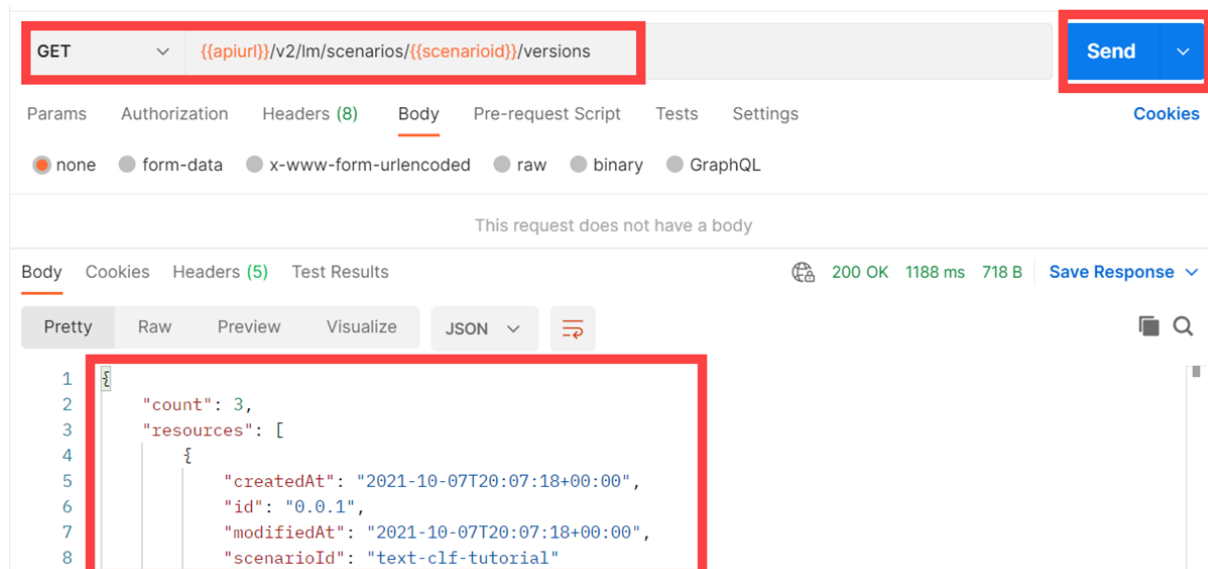
Note

Only scenarios that have a defined training executable can be created. When a new scenario ID is specified in the workflow template for a training executable, a new scenario will be created along with the training executable.

For now, a new scenario with only a deployment executable cannot be created. A possible work-around is to create a scenario with a dummy training executable and then use the same scenario ID in the serving template.

Get Scenario Versions with Postman

Create a new GET request and enter the URL `{{apiurl}}/v2/lm/scenarios/{{scenarioid}}/versions`



Get Scenario Versions with curl

Submit the following:

```
curl --location --request GET '$API_URL/v2/lm/scenarios/$SCENARIO_ID/versions' \
```

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.4 List Executables

An executable is a template that is instantiated for a purpose, such as training a model or creating a deployment. You can list all of the executables in a scenario and get details of specific executables from a scenario. Workflow templates are mapped to training executables.

List Executables with Postman

1. Define your scenario ID as the `scenarioid` environment variable.

Variable	Value
client_secret	xxx
token	xxx
apiurl	
scenarioid	dba85cf3-2d69-498d-... <give your scenario id here>
configurationid	ecff88fb-0c50-4fc3-84... ecff88fb-0c50-4fc3-8439-f87f25e0ba08
executionid	dbba7904390cda65 dbba7904390cda65
variable_key	

Use variables to reuse values in different places. Work with the current value of a variable to prevent sharing sensitive values with your team. [Learn more about variable values](#)

Cancel Update

2. Send a GET request to the endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioid}}/executables`
3. On the [Authorization](#) tab, set the type to [Bearer Token](#).
4. Set the token value to `{{token}}`.

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Code

TYPE: Bearer Token

The authorization header will be automatically generated when you send the request. [Learn more about](#)

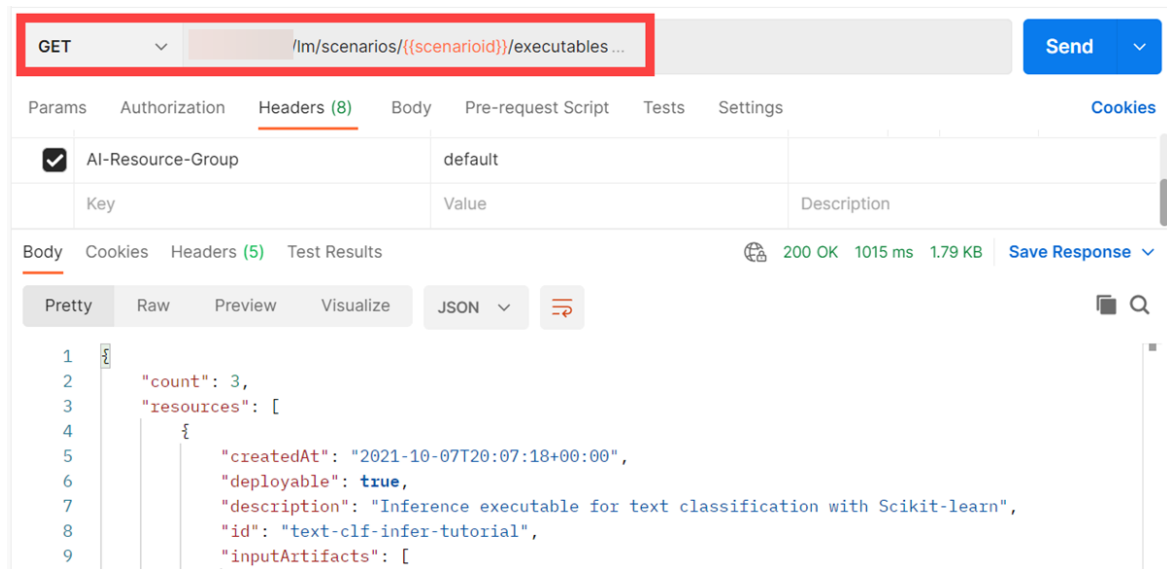
Token: {{token}}

Heads up! These parameters hold sensitive data. To keep this data secure while working in a collaborative environment, we recommend using variables. [Learn more about variables](#)

5. On the [Header](#) tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup>default is used)

6. Send the request.



Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

List Executables with curl

```
curl --request GET "$AI_API_URL/v2/lm/scenarios" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```

{
  "count":4,
  "resources":[
    {
      "createdAt":"2021-02-04T13:11:01+00:00",
      "deployable":true,
      "description":"churn n text class serving executable desc",
      "id":"pytf-serving",
      "inputArtifacts":[
        {
          "name":"model_uri"
        }
      ],
      "labels":[
      ],
      "modifiedAt":"2021-02-04T13:11:01+00:00",
      "name":"churntextclassexecname",
      "parameters":[
        {
          "name":"modelName",
          "type":"string",
          "default": "value",
          "description": "description of the parameter"
        }
      ]
    }
  ]
}
```

```

    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  },
  {
    "createdAt": "2021-02-09T07:35:02+00:00",
    "deployable": true,
    "description": "churn n text class serving executable desc",
    "id": "pytf-serving-tracking",
    "inputArtifacts": [
      {
        "name": "textmodel",
        "kind": "model",
        "description": "artifact description",
        "labels": [
          {
            "key": "ext.ai.sap.com/customkey1",
            "value": "customvalue1"
          },
          {
            "key": "ext.ai.sap.com/customkey2",
            "value": "customvalue2"
          }
        ]
      }
    ],
    "labels": [
      ],
    "modifiedAt": "2021-02-09T07:35:02+00:00",
    "name": "churntextclassexecname",
    "parameters": [
      {
        "name": "modelName",
        "type": "string"
      }
    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  },
  {
    "createdAt": "2021-02-09T07:35:03+00:00",
    "deployable": false,
    "description": "churn and text class executable desc",
    "id": "pytf-training-tracking",
    "inputArtifacts": [
      {
        "name": "churn-data"
      },
      {
        "name": "textclass-data"
      }
    ],
    "labels": [
      ],
    "modifiedAt": "2021-02-09T07:35:03+00:00",
    "name": "churnntextclassexecutablename",
    "outputArtifacts": [
      {
        "name": "churn-pickle",
        "kind": "model",
        "description": "artifact description",
        "labels": [
          {
            "key": "ext.ai.sap.com/customkey1",
            "value": "customvalue1"
          }
        ]
      }
    ],
  },

```

```

        {
          "key": "ext.ai.sap.com/customkey2",
          "value": "customvalue2"
        }
      ],
      {
        "name": "pytf-model"
      }
    ],
    "parameters": [
      {
        "name": "train-epoch",
        "type": "string"
      }
    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  },
  {
    "createdAt": "2021-02-04T14:11:02+00:00",
    "deployable": false,
    "description": "churn and text class executable desc",
    "id": "test-training",
    "inputArtifacts": [
      {
        "name": "churn-data"
      },
      {
        "name": "textclass-data"
      }
    ],
    "labels": [
      ],
    "modifiedAt": "2021-02-04T14:11:02+00:00",
    "name": "churnntextclassexecutablename",
    "outputArtifacts": [
      {
        "name": "churn-pickle"
      },
      {
        "name": "pytf-model"
      }
    ],
    "parameters": [
      {
        "name": "train-epoch",
        "type": "string"
      }
    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  }
]
}

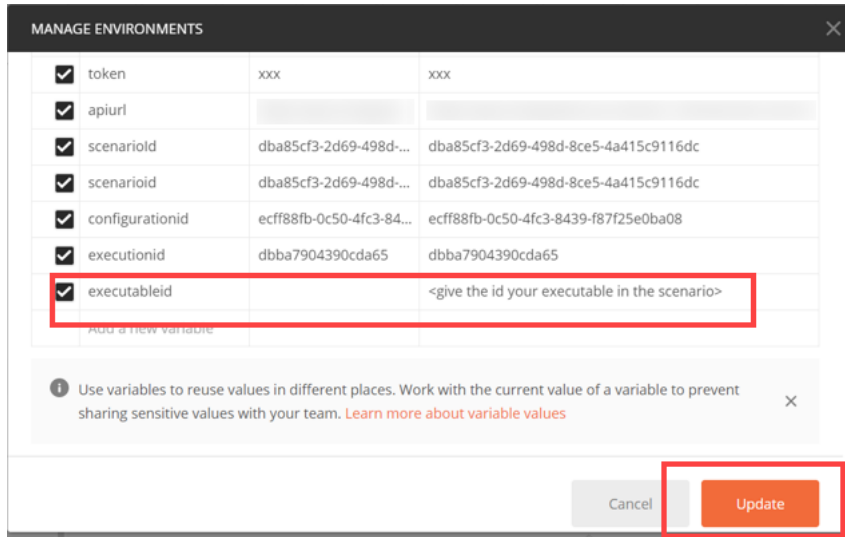
```

📌 Note

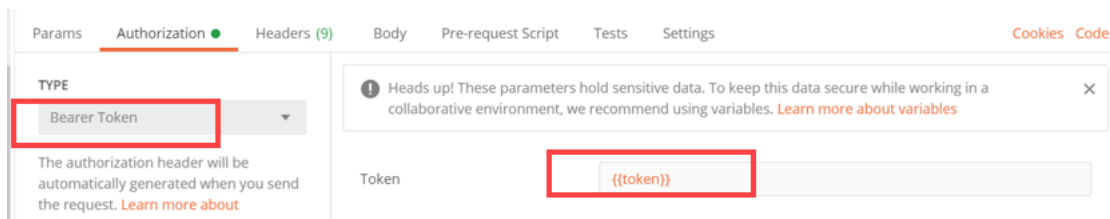
The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

Get Executable Details with Postman

1. Add the environment variable `executableid` and as its value, enter the ID of the executable.



2. Send a GET request to the endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/executables/{{executableid}}`
3. On the *Authorization* tab, set the type to *Bearer Token*.
4. Set the token value to `{{token}}`.



5. On the *Header* tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)

6. Send the request.

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

Get Executable Details with curl

```
curl --request GET "$AI_API_URL/v2/lm/scenarios" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "createdAt":"2021-02-04T14:11:02+00:00",
  "deployable":false,
  "description":"churn and text class executable desc",
  "id":"test-training",
  "inputArtifacts":[
    {
      "name":"churn-data"
    },
    {
      "name":"textclass-data"
    }
  ],
  "labels":[
  ],
  "modifiedAt":"2021-02-04T14:11:02+00:00",
  "name":"churnntextclassexecutable",
  "outputArtifacts":[
    {
      "name":"churn-pickle"
    },
    {
      "name":"pytf-model"
    }
  ],
  "parameters":[
    {
      "name":"train-epoch",
      "type":"string"
    }
  ],
  "scenarioId":"ae0bd260-41ef-4162-81b0-861bd78a8516",
  "versionId":"0.0.1"
}
```

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output 1970-01-01T00:00:00+00:00 indicates an error.

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

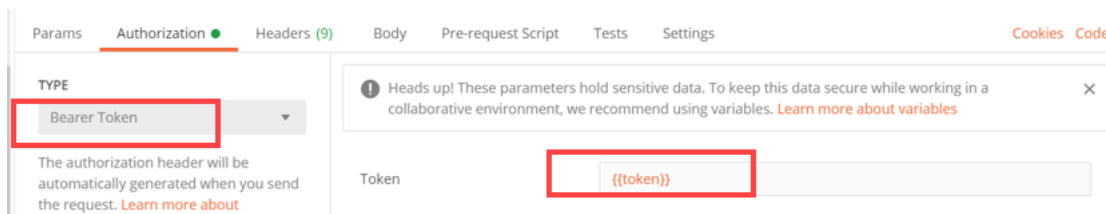
[List Scenarios \[page 167\]](#)
[Create Configurations \[page 177\]](#)
[List Configurations \[page 179\]](#)
[Using Artifact Signatures \[page 182\]](#)
[Start Training \[page 186\]](#)
[Stop Training Instances \[page 189\]](#)
[Delete Training Instances \[page 193\]](#)
[Efficiency Features \[page 196\]](#)
[Retrieve Execution Logs \[page 199\]](#)
[Training Schedules \[page 202\]](#)

9.1.2.5 Create Configurations

A configuration is a collection of parameters, artifact references, and executables that are used to run an execution or deployment.

Using Postman

1. Send a POST request to the endpoint `{{apiurl}}/v2/lm/configurations`
2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.



4. On the [Header](#) tab, add the following entries:

KEY	VALUE
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)
Content-Type	application/json

5. On the [Body](#) tab, select the [raw](#) radio button and add the request body as given below:

```
{
  "name": "configuration-name",
  "executableId": "<executable ID>",
  "scenarioId": "<scenario ID>",
  "parameterBindings": [
    {
```

```

    "key": "<parameter name>",
    "value": "<value>"
  },
  "inputArtifactBindings": [
    {
      "key": "<artifact name>",
      "artifactId": "<artifact ID>"
    }
  ]
}

```

POST /v2/lm/configurations

Params Authorization Headers (11) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL **JSON** ▾

```

10  ....}
11  ..],
12  .."inputArtifactBindings":..[
13  .....{
14  .....  "key": "churn-data",
15  .....  "artifactId": "896e9230-9219-47fa-89c7-c47db7d45d6a"
16  .....}
17  ..]

```

Body Cookies Headers (7) Test Results 🌐 201 Created 663 ms 511 B

Pretty Raw Preview Visualize **JSON** ▾ 

```

1  {
2  |   "id": "47b3eed9-f72f-4a18-b2a5-25b057a3e77f",
3  |   "message": "Configuration created"
4  | }

```

6. Send the request.

Using curl

```

curl --request POST "$AI_API_URL/v2/lm/configurations" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP" --header "Content-Type: application/json" \
-d '{
  "name": "dummy-configuration",
  "executableId": "'"$EXECUTABLE"'",
  "scenarioId": "'"$SCENARIO"'",
  "parameterBindings": [
    {
      "key": "parameter_name_in_template",
      "value": "some_value"
    }
  ],
  "inputArtifactBindings": [
    {
      "key": "input_artifact_name_in_template",
      "artifactId": "a4d62a76-52aa-44cf-a789-743246d6d55b"
    }
  ]
}'

```

```
}',  
}
```

Output Code

```
{  
  "id": "f5bf305f-7c3f-4882-9f6b-8b95e3687b9b",  
  "message": "Configuration created"  
}
```

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

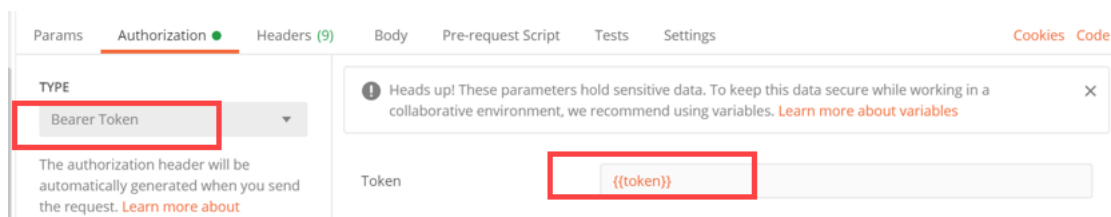
[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.6 List Configurations

Using Postman

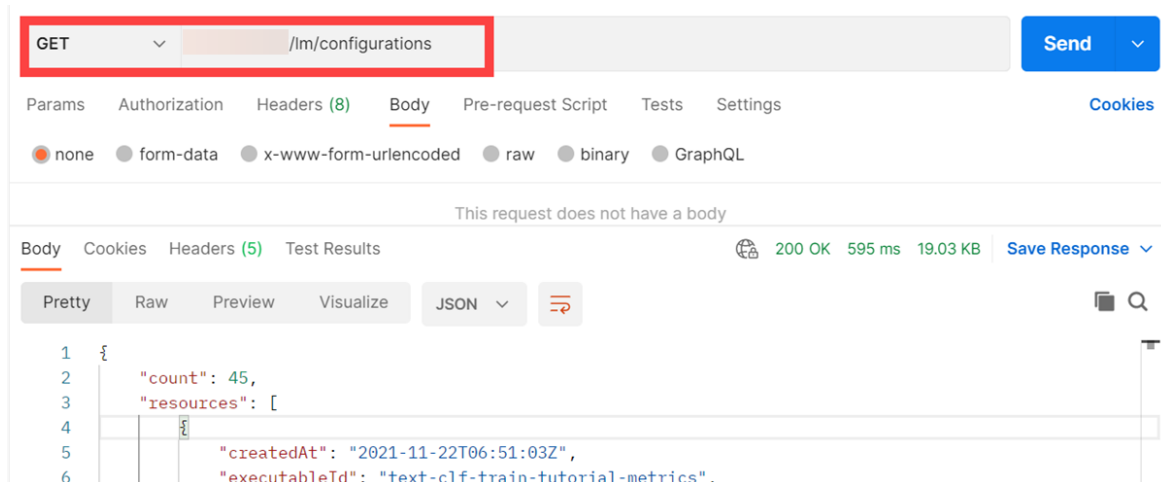
1. Send a GET request to the endpoint `{{apiurl}}/v2/lm/configurations`
2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.



- On the [Header](#) tab, add the following entry:

KEY	VALUE
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)

- Send the request.



Using curl

```
curl --request GET "$AI_API_URL/v2/lm/configurations" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "count": 2,
  "resources": [
    {
      "createdAt": "2021-02-04T11:50:45Z",
      "executableId": "pytf-training",
      "id": "1e6c2a5f-eabe-49a2-88e7-887145a2ef88",
      "inputArtifactBindings": [
        {
          "artifactId": "521f7f17-876e-4369-9162-09748b56d27a",
          "key": "churn-data"
        },
        {
          "artifactId": "521f7f17-876e-4369-9162-09748b56d27a",
          "key": "textclass-data"
        }
      ],
      "name": "pytf-demo-config1",
      "parameterBindings": [
        {
          "key": "train-epoch",
          "value": "100"
        }
      ]
    }
  ]
}
```

```

    ],
    "scenarioId": "84fe6957-1145-4183-b682-8f11ca56d060"
  },
  {
    "createdAt": "2021-02-04T11:59:22Z",
    "executableId": "pytf-training",
    "id": "42147d0d-5e3a-424d-a3a1-545b842379c5",
    "inputArtifactBindings": [
      {
        "artifactId": "521f7f17-876e-4369-9162-09748b56d27a",
        "key": "churn-data"
      },
      {
        "artifactId": "521f7f17-876e-4369-9162-09748b56d27a",
        "key": "textclass-data"
      }
    ],
    "name": "pytf-demo-config2",
    "parameterBindings": [
      {
        "key": "train-epoch",
        "value": "1"
      }
    ],
    "scenarioId": "84fe6957-1145-4183-b682-8f11ca56d060"
  }
]
}

```

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.7 Using Artifact Signatures

Artifact signatures in the form of a hash can be added to output artifacts from executions.

Providing Signatures for Output Artifacts

A hash is an identifier that is generated using the file contents. Hashes can therefore be used to compare two files and verify the integrity of a file. SAP AI Core doesn't create or verify the signatures, but stores them and makes them available. Artifact signatures are created and stored with the artifact.

To provide a signature for an output artifact, specify the `AICORE_ARTIFACT_SIGNATURES` output parameter in the workflow template. For more information, see [Argo Workflows Output Parameters](#). This parameter specifies a path to a file that will contain the signatures as key-value-pairs in JSON format.

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: signature-example
  annotations:
    scenarios.ai.sap.com/description: "Example for signature feature"
    scenarios.ai.sap.com/name: "signature-example-scenario"
    executables.ai.sap.com/description: "Output artifact example for signature feature"
    executables.ai.sap.com/name: "signature-example-output-artifacts"
  labels:
    scenarios.ai.sap.com/id: "signature-example-scenario"
    executables.ai.sap.com/id: "signature-example-out-art"
    ai.sap.com/version: "1.0.0"
spec:
  entrypoint: output-artifact-signatures-example
  templates:
    - name: output-artifact-signatures-example
      outputs:
        # specify an output artifact named "dummy-model"
        artifacts:
          - name: dummy-model
            path: /tmp/model
            globalName: dummy-model
            # specify an output parameter with name 'AICORE_ARTIFACT_SIGNATURES'
            # this parameter should point to a file in valid JSON format, containing
            # an object with key-value pairs, where each key is
            # the globalName of an output artifact and the value is its signature
            parameters:
              - name: AICORE_ARTIFACT_SIGNATURES # the parameter name has to be
                exactly 'AICORE_ARTIFACT_SIGNATURES'
                valueFrom:
                  path: /tmp/signatures.json # the value of the path can be set
arbitrary
  container:
    image: python:3.11-alpine
    imagePullPolicy: Always
    command: ["python", "-c"]
    args:
      - |
        import hashlib, json
        model = "a dummy string serving as output artifact"
        model_signature =
        hashlib.sha256(model.encode("utf-8")).hexdigest() # create a hash of the dummy
        model
        signatures = {
```

```

# the key is the globalName of the output artifact, the value
is its signature
    "dummy-model": model_signature
}
# write the dummy model to disk
with open("/tmp/model", "w") as f:
    f.write(model)
# write the model signatures dictionary to the path specified in
the AICORE_ARTIFACT_SIGNATURES output parameter
with open("/tmp/signatures.json", "w") as f:
    json.dump(signatures, f)
print("success")

```

The example template can be used with multiple output artifacts. For each artifact, you add another entry in the signatures object.

The template produces the output artifact `dummy-model`, with its name-signature key-value pair provided in the `AICORE_ARTIFACT_SIGNATURES` output parameter. The key is the global name of the output artifact, and the value is the signature. For example:

```

{
  "name-of-model1": "signature-of-model1"
}

```

These signatures can then be consumed in another execution.

Note

Signatures must be generated as part of the training workflow, and can't be added retrospectively.

Note

When you query the execution via the GET `{{apiurl}}/v2/lm/executions/{{executionid}}` endpoint you won't see the signatures of the output artifacts, as they're kept internally. However, you'll still be able to use them in other executions or deployments.

Multi-Step Workflows

For multistep workflow templates, the `AICORE_ARTIFACT_SIGNATURES` output parameter needs to be supplied in each step that requires an output artifact and signature to be created. See the following example:

```

apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: signature-example-2
  annotations:
    scenarios.ai.sap.com/description: "Example for signature feature"
    scenarios.ai.sap.com/name: "signature-example-scenario"
    executables.ai.sap.com/description: "Output artifact example for multi
signature feature"
    executables.ai.sap.com/name: "signature-example-multi-output-artifacts"
  labels:
    scenarios.ai.sap.com/id: "signature-example-scenario"
    executables.ai.sap.com/id: "signature-example-multi-out-art"
    ai.sap.com/version: "1.0.0"
spec:
  entrypoint: steps-example

```

```

templates:
  # a template with two steps, each having an output artifact
  - name: steps-example
    steps:
      - - output-artifact-signatures-step1
        - - output-artifact-signatures-step2
    - name: output-artifact-signatures-step1
      outputs:
        # specify an output artifact named "dummy-model1"
        artifacts:
          - name: dummy-model1
            path: /tmp/model
            globalName: dummy-model1
        # the AICORE_ARTIFACT_SIGNATURES parameter can only hold artifact
        signatures of the current step,
        # e.g. it could not hold the signature of "dummy-model2" below, because
        that is created in a different workflow step
        parameters:
          - name: AICORE_ARTIFACT_SIGNATURES
            valueFrom:
              path: /tmp/signatures.json
        container:
          image: python:3.11-alpine
          imagePullPolicy: Always
          command: ["python", "-c"]
          args:
            - >
              # ... skipped, as it will be the same as in the previous example
  - name: output-artifact-signatures-step2
    outputs:
      # specify an output artifact named "dummy-model2"
      artifacts:
        - name: dummy-model2
          path: /tmp/model
          globalName: dummy-model2
      # we need to define the AICORE_ARTIFACT_SIGNATURES parameter in each
      workflow step, where we want to provide
      # signatures for an output artifact
      parameters:
        - name: AICORE_ARTIFACT_SIGNATURES
          valueFrom:
            path: /tmp/signatures.json
      container:
        image: python:3.11-alpine
        imagePullPolicy: Always
        command: ["python", "-c"]
        args:
          - >
            # ... skipped, as it will be the same as in the previous example

```

This workflow template creates two output artifacts: `dummy-model1` and `dummy-model2`, and their name-signature key-value-pairs in JSON format.

Verifying Signatures from Input Artifacts

If you use input artifacts with signatures in another execution, SAP AI Core provides the signatures of the artifacts as an environment variable named `AICORE_ARTIFACT_SIGNATURES`. The variable is a JSON key-value pair, where each key is the input artifact name and the value is its signature. For example:

```

{
  "name-of-model1": "signature-of-model1",
  "name-of-model2": "signature-of-model2"
}

```



```
}
```

To extract the signature, include the variable `AICORE_ARTIFACT_SIGNATURES` in your workflow template, that returns the value from the key of the pair, for example:

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: signature-example-3
  annotations:
    scenarios.ai.sap.com/description: "Example for signature feature"
    scenarios.ai.sap.com/name: "signature-example-scenario"
    executables.ai.sap.com/description: "Input artifact example for signature feature"
    executables.ai.sap.com/name: "signature-example-input-artifacts"
  labels:
    scenarios.ai.sap.com/id: "signature-example-scenario"
    executables.ai.sap.com/id: "signature-example-in-art"
    ai.sap.com/version: "1.0.0"
spec:
  entrypoint: input-artifact-signatures-example
  templates:
    - name: input-artifact-signatures-example
      inputs:
        artifacts:
          - name: dummy-model
            path: /tmp/model
      container:
        image: python:3.11-alpine
        imagePullPolicy: Always
        command: ["python", "-c"]
        args:
          - |
            import hashlib, json, os
            # create a hash of the input artifact to compare against the
            provided signature
            with open("/tmp/model", "rb") as f:
                model_hash_actual = hashlib.file_digest(f,
                "sha256").hexdigest()
            # SAP AI Core will provide the artifact signature via the
            AICORE_ARTIFACT_SIGNATURES environment variable
            # it will be a string in JSON format
            signatures = json.loads(os.environ["AICORE_ARTIFACT_SIGNATURES"])
            model_hash_expected = signatures["dummy-model"] # the name of the
            input artifact is the key
            assert model_hash_expected == model_hash_actual, "signatures did
            not match"
            print("success")
```

This template takes an input artifact with the name `<dummy-model>`, calculates its signature, and compares it to the signature provided by SAP AI Core in the `AICORE_ARTIFACT_SIGNATURES` environment variable. The `AICORE_ARTIFACT_SIGNATURES` environment variable is only set when there is at least one input artifact with a signature, otherwise, it won't be set at all. In a multi-step workflow template, the `AICORE_ARTIFACT_SIGNATURES` environment variable will be supplied to each step of the workflow template, irrespective of where the actual input artifact is defined. In each step, the `AICORE_ARTIFACT_SIGNATURES` variable will hold the signatures of the input artifacts from all steps.

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.8 Start Training

Execute Training with Postman

1. Send a POST request to the endpoint `{{apiurl}}/v2/lm/executions`. Pass the `configurationId` in the request body.

POST ▼ /lm/executions

Params Authorization ● Headers (12) Body ● Pre-request Script Tests ● Se

● none ● form-data ● x-www-form-urlencoded ● **raw** ● binary ● GraphQL JSC

```

1 {
2   "configurationId": "47b3eed9-f72f-4a18-b2a5-25b057a3e77f"
3 }
4

```

Body Cookies Headers (6) Test Results 🌐 202 Ac

Pretty Raw Preview Visualize JSON ▼ ≡

```

1 {
2   "id": "edc4121d1085c20a",
3   "message": "Execution scheduled",
4   "status": "UNKNOWN",
5   "targetStatus": "COMPLETED"

```

2. Check the status of the execution by submitting a GET request to `{{apiurl}}/v2/lm/executions/{{executionid}}`.

GET ▼ /lm/executions/edc4121d1085c20a

Params Authorization Headers (11) Body ● Pre-request Script Tests Settings

Headers 👁 8 hidden

KEY	VALUE
☒ Authorization	{{token}}
☒ al-resource-group	{{resource-group}}
☒ Content-Type	application/json
Key	Value

Note

If the status is dead or pending, there might be errors in the execution. You can check the execution logs for more details, see [Retrieve Execution Logs \[page 199\]](#).

Execute Training with curl

1. Trigger the training workflow via a configuration.

```
curl --location --request POST '$AI_API_URL/v2/lm/executions' \
```

Output Code

```
{
  "id": "dea6263e6283321b",
  "message": "Execution scheduled",
  "status": "UNKNOWN",
  "targetStatus": "COMPLETED"
}
```

2. Check the status of the execution.

```
curl --request GET $AI_API_URL/v2/lm/executions/$EXECUTION \ \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP"
```

Note

If the status is dead or pending, there might be errors in the execution. You can check the execution logs for more details, see [Retrieve Execution Logs \[page 199\]](#).

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.9 Stop Training Instances

You can stop a running execution by submitting a PATCH request to `$AI_API_URL/v2/lm/executions/$EXECUTION \`.

Note

Stop is only enabled if the status is running or pending.

[Stop a Single Training Instance \[page 190\]](#)

[Stop Multiple Training Instances \[page 191\]](#)

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Delete Training Instances \[page 193\]](#)

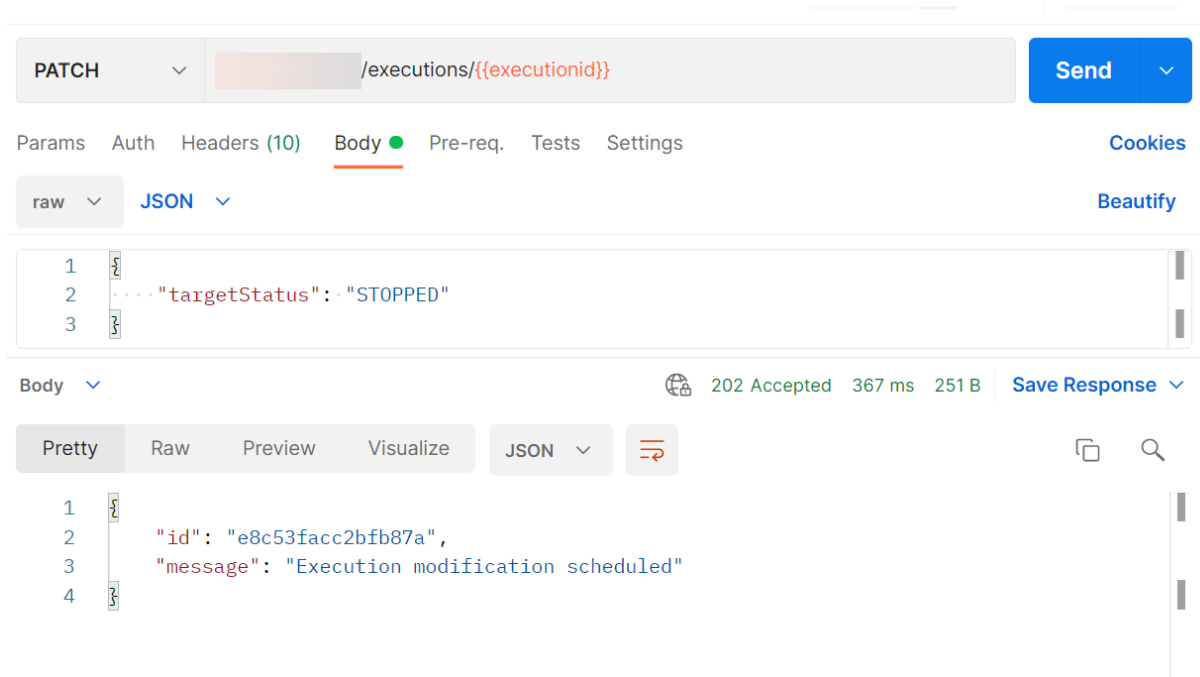
[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.9.1 Stop a Single Training Instance

Using Postman



Using curl

```
curl --request PATCH $AI_API_URL/v2/lm/executions/$EXECUTION \ --header
"Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP" --
header "Content-Type: application/json"
-d '{
  "targetStatus": "STOPPED"
}'
```

Output Code

```
{
  "id": "ee6769e4dc19c0fd",
  "message": "Execution modification scheduled"
}
```

Parent topic: [Stop Training Instances \[page 189\]](#)

Related Information

[Stop Multiple Training Instances \[page 191\]](#)

9.1.2.9.2 Stop Multiple Training Instances

`bulkUpdates` is a meta capability endpoint of the AI API. It enables or disables bulk PATCH operations. For more information, see [AI API Overview \[page 25\]](#).

The feature is set to `false` by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

About `bulkUpdates`:

- The maximum number of updates per request is 100.
- Your bulk update can contain a mixture of STOP and DELETE requests.
- Only [running](#) or [pending](#) executions or deployments can be stopped.
- Only [stopped](#), [dead](#) or [unknown](#) executions or deployments can be deleted.
- An ID can only appear once per bulk request. For multiple modifications of the same ID, multiple requests are needed.

Using Postman

Send a bulk PATCH request to the endpoint: - `/executions`

Update the request body to:

```
{
  "executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

Output Code

```
{
  "executions": [
```

```
{
  "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
  "message": "Execution modification scheduled"
},
{
  "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
  "message": "Execution modification scheduled"
}
]
```

Using curl

Update the request body to:

```
curl --request PATCH - /executions \
  --header {"executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

Output Code

```
{
  "executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "message": "Execution modification scheduled"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "message": "Execution modification scheduled"
    }
  ]
}
```

Parent topic: [Stop Training Instances \[page 189\]](#)

Related Information

[Stop a Single Training Instance \[page 190\]](#)

[AI API Overview \[page 25\]](#)

9.1.2.10 Delete Training Instances

Deleting a training instance releases the SAP AI Core resources that it used.

⚠ Restriction

If your execution is running, you must stop it first. You can stop a running execution by submitting a PATCH request to `$AI_API_URL/v2/lm/executions/$EXECUTION \`. For more information, see [Stop Training Instances \[page 189\]](#)

[Delete a Single Training Instance \[page 193\]](#)

[Delete Multiple Training Instances \[page 195\]](#)

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

9.1.2.10.1 Delete a Single Training Instance

Using curl

```
curl --request DELETE $AI_API_URL/v2/lm/executions/$EXECUTION \ \  
  --header "Authorization: Bearer $TOKEN" \ \  
  --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "id": "ee6769e4dc19c0fd",
  "message": "Deletion scheduled",
  "targetStatus": "DELETED"
}
```

Using Postman

Send a DELETE request to `{{apiurl}}/v2/lm/executions/{{executionid}}`. The header for this request is: `AI-Resource-Group: {YOUR-Resource-Group}`.

The screenshot shows the Postman interface for a DELETE request. The URL is `/lm/executions/{{executionid}}`. The 'Headers' tab is active, showing a single header: `AI-Resource-Group: default`. The 'Body' tab is also active, showing the JSON response in 'Pretty' format:

```
1 {
2   "id": "e8c53facc2bfb87a",
3   "message": "Deletion scheduled",
4   "targetStatus": "DELETED"
5 }
```

Below the response, the status is `202 Accepted`, with a response time of `402 ms` and a size of `266 B`. The 'Save Response' button is visible.

Parent topic: [Delete Training Instances \[page 193\]](#)

Related Information

[Delete Multiple Training Instances \[page 195\]](#)

9.1.2.10.2 Delete Multiple Training Instances

`bulkUpdates` is a meta capability endpoint of the AI API. It enables or disables bulk PATCH operations. For more information, see [AI API Overview \[page 25\]](#).

The feature is set to `false` by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

About `bulkUpdates`:

- The maximum number of updates per request is 100.
- Your bulk update can contain a mixture of STOP and DELETE requests.
- Only *running* or *pending* executions or deployments can be stopped.
- Only *stopped*, *dead* or *unknown* executions or deployments can be deleted.
- An ID can only appear once per bulk request. For multiple modifications of the same ID, multiple requests are needed.

Using curl

Update the request body to:

```
curl --request PATCH - /executions \
  --header {"executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

Output Code

```
{
  "executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "message": "Execution modification scheduled"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "message": "Execution modification scheduled"
    }
  ]
}
```

Using Postman

Send a bulk PATCH request to: - /executions

Update the request body to:

```
{
  "executions": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

↔ Output Code

Parent topic: [Delete Training Instances \[page 193\]](#)

Related Information

[Delete a Single Training Instance \[page 193\]](#)

[AI API Overview \[page 25\]](#)

9.1.2.11 Efficiency Features

Discover features of the SAP AI Core runtime that improve efficiency and help manage resource consumption.

Tenant Warm Node Pool

Tenant warm nodes allow tenants to reserve nodes from specific resource plan types, ensuring that a predefined number of nodes exist in the cluster. The reserved nodes can then be used during model training and serving. Reserving nodes reduces waiting time at workload startup, but incurs costs in line with node use, whether consumed or not.

Mechanism of Node Reservation

1. The tenant specifies number of nodes to reserve.
2. An execution or deployment utilizes a reserved node, a replacement reserve node of the same resource plan type is requested from the hyperscaler.

The number of reserved nodes specified by the tenant are consistently available for use.

The minimum number of reserved nodes is 0.

About Node Reservation

- The number of reserved nodes specified by the tenant are consistently available for use.
- The minimum number of reserved nodes is 0.
- The maximum number of reserved nodes is 10.
- The default number of reserved nodes is 0.

Reserve Nodes Using Postman

1. Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/resources/nodes`
2. Provide the resource plan type and quantity of nodes to reserve in the request body in JSON format:

```
{
  "resourcePlans": [
    {
      "name": "infer.l",
      "request": 1
    },
    {
      "name": "infer.m",
      "request": 1
    },
    {
      "name": "train.l",
      "request": 1
    }
  ]
}
```

Reserve Nodes Using curl

Submit a PATCH request to the endpoint `{{apiurl}}/v2/admin/resources/nodes`

```
curl --request PATCH $AI_API_URL/v2/admin/resources/nodes \
--data-raw '{
  "resourcePlans": [
    {
      "name": "infer.l",
      "request": 1
    },
    {
      "name": "infer.m",
      "request": 1
    },
    {
      "name": "train.l",
      "request": 1
    }
  ]
}'
```

→ Remember

All reserved nodes are charged at the same rate as nodes used during model training and serving.

Check Reserve Node Status Using Postman

Send a GET request to the endpoint`{{apiurl}}/v2/admin/resources/nodes`

Check Reserve Node Status Using curl

```
curl --request GET $AI_API_URL/v2/resources/nodes
```

Output Code

```
{
  "resourcePlans": {
    "infer.l": {
      "provisioned": 1,
      "requested": 1
    },
    "infer.m": {
      "provisioned": 1,
      "requested": 1
    },
    "train.l": {
      "provisioned": 1,
      "requested": 1
    }
  }
}
```

- `requested`: the number of reserve nodes requested by the tenant.
- `provisioned`: the number of reserve nodes that are currently present in the cluster.

Update Quantities of Reserved Nodes

To update the number of nodes reserved, repeat the reservation procedure, with updated quantities in the `request` field.

Delete Reserved Nodes

To delete reserved nodes, repeat the reservation procedure, with quantities in the `request` field set to `0`.

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Retrieve Execution Logs \[page 199\]](#)

[Training Schedules \[page 202\]](#)

[Service Plans \[page 36\]](#)

9.1.2.12 Retrieve Execution Logs

accessed in the deployment and execution logs.

You can retrieve the logs for a specific deployment or execution by submitting a GET request. Use the following endpoints to retrieve the logs:

- GET /v2/lm/deployments/{deploymentId}/logs
- GET /v2/lm/executions/{executionId}/logs

Query parameters include:

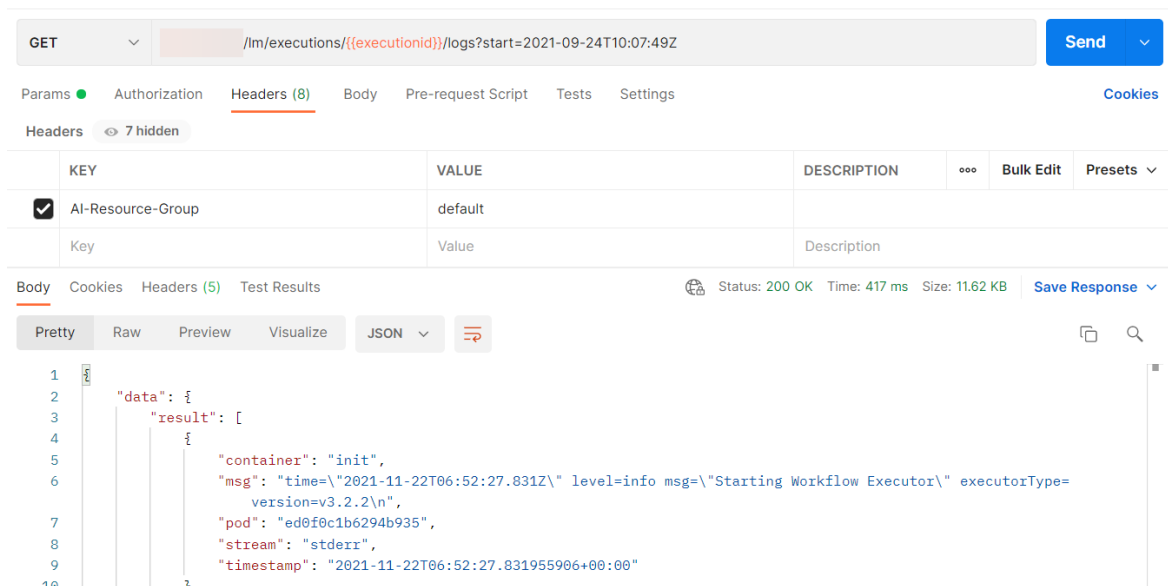
- **start** : the start time for a query as a string, in RFC 3339-compliant datetime format. Defaults to 1 hour before the current. Example: **2021-05-19T00:00:14.347Z**
- **end** : the end time for a query as a string, in RFC 3339-compliant datetime format. Defaults to the current time. Example: **2021-05-19T00:00:14.347Z**
- **\$top** : the maximum number of entries returned. The default value is 1000; the upper limit is 5000.
- **\$order** : the sort order of logs, either **asc** (for ascending, earliest in the order will appear at the top of the list) or **desc** (for descending, most recent in the order will appear at the top of the list). Note the default value is **asc**.

For example:

- /v2/lm/deployments/{deploymentId}/logs?
start=2021-05-19T00:00:14.347Z&end=2021-05-19T01:00:14.347Z&\$top=100&\$order=asc
- returns the first
100 lines of a deployment log between **2021-05-19T00:00:14.347Z** and **2021-05-19T01:00:14.347Z**
- /v2/lm/deployments/{deploymentId}/logs - returns deployment logs from the preceding hour
- /v2/lm/executions/{executionId}/logs - returns execution logs from the preceding hour

Using Postman

1. Send a GET request to the endpoint `{{apiurl}}/v2/lm/executions/{{executionId}}/logs`.



2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.
4. On the [Header](#) tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resource group> (in the example, default is used)

5. Send the request.

Using curl

```
curl --request GET "$AI_API_URL/v2/lm/executions/$EXECUTION_ID/logs?start=2021-05-19T00:00:14.347Z" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Sample Output

For example, see the following JSON output from the API.

Output Code

```

{
  "data": {
    "result": [
      {
        "container": "storage-initializer",
        "msg": "[I 210531 08:20:51 initializer-entrypoint:13]
Initializing, args: src_uri [gs://kserve-samples/models/tensorflow/flowers]
dest_path[ [/mnt/models]\n",

```



```

        "pod": "tfs-dep-i543026-predictor-default-v6nf5-
deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.334+00:00"
    },
    {
        "container": "storage-initializer",
        "msg": "[I 210531 08:20:51 storage:45] Copying contents of
gs://kserve-samp
les/models/tensorflow/flowers to local\n",
        "pod": "tfs-dep-i543026-predictor-default-v6nf5-
deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.335+00:00"
    },
    {
        "container": "storage-initializer",
        "msg": "[W 210531 08:20:51 _metadata:104] Compute Engine
Metadata server unavailable onattempt 1 of 3. Reason: [Errno 111] Connection
refused\n",
        "pod": "tfs-dep-i543026-predictor-default-v6nf5-
deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.338+00:00"
    },
    ...
]
}
}

```

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Training Schedules \[page 202\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

9.1.2.13 Training Schedules

In order to run executions periodically, you can define a schedule to start your executions automatically. To do this, you must have prepared your configuration for the execution.

Training schedules have a start and end timestamp. The start timestamp defines when the schedule will first be run to generate executions. The end timestamp defines when the schedule will expire.

A schedule that has not yet expired has ACTIVE status. Upon expiry, the status will be changed to INACTIVE, and no further executions will start.

The format for these timestamps is defined by RFC3339 section 5.6, without fractions of seconds, for example: 2023-02-09T12:53:47Z. All timestamps are interpreted in UTC time. For more information, see [RFC3339 section 5.6](#) and [AI API Overview \[page 25\]](#).

[Create a Training Schedule \[page 203\]](#)

[List Executions Created by a Training Schedule \[page 205\]](#)

[Change an Existing Training Schedule \[page 207\]](#)

[Delete a Training Schedule \[page 209\]](#)

Parent topic: [Train Your Model \[page 159\]](#)

Related Information

[Choose a Resource Plan \[page 161\]](#)

[Workflow Templates \[page 163\]](#)

[List Scenarios \[page 167\]](#)

[List Executables \[page 171\]](#)

[Create Configurations \[page 177\]](#)

[List Configurations \[page 179\]](#)

[Using Artifact Signatures \[page 182\]](#)

[Start Training \[page 186\]](#)

[Stop Training Instances \[page 189\]](#)

[Delete Training Instances \[page 193\]](#)

[Efficiency Features \[page 196\]](#)

[Retrieve Execution Logs \[page 199\]](#)

9.1.2.13.1 Create a Training Schedule

Using Curl

1. Create a training schedule by submitting a POST request to `$AI_API_URL/v2/lm/executionSchedules`.

In the request body you need to define:

- The schedule in cron format. For more information, see [Cron Formatting](#) 📖
- A name
- The configurationId that you want to use for the training instance
- A start timestamp when the schedule should become active
- An end timestamp when the schedule should become inactive

```
curl --location --request POST "$AI_API_URL/v2/lm/executionSchedules/
$EXECUTION_SCHEDULE" \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{ \
  "cron": "0 * * * *", \
  "name": "Hourly training run", \
  "configurationId": "35b60591-1e48-473b-9b44-d5f8e9e4de32", \
  "start": "2023-02-10T10:50:31Z", \
  "end": "2023-02-10T12:55:31Z" \
}'
```

2. Check the status of the training schedule by submitting a GET request:

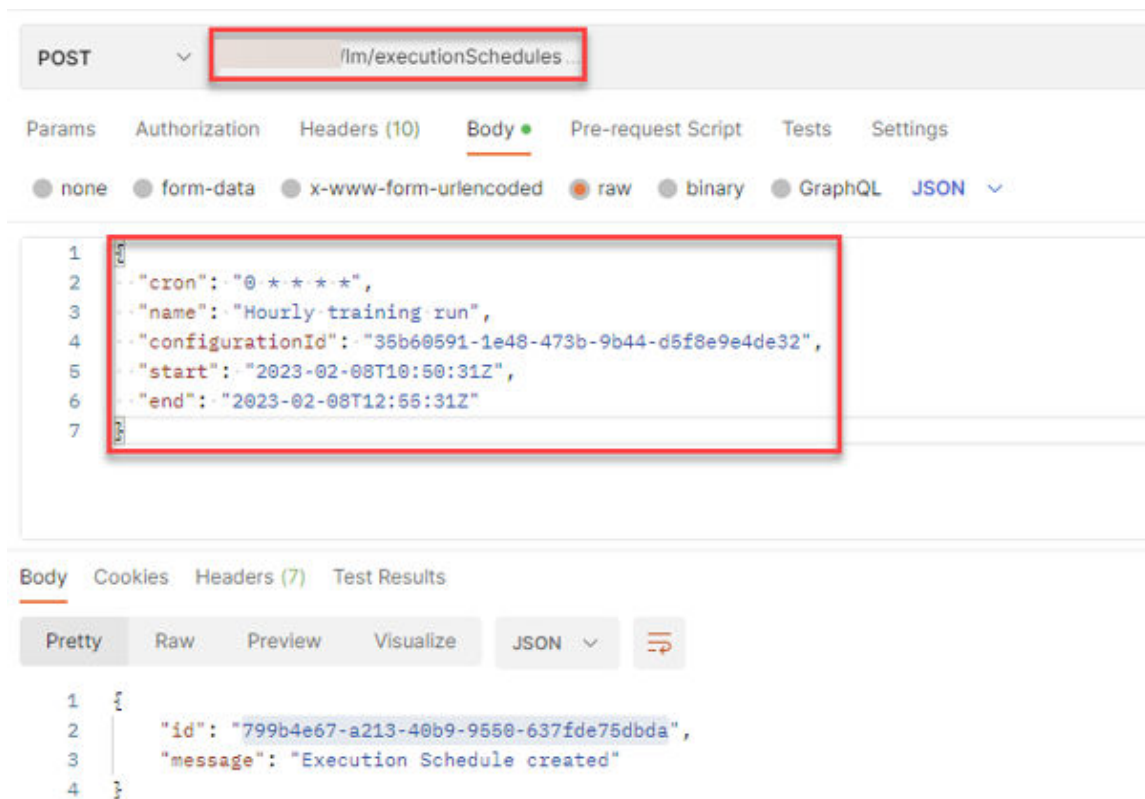
```
curl --location -- request GET "$AI_API_URL/v2/lm/executionSchedules/
$EXECUTION_SCHEDULE" \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP"
```

Using Postman

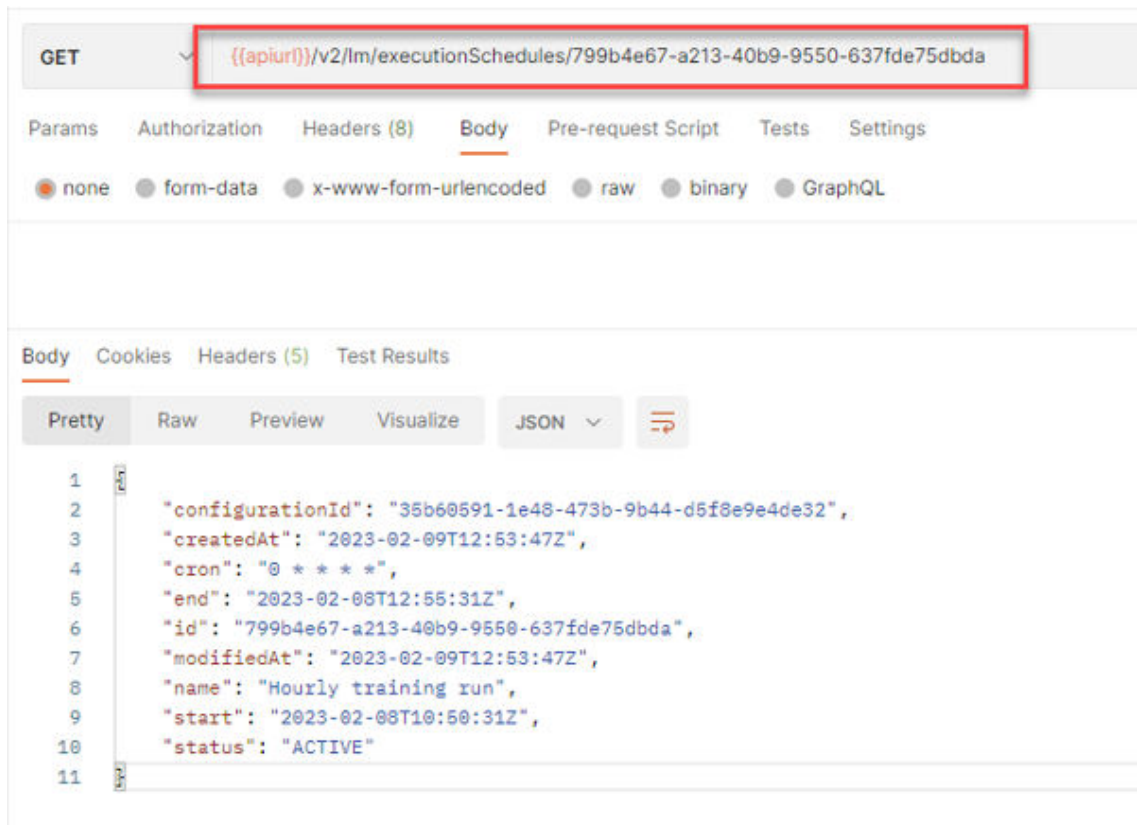
1. Create a training schedule by submitting a POST request to `{{apiurl}}/v2/lm/executionSchedules`.

In the request body you need to define:

- The schedule in cron format. For more information, see [Cron Formatting](#) 📖
- A name
- The configurationId that you want to use for the training instance
- A start timestamp when the schedule should become active
- An end timestamp when the schedule should become inactive



2. Check the status of the training schedule by submitting a GET request to `{{apiurl}}/v2/lm/executionSchedules/{{executionScheduleId}}`



Parent topic: [Training Schedules \[page 202\]](#)

Related Information

[List Executions Created by a Training Schedule \[page 205\]](#)

[Change an Existing Training Schedule \[page 207\]](#)

[Delete a Training Schedule \[page 209\]](#)

9.1.2.13.2 List Executions Created by a Training Schedule

Using Curl

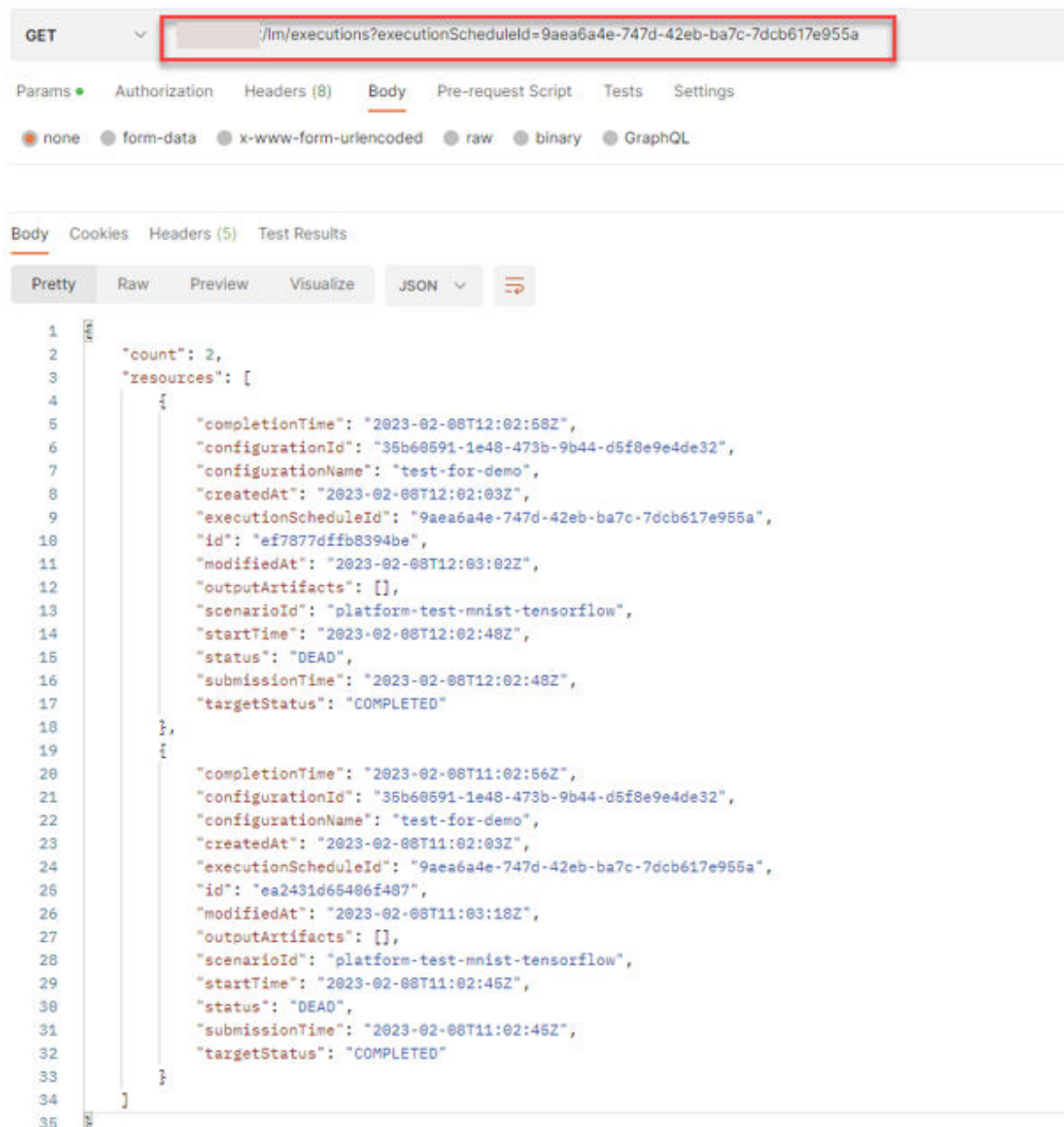
Get a list of executions created by a training schedule by submitting a GET request:

```
curl --location --request GET "$AI_API_URL/v2/lm/executionSchedules/
$EXECUTION_SCHEDULE" \\\
```

```
--header "Authorization: Bearer $TOKEN" \  
--header "ai-resource-group: $RESOURCE_GROUP"
```

Using Postman

Get a list of executions created by a training schedule by submitting a GET request to `{{apiurl}}/v2/lm/executions?executionScheduleId={{executionScheduleId}}`



Parent topic: [Training Schedules \[page 202\]](#)

Related Information

[Create a Training Schedule \[page 203\]](#)

[Change an Existing Training Schedule \[page 207\]](#)

[Delete a Training Schedule \[page 209\]](#)

9.1.2.13.3 Change an Existing Training Schedule

Using Curl

You can change the cron definition, start, end, and configuration of an existing training schedule.

You cannot change the name field.

Submit a PATCH request:

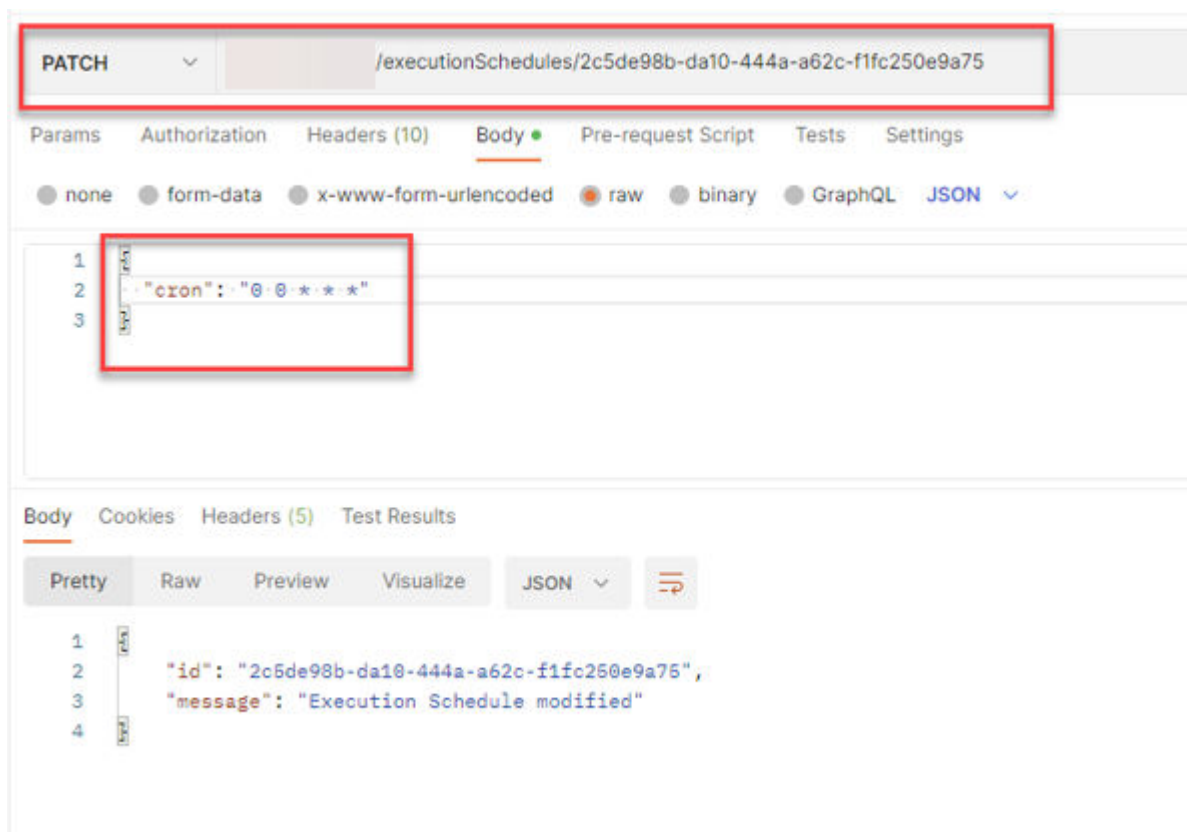
```
curl --location --request PATCH $AI_API_URL/v2/lm/executions?
executionScheduleId=$EXECUTION_SCHEDULE
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{ "cron": "0 0 * * *"'
}'
```

Using Postman

You can change the cron definition, start, end, and configuration of an existing training schedule.

You cannot change the name field.

Send your changes in a PATCH request to `{{apiurl}}/v2/lm/executionSchedules/{{executionScheduleId}}`.



Pause or Resume a Training Schedule

The status parameter pauses or resumes a schedule:

- `<INACTIVE>`: pauses a schedule.
- `<ACTIVE>`: resumes a schedule.

Using Postman

Send a PATCH request to `{{apiurl}}/v2/lm/executionSchedules/{{executionScheduleId}}` with the body:

```
{
  "status": "INACTIVE"
}
```

Using curl

```
curl --location --request PATCH $AI_API_URL/v2/lm/executions?
executionScheduleId=$EXECUTION_SCHEDULE
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--data-raw '{ "status": "INACTIVE"
}'
```


Parent topic: [Training Schedules \[page 202\]](#)

Related Information

[Create a Training Schedule \[page 203\]](#)

[List Executions Created by a Training Schedule \[page 205\]](#)

[Delete a Training Schedule \[page 209\]](#)

9.1.2.13.4 Delete a Training Schedule

Using Curl

You can delete training schedules that are in states ACTIVE and INACTIVE.

Submit a DELETE request:

```
curl --location -- request DELETE "$AI_API_URL/v2/lm/executionSchedules/  
$EXECUTION_SCHEDULE" \\  
--header "Authorization: Bearer $TOKEN" \\  
--header "ai-resource-group: $RESOURCE_GROUP"
```

Using Postman

You can delete training schedules that are in states ACTIVE and INACTIVE.

Send a DELETE request to the endpoint `{{apiurl}}/v2/lm/executionSchedules/
{{executionScheduleId}}`

Parent topic: [Training Schedules \[page 202\]](#)

Related Information

[Create a Training Schedule \[page 203\]](#)

[List Executions Created by a Training Schedule \[page 205\]](#)

[Change an Existing Training Schedule \[page 207\]](#)

9.1.3 Use Your Model

You deploy your AI learning model to run inferences against it.

SAP AI Core provides the means to run AI training and offline batch inference jobs, using the required computational resources (such as GPUs) efficiently.

You can use a model trained in SAP AI Core, or your own pretrained model for deployment and inferencing.

The model serving service includes:

- Serving of AI models through fast, secure, and resource-efficient inference endpoints that can be integrated into online inference scenarios.
- Flexibility to deploy user-supplied Docker images.
- Cost-efficient serving (for example, by autoscaling or serverless support or multimodel serving).
- Parameterized serving templates for main tenants to manage their serving instances.
- Serving instances in resource group namespaces with specific parameters and resource isolation.

The serving templates are used to create model servers. When a model server is up and running, it processes incoming inference requests and returns the results from the AI learning model. Serving templates define how a model is to be deployed. For the model to work, you must provide a spec in line with the [Serving Template API Reference \[page 217\]](#), together with input parameters and artifacts.

Restrict the number of nodes used for processing by specifying parameters minReplicas and maxReplicas.

[Choose a Resource Plan \[page 211\]](#)

You can configure SAP AI Core to use different infrastructure resources for different tasks, based on demand. SAP AI Core provides several preconfigured infrastructure bundles called “resource plans” for this purpose.

[Serving Templates \[page 213\]](#)

You use serving templates to manage your serving instances at the level of the main tenant. Serving templates define how a model is to be deployed.

[List Executables \[page 225\]](#)

An executable is a template that is instantiated for a purpose, such as training a model or creating a deployment. You can list all of the executables in a resource group and get details of specific executables from a resource group. Serving templates are mapped to deployment executables.

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

Discover features of the SAP AI Core runtime that improve efficiency and help manage resource consumption.

[Retrieve Deployment Logs \[page 249\]](#)

accessed in the deployment and execution logs.

Parent topic: [ML Operations \[page 151\]](#)

Related Information

[Connect Your Data \[page 152\]](#)

[Train Your Model \[page 159\]](#)

9.1.3.1 Choose a Resource Plan

You can configure SAP AI Core to use different infrastructure resources for different tasks, based on demand. SAP AI Core provides several preconfigured infrastructure bundles called “resource plans” for this purpose.

Context

Resource plans are used to select resources in workflow and serving templates. Different steps of a workflow can have different resource plans.

In general, if your workload needs GPU acceleration, you should use one of the GPU-enabled resource plans. Otherwise, choose a resource plan based on the anticipated CPU and memory need of your workloads.

Within SAP AI Core, the resource plan is selected via the `ai.sap.com/resourcePlan` label at pod level. It maps the selected resource plan and takes a string value, which can be any of the following resource plan IDs:

Resource Plan Specifications for AWS

Resource Plan IDs	GPUs	CPU Cores	Memory GBs	Code to Allocate Re- sources
				in Workflow Tem- plates
Starter	–	1	3	<code>ai.sap.com/ resourcePlan: starter</code>
Basic	–	3	11	<code>ai.sap.com/ resourcePlan: basic</code>
Basic-8x	–	31	116	<code>ai.sap.com/ resourcePlan: basic.8x</code>
Train-L	1 V100	7	55	<code>ai.sap.com/ resourcePlan: train.l</code>
Infer-S	1 T4	3	10	<code>ai.sap.com/ resourcePlan: infer.s</code>

Resource Plan IDs	GPUs	CPU Cores	Memory GBs	Code to Allocate Resources in Workflow Templates
Infer-M	1 T4	7	26	<code>ai.sap.com/ resourcePlan: infer.m</code>
Infer-L	1 T4	15	58	<code>ai.sap.com/ resourcePlan: infer.l</code>

⚠ Restriction

For the Free Tier service plan, only the Starter resource plan is available. Specifying other plans will result in error. For the Standard service plan, all resource plans are available. For more information, see [Free Tier \[page 38\]](#) and [Service Plans \[page 36\]](#).

📘 Note

There are limits to the default disk storage size for all of these nodes. Datasets that are loaded to the nodes will consume disk space. If you have large data sets (larger than 30 GB), or have large models, you may have to increase the disk size. To do so, use the persistent volume claim in Argo Workflows to specify the required disk size (see [Volumes](#) ➦).

Task overview: [Use Your Model \[page 210\]](#)

Related Information

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

Service Usage Reporting

Usage consumption of services is reported in the SAP BTP cockpit on the [Overview](#) page for your global account and on the [Overview](#) and [Usage Analytics](#) pages of your subaccount. The usage report lists usage

in billable measures and non-billable measures. Your final monthly bill is based on the billable measure only. Non-billable measures are displayed for reporting purposes only.

9.1.3.2 Serving Templates

You use serving templates to manage your serving instances at the level of the main tenant. Serving templates define how a model is to be deployed.

Serving templates are used to deploy one or more trained models on a model server, which process incoming inference requests and returns the results from the machine learning model. Serving templates are stored in your git repository, where you can version them as required.

In SAP AI Core, serving templates are mapped as executables. Mapping requires certain attributes in the `metadata` section of your template.

Models are deployed using a simple Kubernetes Custom Resource Definition (CRD), which is provided by KServe. To deploy a model, you must provide a YAML specification that follows the KServe specification and that defines the required input parameters and artifacts.

⚠ Restriction

The maximum number of serving templates is limited at tenant level to 50. If you reach this limit, you will receive an error message. To free up space, delete some serving templates. Alternatively, raise a ticket to increase your quota.

To get started, copy the generic serving template below and add your own values as required. You can use any text editor with a YAML plugin to create the templates. Your YAML specification must follow the KServe specs and define the required input parameters and artifacts.

Serving Template Parameter Details

Type	Parameter	Description
name (mandatory)		The executable ID. The executable ID must be unique among all executables available within the SAP AI Core main tenant.
metadata (mandatory)	name	The technical ID of the serving template that is used to uniquely identify the serving template. It is specified as the serving Template's executable ID, and therefore must be unique among all executables available within the SAP AI Core main tenant.
annotations	scenarios.ai.sap.com/ description (optional)	A description of the scenario to which this executable belongs.
	scenarios.ai.sap.com/name (mandatory)	Scenario name, which is necessary for the AI API to discover a scenario.

Type	Parameter	Description
	<code>executables.ai.sap.com/description</code> (optional)	A description of the serving template.
	<code>executables.ai.sap.com/name</code> (mandatory)	The name of the serving template.
	<code>executables.ai.sap.com/cascade-update-deployments</code> (optional)	Setting to allow changes to the serving template to be cascaded to associated deployments. See Change Serving Template and Update Deployments [page 224] .
	<code>artifacts.ai.sap.com/<argo_artifact_name>.description</code> (optional) <code>artifacts.ai.sap.com/<argo_artifact_name>.labels: {"ext.ai.sap.com/customkey1":"customvalue1", "ext.ai.sap.com/customkey2":"customvalue2"}</code> (optional)	You can add further metadata for an artifact using these annotations.
labels	<code>scenarios.ai.sap.com/id</code> (mandatory)	The scenario ID to which the serving template belongs
	<code>ai.sap.com/version</code> (mandatory)	The version is intended for customer usage to identify compatible serving template versions.
spec	<code>imagePullSecret</code> (Optional) Not required if your Docker image is fetched from a public docker repository.	Register Your Docker Registry Secret [page 87] .
	<code>minReplicas</code> (Optional)	Deployments run continually by default, but can be stopped manually when inferences are not needed. Setting minReplicas to 0 allows the inference server to enter the stopped state, when not in use. When needed, it is restarted and nodes scaled, based on demand and the min/maxReplicas parameter. For more information see Efficiency Features [page 246] .
	<code>maxReplicas</code> (Optional)	Limit the number of nodes that the inference server scales up to. For more information see Efficiency Features [page 246] .

Type	Parameter	Description
labels (mandatory)	ai.sap.com/resourcePlan	You must specify the chosen resourcePlan . The value is the string value of the selected resource plan (see Choose a Resource Plan [page 161]).

Generic Serving Template

```

apiVersion: ai.sap.com/v1alpha1
kind: ServingTemplate
metadata:
  name: text-clf-infer-tutorial
  annotations:
    scenarios.ai.sap.com/description: "SAP developers tutorial scenario"
    scenarios.ai.sap.com/name: "text-clf-tutorial-scenario"
    executables.ai.sap.com/description: "Inference executable for text
classification with Scikit-learn"
    executables.ai.sap.com/name: "text-clf-infer-tutorial-exec"
    artifacts.ai.sap.com/textmodel.kind: "model"
    artifacts.ai.sap.com/textmodel.description: "artifact description"
    artifacts.ai.sap.com/textmodel.labels: |
      {"ext.ai.sap.com/customkey1":"customvalue1", "ext.ai.sap.com/
customkey2":"customvalue2"}
  labels:
    scenarios.ai.sap.com/id: "text-clf-tutorial"
    ai.sap.com/version: "1.0.0"
spec:
  inputs:
    parameters:
      - name: modelName
        default: value
        type: string
        description: description of the parameter
    artifacts:
      - name: textmodel
template:
  apiVersion: "serving.kserve.io/v1beta1"
  metadata:
    annotations: |
      autoscaling.knative.dev/metric: concurrency
      autoscaling.knative.dev/target: 1
      autoscaling.knative.dev/targetBurstCapacity: 0
    labels: |
      ai.sap.com/resourcePlan: starter
  spec: |
    predictor:
      imagePullSecrets:
        - name: <Name of your Docker registry secret>
      minReplicas: 0
      maxReplicas: 5
      containers:
        - name: kserve-container
          image: "<DOCKER IMAGE URL GOES HERE>"
          ports:
            - containerPort: 9001
              protocol: TCP
      env:
        - name: STORAGE_URI
          value: "{{inputs.artifacts.textmodel}}"

```

- STORAGE_URI environment variable name is currently hard-coded in KServe to indicate that the model should be downloaded when a custom predictor is configured with the env var STORAGE_URI.
- /mnt/models is currently hard-coded in KServe. When you try this example with your own docker container, read the models from that path.

Sync an Application Manually

Applications sync with your GitHub repository automatically at intervals of ~3 minutes. Use the endpoint below to manually request a sync:

```
{{apiurl}}/admin/applications/{{appName}}/refresh
```

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

Stop or Delete Multiple Deployments

The feature is set to false by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

Related Information

[Stop Multiple Deployments \[page 240\]](#)

9.1.3.2.1 Serving Template API Reference

[API Schema Spec ai.sap.com/v1alpha1 \[page 217\]](#)

Package `v1alpha1` contains API Schema definitions for the serving `v1alpha1` API group (`ai.sap.com/v1alpha1`).

[KServe Spec serving.kserve.io/v1beta1 \[page 220\]](#)

Package `v1beta1` contains API Schema definitions for the serving `v1beta1` API group (`serving.kserve.io/v1beta1`).

9.1.3.2.1.1 API Schema Spec ai.sap.com/v1alpha1

Package `v1alpha1` contains API Schema definitions for the serving `v1alpha1` API group (`ai.sap.com/v1alpha1`).

ServingTemplate

`ServingTemplate` is a type of executable that specifies how a model is to be served.

Field	Description
<code>metadata</code> Kubernetes meta/v1.ObjectMeta ➡	<code>name</code> (Mandatory) Technical ID of the serving template that is used to uniquely identify the serving template.
	<code>annotations</code> ServingTemplate Metadata Annotations [page 218] Annotations to define the Scenario and Executable info.
	<code>labels</code> ServingTemplate Metadata Labels [page 218] Labels to define the Scenario and Executable info.
<code>spec</code> ServingTemplate Spec [page 218]	<code>ServingTemplateSpec</code> contains a template for KServe, input parameters and artifacts required to create a KServe CR.

ServingTemplate Metadata Annotations

(Appears on [ServingTemplate \[page 217\]](#))

A subset of supported Kubernetes Metadata Annotations

Field	Description
<code>scenarios.ai.sap.com/description</code>	A description of the Scenario (for example, "SAP developers tutorial scenario")
<code>scenarios.ai.sap.com/name</code>	The name of the serving template (for example, "text-clf-tutorial-scenario")
<code>executables.ai.sap.com/description</code>	A description of the serving template. (for example, "Inference executable for text classification with Scikit-learn")
<code>executables.ai.sap.com/name</code>	The name of the serving template (for example, "text-clf-infer-tutorial-exec")

ServingTemplate Metadata Labels

(Appears on [ServingTemplate \[page 217\]](#))

A subset of supported Kubernetes Metadata Labels

Field	Description
<code>ai.sap.com/version</code>	Version of the Scenario (for example, "1.0.0")
<code>scenarios.ai.sap.com/id</code>	Unique ID of the Scenario (for example, "1234-abcd-efg")

ServingTemplate Spec

(Appears on [ServingTemplate \[page 217\]](#))

ServingTemplate spec is the top level type for this resource.

Field	Description
inputs	parameters
	Input parameters for the KServe InferenceService Template [page 219]
	ServingTemplate Input Parameters [page 219]
	artifacts
	Input artifacts for the KServe InferenceService Template [page 219]
	ServingTemplate Input Artifacts [page 219]
template	KServe CR template
	KServe InferenceService Template [page 219]

ServingTemplate Input Parameters

(Appears on [ServingTemplate Spec \[page 218\]](#))

Input parameters required for [KServe InferenceService Template \[page 219\]](#) spec

Field	Description
name	Parameter name
default	(Optional) Default value for the parameter
type	(Optional) Type of the parameter. Only "string" type is supported

ServingTemplate Input Artifacts

(Appears on [ServingTemplate Spec \[page 218\]](#))

Input artifacts required for [KServe InferenceService Template \[page 219\]](#) spec

Field	Description
name	Artifact name

KServe InferenceService Template

(Appears on: [ServingTemplate Spec \[page 218\]](#))

KServe InferenceService template

Field	Description
apiVersion String	KServe API version. For example, <code>serving.kserve.io/v1beta1</code> .
metadata	Metadata for KServe InferenceService CR. See KServe Metadata Annotations [page 221] and KServe Metadata Labels [page 221] . Annotations and labels in metadata can be parameterized by using <code>{{inputs.parameters.Parameter-Name-Here}}</code>
spec Multiline string	A multiline string template for KServe InferenceServiceSpec [page 222] with optional placeholders for parameters and artifacts

Parent topic: [Serving Template API Reference \[page 217\]](#)

Related Information

[KServe Spec serving.kserve.io/v1beta1 \[page 220\]](#)

9.1.3.2.1.2 KServe Spec serving.kserve.io/v1beta1

Package `v1beta1` contains API Schema definitions for the serving `v1beta1` API group (`serving.kserve.io/v1beta1`).

InferenceService

InferenceService is the schema for the InferenceServices API.

Field	Description
metadata	Refer to the Kubernetes API documentation for the fields of the <code>metadata</code> field. Kubernetes meta/v1.ObjectMeta ➡

Field	Description
spec	
KServe InferenceServiceSpec [page 222]	predictor Predictor defines the model serving spec KServe PredictorSpec [page 222]

KServe Metadata Annotations

(Appears on [InferenceService \[page 220\]](#))

A multiline string containing a subset of supported KServe Metadata Labels. You can add a placeholder using `{{variable_name}}` to define a variable and replace the value dynamically. For example:

```
autoscaling.knative.dev/target: "{{inputs.parameters.MyAutoScalingTarget}}"
```

Field	Description
<code>autoscaling.knative.dev/metric</code>	Knative autoscaling metric to be used for autoscaling (for example, "rps", "concurrency")
<code>autoscaling.knative.dev/target</code>	Knative autoscaling target number (for example, "1")
<code>autoscaling.knative.dev/targetBurstCapacity</code>	Knative autoscaling target burst capacity number (for example, "70")
<code>autoscaling.knative.dev/window</code>	Knative autoscaling window (for example, "10s")
<code>autoscaling.knative.dev/scaleToZeroPodRetentionPeriod</code>	Determines the minimum amount of time that the last pod will remain active after the Autoscaler decides to scale pods to zero. Default: 0s

❖ Example

```
metadata:
  annotations: |
    autoscaling.knative.dev/metric: rps
    autoscaling.knative.dev/target:
      {{inputs.parameters.MyAutoScalingTarget}}
    autoscaling.knative.dev/targetBurstCapacity: 70
    autoscaling.knative.dev/window: 10s
```

KServe Metadata Labels

(Appears on [InferenceService \[page 220\]](#))

A multiline string containing a subset of supported KServe Metadata Labels. You can add a placeholder using `{{variable_name}}` to define a variable and replace the resource plan dynamically. For example:

```
ai.sap.com/resourcePlan:
    "{{inputs.parameters.MyResourcePlan}}"
```

Field	Description
ai.sap.com/resourcePlan	Resource Plan to be used in creating the KServe-based Model Server (for example, "basic")

❖ Example

```
metadata:
  labels: |
    ai.sap.com/resourcePlan: basic
```

KServe InferenceServiceSpec

(Appears on [InferenceService](#) [page 220])

InferenceServiceSpec is the top level type for this resource.

Field	Description
predictor	Predictor defines the model serving spec KServe PredictorSpec [page 222]

KServe PredictorSpec

(Appears on [KServe InferenceServiceSpec](#) [page 222])

PredictorSpec defines the configuration for a predictor. The following fields follow a "1-of" semantic. Users must specify exactly one spec.

Field	Description
containers Kubernetes core/v1.Container ➡	List of containers belonging to the pod. Containers cannot currently be added or removed. There must be at least one container in a pod, cannot be updated.

Field	Description
imagePullSecrets Kubernetes core/v1.LocalObjectReference	List of references to secrets in the same resource group, and used for pulling any of the images used by this spec. The name of the secret must be the same as the one created via Docker Registry Secret creation API. For more information, see Specifying imagePullSecrets on a Pod .
minReplicas (optional) int	Minimum number of replicas. Defaults to 1 but can be set to 0 to enable scale-to-zero. AI Core will attempt, on best-case effort, to schedule your deployments in different availability zone when you have more than one replica. This way, you can make your deployment High-Available (HA) by configuring min and max-replica accordingly.
maxReplicas (optional) int	Maximum number of replicas for autoscaling. Default: if minReplicas >= 2 then maxReplicas = minReplicas else 2. AI Core will attempt, on best-case effort, to schedule your deployments in different availability zone when you have more than one replica. This way, you can make your deployment High-Available (HA) by configuring min and max-replica accordingly.
containerConcurrency (optional) int64	Specifies how many requests can be processed concurrently, this sets the hard limit of the container concurrency. (https://knative.dev/docs/serving/autoscaling/concurrency).
timeout (optional) int64	Specifies the number of seconds to wait before timing out a request to the component.
terminationGracePeriodSeconds (optional) int64	Optional duration in seconds that the pod needs to terminate gracefully, may be decreased in delete request. Value must be non-negative integer. The value zero indicates delete immediately. If this value is nil, the default grace period will be used instead. The grace period is the duration in seconds after the processes running in the pod are sent a termination signal and the time when the processes are forcibly halted with a kill signal. Set this value longer than the expected cleanup time for your process. Defaults to 30 seconds.

❖ Example

```
spec: |
  predictor:
    imagePullSecrets:
      - name: [DOCKER REGISTRY SECRET GOES HERE]
    minReplicas: 1
    maxReplicas: 5
    containers:
      - name: kserve-container
        image: "[DOCKER IMAGE URL GOES HERE]"
        ports:
          - containerPort: 9001
            protocol: TCP
        env:
          - name: STORAGE_URI
            value: "{{inputs.artifacts.textmodel}}"
```

Parent topic: [Serving Template API Reference \[page 217\]](#)

Related Information

[API Schema Spec ai.sap.com/v1alpha1 \[page 217\]](#)

9.1.3.2.2 Change Serving Template and Update Deployments

If you change a serving template, you can automatically update the deployments that are associated with that template.

Prerequisites

The `executables.ai.sap.com/cascade-update-deployments` parameter is present and set to **true** in the serving template. This allows any change to the serving template to trigger an automatic update of the associated deployments.

Procedure

1. Make any required changes to the fields for the serving template. Changes to the following are supported:
 - `spec.template.spec.default.predictor.minReplicas`
 - `spec.template.spec.default.predictor.custom.container.image`

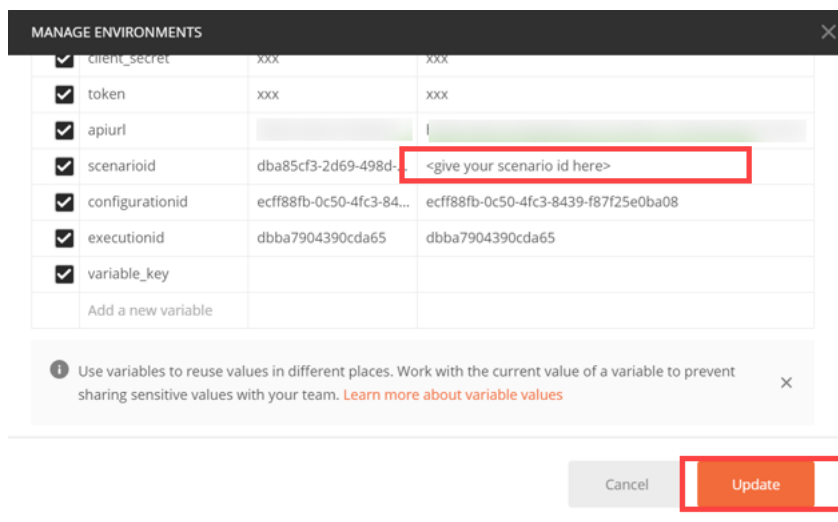
- `spec.template.spec.default.predictor.custom.container.ports`
 - `spec.template.spec.default.predictor.custom.container.env`
2. Make any required changes to the fields for the beta serving template. Changes to the following are supported:
 - `spec.template.spec.predictor.containers.image`
 - `spec.template.spec.predictor.containers.ports`
 - `spec.template.spec.predictor.containers.env`
 - Modify a value
 - Add a name-value pair
 - Parameterized values can be updated to hardcoded and vice versa

9.1.3.3 List Executables

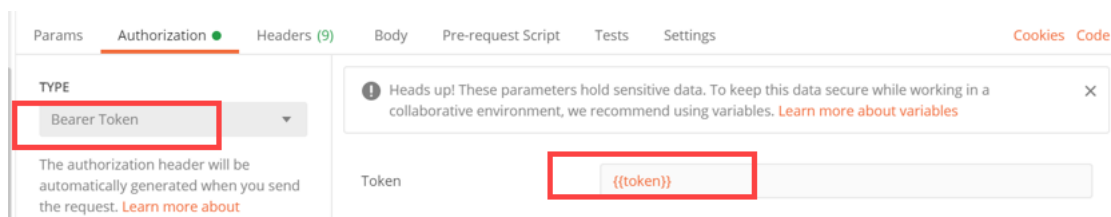
An executable is a template that is instantiated for a purpose, such as training a model or creating a deployment. You can list all of the executables in a resource group and get details of specific executables from a resource group. Serving templates are mapped to deployment executables.

List Executables with Postman

1. Define your scenario ID as the `scenarioId` environment variable.



2. Send a GET request to the endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioId}}/executables`
3. On the [Authorization](#) tab, set the type to [Bearer Token](#).
4. Set the token value to `{{token}}`.



- On the [Header](#) tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup>default is used)

- Send the request.

The screenshot shows the SAP API Explorer interface. The top bar indicates a GET request to the endpoint `/lm/scenarios/{{scenarioId}}/executables...`. The **Headers** tab is selected, showing a single header: `AI-Resource-Group` with the value `default`. Below the headers, the **Body** tab is active, displaying a JSON response in 'Pretty' format. The response includes a `count` of 3 and a `resources` array. One resource is shown with details like `createdAt`, `deployable` (true), `description`, `id` ("text-clf-infer-tutorial"), and `inputArtifacts`.

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

List Executables with curl

```
curl --request GET "$AI_API_URL/v2/lm/scenarios" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "count":4,
  "resources":[
    {
      "createdAt":"2021-02-04T13:11:01+00:00",
      "deployable":true,
      "description":"churn n text class serving executable desc",
      "id":"pytf-serving",
      "inputArtifacts":[
        {
          "name":"model_uri"
        }
      ],
      "labels":[]
    }
  ]
}
```

```

    ],
    "modifiedAt": "2021-02-04T13:11:01+00:00",
    "name": "churntextclassexecname",
    "parameters": [
      {
        "name": "modelName",
        "type": "string",
        "default": "value",
        "description": "description of the parameter"
      }
    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  },
  {
    "createdAt": "2021-02-09T07:35:02+00:00",
    "deployable": true,
    "description": "churn n text class serving executable desc",
    "id": "pytf-serving-tracking",
    "inputArtifacts": [
      {
        "name": "textmodel",
        "kind": "model",
        "description": "artifact description",
        "labels": [
          {
            "key": "ext.ai.sap.com/customkey1",
            "value": "customvalue1"
          },
          {
            "key": "ext.ai.sap.com/customkey2",
            "value": "customvalue2"
          }
        ]
      }
    ],
    "labels": [
      ],
    "modifiedAt": "2021-02-09T07:35:02+00:00",
    "name": "churntextclassexecname",
    "parameters": [
      {
        "name": "modelName",
        "type": "string"
      }
    ],
    "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
    "versionId": "0.0.1"
  },
  {
    "createdAt": "2021-02-09T07:35:03+00:00",
    "deployable": false,
    "description": "churn and text class executable desc",
    "id": "pytf-training-tracking",
    "inputArtifacts": [
      {
        "name": "churn-data"
      },
      {
        "name": "textclass-data"
      }
    ],
    "labels": [
      ],
    "modifiedAt": "2021-02-09T07:35:03+00:00",
    "name": "churnntextclassexecutablename",
  }

```

```

        "outputArtifacts": [
            {
                "name": "churn-pickle",
                "kind": "model",
                "description": "artifact description",
                "labels": [
                    {
                        "key": "ext.ai.sap.com/customkey1",
                        "value": "customvalue1"
                    },
                    {
                        "key": "ext.ai.sap.com/customkey2",
                        "value": "customvalue2"
                    }
                ]
            },
            {
                "name": "pytf-model"
            }
        ],
        "parameters": [
            {
                "name": "train-epoch",
                "type": "string"
            }
        ],
        "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
        "versionId": "0.0.1"
    },
    {
        "createdAt": "2021-02-04T14:11:02+00:00",
        "deployable": false,
        "description": "churn and text class executable desc",
        "id": "test-training",
        "inputArtifacts": [
            {
                "name": "churn-data"
            },
            {
                "name": "textclass-data"
            }
        ],
        "labels": [
            {
                "name": "churn-pickle"
            },
            {
                "name": "pytf-model"
            }
        ],
        "parameters": [
            {
                "name": "train-epoch",
                "type": "string"
            }
        ],
        "scenarioId": "ae0bd260-41ef-4162-81b0-861bd78a8516",
        "versionId": "0.0.1"
    }
]
}

```

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

Get Executable Details with Postman

1. Add the environment variable `executableid` and as its value, enter the ID of the executable.

Variable	Value
token	xxx
apiurl	
scenarioid	dba85cf3-2d69-498d-...
scenarioid	dba85cf3-2d69-498d-8ce5-4a415c9116dc
configurationid	ecff88fb-0c50-4fc3-84...
configurationid	ecff88fb-0c50-4fc3-8439-f87f25e0ba08
executionid	dbba7904390cda65
executionid	dbba7904390cda65
executableid	<give the id your executable in the scenario>

2. Send a GET request to the endpoint `{{apiurl}}/v2/lm/scenarios/{{scenarioid}}/executables/{{executableid}}`
3. On the *Authorization* tab, set the type to *Bearer Token*.
4. Set the token value to `{{token}}`.

TYPE: Bearer Token

Token: {{token}}

5. On the *Header* tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)

6. Send the request.

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output `1970-01-01T00:00:00+00:00` indicates an error.

Get Executable Details with curl

```
curl --request GET "$AI_API_URL/v2/lm/scenarios" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "createdAt":"2021-02-04T14:11:02+00:00",
  "deployable":false,
  "description":"churn and text class executable desc",
  "id":"test-training",
  "inputArtifacts":[
    {
      "name":"churn-data"
    },
    {
      "name":"textclass-data"
    }
  ],
  "labels":[
  ],
  "modifiedAt":"2021-02-04T14:11:02+00:00",
  "name":"churnntextclassexecutable",
  "outputArtifacts":[
    {
      "name":"churn-pickle"
    },
    {
      "name":"pytf-model"
    }
  ],
  "parameters":[
    {
      "name":"train-epoch",
      "type":"string"
    }
  ],
  "scenarioId":"ae0bd260-41ef-4162-81b0-861bd78a8516",
  "versionId":"0.0.1"
}
```

Note

The `<modifiedAt>` field denotes the timestamp of the latest successful sync. The output 1970-01-01T00:00:00+00:00 indicates an error.

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

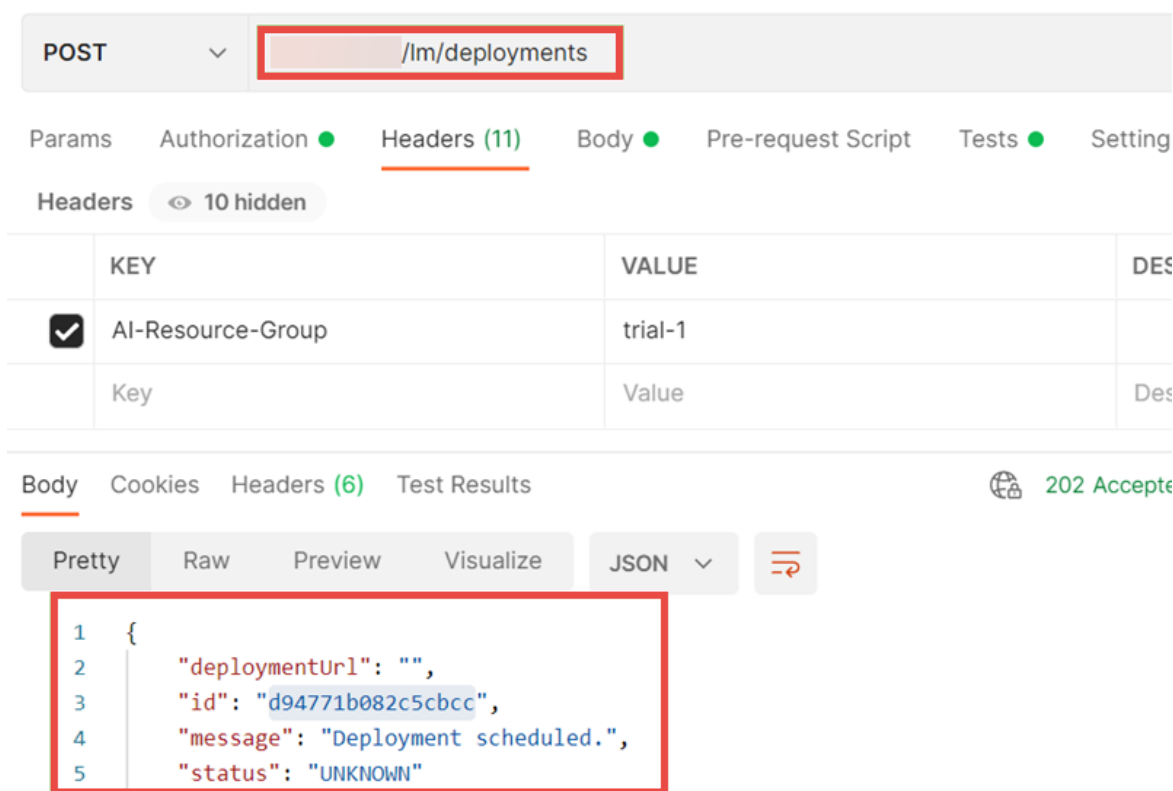
[Serving Templates \[page 213\]](#)

- [Deploy Models \[page 231\]](#)
- [Inferencing \[page 234\]](#)
- [Update a Deployment \[page 236\]](#)
- [Stop Deployments \[page 238\]](#)
- [Delete Deployments \[page 242\]](#)
- [Efficiency Features \[page 246\]](#)
- [Retrieve Deployment Logs \[page 249\]](#)

9.1.3.4 Deploy Models

Using Postman

- Send a POST request to the endpoint `{{apiurl}}/v2/lm/deployments`.
- Pass the `configurationId` in the request body.



- Check the status of the deployment by sending a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

GET ▼ /lm/deployments/d94771b082c5cbcc Send

Params Authorization ● Headers (9) Body Pre-request Script Tests ● Settings ⌵

Headers 8 hidden

	KEY	VALUE	DESCRIPTION	Bulk Edit	Pre
☒	AI-Resource-Group	trial-1			
	Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 877 ms 673 B Save Response

Pretty Raw Preview Visualize JSON ▼ ⌵

```

5  "deploymentUrl": "",
6  "id": "d94771b082c5cbcc",
7  "modifiedAt": "2021-09-27T10:48:51Z",
8  "scenarioId": "b5379278-887c-4156-9a65-a6c11d6f1a71",
9  "startTime": "2021-09-27T10:33:12Z",
10 "status": "RUNNING".

```

Note

Configurations are checked when POST requests are submitted. If a configuration is incorrect configuration, an error message outlining the details will be returned..

If the status of the deployment is dead or pending after successful submission, there might be errors in the deployment. You can check the deployment logs for more details, see [Retrieve Deployment Logs \[page 249\]](#).

Using curl

1. Trigger the deployment.

```

curl --request POST $AI_API_URL/v2/lm/deployments \
  --header "Authorization: Bearer $TOKEN" \
  --header "ai-resource-group: $RESOURCE_GROUP" \
  --data-raw '{
    "configurationId": " 2b72d740-5a89-4cf7-b37c-85973eed6ae "
  }'

```

Output Code

```

{
  "deploymentUrl": "",
  "id": "dda5d19065d5b1f4",
  "message": "Deployment created.",
  "status": "UNKNOWN"
}

```

2. Note the deploy environment variable for later use.

3. Check the status of the deployment.

```
curl --request GET $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP"
```

Output Code

```
{
  "configurationExecutableId": "hello-tf-1-15",
  "configurationId": "2b72d740-5a89-4cf7-b37c-85973eed6ae",
  "configurationName": "hello-tf-1-15-config",
  "createdAt": "2020-11-11T06:34:24Z",
  "createdBy": "user",
  "deploymentUrl": "https://my-deployment-url.com",
  "id": "d291766fd1072b3f",
  "modifiedAt": "2020-11-11T06:37:29Z",
  "modifiedBy": "user",
  "scenarioId": "dba85cf3-2d69-498d-8ce5-4a415c9116dc",
  "status": "RUNNING",
  "targetStatus": "RUNNING",
  "versionId": "0.1.0"
}
```

Note

Configurations are checked when POST requests are submitted. If a configuration is incorrect configuration, an error message outlining the details will be returned..

If the status of the deployment is dead or pending after successful submission, there might be errors in the deployment. You can check the deployment logs for more details, see [Retrieve Deployment Logs \[page 249\]](#).

Optional Parameters

The duration of a deployment can be limited using the `t t l` parameter. It takes an integer for quantity, and a single letter to specify units of time. Only minutes (**m**), hours (**h**) and days (**d**), are supported, and values must be natural numbers. For example, "`t t l`": "`5h`" gives the deployment a duration of 5 hours. **4.5h** and **4h30m** are not valid inputs. If no value is passed, the duration of the deployment is indefinite. Once the duration expires, the deployment is stopped and deleted.

Deployment Quotas

Each tenant is assigned a default quota that limits the number of deployments and replicas per deployment. If you reach this quota, your deployment will not be created, and you will be notified. You can free up your quota by deleting existing deployments.

Alternatively, you can request a quota increase by creating a ticket. The component name is **CA-ML-AIC** and ticket title is **Request to Increase Quota**.

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

[Support.](#) 

[Service Plans \[page 36\]](#)

9.1.3.5 Inferencing

Use the URL from your model deployment to access the results of your model.

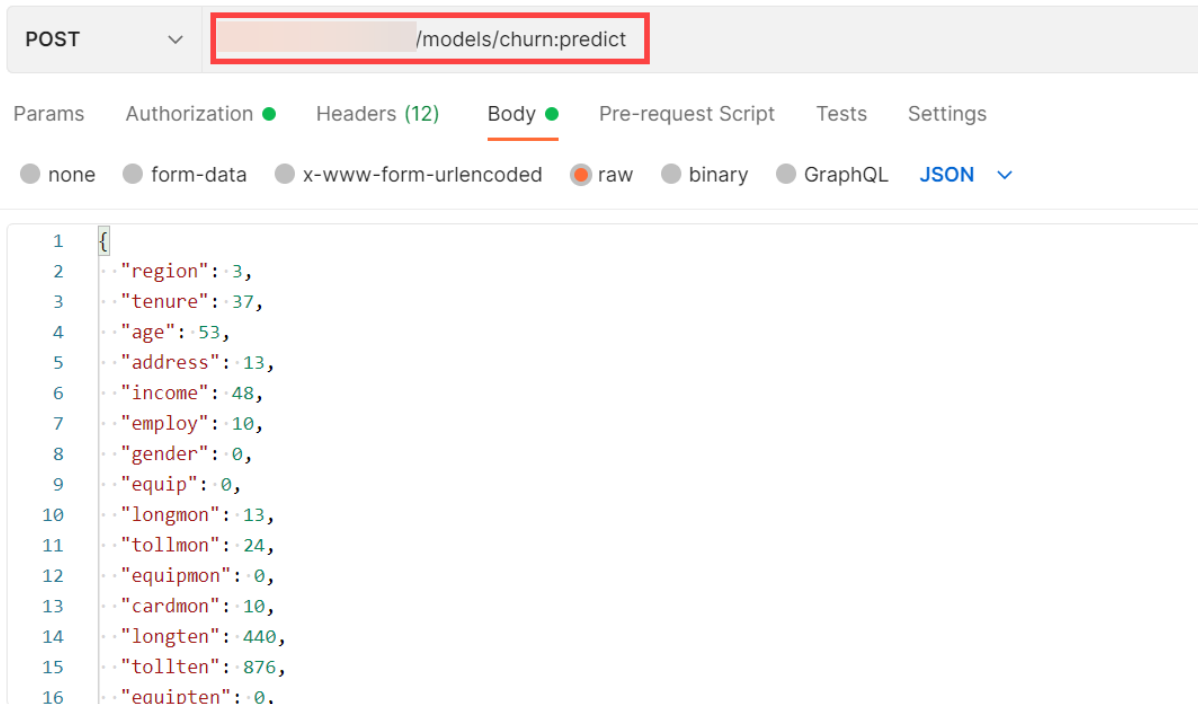
To inference your model, send a POST request to "\$DEPLOYMENT_URL/<path that you have defined>"

For example, TensorFlow models follow the path `v1/models/$MODEL_NAME:predict`.

Inference with Postman

As the deployment URL, pass the URL that was returned in the response body in the `{{apiurl}}/v2/lm/deployments` call (see [Deploy Models \[page 231\]](#)).

As the request body, enter a sample instance in JSON format.



Inference with curl

In this example, the name of the model is “churn”.

```
curl --location --request POST '$AI_API_URL/v1/models/churn:predict' \
```

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

9.1.3.6 Update a Deployment

You can update a deployment with a new configuration while retaining the inference URL.

During the transition to the new deployment configuration, the inference requests will continue to work.

When the new configuration is deployed, the deployment may end in a “dead” status, or get stuck in the “pending” status, and never get to a “running” state, due to incorrect configuration. In this case, inference requests will no longer work. The last known running configuration ID is documented in field *latestRunningConfigurationId* and can be used in another PATCH request to return to the last running configuration.

If the updated deployment reaches the “running” state, *latestRunningConfigurationId* will be updated to the new configuration.

Prerequisites

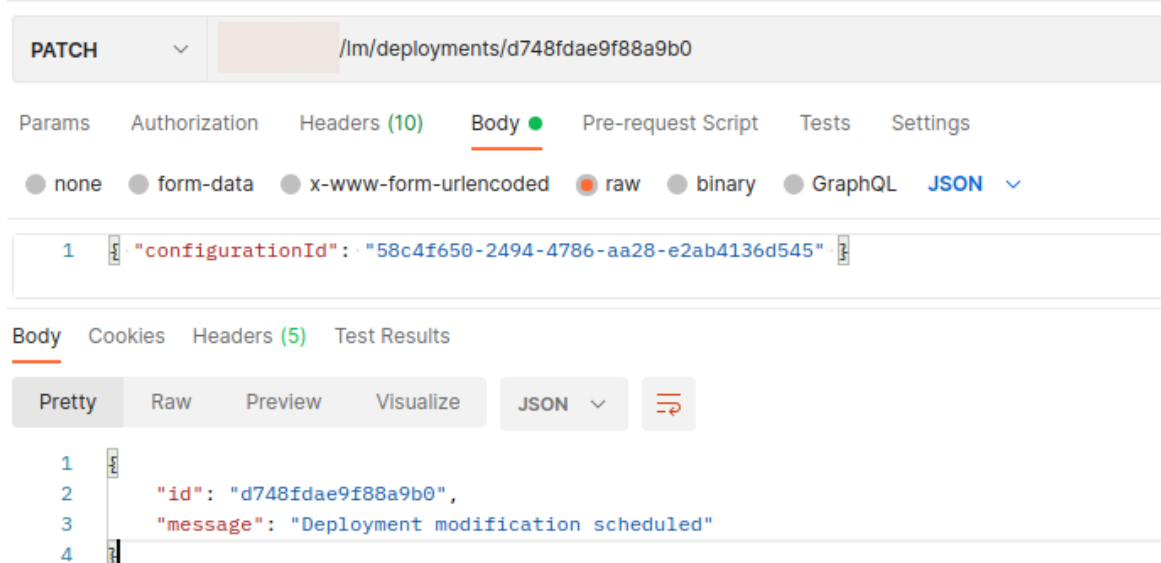
- The deployment must have the state “pending”, “running” or “dead”.
- The new configuration contains the same `scenarioId` and `executableId` as the currently active configuration.

Note

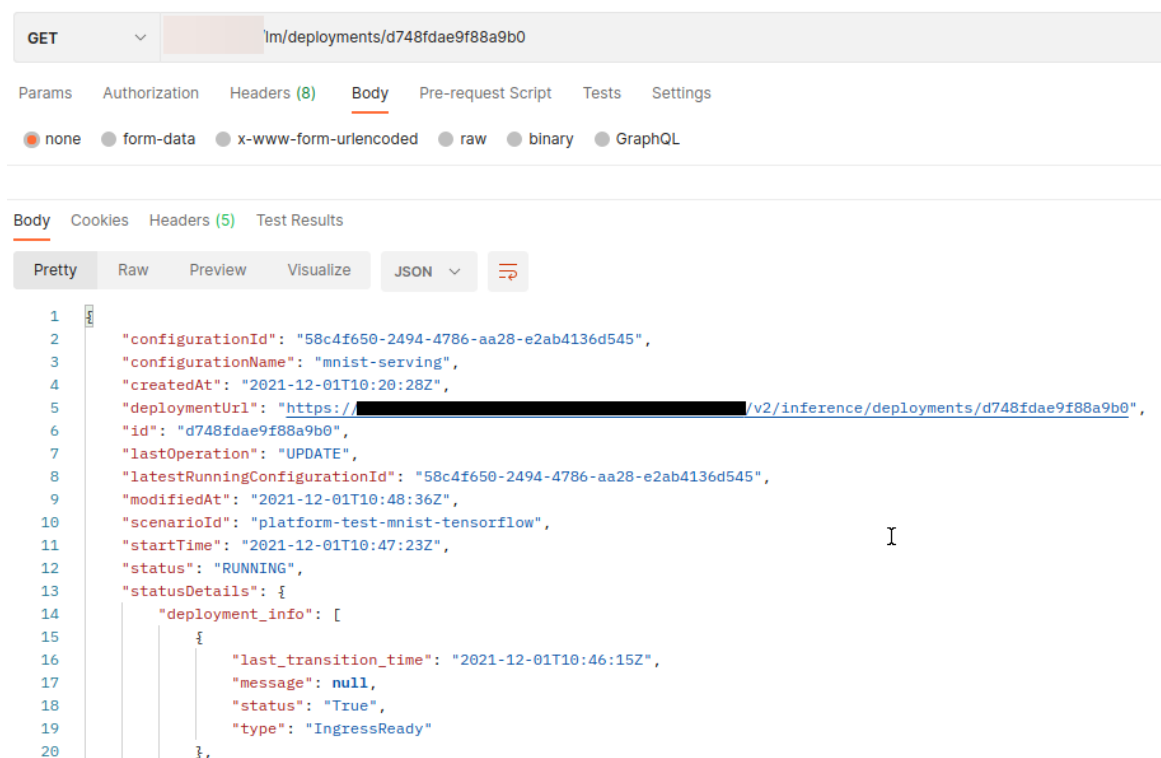
Dead deployments can only be patched within 7 days. After 7 days from the time that the deployment reached DEAD status, it will be deleted and will no longer be available.

Using Postman

1. Update the deployment by submitting a PATCH request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.
2. Pass the new `configurationId` in the request body.



3. Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.



Using curl

1. Update the deployment by submitting a PATCH request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.
2. Pass the new `configurationId` in the request body.

```

curl --request PATCH $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \
--header "Authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--header 'Content-Type: application/json' \
--data-raw '{
  "configurationId": "490a02b0-4b97-48e8-b905-1c6ae5ea5b1c"
}'

```

Output Code

```

{
  "id": "d748fdae9f88a9b0",
  "message": "Deployment modification scheduled"
}

```

3. Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

```

curl --request GET $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \ --header
"Authorization: Bearer $TOKEN" \ --header "ai-resource-group: $RESOURCE_GROUP"

```

Output Code

```
{
  "configurationId": "490a02b0-4b97-48e8-b905-1c6ae5ea5b1c",
  "configurationName": "mnist-serving",
  "createdAt": "2021-12-01T10:20:28Z",
  "deploymentUrl": "https://my-deployment-url.com",
  "id": "d748fdae9f88a9b0",
  "lastOperation": "UPDATE",
  "latestRunningConfigurationId": "490a02b0-4b97-48e8-b905-1c6ae5ea5b1c",
  "modifiedAt": "2021-12-01T11:19:21Z",
  "scenarioId": "platform-test-mnist-tensorflow",
  "startTime": "2021-12-01T10:47:23Z",
  "status": "PENDING",
  "statusDetails": {},
  "submissionTime": "2021-12-01T11:03:55Z",
  "targetStatus": "RUNNING"
}
```

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Efficiency Features \[page 246\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

9.1.3.7 Stop Deployments

Stopping a deployment releases the SAP AI Core runtime computing resources that it used. A stopped deployment does not incur costs.

Note

Stop is only enabled if the status is running or pending.

[Stop a Single Deployment \[page 239\]](#)

[Stop Multiple Deployments \[page 240\]](#)

Parent topic: [Use Your Model \[page 210\]](#)

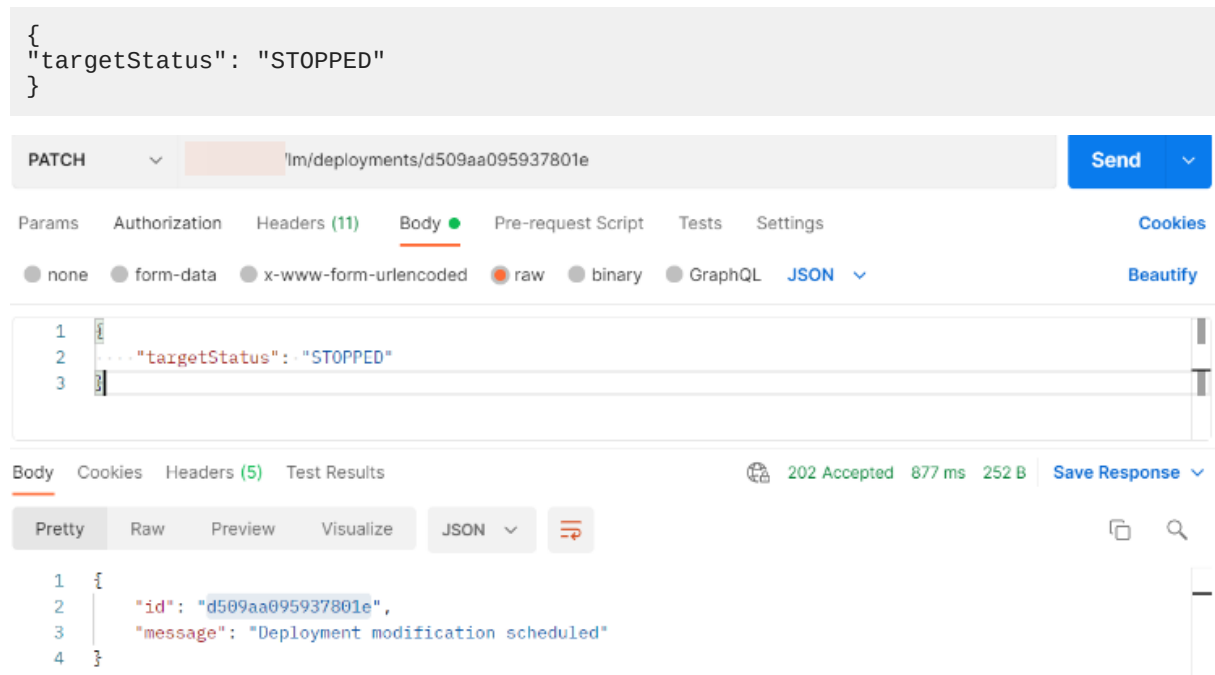
Related Information

[Choose a Resource Plan \[page 211\]](#)
[Serving Templates \[page 213\]](#)
[List Executables \[page 225\]](#)
[Deploy Models \[page 231\]](#)
[Inferencing \[page 234\]](#)
[Update a Deployment \[page 236\]](#)
[Delete Deployments \[page 242\]](#)
[Efficiency Features \[page 246\]](#)
[Retrieve Deployment Logs \[page 249\]](#)

9.1.3.7.1 Stop a Single Deployment

Using Postman

Stop the deployment by submitting a PATCH request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`. The header for this request is: `AI-Resource-Group: {YOUR-Resource-Group}`. The Body for this request is



Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

Using curl

1. Update the deployment by submitting a PATCH request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.
2. Update the request body to:

```
curl --request PATCH $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \
--header "authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--header 'content-Type: application/json' \
--data-raw '{
  "targetStatus": "STOPPED"
}'
```

Output Code

```
{
  "id": "d748fdae9f88a9b0",
  "message": "Deployment modification scheduled"
}
```

3. Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

```
curl --request GET $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \ --header
"Authorization: Bearer $TOKEN" \ --header "ai-resource-group: $RESOURCE_GROUP"
```

Parent topic: [Stop Deployments \[page 238\]](#)

Related Information

[Stop Multiple Deployments \[page 240\]](#)

9.1.3.7.2 Stop Multiple Deployments

`bulkUpdates` is a meta capability endpoint of the AI API. It enables or disables bulk PATCH operations. For more information, see [AI API Overview \[page 25\]](#).

The feature is set to `false` by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

About `bulkUpdates`:

- The maximum number of updates per request is 100.
- Your bulk update can contain a mixture of STOP and DELETE requests.
- Only *running* or *pending* executions or deployments can be stopped.
- Only *stopped*, *dead* or *unknown* executions or deployments can be deleted.
- An ID can only appear once per bulk request. For multiple modifications of the same ID, multiple requests are needed.

Using curl

Update the request body to:

```
curl --request PATCH - /deployments \
  --header {"deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

Output Code

```
{
  "deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "message": "Deployment modification scheduled"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "message": "Deployment modification scheduled"
    }
  ]
}
```

Using Postman

Send a bulk PATCH request to: - /deployments

```
{
  "deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

```
}  
]  
}  
'
```

Output Code

```
{  
  "deployments": [  
    {  
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",  
      "message": "Deployment modification scheduled"  
    },  
    {  
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",  
      "message": "Deployment modification scheduled"  
    }  
  ]  
}
```

Parent topic: [Stop Deployments \[page 238\]](#)

Related Information

[Stop a Single Deployment \[page 239\]](#)

[AI API Overview \[page 25\]](#)

9.1.3.8 Delete Deployments

Deleting a deployment releases the SAP AI Core resources that it used.

⚠ Restriction

If your deployment is running, you must stop it first. You can stop a deployment by submitting a PATCH request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`. For more information, see [Stop Deployments \[page 238\]](#)

[Delete a Single Deployment \[page 243\]](#)

[Delete Multiple Deployments \[page 244\]](#)

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)
[List Executables \[page 225\]](#)
[Deploy Models \[page 231\]](#)
[Inferencing \[page 234\]](#)
[Update a Deployment \[page 236\]](#)
[Stop Deployments \[page 238\]](#)
[Efficiency Features \[page 246\]](#)
[Retrieve Deployment Logs \[page 249\]](#)

9.1.3.8.1 Delete a Single Deployment

Using Postman

Delete the deployment by submitting a DELETE request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`. The header for this request is: `AI-Resource-Group: {YOUR-Resource-Group}`.

Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

Using curl

1. Update the deployment by submitting a DELETE request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.
2. Update the request body to:

```
curl --request DELETE $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \
--header "authorization: Bearer $TOKEN" \
--header "ai-resource-group: $RESOURCE_GROUP" \
--header 'content-Type: application/json' \
--data-raw '{
  "targetStatus": "DELETED"
}'
```

Output Code

```
{
  "id": "d748fdae9f88a9b0",
  "message": "Deployment modification scheduled"
}
```

3. Check the status of the deployment by submitting a GET request to `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`.

```
curl --request GET $AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID \ --header
"Authorization: Bearer $TOKEN" \ --header "ai-resource-group: $RESOURCE_GROUP"
```

Parent topic: [Delete Deployments \[page 242\]](#)

Related Information

[Delete Multiple Deployments \[page 244\]](#)

9.1.3.8.2 Delete Multiple Deployments

`bulkUpdates` is a meta capability endpoint of the AI API. It enables or disables bulk PATCH operations. For more information, see [AI API Overview \[page 25\]](#).

The feature is set to `false` by default. To enable bulk PATCH operations, your template must contain the following snippet, with the relevant values set to `true`.

```
Meta:
  "bulkUpdates": {
    "executions": false,
    "deployments": false
  }
```

About `bulkUpdates`:

- The maximum number of updates per request is 100.
- Your bulk update can contain a mixture of STOP and DELETE requests.
- Only *running* or *pending* executions or deployments can be stopped.
- Only *stopped*, *dead* or *unknown* executions or deployments can be deleted.
- An ID can only appear once per bulk request. For multiple modifications of the same ID, multiple requests are needed.

Using curl

Update the request body to:

```
curl --request PATCH - /deployments \
  --header {"deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
  ],
```

```
{
  "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
  "targetStatus": "DELETED"
}
]
```

Output Code

```
{
  "deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "message": "Deployment modification scheduled"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "message": "Deployment modification scheduled"
    }
  ]
}
```

Using Postman

Send a bulk PATCH request to: - /deployments

Update the request body to:

```
{
  "deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "targetStatus": "STOPPED"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "targetStatus": "DELETED"
    }
  ]
}
```

Output Code

```
{
  "deployments": [
    {
      "id": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "message": "Deployment modification scheduled"
    },
    {
      "id": "qweq32131-qwee-1231-8543-0f91a7a2e2e",
      "message": "Deployment modification scheduled"
    }
  ]
}
```

Parent topic: [Delete Deployments \[page 242\]](#)

Related Information

[Delete a Single Deployment \[page 243\]](#)

[AI API Overview \[page 25\]](#)

9.1.3.9 Efficiency Features

Discover features of the SAP AI Core runtime that improve efficiency and help manage resource consumption.

Autoscaling

SAP AI Core includes parameters to reduce the number of nodes used based on current consumption, or impose usage limits during periods of high consumption. These parameters allow your workload the flexibility to scale based on demand, and for consumption to be capped, limiting your consumption and therefore costs. For more information, see [Serving Templates](#).

Scaling to 0

Where non-uniform loads are expected, scaling to 0 allows nodes to enter a sleeping state when demand allows, limiting your consumption, and therefore costs. Nodes wake when demand increases, which has an increased response time. The Global Node pool reduces, this cold start time. For more information, see [Serving Templates](#).

Global Node Pool

When the inference server scales up from a sleeping state, there is some additional response time. To reduce this, SAP AI Core has a Global Node Pool, which keeps commonly used nodes reserved to allow for shorter response times. You do not need to do anything to make use of the Global Node Pool, it is already in place. To reduce response times further, avoid a cold start altogether by setting your autoscaling parameter to 1. For more information, see [Serving Templates](#).

Scaling to 1

Cold starts can be avoided completely by scaling to 1. This keeps a single node warm, even when it is not needed, reducing response time. However, it does not offer the consumption and cost savings associated with scaling to 0. For more information, see [Serving Templates](#).

Duration

The default duration is indefinite, however the `t t l` parameter limits the duration of a deployment to minutes, hours, or days. This parameter allows you to plan the deletion of your model servers and model deployment URL, allowing for an expected period of use and avoiding unnecessary consumption and costs afterwards. For more information, see [Deploy Models](#) and [About the AI API](#).

Tenant Warm Node Pool

Tenant warm nodes allow tenants to reserve nodes from specific resource plan types, ensuring that a predefined number of nodes exist in the cluster. The reserved nodes can then be used during model training and serving. Reserving nodes reduces waiting time at workload startup, but incurs costs in line with node use, whether consumed or not.

Mechanism of Node Reservation

1. The tenant specifies number of nodes to reserve.
2. An execution or deployment utilizes a reserved node, a replacement reserve node of the same resource plan type is requested from the hyperscaler.

The number of reserved nodes specified by the tenant are consistently available for use.

The minimum number of reserved nodes is 0.

About Node Reservation

- The number of reserved nodes specified by the tenant are consistently available for use.
- The minimum number of reserved nodes is 0.
- The maximum number of reserved nodes is 10.
- The default number of reserved nodes is 0.

Reserve Nodes Using Postman

1. Send a PATCH request to the endpoint `{{apiurl}}/v2/admin/resources/nodes`
2. Provide the resource plan type and quantity of nodes to reserve in the request body in JSON format:

```
{
  "resourcePlans": [
    {
      "name": "infer.l",
      "request": 1
    },
    {
      "name": "infer.m",
      "request": 1
    },
    {
      "name": "train.l",
      "request": 1
    },
    ...
  ]
}
```

Reserve Nodes Using curl

Submit a PATCH request to the endpoint`{{apiurl}}/v2/admin/resources/nodes`

```
curl --request PATCH $AI_API_URL/v2/admin/resources/nodes \
--data-raw '{
  "resourcePlans": [
    {
      "name": "infer.l",
      "request": 1
    },
    {
      "name": "infer.m",
      "request": 1
    },
    {
      "name": "train.l",
      "request": 1
    }
  ]
}'
```

→ Remember

All reserved nodes are charged at the same rate as nodes used during model training and serving.

Check Reserve Node Status Using Postman

Send a GET request to the endpoint`{{apiurl}}/v2/admin/resources/nodes`

Check Reserve Node Status Using curl

```
curl --request GET $AI_API_URL/v2/resources/nodes
```

Output Code

```
{
  "resourcePlans": {
    "infer.l": {
      "provisioned": 1,
      "requested": 1
    },
    "infer.m": {
      "provisioned": 1,
      "requested": 1
    },
    "train.l": {
      "provisioned": 1,
      "requested": 1
    }
  }
}
```

- requested: the number of reserve nodes requested by the tenant.
- provisioned: the number of reserve nodes that are currently present in the cluster.

Update Quantities of Reserved Nodes

To update the number of nodes reserved, repeat the reservation procedure, with updated quantities in the request field.

Delete Reserved Nodes

To delete reserved nodes, repeat the reservation procedure, with quantities in the `request` field set to `0`.

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)

[Retrieve Deployment Logs \[page 249\]](#)

[Service Plans \[page 36\]](#)

9.1.3.10 Retrieve Deployment Logs

accessed in the deployment and execution logs.

You can retrieve the logs for a specific deployment or execution by submitting a GET request. Use the following endpoints to retrieve the logs:

- GET `/v2/lm/deployments/{deploymentId}/logs`
- GET `/v2/lm/executions/{executionId}/logs`

Query parameters include:

- `start` : the start time for a query as a string, in RFC 3339-compliant datetime format. Defaults to 1 hour before the current. Example: `2021-05-19T00:00:14.347Z`
- `end` : the end time for a query as a string, in RFC 3339-compliant datetime format. Defaults to the current time. Example: `2021-05-19T00:00:14.347Z`
- `$top` : the maximum number of entries returned. The default value is 1000; the upper limit is 5000.
- `$order` : the sort order of logs, either `asc` (for ascending, earliest in the order will appear at the top of the list) or `desc` (for descending, most recent in the order will appear at the top of the list). Note the default value is `asc`.

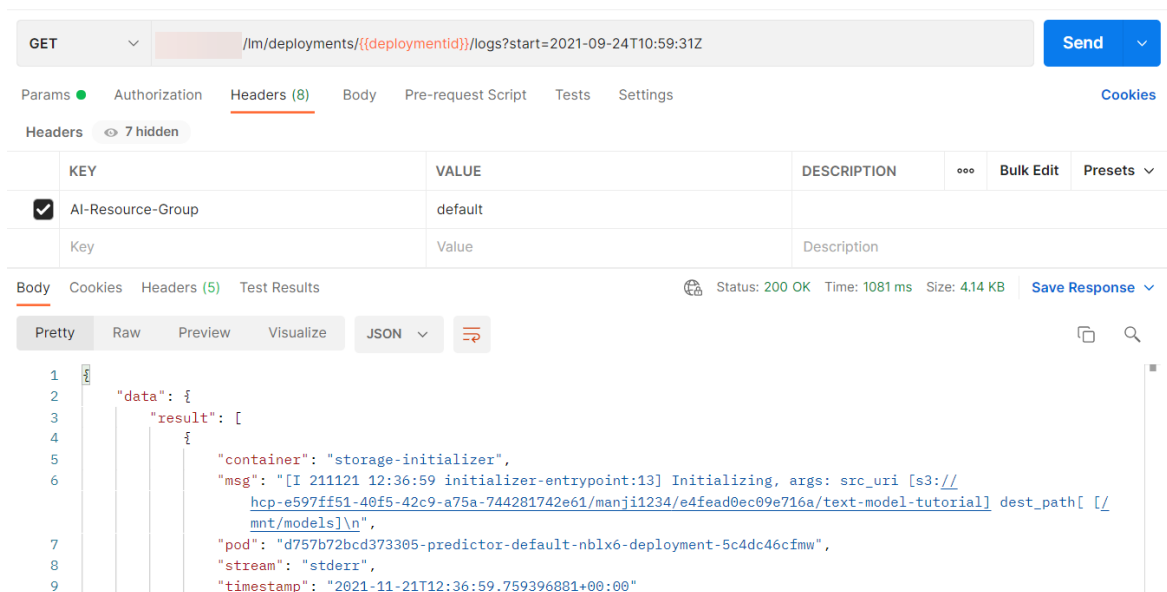
For example:

- `/v2/lm/deployments/{deploymentId}/logs?start=2021-05-19T00:00:14.347Z&end=2021-05-19T01:00:14.347Z&$top=100&$order=asc`
- returns the first
100 lines of a deployment log between `2021-05-19T00:00:14.347Z` and `2021-05-19T01:00:14.347Z`

- `/v2/lm/deployments/{deploymentId}/logs` - returns deployment logs from the preceding hour
- `/v2/lm/executions/{executionId}/logs` - returns execution logs from the preceding hour

Using Postman

1. Create a new GET request and enter the URL `{{apiurl}}/v2/lm/deployments/{{deploymentid}}/logs`.



2. On the *Authorization* tab, set the type to *Bearer Token*.
3. Set the token value to `{{token}}`.
4. On the *Header* tab, add the following entry:

Key	Value
ai-resource-group	<Name of your resourceGroup> (in the example, default is used)

5. Send the request.

Using curl

```
curl --request GET "$AI_API_URL/v2/lm/deployments/$DEPLOYMENT_ID/logs?start=2021-05-19T00:00:14.347Z" --header "Authorization: Bearer $TOKEN" --header "ai-resource-group: $RESOURCE_GROUP"
```

Sample Output

For example, see the following JSON output from the API.

Output Code

```
{
  "data": {
    "result": [
      {
        "container": "storage-initializer",
        "msg": "[I 210531 08:20:51 initializer-entrypoint:13] Initializing, args: src_uri [gs://kserve-samples/models/tensorflow/flowers] dest_path[ [/mnt/models]\n",
        "pod": "tfs-dep-i543026-predictor-default-v6nf5-deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.334+00:00"
      },
      {
        "container": "storage-initializer",
        "msg": "[I 210531 08:20:51 storage:45] Copying contents of gs://kserve-samples/models/tensorflow/flowers to local\n",
        "pod": "tfs-dep-i543026-predictor-default-v6nf5-deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.335+00:00"
      },
      {
        "container": "storage-initializer",
        "msg": "[W 210531 08:20:51 _metadata:104] Compute Engine Metadata server unavailable onattempt 1 of 3. Reason: [Errno 111] Connection refused\n",
        "pod": "tfs-dep-i543026-predictor-default-v6nf5-deployment-8b58c8ddcfdx",
        "stream": "stderr",
        "timestamp": "2021-05-31T08:20:51.338+00:00"
      },
      ...
    ]
  }
}
```

Parent topic: [Use Your Model \[page 210\]](#)

Related Information

[Choose a Resource Plan \[page 211\]](#)

[Serving Templates \[page 213\]](#)

[List Executables \[page 225\]](#)

[Deploy Models \[page 231\]](#)

[Inferencing \[page 234\]](#)

[Update a Deployment \[page 236\]](#)

[Stop Deployments \[page 238\]](#)

[Delete Deployments \[page 242\]](#)
[Efficiency Features \[page 246\]](#)
[Retrieve Execution Logs \[page 199\]](#)

9.2 Metrics

The AI API provides the ability to track metrics, and to customize or filter which metrics are reported.

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[ML Operations \[page 151\]](#)
[Advanced Features \[page 269\]](#)
[APIs and API Extensions \[page 276\]](#)
[Libraries and SDKs \[page 277\]](#)
[Content Packages \[page 278\]](#)

9.2.1 Metrics Tracking with AI API

You can use AI API to track and fetch metrics for executions and models. In addition, metrics can be compared using the SAP AI Launchpad interface.

Usage Authorization

The following role collections are required to use the AI API to track metrics.

Function	Role Collection
Create Metrics	aicore_admin
	or
	aicore_scenario_editor
View Metrics	aicore_viewer
	or
	aicore_scenario_viewer

9.2.1.1 Get Metrics

The following demonstrates how you can manually track metrics information, and patch and delete metrics.

Get Metrics Using Postman

1. Prepare a GET request to the endpoint `{{apiurl}}/v2/lm/metrics` with the following query parameters and headers:

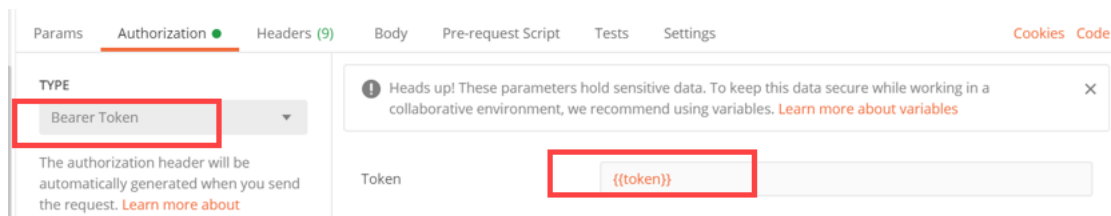
Query Parameters

Parameter	Required	Data Type	Description
executionIds	Yes	Array of String	ID of execution for which to list metrics information.
\$select	No	Array of String	Projects only the resources that are specified in the \$select parameter, and executionId. The supported values of \$select are metrics, tags and customInfo or any combination of these and *. If the value is *, all metric resource data is returned.

Header

Field	Required	Data Type	Description
AI-Resource-Group	Yes	String	ID of the resource group that contains the execution.

2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.



4. Set the query parameters in [Params](#).

GET ▼ `{{apiurl}}/v2/lm/metrics?$select=metrics&executionIds=e3de6dbbca8621c5,ed4bb3c7a4ba64b9` Send ▼

Params 6 1 Authorization ● Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	...	Bulk Edit
☒ \$select	metrics	Projects only the resources that are specified in \$select along with		
☒ executionIds	e3de6dbbca8621c5,ed4bb3c7a4ba64b9	ID of execution for which to list metrics information.		
Key	Value	Description		

5. Set the resource group *Header*.

GET ▼ `{{apiurl}}/v2/lm/metrics?executionIds=ec83b8e837fe4a56,ea4af58c56c26384`

Params ● Authorization Headers (8) 7 hidden Body Pre-request Script Tests Settings

Headers

	KEY	VALUE	DESCRIPTION	...	Bulk
☒	AI-Resource-Group	default			
	Key	Value	Description		

6. Send the request.
You will receive the following response:

Response

JSON Field	Description
customInfo	Array of dictionary. Each dictionary is a contains custom information in string.
executionId	ID of the execution, which the metrics belong to.
metrics	Array of dictionary of metric.
tags	

Body Cookies Headers (3) Test Results Status: 200 OK Time: 2.65 s Size: 1.78 KB Save Response

Pretty Raw Preview Visualize JSON ▼

```

1  {
2    "count": 2,
3    "resources": [
4      {
5        "customInfo": [
6          {
7            "executionId": "e3de6dbbca8621c5",
8            "metrics": [
9              {
10               "count": 1,
11               "tags": {
12                 "type": "Metric"
13               }
14             }
15            ]
16          },
17          {
18            "executionId": "ed4bb3c7a4ba64b9",
19            "metrics": [
20              {
21               "count": 1,
22               "tags": {
23                 "type": "Metric"
24               }
25             }
26            ]
27          }
28        ]
29      },
30      {
31        "customInfo": [
32          {
33            "executionId": "e3de6dbbca8621c5",
34            "metrics": [
35              {
36               "count": 1,
37               "tags": {
38                 "type": "Metric"
39               }
40             }
41            ]
42          },
43          {
44            "executionId": "ed4bb3c7a4ba64b9",
45            "metrics": [
46              {
47               "count": 1,
48               "tags": {
49                 "type": "Metric"
50               }
51             }
52            ]
53          }
54        ]
55      }
56    ]
57  }

```

Note

It is possible to use more than one metric in the \$select parameter by separating valid entries with a comma.

Params ● Authorization ● Headers (9) Body Pre-request Script Tests Settings Cookies			
Query Params			
KEY	VALUE	DESCRIPTION	⋮ Bulk Edit
☒ \$select	metrics,customInfo	Projects only the resources that are specified in \$select along with ↗ ...	
☒ executionIds	e3de6dbbca8621c5,ed4bb3c7a4ba64b9	ID of execution for which to list metrics information.	
Key	Value	Description	

Get Metrics Using curl

```
curl --location -g --request GET '$AI_API_URL/v2/lm/metrics?
$select=metrics,tags,customInfo&executionIds=e3de6dbbca8621c5,ed4bb3c7a4ba64b9'
--header 'AI-Resource-Group: default' \
--header 'Authorization: Bearer $TOKEN'
```

Note

It is possible to use any one of the allowed values or combination of the allowed values (metrics, tags, customInfo) or * in the \$select parameter.

Output Code

```
{
  "count": 2,
  "resources": [
    {
      "customInfo": [
        {
          "name": "confusion matrix",
          "value": "[{'Predicted': 'False', 'Actual': 'False', 'value': 34}, {'Predicted': 'False', 'Actual': 'True', 'value': 124}, {'Predicted': 'True', 'Actual': 'False', 'value': 165}, {'Predicted': 'True', 'Actual': 'True', 'value': 36}]"
        }
      ],
      "executionId": "ec83b8e837fe4a56",
      "metrics": [
        {
          "labels": [
            {
              "name": "m11",
              "value": "alpha"
            },
            {
              "name": "metrics.ai.sap.com/Artifact.name",
              "value": "text-model-tutorial"
            }
          ],
          "name": "Error Rate",
          "step": 2,
          "timestamp": "2020-09-29T11:40:10.330000Z",
          "value": 0.98
        }
      ],
      "tags": [
        {
          "name": "text-model-tutorial",
          "value": "beta-3"
        }
      ]
    }
  ]
}
```

```

    },
    {
      "customInfo": [
        {
          "name": "confusion matrix",
          "value": " precision recall f1-score support\n\n 0 0.85 0.97 0.91
273\n 1 0.96 0.79 0.87 227\n\n accuracy 0.89 500\n macro avg 0.90 0.88 0.89
500\nweighted avg 0.90 0.89 0.89 500\n"
        }
      ],
      "executionId": "ea4af58c56c26384",
      "metrics": [
        {
          "labels": [
            {
              "name": "train",
              "value": "range 0-400"
            }
          ],
          "name": "Accuracy",
          "step": 1,
          "timestamp": "2021-12-03T06:23:57.945336Z",
          "value": 0.9675
        },
        {
          "labels": [
            {
              "name": "train",
              "value": "range 400-800"
            }
          ],
          "name": "Accuracy",
          "step": 2,
          "timestamp": "2021-12-03T06:23:58.089152Z",
          "value": 0.94
        },
        {
          "labels": [
            {
              "name": "train",
              "value": "range 800-1200"
            }
          ],
          "name": "Accuracy",
          "step": 3,
          "timestamp": "2021-12-03T06:23:58.195216Z",
          "value": 0.9625
        },
        {
          "labels": [
            {
              "name": "train",
              "value": "range 1200-1600"
            }
          ],
          "name": "Accuracy",
          "step": 4,
          "timestamp": "2021-12-03T06:23:58.306945Z",
          "value": 0.9425
        },
        {
          "labels": [
            {
              "name": "train",
              "value": "range 1600-2000"
            }
          ],
          "name": "Accuracy",

```



```

        "step": 5,
        "timestamp": "2021-12-03T06:23:58.421745Z",
        "value": 0.945
    },
    {
        "labels": [
            {
                "name": "metrics.ai.sap.com/Artifact.name",
                "value": "text-model-tutorial"
            }
        ],
        "name": "Error Rate",
        "step": 0,
        "timestamp": "2021-12-03T06:23:58.569476Z",
        "value": 0.10999999999999999
    },
    {
        "labels": [],
        "name": "n_compliments",
        "step": 0,
        "timestamp": "2021-12-03T06:23:57.576040Z",
        "value": 1173.0
    },
    {
        "labels": [],
        "name": "n_complaints",
        "step": 0,
        "timestamp": "2021-12-03T06:23:57.575618Z",
        "value": 1327.0
    },
    {
        "labels": [],
        "name": "n_samples",
        "step": 0,
        "timestamp": "2021-12-03T06:23:57.574852Z",
        "value": 2500.0
    },
    {
        "labels": [
            {
                "name": "train",
                "value": "80%"
            },
            {
                "name": "step",
                "value": "preprocessing"
            }
        ],
        "name": "split_samples",
        "step": 0,
        "timestamp": "2021-12-03T06:23:57.757594Z",
        "value": 2000.0
    },
    {
        "labels": [
            {
                "name": "validation",
                "value": "20%"
            },
            {
                "name": "step",
                "value": "preprocessing"
            }
        ],
        "name": "split_samples",
        "step": 0,
        "timestamp": "2021-12-03T06:23:57.757608Z",
        "value": 500.0
    }

```

```

    }
  ],
  "tags": [
    {
      "name": "Reading Model",
      "value": "OK"
    },
    {
      "name": "Test Inference",
      "value": "checked"
    }
  ]
}
]
}

```

```

curl --location -g --request GET '$AI_API_URL/v2/lm/metrics?
$select=tags,customInfo&executionIds=e3de6dbbca8621c5,ed4bb3c7a4ba64b9'
--header 'AI-Resource-Group: default' \
--header 'Authorization: Bearer $TOKEN'

```

Output Code

```

{
  "count": 2,
  "resources": [
    {
      "customInfo": [
        {
          "name": "confusion matrix",
          "value": "[{'Predicted': 'False', 'Actual': 'False', 'value':
34},{'Predicted': 'False', 'Actual': 'True', 'value': 124}, {'Predicted':
'True', 'Actual': 'False', 'value': 165},{'Predicted': 'True', 'Actual':
'True', 'value': 36}]"
        }
      ],
      "executionId": "ec83b8e837fe4a56",
      "tags": [
        {
          "name": "text-model-tutorial",
          "value": "beta-3"
        }
      ]
    },
    {
      "customInfo": [
        {
          "name": "confusion matrix",
          "value": " precision recall f1-score support\n\n 0  0.85  0.97  0.91
273\n 1  0.96  0.79  0.87  227\n\n accuracy  0.89  500\n macro avg  0.90  0.88  0.89
500\nweighted avg  0.90  0.89  0.89  500\n"
        }
      ],
      "executionId": "ea4af58c56c26384",
      "tags": [
        {
          "name": "Reading Model",
          "value": "OK"
        },
        {
          "name": "Test Inference",
          "value": "checked"
        }
      ]
    }
  ]
}

```

```
}
]
```

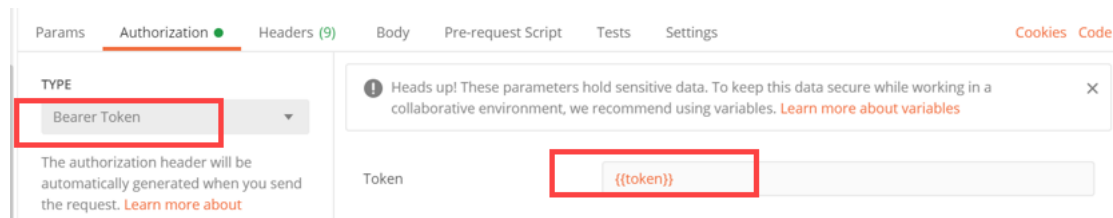
Patch Metrics Using Postman

1. Prepare a PATCH request to the endpoint `{{apiurl}}/v2/lm/metrics` with the following headers:

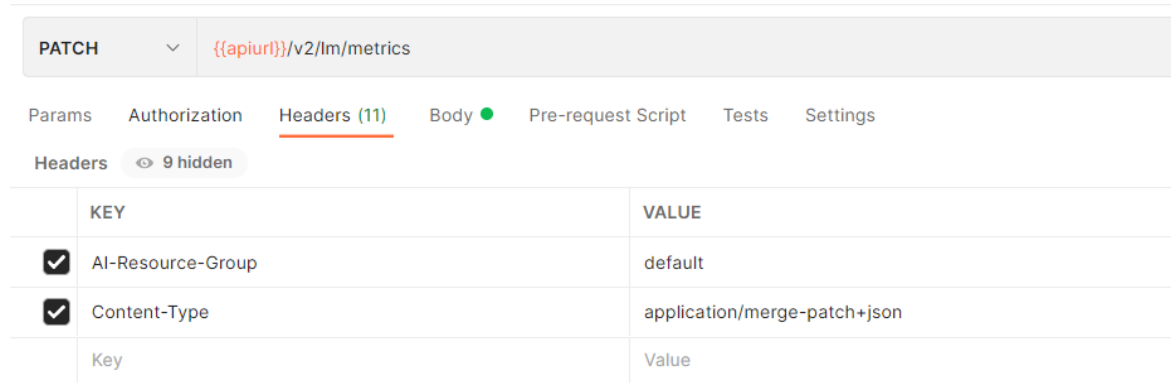
Header

Field	Required	Data Type	Description
AI-Resource-Group	Yes	String	ID of the resource group that contains the execution.
Content-Type	Yes	value = <code>application/merge-patch+json</code>	

2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.



4. Set the [Header](#).



5. Add the [Body](#).

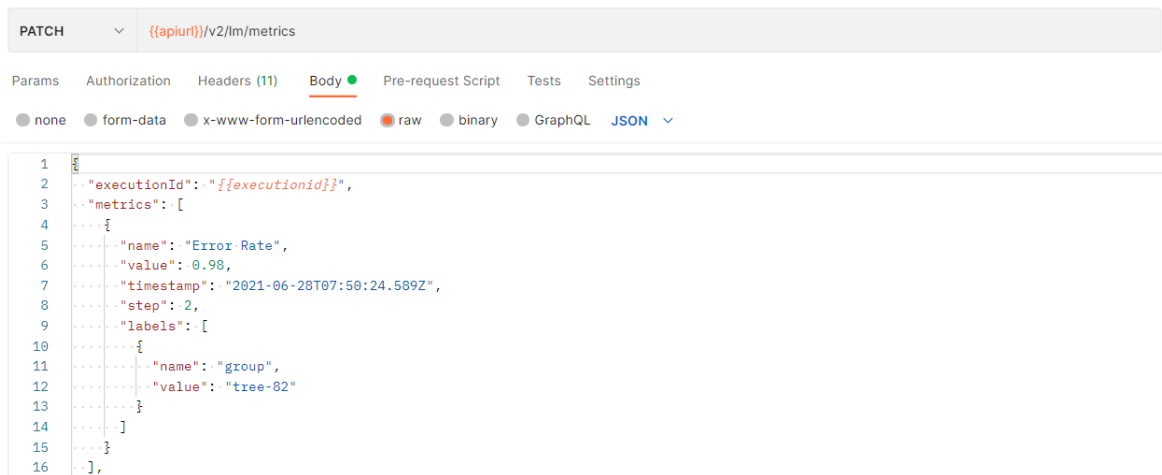
Sample Code

```
{
  "executionId": "{{executionid}}",
  "metrics": [
    {
      "name": "Error Rate",
      "value": 0.98,
      "timestamp": "2021-06-28T07:50:24.589Z",
      "step": 2,
```

```

    "labels": [
      {
        "name": "group",
        "value": "tree-82"
      }
    ]
  },
  "tags": [
    {
      "name": "Artifact Group",
      "value": "RFC-1"
    }
  ],
  "customInfo": [
    {
      "name": "Confusion Matrix",
      "value": "[{'Predicted': 'False', 'Actual': 'False', 'value': 34},
{'Predicted': 'False', 'Actual': 'True', 'value': 124}, {'Predicted':
'True', 'Actual': 'False', 'value': 165}, { 'Predicted': 'True', 'Actual':
'True', 'value': 36}]"
    }
  ]
}

```



- Send the request.
As the response, you will receive a 204 (no content) response.

Patch Metrics Using curl

```

curl --location --request GET '$AI_API_URL/v2/lm/metrics?
executionId=e1c49497ccf6dde8' \
--header 'AI-Resource-Group: default' \
--header 'Authorization: Bearer $TOKEN' \
--data-raw '{
"executionId": "e1c49497ccf6dde8",
"metrics": [
{
"name": "Error Rate",
"value": 0.98,
"timestamp": "2021-06-28T07:50:24.589Z",

```

```

"step": 2,
"labels": [
  {
    "name": "group",
    "value": "tree-82"
  }
],
"tags": [
  {
    "name": "Artifact Group",
    "value": "RFC-1"
  }
],
"customInfo": [
  {
    "name": "Confusion Matrix",
    "value": "[{'\\'Predicted\\': '\\'False\\', '\\'Actual\\': '\\'False\\', '\\'value\\': 34}, {'\\'Predicted\\': '\\'False\\', '\\'Actual\\': '\\'True\\', '\\'value\\': 124}, {'\\'Predicted\\': '\\'True\\', '\\'Actual\\': '\\'False\\', '\\'value\\': 165}, {'\\'Predicted\\': '\\'True\\', '\\'Actual\\': '\\'True\\', '\\'value\\': 36}]"
  }
]
}'

```

9.2.1.2 Query Metric Data

You can query metrics by submitting a GET request to the endpoint `/v2/lm/metrics`. To fetch the tracking data, you can use the following parameters:

- **AI-Resource-Group** (header) - string – UUID
- **executionIds** (query) – string – Retrieve metrics based on up to 10 execution IDs (comma-separated list)
- **\$select** (query) – Selectively retrieve metric resource data, such as metrics and custom info. For example, if the parameter value is a wildcard (*), then all metric resource data is returned. If the parameter value is **custom info**, then only custom info data is returned. Values should be entered as an array. Only the values **metrics**, **tags**, **customInfo** and ***** are supported.

Examples

Responses of GET API

In this example, the **\$select** parameter value is a wildcard (*), so all metric resource data is returned.

Response Code	Description
200	List of tracking metadata, where each item includes metrics, labels, parameters, and tags.

Sample Code

```
{
  "count": 1,
  "resources": [
    {
      "executionId": "aa97b177-9383-4934-8543-0f91a7a0283a",
      "metrics": [
        {
          "name": "Error Rate",
          "value": 0.98,
          "timestamp": "2020-11-12T12:36:13.730Z",
          "step": 2,
          "labels": [
            {
              "name": "group",
              "value": "tree-82"
            }
          ]
        }
      ]
    },
    {
      "tags": [
        {
          "name": "Artifact Group",
          "value": "RFC-1"
        }
      ]
    },
    {
      "customInfo": [
        {
          "name": "Confusion Matrix",
          "value": "[{'Predicted': 'False', 'Actual': 'False', 'value': 34}, {'Predicted': 'False', 'Actual': 'True', 'value': 124}, {'Predicted': 'True', 'Actual': 'False', 'value': 165}, {'Predicted': 'True', 'Actual': 'True', 'value': 36}]"
        }
      ]
    }
  ]
}
```

Response Code

400

Description

The specification of the resource was incorrect.

Sample Code

```
{
  "error": {
    "code": "<error code>",
    "message": "<error message>.",
    "requestId": "9832bf934f3743v3948v3",
    "target": "/metrics",
    "details": [
      {
        "code": "<error code>",
        "message": "<error message>."
      }
    ]
  }
}
```

Response Code	Description
501	The operation is not supported.

Sample Code

```
{
  "error": {
    "code": "02010055",
    "message": "Metrics was not found.",
    "requestId": "9832bf934f3743v3948v3",
    "target": "/metrics",
    "details": [
      {
        "code": "9827389374",
        "message": "Empty result set."
      }
    ]
  }
}
```

9.2.1.3 Store Metric Data

You can store metrics by submitting a PATCH request to the endpoint `/v2/lm/metrics`. To store tracking data, you must provide the parameter `AI-Resource-Group` header string.

The `executionId`, `metrics labels`, `tags`, and `customInfo` together form a `MetricResource` that is used as the request body for the PATCH API to persist tracking data.

The following limits apply to each of the attributes in the `MetricResource`:

- `metrics` – maximum 1000
- `labels` – maximum 20, with a maximum character limit of 256 for each label
- `tags` – maximum 100, with a maximum character limit of 256 for each tag
- `customInfo` – maximum 100, with a maximum size of 5 MB for the entire `customInfo` object

→ Recommendation

Do not use the tracking functions provided by SAP AI Core to track sensitive information. For more information, see [Security \[page 280\]](#).

`executionId`

A unique identifier for an execution.

metrics

A metric is a key/value pair where the value is numeric. Metrics can have optional step, timestamp, and label fields. Metrics are captured as part of a training execution to evaluate a model's performance. Every metric (and associated labels), tag, and custom info must be associated with an execution. Once the metric, tag, and custom info are saved, they can be queried with an execution ID.

Metrics Attributes

Attribute Name	Description	Type	Constraints
Name (mandatory)	Name of the metric	String	Pattern: [\w-]{1,64} Maxlength: 256
Value (mandatory)	Numeric value of the metric	Number	
Timestamp	Time when the metric was created or logged in RFC3339 format	String (\$date-time)	
Step	Step is an optional integer that represents any measurement of training progress (number of training iterations, number of epochs, and so on) for the metric	Integer	Minimum: 0
Labels	A list of name-value object pairs associated with some metric.	See Label Attributes [page 264]	

labels

A label is a classifying phrase or name applied to a metric. A set of labels can be used to provide each instance of a metric record with extra information.

Label Attributes

Attribute Name	Description	Type	Constraints
Name (mandatory)	Name of the label	String	Minlength: 1 Maxlength: 256
Value (mandatory)	Value of the label	String	Minlength: 1 Maxlength: 256

For a metric to be associated with a model, you must explicitly associate it with a model artifact:

Sample Code

```
"labels": [
  {
    "name": "group",
    "value": "tree-82"
  },
  {
    "name": "metrics.ai.sap.com/Artifact.name",
    "value": "my_model_name"
  }
]
```

When an execution produces only one output model artifact, all metrics captured in the execution are associated with that output artifact. If an execution produces more than one output model artifact, each metric captured must be explicitly associated with a model artifact (as shown in the example above).

tags

A tag is a name/value pair that is used to support segregation at execution level. For example, you can assign tags to a group of selected test executions. A set of tags can be associated with a **MetricResource**, which in turn is linked to an execution.

Tag Attributes

Attribute Name	Description	Type	Constraints
Name (mandatory)	Name of the tag	String	Minlength: 1 Maxlength: 256
Value (mandatory)	Value of the tag	String	Minlength: 1 Maxlength: 256

customInfo

Custom info key/value pairs that enable the capture of large amounts of metadata, typically associated with an execution. Custom info provides rendering or semantic information regarding a metric for a consuming application or complex metrics in JSON format.

A set of such custom info objects can be associated with a **MetricResource**.

CustomInfo Attributes

Attribute Name	Description	Type	Constraints
Name (mandatory)	Name of the CustomInfo	String	Minlength: 1 Maxlength: 256
Value (mandatory)	Value of the CustomInfo	String	Minlength: 1 Maxlength: 256

Examples

Example Request Body for PATCH API

```
{
  "executionId": "aa97b177-9383-4934-8543-0f91a7a0283a",
  "metrics": [
    {
      "name": "Error Rate",
      "value": 0.98,
      "timestamp": "2020-11-12T12:18:01.539Z",
      "step": 2,
      "labels": [
        {
          "name": "group",
          "value": "tree-82"
        },
        {
          "name": "metrics.ai.sap.com/Artifact.name",
          "value": "my_model_name"
        }
      ]
    }
  ],
  "tags": [
    {
      "name": "Artifact Group",
      "value": "RFC-1"
    }
  ],
  "customInfo": [
    {
      "name": "Confusion Matrix",
      "value": "[{'Predicted': 'False', 'Actual': 'False', 'value': 34},
{'Predicted': 'False', 'Actual': 'True', 'value': 124}, {'Predicted': 'True',
'Actual': 'False', 'value': 165}, {'Predicted': 'True', 'Actual': 'True',
'value': 36}]"
    }
  ]
}
```

Responses of PATCH API

Response Code	Description
204	Metrics was successfully updated/created
400	The specification of the resource was incorrect

Sample Code

```
{
  "error": {
    "code": "<error code>",
    "message": "<error message>.",
    "requestId": "9832bf934f3743v3948v3",
    "target": "/metrics",
    "details": [
      {
        "code": "<error code>",
        "message": "<error message>"
      }
    ]
  }
}
```

Response Code	Description
413	Request entity is larger than limits defined by server.

Sample Code

```
{
  "error": {
    "code": "02000005",
    "message": "PayloadLimitException",
    "requestId": "9832bf934f3743v3948v3",
    "target": "/metrics",
    "details": [
      {
        "code": "02000005",
        "message": "PayloadLimitException"
      }
    ]
  }
}
```

9.2.1.4 Delete Metrics

You can delete metrics by submitting a DELETE request to the endpoint `/v2/lm/metrics`. To delete tracking data, you must provide the following parameters:

- `AI-Resource-Group` (header) – string
- `executionId` (query)

Query Parameters

Parameter	Required	Data Type	Description
<code>executionIds</code>	Yes	Array of String	ID of execution

Header

Field	Required	Data Type	Description
AI-Resource-Group	Yes	String	ID of the resource group that contains the execution.

Responses of DELETE API

Response Code	Description
200	Metric resource was successfully deleted
404	The specified resource was not found

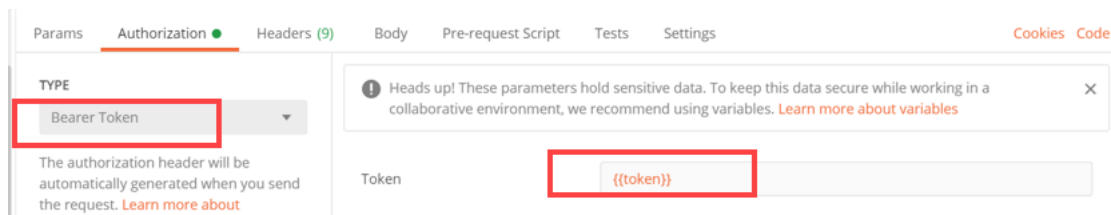
Sample Code

Sample 404 response:

```
{
  "error": {
    "code": "02010055",
    "message": "Metrics was not found.",
    "requestId": "9832bf934f3743v3948v3",
    "target": "/metrics",
    "details": [
      {
        "code": "9827389374",
        "message": "Empty result set."
      }
    ]
  }
}
```

Using Postman

1. Send a DELETE request to the endpoint `{{apiurl}}/v2/lm/metrics`
2. On the [Authorization](#) tab, set the type to [Bearer Token](#).
3. Set the token value to `{{token}}`.



4. Set the query parameters in [Params](#).

DELETE ▼ `{{apiurl}}/v2/lm/metrics?executionId={{executionid}}`

Params ● Authorization Headers (8) Body Pre-request Script Tests Settings

Query Params

	KEY	VALUE	DESCRIPTION
☒	executionId	<code>{{executionid}}</code>	
	Key	Value	Description

5. Set the *Header*.

DELETE ▼ `{{apiurl}}/v2/lm/metrics?executionId={{executionid}}`

Params ● Authorization Headers (8) Body Pre-request Script Tests Settings

Headers 👁 7 hidden

	KEY	VALUE
☒	AI-Resource-Group	default
	Key	Value

6. Send the request.

You should receive the message `Metric Resource was successfully deleted`.

Body Cookies Headers (5) Test Results 🌐 Status: 200 OK Time: 458 ms Size: 225 B

Pretty Raw Preview Visualize JSON ▼ 🔄

```

1  {
2    "message": "Metric Resource was successfully deleted"
3  }

```

Using curl

Send a DELETE request to the endpoint `{{apiurl}}/v2/lm/metrics` with the required query parameters and headers.

```
curl --location --request DELETE '$AI_API_URL/v2/lm/metrics?
executionId=e1c49497ccf6dde8' \
--header 'AI-Resource-Group: default' \
--header 'Authorization: Bearer $TOKEN'
```

9.3 Advanced Features

AI Content as a Service [\[page 270\]](#)

SAP AI Core helps users provide their AI content available as a service on the [Service Marketplace](#), using GitOps.

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[ML Operations \[page 151\]](#)

[Metrics \[page 252\]](#)

[APIs and API Extensions \[page 276\]](#)

[Libraries and SDKs \[page 277\]](#)

[Content Packages \[page 278\]](#)

9.3.1 AI Content as a Service

SAP AI Core helps users provide their AI content available as a service on the [Service Marketplace](#), using GitOps.

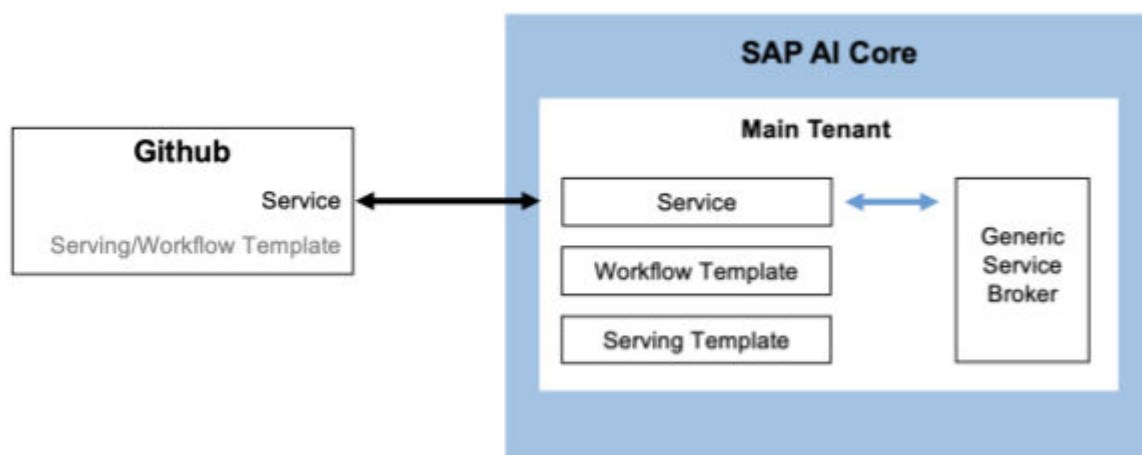
Consumers can create service instances using SAP AI Core to use available content, and create their own executions or deployments. If the person acting as the main tenant chooses to make this available externally through the [Service Marketplace](#), they become a service provider. The onboarding process for service providers and their content is as follows:

1. Create AI content such as a workflow, serving template, or Docker registry that is consumer ready.
2. Create a generic secret for broker registration.
3. Provide a service custom resource YAML in a registered git repository.
4. SAP AI Core creates a service broker for the content to be made available to the . The service broker handles onboarding and offboarding for end consumers.
5. Fetch the service broker information by calling the endpoint: `{{apiurl}}/v2/admin/services`.
6. Register the service broker in the and SAP Cloud Management service.
7. The consumer creates service instance from the [Service Marketplace](#).
8. SAP AI Core creates a resource group for the consumer with `<resourceGroupId> == serviceInstanceId`.
9. The consumer creates a service key and starts using the service.

Note

The service provider main tenant is allowed to provision only 1 Service.

SAP AI Core deploys a generic service broker instance for the main tenant as follows:



Parent topic: [Advanced Features \[page 269\]](#)

9.3.1.1 Service Custom Resource

The service provider main tenant needs to prepare the service custom resource. The custom resource contains service details, reference to broker credentials or secrets, and capabilities configured for service consumers.

An example service custom resource is provided in the following code block:

```
apiVersion: ai.sap.com/v1alpha1
kind: Service
metadata:
  name: sample-service
spec:
  brokerSecret:
    name: broker-credentials
    usernameKeyRef: username
    passwordKeyRef: password
  description: Service used for demos
  capabilities:
    basic:
      staticDeployments: true
      userDeployments: true
      createExecutions: true
    logs:
      executions: true
      deployments: true
  serviceCatalog:
    - extendCredentials:
        shared:
          serviceUrls:
            AI_API_URL: https://api.ai.internalprod.eu-central-1.aws.ml.hana.ondemand.com
  extendCatalog:
    name: sample-service
    id: sample-service-broker-id
    description: sample service
    bindable: true
    plans:
```

```

- id: sample-service-standard
description: Standard plan for sample service
name: standard
free: false
metadata:
  supportedPlatforms:
    - cloudfoundry
    - kubernetes
    - sapbtp

```

This can be used as a guide, with values amended as required. To create YAML descriptors, use any text editor with a YAML plugin.

For details about parameters, refer to the following table:

Service Parameter Details

Type	Parameter		Description
metadata	name		Name for the service
brokerSecret	name		Secret's name containing credentials to register Service Broker.
	usernameKeyRef		Key reference for username from registered Secret.
	passwordKeyRef		Key reference for password from registered Secret.
capabilities	basic	staticDeployments	Consumers can use existing deployments, but not allowed to Create, Update or Delete (default: true).
		userDeployments	Consumers can create deployments (default: true).
		createExecutions	Consumers can create executions (default: true).
	logs	executions	Consumers can access execution logs.
		deployments	Consumers can access deployment logs.

Note

It is mandatory to have this secret registered as a generic secret.

Type	Parameter	Description
	by default	Consumer has access to: - Create, read artifacts & configurations - Download, upload and delete datasets - Create, read, update & delete object store secrets
serviceCatalog	extendCredentials	Used to mention to be consumed Service URL. It will reflect in the service-key.
	extendCatalog	Used to extend service catalog.

⚠ Restriction

Updating of Service Custom Resource is not Supported

Once a service custom resource has synced successfully, updates of any parameters in templates will not take any effect.

9.3.1.2 Onboarding

To onboard a service, complete the following:

1. Create a brokerSecret to be used as credentials when registering the service broker.
 1. Use a new POST request to URL `{{apiurl}}/v2/admin/secrets`
 2. Provide credentials (base64 encoded) in the request body in JSON format:

```
{
  "name": "broker-credentials",
  "data": {
    "username": "bXktc2VjcmV0LWNyZWRLbnRpYWw=",
    "password": "bXktc2VjcmV0LW90aGVyLWNyZWRLbnRpYWw="
  }
}
```

3. Specify the scope of the request in the header: `AI-Tenant-Scope: true`
4. Send the request.
2. Modify the brokerSecret specification section using these details:

```
apiVersion: ai.sap.com/v1alpha1
kind: Service
metadata:
  name: sample-service
spec:
  brokerSecret:
    name: broker-credentials
    usernameKeyRef: username
    passwordKeyRef: password
```

```

description: Service used for demos
capabilities:
  basic:
    staticDeployments: true
    userDeployments: true
    createExecutions: true
  logs:
    executions: true
    deployments: true
serviceCatalog:
  - extendCredentials:
    shared:
      serviceUrls:
        AI_SVC_URL: https://api.ai.internalprod.eu-central-1.aws.ml.hana.ondemand.com
    extendCatalog:
      name: sample-service
      id: sample-service-broker-id
      description: sample service
      bindable: true
      plans:
        - id: sample-service-standard
          description: Standard plan for sample Service
          name: standard
          free: false
          metadata:
            supportedPlatforms:
              - cloudfoundry
              - kubernetes
              - sapbtp

```

Note

The username and password are key names from the broker credentials in the previous step.

3. Update the `spec.serviceCatalog[].extendCredentials` with the service URL you want to provide to the consumer, which will be part of the service key. Provide catalog details under `spec.serviceCatalog[].extendCatalog`.
4. Push your service custom resource to your registered GitHub repository, and wait for the sync to be successful.
5. Once it has synced, Check the service details by sending a GET request to URL `{{apiurl}}/v2/admin/services`.

```

{
  "count": 1
  "resources": [
    {
      "name": "sample-service",
      "description": "Service used for demos",
      "status": "PROVISIONED",
      "url": "https://aif-xyzabc.servicebroker.internalprod.eu-central-1.aws.ml.hana.ondemand.com"
    }
  ]
}

```

Take note of the service broker URL.

6. Register the service broker using `smctl` as subaccount-scoped.
 1. Test the registration of the service broker first as subaccount-scoped, before you register it globally. Subaccount-scoped means that your service is automatically visible in the catalog of environments where it's registered. You can follow steps described in Service Manager Guide to [set up for smctl](#) .

Once smctl is installed, login as shown:

```
# env variables
SERVICE_MANAGER_URL=<sm url e.g. https://service-
manager.cfapps.sap.hana.ondemand.com/>
SVC_SUBACCOUNT_USER=<user-with-servicemanager-
role>SVC_SUBACCOUNT_PWD=<password + 2FA>
SERVICE_BROKER_URL=https://aif-xyzabc.servicebroker.internalprod.eu-
central.aws.ml.hana.ondemand.com
SVC_SUBACCOUNT_SUBDOMAIN=<subaccount e.g. subaccountxyz>
SERVICE_BROKER_USER=<broker username provided in secret>
SERVICE_BROKER_PWD=<broker password provided in secret>
# smctl login
smctl login -a $SERVICE_MANAGER_URL \
  --param subdomain=$SVC_SUBACCOUNT_SUBDOMAIN \
  -u=$SVC_SUBACCOUNT_USER \
  -p=$SVC_SUBACCOUNT_PWD
```

2. Register your service-broker by providing the broker-name, URL, and credentials.

```
# register service-broker
smctl register-broker sample-service $SERVICE_BROKER_URL -b
$SERVICE_BROKER_USER:$SERVICE_BROKER_PWD
# service-broker registration should complete successfully
```

With the service-broker registered successfully, the service is available in the [Service Marketplace](#).

```
# assuming you are logged in to Cloud Foundry, provided correct subaccount, get
service plan information
cf marketplace -s sample-service
```

Consumers can now create a service instance and service-key. On creation of service instance, SAP AI Core will create a corresponding resource-group with id = instance id, and the service is now ready for use.

9.3.1.3 Offboarding

To prevent accidental deletion of the service, service providers must provide a deletion strategy as follows:

```
metadata:
  name: sample-service
  annotations:
    ai.sap.com/serviceDeletionStrategy: "delete"
```

With the serviceDeletionStrategy annotation, service providers can delete service custom resources from the git repository and proceed for offboarding. For successful service offboarding, all the consumer service instances should be deleted.

9.4 APIs and API Extensions

Explore APIs and API extensions that you can use with SAP AI Core.

APIs

Resource	Description	More Information
AI Core API	Explore the entities, methods, and endpoints provided by the API for SAP AI Core.	AI Core API 
AI API	Explore the entities, methods, and endpoints provided by the runtime-agnostic AI API.	AI API swagger specification

API Extensions

Resource	Description	More Information
Dataset Management Extension	Manage your dataset files.	AI API Dataset Extension swagger specification.
Resource Groups Extension	Access usage information for a tenant using the AI API.	AI API Admin API swagger specification
Analytics Extension	Add analytics capabilities to the AI API.	AI API Analytics Extension swagger specification
Metrics Extension	Query which capabilities of the metrics endpoint are supported.	AI API Metrics API swagger specification

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[ML Operations \[page 151\]](#)

[Metrics \[page 252\]](#)

[Advanced Features \[page 269\]](#)

[Libraries and SDKs \[page 277\]](#)

[Content Packages \[page 278\]](#)

9.5 Libraries and SDKs

Explore additional SDKs and Libraries, for use with SAP AI Core.

SDKs Available with SAP AI Core

Resource	Description	More Information
SAP AI Core SDK	The SAP AI Core SDK is a Python-based SDK that lets you access SAP AI Core using Python methods and data structures. It can be used to interact with SAP AI Core, and provides access to all public lifecycle and administration APIs.	• SAP AI Core SDK ➡ • Business Accelerator Hub ➡
AI API Client SDK	The AI API client SDK is a Python-based SDK that enables you to access the AI API using Python methods and data structures.	• AI API Client SDK ➡ • AI API
SAP generative AI hub SDK	With this SDK you can leverage the power of generative AI models available in the generative AI hub.	Generative AI hub SDK ➡

Libraries Available with SAP AI Core

Resource	Description	More Information
Metaflow Python Library for SAP AI Core	You can download the software development kit (SDK) for SAP AI Core from the Python Package Index (PyPI). This Python library is a plugin that lets you generate workflow templates that can be executed in SAP AI Core Metaflow is an open source tool to boost the productivity of data scientists. The Metaflow Python library for SAP AI Core extends Metaflow's capabilities so that machine learning pipelines can be run on Argo Workflows (which are also used in SAP AI Core).	• PyPi Metaflow ➡ • Metaflow ➡ • What are Argo workflows? ➡

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[ML Operations \[page 151\]](#)

[Metrics \[page 252\]](#)

[Advanced Features \[page 269\]](#)

[APIs and API Extensions \[page 276\]](#)

[Content Packages \[page 278\]](#)

[Metrics Tracking with AI API \[page 252\]](#)

9.6 Content Packages

Explore additional Content Packages for use with SAP AI Core.

Content Packages Available with SAP AI Core

Resource	Description	More Information
Data robot package	The content package for DataRobot integration for SAP AI Core.	Data robot package for SAP AI Core ➡
Computer vision package	The content package for image use cases in SAP AI Core adds image classification and feature extraction and is used with the AI Core SDK.	Computer vision package for SAP AI Core ➡

Parent topic: [Predictive AI \[page 151\]](#)

Related Information

[ML Operations \[page 151\]](#)

[Metrics \[page 252\]](#)

[Advanced Features \[page 269\]](#)

[APIs and API Extensions \[page 276\]](#)

[Libraries and SDKs \[page 277\]](#)

10 Tutorials

All available tutorials for SAP AI Core.

Starter Tutorials

[Quick Start for SAP AI Core](#)

Provision your account, register your keys and set up tools to connect with and operate the SAP AI Core SDK. Learn the basics by using SAP AI Core from beginning to end in a simple use case. The necessary codes are all provided directly in the steps of the tutorials, with YAML files made available in GitHub additionally.

[Quick Start for SAP AI Core with VS Code](#)

Provision your account, register your keys and set up tools to connect with and operate the SAP AI Core SDK. Learn the basics by using SAP AI Core from beginning to end in a simple use case. The necessary codes are all provided directly in the steps of the tutorials, with YAML files made available in GitHub additionally. This tutorial is the starter tutorial using VS Code.

[Metrics](#)

Explore different ways of logging metrics and comparing models with SAP AI Core.

Advanced Tutorials

In addition to the basic tutorials, the following tutorials are available for you in SAP Developer Center:

[Computer Vision Package for SAP AI Core](#)

Discover the Computer Vision Package with SAP AI Core.

[Pretrained Tensorflow Models](#)

Use Pre-Trained Tensorflow Models with a GPU in SAP AI Core.

Generative AI Hub Tutorials

[Generative AI Hub Starter Tutorial](#)

Activate the generative AI hub and learn the basics of composing prompts.

[Using foundation models with SAP AI Core](#)

Learn how to consume foundation models using SAP AI Core.

11 Security

Here, we'll explain some of the security aspects of SAP AI Core.

11.1 Security Features of Data, Data Flow, and Processes

The table below shows an overview of the data flow for SAP AI Core.

Overview of Data and Security Measures

Step	Description	Security Measure
1	Transmission control/communication security	Encrypted (HTTPS) communication. Data-in-transit is encrypted using state-of-the-art TLS settings.
2	Application data residing in persistence layer	Encrypted at rest with state-of-the-art encryption keys generated and maintained by SAP.
3	Application data residing in persistence layer	Backup and restore capabilities are implemented and tested regularly. Backups are stored in remote locations and backups are encrypted at rest with state-of-the-art encryption keys generated and maintained by SAP.
4	Access control and separation by purpose	Roles and scopes are available for implementing access control. Customer admin can use standard BTP security administration capabilities to assign roles to users to ensure "least privilege" and "segregation of duties".

11.2 Encryption in Transit

Communication with the service, including data upload and download, is encrypted using the transport layer security (TLS) protocol. SAP services support only the latest protocol versions (that is, TLS v1.2 and later) and strong cipher suites. Your systems must use the supported protocol versions and cipher suites to set up secure communication with the services. They must also validate the certificates against the services' domain names to avoid man-in-the-middle attacks.

11.3 User Authentication and Administration

SAP AI Core uses the user management and authentication mechanisms provided by SAP Authorization and Trust Management service (XSUAA).

For information about SAP Authorization and Trust Management service (XSUAA) in SAP BTP, see [What Is the SAP Authorization and Trust Management Service?](#)

Note

The data security provisions of SAP AI Core also apply to the SAP AI Core toolkit.

The SAP AI Core extension persists the connection details of the SAP AI Core engine. The connection details are persisted in an encrypted format in the application memory of the machine from which the user launches Visual Studio Code and installs the SAP AI Core toolkit extension.

User Authentication

Authentication in SAP AI Core is based on JSON web tokens (JWTs).

For named user communication, obtain a JWT token using password grant OAuth2.0 flow. The user's JWT token contains the scope as defined by the role or role collection assigned (see [User Roles \[page 281\]](#)). Consuming applications must have an application router or REST gateway that accepts the user name and password, and that obtains the user token.

For technical user communication, obtain a JWT token using the client credentials OAuth 2.0 flow.

User Roles

SAP AI Core provides role templates and role collections that are available under the SAP Authorization and Trust Management service (XSUAA). These templates and collections help you to manage and control access to SAP AI Core functionality and resources, such as scenarios, deployments, and executions.

A role template is a collection of scopes, references, and optional attribute references. A role collection is a set of role templates. Your security administrator can assign a role collection to a specific user or to a group of users.

Note

You must have Security Administrator permissions to create users and assign roles.

SAP AI Core supports the user roles outlined in [Roles and Authorizations \[page 282\]](#).

11.3.1 Roles and Authorizations

SAP AI Core provides default role collections that you can assign to users. The role collections determine which actions a user is able to carry out in SAP AI Core. You can also create your own role collections and assign the required roles to them.

Default Roles

SAP AI Core provides the following default roles:

Default Roles

Role	Description
aicore_resourcegroup_viewer	View resource groups of the SAP AI Core tenant
aicore_resourcegroup_editor view	Create and delete resource groups of the SAP AI Core tenant
aicore_repository_viewer	View GitOps repositories of the SAP AI Core tenant
aicore_repository_admin view	Create, update, and delete GitOps repositories of the SAP AI Core tenant
aicore_application_viewer	View GitOps applications of the SAP AI Core tenant
aicore_application_admin view	Create, update, and delete GitOps applications of the SAP AI Core tenant
aicore_log_viewer	View application logs of the SAP AI Core tenant
aicore_credential_viewer	View object store credentials and docker registry credentials (metadata only) of the SAP AI Core tenant
aicore_credential_admin	View, create, update, and delete object store credentials and docker registry credentials (metadata only) of the SAP AI Core tenant
scenario_metadata_viewer	View scenarios and scenario versions
scenario_executable_viewer	View executables of a scenario
scenario_configuration_viewer	View configurations of a scenario
scenario_configuration_editor	Edit configurations of a scenario
scenario_deployment_viewer	View or list deployments
scenario_deployment_editor view	Create, update, and delete deployments; view the application logs of the deployments

Role	Description
scenario_deployment_predictor	Invoke deployments of a scenario (for model inference purposes)
scenario_execution_viewer	View executions of a scenario and view tracking metrics
scenario_execution_editor view	Create, update, and delete executions and tracking metrics; view the application logs of the executions
scenario_artifact_viewer	View or list artifacts
scenario_artifact_editor	View, create, update, and delete artifacts
scenario_metric_viewer	View tracking metrics of an execution

Role Collections

SAP AI Core provides the following default role collections:

Role Collections and Roles

Role Collection	Roles
aicore_viewer	- scenario_metadata_viewer - scenario_executable_viewer - scenario_configuration_viewer - scenario_deployment_viewer - scenario_artifact_viewer - scenario_execution_viewer - scenario_metric_viewer - aicore_credential_viewer - aicore_connection_viewer - aicore_resourcegroup_viewer - aicore_application_viewer

Role Collection	Roles
aicore_admin	- scenario_configuration_editor - scenario_deployment_editor - scenario_deployment_predictor - scenario_execution_editor - scenario_artifact_editor - aicore_credential_admin - aicore_connection_admin - aicore_resourcegroup_editor - aicore_log_viewer - aicore_application_admin
aicore_scenario_viewer	- scenario_metadata_viewer - scenario_executable_viewer - scenario_configuration_viewer - scenario_deployment_viewer - scenario_artifact_viewer - scenario_execution_viewer - scenario_metric_viewer
aicore_scenario_editor	- scenario_configuration_editor - scenario_deployment_editor - scenario_deployment_predictor - scenario_execution_editor - scenario_artifact_editor
aicore_resourcegroup_viewer	aicore_resourcegroup_viewer
aicore_resourcegroup_editor	aicore_connection_admin
aicore_application_admin	aicore_application_admin
aicore_repository_admin	aicore_repository_admin

Related Information

[User Authentication and Administration \[page 280\]](#)

11.4 Docker Images

SAP AI Core supports tenant-specific Docker registries (registered via the administration APIs). Additional tenant workloads, such as for execution and deployments, can be created by referencing the Docker images from this Docker registry.

Docker images are cached on virtual machines. These cached Docker images cannot be accessed by other tenants and will not be accessed by SAP.

Cached Docker images are not deleted immediately upon tenant offboarding but are cleaned up as part of operational events such as cluster scaling-down behavior, maintenance, and upgrade of virtual machines.

With every service that you consume, there is a shared security responsibility between you and SAP. Because the creation of a Docker image is the responsibility of the tenant, we strongly recommend that you do not embed or hard-code personal data, sensitive data, or machine learning models inside your Docker images.

For security reasons, Docker containers in SAP AI Core are run as non-root only. For more information, see [Workflow Templates \[page 163\]](#).

11.5 AI Content Security

AI content covers workflow templates and serving templates, as well as the Docker images used in them. Docker images contain the machine-learning algorithms or code, along with the machine-learning libraries, frameworks, and other dependent packages. Ensure that you follow standard practices for developing secure software when working with AI content.

Security Practices

Practice	Description
Threat Modeling	Hold threat modeling workshops to identify and assess security risks or threats in the AI content.
Static Code Scans	Use static code scan (SAST) tools to scan and analyze the code for vulnerabilities.
Open-Source Vulnerability Scan	Scan any open-source components used by the product for vulnerabilities, and patch any vulnerable OSS components found.
Open-Source Strategy	Develop an update strategy that defines when the open-source components used in a product or service are updated to the latest secure version.
Code Reviews	Perform peer code reviews of each code change. The reviewer should take a closer look at the code from a security perspective.
Malware Scanning	Scan for malware in the data uploaded for AI content.
Secure Code Protection	Support security assurance starting with source code through deployed service. For example, use Docker image digest and image sign verifications.
Docker Base Image Security	Use a secure, light base image for building the Docker images for the AI content. Ensure you use the latest available base image and remove unused components from the Docker image.

11.6 Kubernetes Security

We recommend that you enable the relevant and applicable Kubernetes security features on your workflow templates and serving templates. Ensure that you enable the appropriate Kubernetes features for your workloads.

Related Information

[Security Best Practices for Kubernetes Deployment](#) ➡

11.7 Configuration Data and Secrets

Workloads can access network resources other than object stores by using credentials at runtime. The ways of including this information at runtime have different standards when it comes to confidentiality.

- For **sensitive** information:
SAP AI Core allows you to include secrets in the form of generic secrets. Generic secrets are created and managed by the REST APIs in SAP AI Core and consumed securely in a workload.
- For **non-sensitive** information:
You can include non-sensitive parameters using configurations or labels.

Note

These parameters may be returned in clear text (for example, in GET requests).

11.8 Output Encoding

To avoid breaking the business functionality, SAP AI Core does not sanitize any user input. Consumers or applications that consume the AI API are expected to perform necessary output encoding based on the usage context to prevent XSS attacks.

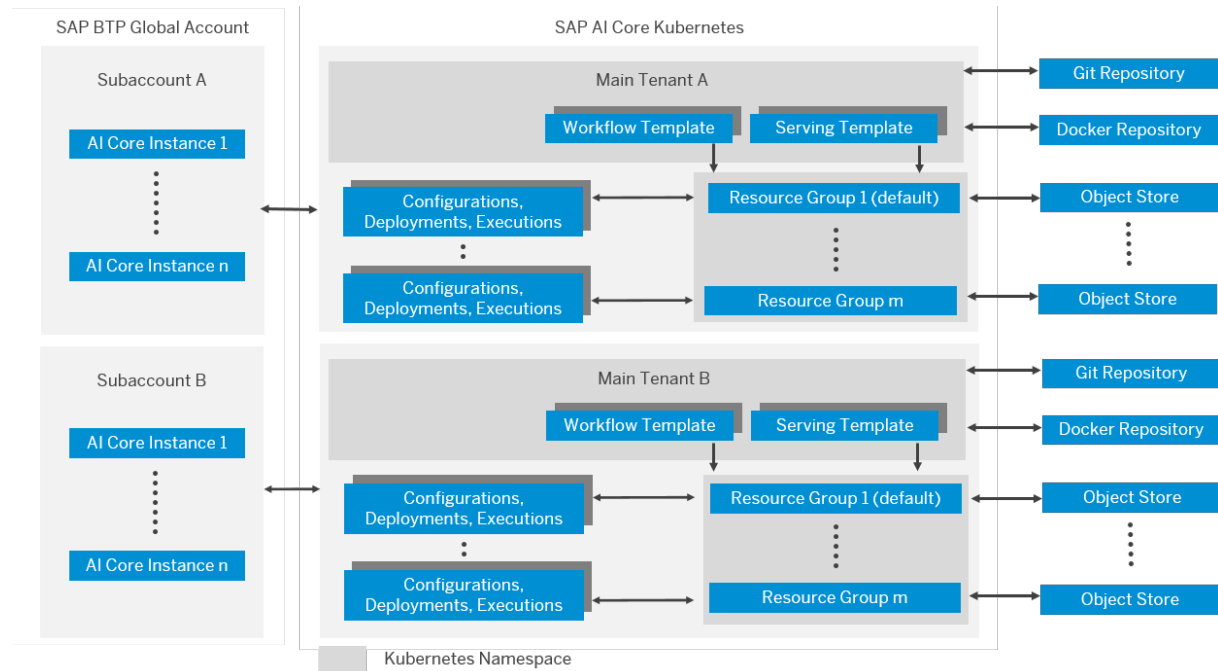
Related Information

[Cross Site Scripting \(XSS\) – OWASP Site](#) ➡

[Cross Site Scripting Prevention Cheat Sheet – OWASP Site](#) ➡

11.9 Multitenancy

SAP AI Core is a tenant-aware BTP reuse service, supporting main tenants and resource groups. Resources are defined for tenants or resource groups as outlined below:



Each main tenant and resource group is mapped to a namespace. The main tenant namespace only contains templates for workflows and model serving. The instances of these objects are created in the respective resource group namespaces and reference the corresponding templates in the main tenant namespace. Each main tenant has a default resource group, which can be used for workloads from the main tenant.

Tenant-Level Resources

Tenant-level resources include executables such as:

- Workflow templates
- Serving templates
- Docker registries that contain Docker images
- User authentication and authorization (UAA)

User authentication and authorization are based on the SAP AI Core tenant (access token obtained using the service key for SAP AI Core). At runtime or when managing the lifecycle via AI API, the SAP AI Core tenant must set the appropriate resource group in the request header.

Resource-Group-Level Resources

Executables at tenant level are shared across all resource groups. In contrast, runtime entities such as executions, deployments, configurations, and artifacts belong to a specific resource group and cannot be shared across resource groups. Similarly, generic secrets created within a resource group and be used only for workloads within that group.

You can register an object store at resource-group level by setting the resource group header. We recommend that you do not use the same object store bucket with the same IAM user for multiple resource groups.

Tenant Isolation of Workloads

Workloads run in a sandbox environment and cannot access workflows of other tenants or resource groups. Only TCP is supported for inbound or outbound traffic from a workload. Opening workloads to open Sockets on UDP ports is strongly discouraged. They are not usable, but may pose a theoretical security problem for the workload.

11.10 Auditing and Logging Information

Here you can find a list of the security events that are logged by SAP AI Core.

Security Events Written in Audit Logs

What Events Are Logged	How to Identify Related Log Events
Creation of object store secret	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new" : "Succeeded".
Deletion of object store secret	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new" : "Succeeded".
Successful retrieval of object store secret	Successfully read object store connection details for resource group <resource group name> and object store <object store name>
Provisioning of resource group	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new" : "Succeeded".
Deprovisioning of resource group	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new" : "Succeeded".

What Events Are Logged	How to Identify Related Log Events
Creation of docker registry secret	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Deletion of docker registry secret	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Creation of deployments	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Deletion of deployments	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Creation of executions	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Deletion of executions	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Creation of ArgoCD application	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Deletion of ArgoCD application	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Creation of repositories	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Deletion of repositories	Message containing a time stamp, tenant and user IDs, and attributes containing the API call and "new": "Succeeded".
Provisioning of tenant	Message containing a time stamp, tenant and user IDs, and attributes containing "name": "state", "new": "PENDING" and "name": "cfAccountState", "new": "ACTIVE" and success value true
Retrieval of tenant	Message containing Attribute <attribute detail> was read.

What Events Are Logged	How to Identify Related Log Events
Deprovisioning of tenant	Message containing a time stamp, tenant and user IDs, and attributes containing "name": "state", "new": "PENDING" and "name": "cfAccountState", "new": "DELETED" and success value <code>true</code> .
List Get Watch	Message containing a time stamp, tenant and user IDs, source IPs, the request URI, and the level.
Create Update Patch	Message containing a time stamp, tenant and user IDs, source IPs, the request URI, the level, and the request and response objects.
Delete	Message containing a time stamp, tenant and user IDs, source IPs, the request URI, the level, the and response object.
Token expired	Jwt is expired
No authorization header	RBAC: access denied
Invalid token	Jwt issuer is not configured
Wrong <code>tenantId</code>	Jwt verification fails

The following information is described in the table columns:

- [Event grouping](#) - Events that are logged with a similar format or are related to the same entities.
- [What events are logged](#) - Description of the security or data protection and privacy related event that is logged.
- [How to identify related log events](#) - Search criteria or key words, that are specific for a log event that is created along with the logged event.
- [Additional information](#) - Any related information that can be helpful.

Related Information

[Audit Logging in the Cloud Foundry Environment](#)

[Audit Logging in the Neo Environment](#)

11.11 Data Protection and Privacy

For general information about data protection and privacy on SAP Business Technology Platform, see [Data Protection and Privacy](#).

Data protection is associated with numerous legal requirements and privacy concerns. In addition to compliance with general data protection and privacy acts, it is necessary to consider compliance with industry-specific legislation in different countries/regions. This section describes the specific features and functions that SAP AI Core provides to support compliance with the relevant legal requirements and data privacy.

This guide does not give advice on whether these features and functions are the best method to support company, industry, regional, or country/region-specific requirements. Furthermore, this guide does not give advice or recommendations about additional features that would be required in a particular environment. Decisions related to data protection must be made on a case-by-case basis and under consideration of the given system landscape and the applicable legal requirements.

Note

SAP does not provide legal advice in any form. SAP software supports data protection compliance by providing security features and specific data protection-relevant functions, such as simplified blocking and deletion of personal data. In many cases, compliance with applicable data protection and privacy laws will not be covered by a product feature. Definitions and other terms used in this document are not taken from a particular legal source.

The extent to which data protection is ensured depends on secure system operation. Network security, security note implementation, adequate logging of system changes, and appropriate usage of the system are the basic technical requirements for compliance with data privacy legislation and other legislation.

For a glossary of Data Protection and Privacy terms in SAP BTP, see the SAP BTP [Glossary for Data Protection and Privacy](#).

11.11.1 Data Storage and Processing

SAP AI Core provides functionality that allows you to process data, such as configuration files or Machine Learning (ML) Training or ML Serving.

SAP AI Core acts as the data processor and is not aware of the type of data or category of data. SAP AI Core customers, as Data Controllers, are responsible for fulfilling data protection and privacy (DPP) responsibilities for data storage and processing requirements.

11.11.2 Change Logging and Read-Access Logging

SAP AI Core acts as the data processor and is not aware of the type of data or category of data. SAP AI Core customers, as Data Controllers, are responsible for fulfilling data protection and privacy (DPP) responsibilities for data storage and processing requirements.

For any applications or services you develop using SAP AI Core, you must ensure that they include relevant logging functions, and ensure compliance with the data privacy laws by making sure that the data is properly logged.

11.11.3 Consent

SAP AI Core acts as the data processor and is not aware of the type of data or category of data. SAP AI Core customers, as Data Controllers, are responsible for fulfilling asking for consent from data subjects before collecting any personal data.

11.11.4 Deletion

SAP AI Core supports **Bring your object store**, whereby customers register the object store secret where artifact relevant files (such as training data or machine learning models or other types) are stored. Such data is used by the ML workloads during processing, such as ML Training or ML Serving.

The notion of artifacts is limited to capture the metadata of the data. Data is physically stored in the object store and SAP AI Core is not responsible for deletion of files. When artifacts are deleted using AI API, corresponding metadata are deleted from SAP AI Core service and the actual files are not deleted from the object store. For more information about the AI API, see [AI API Overview \[page 25\]](#).

Upon offboarding, SAP AI Core will clean up the cached data within AI Core used for processing purposes.

SAP AI Core customers, as Data Controllers, are responsible for deletion of data from the registered object store.

11.11.5 Security and Customer Data Protection

SAP product standard security and the data protection and privacy (DPP) requirements set high standards and obligations when it comes to securing and protecting customer data that is entrusted to SAP.

Customer data protection is handled in three ways:

- Customer data is imported, output, and processed by the services for no purpose beyond that to which the customer has subscribed.
- Customer data is protected from malicious access by security technologies that include authentication and authorization.
- Customer data is protected from accidental exposure to SAP administrators or support persons by security policies, access controls, and monitoring.

12 Accessibility Features in SAP AI Core

To optimize your experience of SAP AI Core, SAP AI Core provides features and settings that help you use the software efficiently.

Note

SAP AI Core uses SAP AI Launchpad for its interface, which is based on SAPUI5. For this reason, accessibility features for SAPUI5 also apply. See the accessibility documentation for SAPUI5 on SAP Help Portal at [Accessibility for End Users](#).

For more information on-screen reader support and keyboard shortcuts, see [Screen-Reader Support for SAPUI5 Controls](#) and [Keyboard Handling for SAPUI5 Elements](#).

13 Monitoring and Troubleshooting

Explore solutions to potential issues, and find out how to get support.

Getting Support

If you encounter an issue with this service, we recommend that you follow the procedure below.

Check Platform Status

Check the availability of the platform at [SAP Trust Center](#).

For more information about platform availability, updates and notifications, see [Platform Updates and Notifications in the Cloud Foundry Environment](#).

Check Guided Answers

In the SAP Support Portal, check the [Guided Answers](#) section for SAP Cloud Platform. You can find solutions for general SAP Cloud Platform issues as well as for specific services there.

Contact SAP Support

You can report an incident or error through the [SAP Support Portal](#).

Please use the following component(s) for your incident:

Component Name	Component Description
CA-ML-AIC	SAP AI Core

We recommend that you include the following information when you submit the incident:

- Region information (Canary, EU10, US10)
- Subaccount technical name
- URL of the page where the incident or error occurs
- Steps or clicks used to replicate the error
- Screenshots, videos, or the code entered

13.1 Troubleshooting

For troubleshooting information, see the following sections:

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.1 Repository

Repository ra-aicore-test not found for tenant

You get the result:

```
{
  "error": {
    "code": "500",
    "details": [
      {
        "code": null,
        "message": "Repository ra-aicore-test not found for tenant
b82a8318"
      }
    ],
    "message": "Repository ra-aicore-test not found for tenant b82a8318"
  },
  "request_id": null,
  "target": "/api/v4alpha/repositories"
}
```

and:

```
AIAPIServerException: Failed to post /admin/repositories: Repository ra-aicore-
test not found for tenant 68
```

Follow the solution:

Use a different name for the value of the `name` parameter. The exception is raised by reuse of the name `aicore-test`.

```
response = ai_api_client.rest_client.post(
    path="/admin/repositories",
    body={
        "name": "aicore-test-1",
        "url": "https://github.com/John/aicore-test",
    }
)
print(response)
{'message': 'Repository has been on-boarded.'}
```

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.2 Configuration

Could not create configuration, because executable <x> for scenario <y> is not found

When you try to create a configuration, you are told that an executable cannot be found for your scenario.

Check the following:

1. Check that you are using the name value from your workflow for the executable ID in the configuration.

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: text-clf-train-tutorial
  annotations:
    scenarios.ai.sap.com/description: "SAP developers tutorial"
scenario"
...
```

Note

Do not use the value from `executables.ai.sap.com/id` as an executable ID.

2. Check that you are using the value of `executables.ai.sap.com/id` from your workflows as your scenario ID.

```
...
  artifacts.ai.sap.com/text-data.kind: "dataset"
  artifacts.ai.sap.com/text-model-tutorial.kind: "model"
labels:
  scenarios.ai.sap.com/id: "text-clf-tutorial"
  ai.sap.com/version: "2.1.0"
spec:
```


...

Log message: using minio client

Check the following:

1. Check that you are using the name value from your workflow for the executable ID in the configuration.

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: text-clf-train-tutorial
  annotations:
    scenarios.ai.sap.com/description: "SAP developers tutorial
scenario"
...
```

Note

Do not use the value from `executables.ai.sap.com/id` as an executable ID.

2. Check that you are using the value of `executables.ai.sap.com/id` from your workflows as your scenario ID.

```
...
  artifacts.ai.sap.com/text-data.kind: "dataset"
  artifacts.ai.sap.com/text-model-tutorial.kind: "model"
  labels:
    scenarios.ai.sap.com/id: "text-clf-tutorial"
    ai.sap.com/version: "2.1.0"
  spec:
...
```

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.3 Artifacts

No output artifact has been generated

Complete the following:

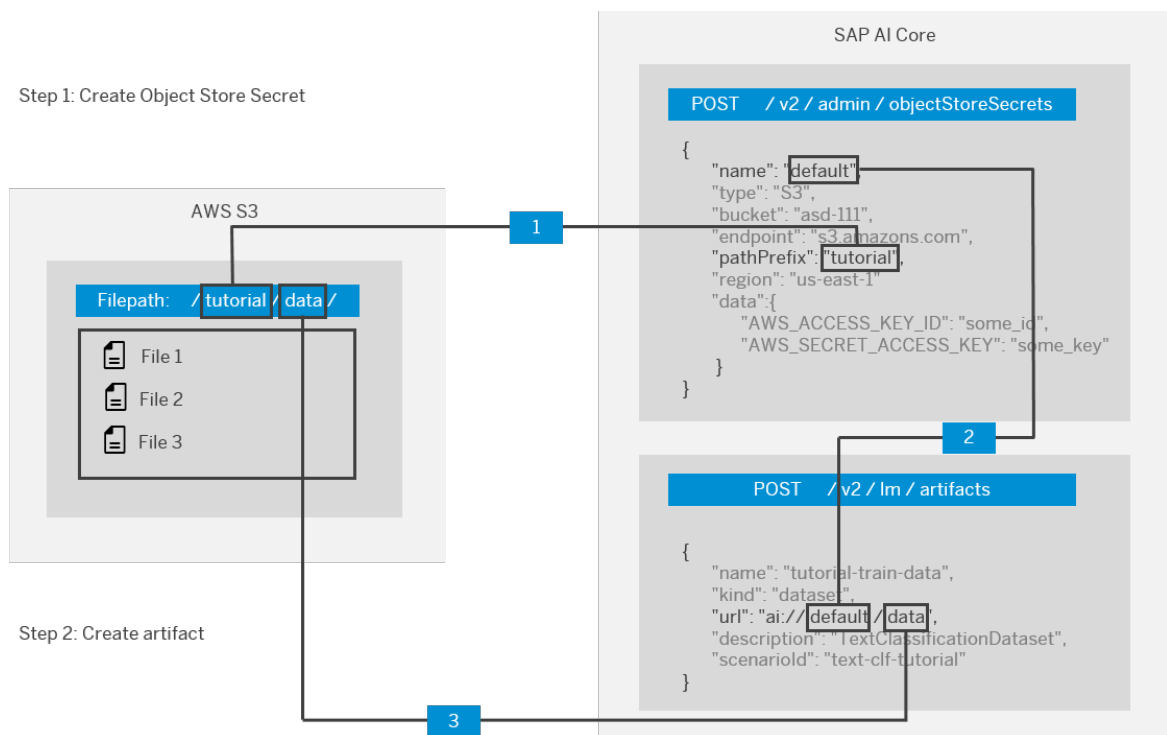
Define the `globalName` parameter for the output artifact in your workflow:

```
...
  executables.ai.sap.com/description: "Text classification Scikit training
  executable"
  executables.ai.sap.com/name: "text-clf-train-tutorial-exec"
  artifacts.ai.sap.com/text-data.kind: "dataset"
  artifacts.ai.sap.com/text-model-tutorial.kind: "mind"
  labels:
    scenarios.ai.sap.com/id: "text-clf-tutorial"
    ai.sap.com/version: "2.1.0"
  ...
  ...
    outputs:
      artifacts:
        -name: text-model-tutorial
        path: /app/model
        global name: text-model-tutorial
        archive:
          none: {}
      container:
    ...
```

Failed to load artifacts: The specified key does not exist

Complete the following:

1. Ensure you have created an object store secret using the naming convention `<name>` and the `pathPrefix` from your AWS S3 path. Refer to the following diagram:



- When creating the artifact, don't add the trailing forward slash (/) in URL parameter:
 - Incorrect usage: `"url": "ai://yourObjectStoreSecretName/folder/subfolder/"`
 - Correct usage: `"url": "ai://yourObjectStoreSecretName/folder/subfolder"`

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)
[Configuration \[page 296\]](#)
[Application \[page 300\]](#)
[Execution \[page 305\]](#)
[Docker \[page 307\]](#)
[Deployment \[page 308\]](#)
[Miscellaneous \[page 309\]](#)

13.1.4 Application

Templates are not synced via applications

After you have deleted or created an application, templates are not synced.

Follow the solution:

1. Delete the synced application using the endpoint:
DELETE `{{apiurl}}/v2/admin/applications/{{appName}}`
2. Offboard the associated GitHub repository using the endpoint:
DELETE `{{apiurl}}/v2/admin/repositories/{{repositoryName}}`
3. Onboard the associated GitHub repository using a personal access token instead of your GitHub password.
For more information, see [Creating a personal access token](#) .
POST `{{apiurl}}/v2/admin/repositories`

Sample Code

Body:

```
{
  "name": "aicore-test",
  "url": "https://github.com/john/aicore-test",
  "username": "john",
  "password": "yourGitHubPersonalAccessToken"
}
```

4. Create the application using the endpoint:
POST `{{apiurl}}/v2/admin/applications`
5. Check the ArgoCD applications to determine if the repository has been synchronized correctly for the tenant. For example, check that there are no duplicated workflow names. The value of the `name` parameter is considered as an executable ID.

```
name: text-clf-train-tutorial
  annotations:
    ...apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
```

6. Check that you're calling SAP AI Core using the expected tenant.
7. Check if the workflow templates contain the correct scenario label.
8. Get your application sync status using the endpoint:
GET `{{apiurl}}/v2/admin/applications/{{appName}}/status`
The status will any return errors in your templates. When your templates are updated, the application will resync automatically after approximately three minutes.

Executables do not appear when you retrieve them from a scenario

Follow the solution:

1. Delete the synced application using the endpoint:
DELETE `{{apiurl}}/v2/admin/repositories/{{appName}}`
2. Offboard the associated GitHub repository using the endpoint:
DELETE `{{apiurl}}/v2/admin/repositories/{{repositoryName}}`
3. Onboard the associated GitHub repository using a personal access token instead of your GitHub password.
For more information, see [Creating a personal access token](#) .
POST `{{apiurl}}/v2/admin/repositories`

Sample Code

Body:

```
apiVersion: argoproj.io/v1alpha1{
  "name": "aicore-test",
  "url": "https://github.com/john/aicore-test",
  "username": "john",
  "password": "yourGitHubPersonalAccessToken"
}
```

4. Create the application using the endpoint:
POST `{{apiurl}}/v2/admin/applications`
5. Check the ArgoCD applications to determine if the repository has been synchronized correctly for the tenant. For example, check that there are no duplicated workflow names. The value of the `name` parameter is considered as an executable ID.

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: text-clf-train-tutorial
  annotations:
    ...
```

6. Check if the user is calling SAP AI Core using the expected tenant.
7. Check that the scenario label is correct in the workflow templates.
8. Get your application sync status using the endpoint:
GET `{{apiurl}}/v2/admin/applications/{{appName}}/status`
The status will any return errors in your templates. When your templates are updated, the application will resync automatically after approximately three minutes.

Application status returns `healthy`, but most other properties are `unknown`

Follow the solution:

1. Delete the synced application using the endpoint:
DELETE `{{apiurl}}/v2/admin/applications/{{appName}}`
2. Offboard the associated GitHub repository using the endpoint:
DELETE `{{apiurl}}/v2/admin/repositories/{{repositoryName}}`

3. Onboard the associated GitHub repository using a personal access token instead of your GitHub password. For more information, see [Creating a personal access token](#) .

POST `{{apiurl}}/v2/admin/repositories`

Sample Code

Body:

```
{
  "name": "aicore-test",
  "url": "https://github.com/john/aicore-test",
  "username": "john",
  "password": "yourGitHubPersonalAccessTokenHere"
}
```

4. Create the application using the endpoint:
POST `{{apiurl}}/v2/admin/applications`
5. Check the ArgoCD applications to determine if the repository has been synchronized correctly for the tenant. For example, check that there are no duplicated workflow names. The value of the `name` parameter is considered as an executable ID.

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: text-clf-train-tutorial
  annotations:
    ...
```

6. Check if the user is calling SAP AI Core using the expected tenant.
7. Check if the workflow templates contain the correct scenario label.
8. Get your application sync status using the endpoint:
GET `{{apiurl}}/v2/admin/applications/{{appName}}/status`
The status will any return errors in your templates. When your templates are updated, the application will resync automatically after approximately three minutes.

Application status message: `rpc error: code = Unknown desc = my-path: app path does not exist`

The specified path in your application doesn't exist in your repository.

Follow the solution:

Delete your application and create a new one using the correct path.

Application status message: `application repo <your git repository> is not permitted in project 'xyz'`

The repository URL cannot be found in your onboarded repositories.

Check the following:

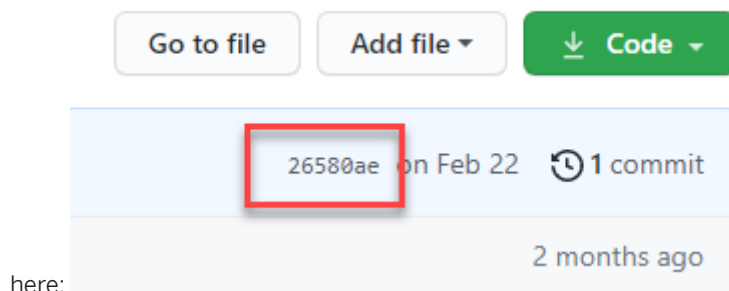
Make sure that the repository specified in your application has been successfully onboarded by using **GET** `{{apiurl}}/v2/admin/applications` and checking that the repository url has "status": "COMPLETED"

Application status message: rpc error: code = Unknown desc = Unable to resolve 'not-existing-branch' to a commit SHA

The revision you specified in your application doesn't exist in your repository.

Follow the solution:

Delete your application and create a new one using the correct revision. The revision number is found in GitHub



here:

Alternatively, enter **HEAD** to refer to the latest commit.

Application status message: rpc error: code = FailedPrecondition desc = Failed to unmarshal \"workflow.yaml\": failed to unmarshal manifest: error converting YAML to JSON: yaml: line 7: mapping values are not allowed in this context

You have a syntax error in your workflow template.

Follow the solution:

Use Argo Lint to identify syntax errors in your workflow template. To set up the Argo Lint IDE, refer to [Argo Lint IDE setup](#)

Application status message: spec.source.repoURL and spec.source.path either spec.source.chart are required

You specified an empty path in your application.

```
{  
  "healthStatus": "Unknown",
```

```

    "message": "spec.source.repoURL and spec.source.path either
spec.source.chart are required",
    "reconciledAt": "Unknown",
    "source": {
      "path": "Unknown",
      "repoURL": "Unknown",
      "revision": "Unknown"
    },
    "syncFinishedAt": "Unknown",
    "syncRessourcesStatus": [],
    "syncStartedAt": "Unknown",
    "syncStatus": "Unknown"
  }
}

```

Follow the solution:

Delete your application and create a new one, specifying a path. Check its status by using the endpoint:
`{{apiurl}}/v2/admin/applications/{{appName}}/status`

Sync an Application Manually

Applications sync with your GitHub repository automatically at intervals of ~3 minutes. Use the endpoint below to manually request a sync:

`{{apiurl}}/admin/applications/{{appName}}/refresh`

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.5 Execution

Execution status is DEAD or PENDING for a long time

Check the following:

1. Check the execution logs.
2. Check that the parameter name and execution name that you provided match those in your template. Note that the names are case-sensitive.

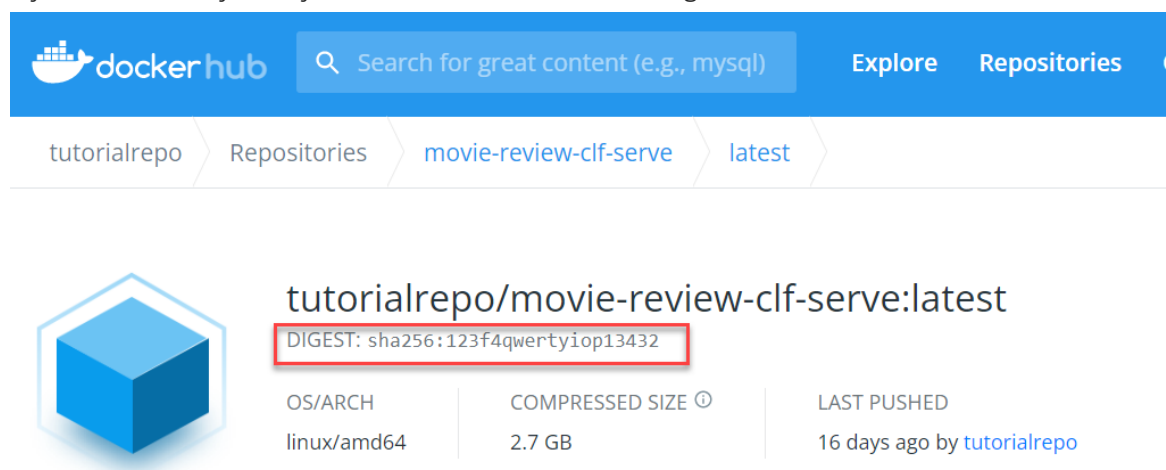
Images from private docker.io cannot be pulled for execution

Follow the solution:

1. Specify the domain name of your Docker Hub. For example:

```
...
      none: {}
    container:
      image: "docker.io/tutorialrepo/movie-review-clf-serve@sha256:123f4qwertyiop13432"
      imagePullPolicy: Always
...
```

If you do not already know your domain name, refer to the image:



2. Specify your Docker Image digest instead of the image version in your templates:
image tutorialrepo/movie-review-clf-serve@sha256:123f4qwertyiop13432

Output artifacts in the execution list are empty and have status UNKNOWN for a long time

Check the following:

1. Check that an object secret with the name `default` exists in the current resource group. For more information, see [Register an Object Store Secret \[page 79\]](#)
2. Check that you have created the Docker Registry secrets required to pull your Docker Images. Logs will be available only after the execution has started.

GET execution has the status UNKNOWN for a long time

Check the following:

1. Check if an object secret with the name `default` exists in the current resource group. For more information, see [Register an Object Store Secret \[page 79\]](#)
2. Check that you have created the Docker Registry secrets required to pull your Docker Images. Logs will be available only after the execution has started.

Execution status changes to DEAD without any log

Check the following:

1. Check that your templates meet the Argo specifications and that they can be executed by SAP AI Core.
2. To identify errors automatically in your templates, use the Argo linter.
3. Check that you have created Docker folders, to store artifacts that will be created during an execution.

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.6 Docker

Error pulling container main logs. Back-off pulling image

You push a template to SAP AI Core and you get the error "Error pulling container main logs".
"Back-off pulling image".

Complete the following steps:

1. Make sure that your Docker Registry is public-facing and not protected behind the firewall of your organization.
If your Docker Image isn't public, verify that you have created a Docker Registry secret in SAP AI Core.
2. Check that you can pull your Docker Image on your local computer using the credentials from the previous step.
3. Specify the same Docker Registry secret in your executable.
4. Specify the path to your Docker Image in your executable in following format:
<DOCKER_REGISTRY>/<REPO_NAME>/<DOCKER_IMAGE>:<TAGNAME>

❖ Example

```
docker.io/tutorialrepo/text-clf-train:0.0.1
```

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Deployment \[page 308\]](#)

[Miscellaneous \[page 309\]](#)

13.1.7 Deployment

You want to force a deployment with status UNKNOWN to stop

You want to stop and delete a deployment but you are unable to because the deployment status is “Unknown”. You have tried to submit a PATCH request as follows:

PATCH `{{apiurl}}/lm/deployments/d4fec9c24c54f87e`

However, you receive the following response:

```
{
  "error": {
    "code": "01010076"
    "message": "Invalid Request, Current status UNKNOWN cannot be changed..",
    "requestID": "e110820e-1cfe-456a-bb0e-77907b36422c",
    "target": "/apu/v2/deployments/d4fec9c24c54f87e"
  }
}
```

Complete the following steps:

1. Find out why your deployment status is “Unknown” by using the endpoint:
GET `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`
2. Delete the deployment without trying to stop it (stopping a deployment is necessary only when it's running):
DELETE `{{apiurl}}/v2/lm/deployments/{{deploymentid}}`

Deployment remains in status PENDING

Check the following:

1. Check that the Docker Registry secret exists when using your private Docker Image.
2. Check that your Docker Image can be downloaded to your local system.

Deployment ID `<abc>` not found

This message appears when you have just started the deployment. Wait a few minutes and the message will resolve itself automatically.

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Miscellaneous \[page 309\]](#)

13.1.8 Miscellaneous

403 - forbidden: RBAC Access denied

When you submit a POST request for an execution or a configuration, you get the error **403 - forbidden: RBAC Access denied**.

Check the following:

1. Check that you are passing the correct token and `ai-resource-group` header.
2. Check your tenant provisioning.

You get a runtime adapter exception

You get the error: `"Runtime Adapter Exception; Failed to post deployments : {\n\n\"code\": \"400\", \n\n\"message\": \"Missing input parameter or artifacts, one or more placeholder values are not resolved in the serving spec and error is 'dict object' has no attribute '\".\\n}\\n\""`

Check the following:

Check that your input artifact key doesn't contain any separators, such as `“example-artifact-key”`. If it does, rename the key (for example, `“exampleArtifactKey”`)

You have pushed your template to your GitHub repository but you can't see the executable created via the API

There may be an error in your template. Check the following:

- Serving templates always expect an input parameter to be configured. If you don't have a parameter in your Serving template, add a dummy parameter and create a configuration for it.
- Hyphens (-) are the only separator as permitted in the template name.

You have created a scenario in a template but you cannot see it in the AI API calls

Use the following solution:

Add a workflow template with the new `scenario_id` (not a serving template) to make the scenario visible.

Error: getaddrinfo ENOTFOUND

Check the following:

1. Check that all environment variables match your SAP AI Core service keys. Specifically, check:
 - `auth_url`
 - `client_id`
 - `client_secret`
 - `apiurl`
2. Submit a GET request to the endpoint `{{apiurl}}/v2/admin/repositories`
3. If the issue persists, contact SAP support as described at [Monitoring and Troubleshooting \[page 294\]](#).

Git repo doesn't synchronize with SAP AI Core instances.

When you try to synchronize your git repository with SAP AI Core, the response shows empty fields.

Check the following:

1. Check that you are using a GitHub personal access token and not your GitHub password.

If you are already using a personal access token, proceed as follows:

1. Delete all UNKNOWN applications from your SAP AI Core instance using the endpoint:
`DELETE {{apiurl}}/v2/admin/applications/{{appName}}`
2. Offboard your GitHub repo from SAP AI Core by calling the endpoint:
`DELETE {{apiurl}}/v2/admin/repositories/{{repositoryName}}`
3. Generate a GitHub personal access token and use it to onboard your git repo to SAP AI Core as before. For more information, see [Creating a personal access token](#) 📖.

4. Sync your application again.

Parent topic: [Troubleshooting \[page 294\]](#)

Related Information

[Repository \[page 295\]](#)

[Configuration \[page 296\]](#)

[Artifacts \[page 298\]](#)

[Application \[page 300\]](#)

[Execution \[page 305\]](#)

[Docker \[page 307\]](#)

[Deployment \[page 308\]](#)

14 Service Offboarding

Tenant offboarding occurs when a customer deletes a subaccount. SAP AI Core polls for the subaccount deletion event and performs the necessary deprovisioning and deletion activities.

ⓘ Note

Data and resources are not deleted when a service instance is deleted (because we don't isolate based on the service instance). If you want to keep the subaccount but still deprovision SAP AI Core, create a **medium support ticket** on component CA-ML-AIC with the title **Service Offboarding** and request that your data and resources be deleted manually.

Important Disclaimers and Legal Information

Hyperlinks

Some links are classified by an icon and/or a mouseover text. These links provide additional information.

About the icons:

- Links with the icon  : You are entering a Web site that is not hosted by SAP. By using such links, you agree (unless expressly stated otherwise in your agreements with SAP) to this:
 - The content of the linked-to site is not SAP documentation. You may not infer any product claims against SAP based on this information.
 - SAP does not agree or disagree with the content on the linked-to site, nor does SAP warrant the availability and correctness. SAP shall not be liable for any damages caused by the use of such content unless damages have been caused by SAP's gross negligence or willful misconduct.
- Links with the icon  : You are leaving the documentation for that particular SAP product or service and are entering an SAP-hosted Web site. By using such links, you agree that (unless expressly stated otherwise in your agreements with SAP) you may not infer any product claims against SAP based on this information.

Videos Hosted on External Platforms

Some videos may point to third-party video hosting platforms. SAP cannot guarantee the future availability of videos stored on these platforms. Furthermore, any advertisements or other content hosted on these platforms (for example, suggested videos or by navigating to other videos hosted on the same site), are not within the control or responsibility of SAP.

Beta and Other Experimental Features

Experimental features are not part of the officially delivered scope that SAP guarantees for future releases. This means that experimental features may be changed by SAP at any time for any reason without notice. Experimental features are not for productive use. You may not demonstrate, test, examine, evaluate or otherwise use the experimental features in a live operating environment or with data that has not been sufficiently backed up.

The purpose of experimental features is to get feedback early on, allowing customers and partners to influence the future product accordingly. By providing your feedback (e.g. in the SAP Community), you accept that intellectual property rights of the contributions or derivative works shall remain the exclusive property of SAP.

Example Code

Any software coding and/or code snippets are examples. They are not for productive use. The example code is only intended to better explain and visualize the syntax and phrasing rules. SAP does not warrant the correctness and completeness of the example code. SAP shall not be liable for errors or damages caused by the use of example code unless damages have been caused by SAP's gross negligence or willful misconduct.

Bias-Free Language

SAP supports a culture of diversity and inclusion. Whenever possible, we use unbiased language in our documentation to refer to people of all cultures, ethnicities, genders, and abilities.

© 2024 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

Please see <https://www.sap.com/about/legal/trademark.html> for additional trademark information and notices.